

Introduction to Statistical Modeling

Michael R. Hughes and Thomas J. Fisher

2025-06-26

Contents

Preface

DatasauRus Rex



Source: `datasaurus_dozen` from `datasauRus` package

Figure 1: A 'fun' plot: A scatterplot resulting in the shape of a Tyrannosaurus [R-datasauRus] with shaded Voronoi cells using the 'ggvoronoi' package [R-ggvoronoi]

This online text was designed for **STA 363 - Introduction to Statistical Modeling** at Miami University and has been used as a resource in other courses at Miami University (*e.g.*, STA 672 - Statistical Modeling and Study Design, a service course for graduate students in the life sciences). The original version of this document was not intended for broad publication but, as is common, has evolved into an textbook of sorts. What is now this text was originally a set of class notes written by Mr. Mike Hughes for use during class. Many of the data

and coding examples have been updated and everything has been structured such that this work can largely *stand alone*.

The bulletin description for the STA 363 course states:

Applications of statistics using regression and design of experiments techniques. Regression topics include simple linear regression, correlation, multiple regression and selection of the best model. Design topics include the completely randomized design, multiple comparisons, blocking and factorials.

The book and course have been designed to be a follow-up to a standard introductory statistics course (in many ways, this course can be considered “Intro Stat 2”). The course and text assumes the reader has a solid foundation in two-sample inference and some basic computing skills.

The book mixes statistical background with applications using the R Project for Statistical Computing (?). We have attempted to perform all data processing and graphics in this text using the `tidyverse` (?); thus all plots and data processing should follow the *tidy* grammar in R (`ggplot`, piping `|>` and `dplyr` function calls). All statistical modeling uses base R function calls (*e.g.*, `lm()` and `aov()`) and the `emmeans` package (?) is used for follow-up comparisons.

The book was designed with 13 chapters with the intent to be used during the 14-week semester at Miami University (one week reserved for midterm examination and end-of-semester review). The first two chapters are very comprehensive. Chapter 1 reviews introductory statistics material and provides a crash course in the basics of R, some elementary data handling with `dplyr` (?) and making plots with `ggplot2` (?). The development of the two-sample *t*-test is reviewed. Chapter 2 discusses the difference between designed experiments and observational studies, and extends the idea of two-sample inference into *multiple samples*. The key concepts of experimental design are introduced, some mathematical foundation for statistical models is introduced, and One-Way ANOVA is fully covered, including follow-up multiple comparisons. Chapters 3 and 4 cover some advanced design ideas (multiple factors, blocking and repeated measures), presenting all inference in the form of a linear model. Chapters 5–10 cover the *classic* topics around multiple linear regression as well as some *data science* concepts (prediction, variable selection and model validation). Chapters 11 and 12 provides an overview of statistical odds and logistic regression. Chapter 13 provides an overview of Generalized Linear Modeling with examples of both Poisson and Negative Binomial regression.

About the Authors

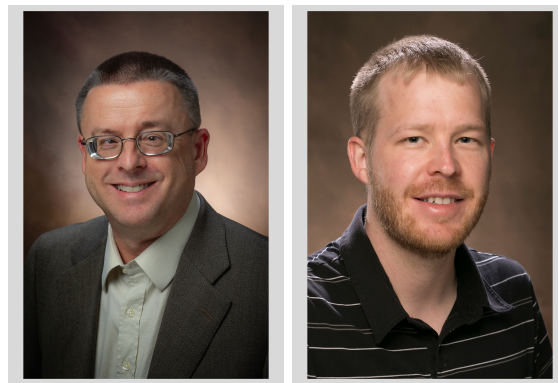


Figure 2: Photographs of the authors: Michael R. Hughes (left) and Thomas J. Fisher (right)

Michael R. Hughes is an instructor and manager of the statistical consulting center at Miami University. He has a B.S. in Mathematics and Statistics ('85) and an M.S. in Statistics ('87) from Miami University and typically teaches the nonparametric, statistical modeling and data practicum courses. He spends his free time snuggling up to a roaring fireplace with the most recent texts, and loves long walks on the beach thinking about the latest modeling techniques and applications. He seeks a ambitious group of students who wish to be taught the subject of statistical modeling.

Thomas J. Fisher is a Pisces who enjoys hockey and hiking. He has a PhD in Mathematical Sciences ('09) from Clemson University and a B.S. in Computer Science ('03) from the University of Maryland, Baltimore County. He has taught a wide-range of courses from *intro stat* to graduate-level probability and inference. When not teaching Statistical modeling, you'll most likely find him teaching one of the coding courses or skating in the Goggin Ice Center.

Important Preliminary Review

Statistics background

This text assumes the reader has completed an introductory statistics course and a college algebra course. From a mathematics perspective, the reader should be comfortable with linear equations, power and logarithmic functions, and general mathematical operations. No calculus is required although in a few cases we may *hint* at its connection to a specific topic.

For those needing a refresher on introductory statistics, two links are provided below.

<https://bookdown.org/chesterismay/rbasics/>

<http://moderndive.com/index.html>

Software

The text utilizes the R computing system. The courses, STA 363 - Introduction to Statistical Modeling and STA 672 - Statistical Modeling and Study Design, both utilizes RStudio and Rmarkdown.

R is a free statistical software platform that can be easily downloaded from the Comprehensive R Archive Network (CRAN): <https://cran.r-project.org/>

You can download RStudio from here: <https://rstudio.com/products/rstudio/download/#download>

Detailed instructions for installing R and RStudio can be found in this video: https://www.youtube.com/watch?v=d-u_7vdag-0

An overview of using R and RStudio can be found here: <https://www.youtube.com/watch?v=IVKMsaWju8w&t=458s>

Add-on packages

We will be using the following add-on packages in this textbook (*note*: additional packages may be used in the course).

- tidyverse (?)
- knitr (?)
- kableExtra (?)
- GGally (?)
- ggfortify (?)
- gridExtra (?)
- emmeans (?)
- lindia (?)
- car (?)
- leaps (?)
- caret (?)

You can install all of these packages by following the directions here: <https://www.youtube.com/watch?v=u1r5XTqrCTQ>

Or by running the following command:

```
install.packages(c("tidyverse",  
                  "knitr",  
                  "kableExtra",  
                  "GGally",  
                  "ggfortify",  
                  "gridExtra",  
                  "emmeans",  
                  "lindia",  
                  "car",  
                  "leaps",  
                  "caret"))
```

IMPORTANT NOTE: If running on a Mac, when asked, “Do you want to install from source?” In general it is better to select “No”.

Help with RMarkdown

Throughout the STA 363 course, Rmarkdown is utilized. In fact, this textbook was written using RMarkdown. You can find a tutorial of RMarkdown here: <https://www.youtube.com/watch?v=tKUufzpoHDE>

RStudio provides tutorials as well: <https://rmarkdown.rstudio.com/lesson-1.html>

Managing your work in R

We recommend using ‘Projects’ in RStudio, which links all code and data to a folder in your computer: <https://www.youtube.com/watch?v=pyJMWIDptYw>

Better yet, create a Github account, and github repository to manage your code and analysis: https://www.youtube.com/watch?v=kL6L2MNqPHg&ab_channel=IDGTECHtalk

Data in this text

The data used in this text is available on the hosting website (<https://github.com/tjfisher19/introStatModeling/docs/data>) and links are provided throughout the text. Alternatively, all data used in the text can be downloaded as a single zip file: [zipFileLinkHere](#)

Chapter 1

Introductory Statistics in R

We begin our dive into Statistical Modeling by first reviewing content from introductory statistics and introduce fundamental elements of the R programming language (?). The learning objective of this chapter include:

- Review of the fundamental concepts of *populations*, *samples*, *parameters* and *statistics*.
- Review of exploratory and descriptive statistics: numerical quantities and graphs.
- An introduction and overview of the R software for statistical computing.
- Basic statistical inference through a two-sample *t*-test and confidence intervals.

1.1 Goals of a statistical analysis

The field of **Statistics** can broadly be defined as the study of variation.

We collect **sample** data from a parent **population** of interest. Ultimately, we are interested in characteristics or **parameters** about the **population**. We use a **sample** because it is unfeasible or overly costly to collect data from every member of a **population** (that process is known as a *census*).

Good statistical practice demands that we collect the **sample** data from the **population** in such a manner so as to ensure that the **sample** is representative; *i.e.*, it is not presenting a systematic bias in its representation. With a *good* sample, we can make decisions about the population using that sample.

There are two main sources of variation that arise in statistical analysis:

1. Measurements of the same attribute will vary from observation to observation. This is natural and is to be expected. Consider how your height may vary compared to your parents and siblings. This *random* (or *stochastic*)

component is typically modeled through probabilistic assumptions (think the *Normal* distribution from your Intro Stat course). This component is often called *noise* or *error*, although the later is a misnomer.

2. We are interested in some **parameter** that helps describes characteristics of the population. Any characteristic computed from the sample is known as a sample **statistic**. Ideally, this **statistic** will reasonably approximate the **parameter** of interest. Since only a subset of the population is used in the calculation of a **statistic**, it will *vary* from their corresponding population **parameter**. In fact, different samples from the same population will produce different estimates of the same parameter, so sample statistics also vary from sample to sample.

This variation gives rise to the concept of uncertainty in any findings we make about a population based on sample data. A meaningful statistical analysis helps quantify this uncertainty.

The **goals** of a statistical analysis typically are

- to make sense of data in the face of uncertainty,
- to meaningfully describe the patterns of variation in collected sample data,
 - and use this information to make reliable inferences/generalizations about the parent population.

1.1.1 Conducting a statistical analysis

The practice of statistics starts with an underlying problem or contextual question. Prior to performing an analysis, we must that original question. Some examples may include:

- Are fish sensitive to Ultraviolet light?
- What socio-economic variables and demographics predict college graduation rates?
- Can we determine if there truly is a *home field* advantage in sports?
- Does a new experimental drug improve over an existing drug?

From there, data is collected through an experiment, by observation or via a survey. You then proceed with a data analysis, and then finish with statistical and contextual conclusions.

Unfortunately, it is common for inexperienced statisticians to start with a complex analysis without ever considering the objectives of the study. It is also common for inexperienced statisticians to not consider whether the data are appropriate for the chosen analysis and whether the underlying question can be answered.

We recommend you follow the guidance (outlined below) provided in Chapter 1 of the text by ? – Look before you leap!

Problem formulation - To perform statistical analysis correctly, you should:

1. **Understand the context.** Statisticians often work in collaboration with researchers in other disciplines and need to understand something about the subject area. You should regard this as an opportunity to learn something new. In essence, a Statistician must be a renaissance person – some level of *elementary* expertise is needed in the subject area.
2. **Understand the objectives.** Again, often you will be working with a collaborator who may not be clear about their objectives. Ideally they approach a statistician before data collection. This helps alleviate several potential issues. The statistician can gain an understanding of the contextual problem, and assess the appropriateness of data collection procedure to answer the underlying contextual questions.

Beware of contextual problems that are “fishing expeditions”: if you look hard enough, you’ll almost always find something. But... that something may just be a coincidence of no meaningful consequence. Also, sometimes collaborators request a more complicated analysis is performed than is necessary. You may find that simple descriptive statistics are all that are really required. Even if a more complex analysis is necessary, simple descriptive statistics can provide a valuable supplement.

3. **Translate the problem into statistical terms.** This can be a challenging step and oftentimes requires creativity on the part of the statistician (in other words, not every problem fits into a neat little box like in an intro stats course!). Care must be taken at this step.

Connected to the above guidance is a good understanding of the **Data Collection** - It is vitally important to understand how the data was collected. Consider:

- Are the data observational or experimental? Are the data a “sample of convenience”, or were they obtained via random sampling? Did the experiment include randomization? How the data were collected has a crucial impact on what conclusions can be meaningfully made. We discuss this topic more in Section ??.
- Is there “non-response”? The data you don’t see may be just as important as the data you do see! This aligns with the concern of whether a sample is representative of the intended target population.
- Are there missing values? This is a common problem that can be troublesome in some applications but not a concern in others. Data that are “missing” according to some pattern or systematic procedure are particularly troublesome.
- How are the data coded? In particular, how are the qualitative or categorical variables represented? This is important on the implementation of statistical methods in R (and other languages).
- What are the units of measurement? Sometimes data are collected or represented using far more digits than are necessary. Consider rounding if this will help with the interpretation.

- Beware of data entry errors. This problem is all too common, and is almost a certainty in any real dataset of at least moderate size. Perform some data sanity checks.

1.2 Introduction to R

Much like a chemist using test tubes & beakers, or a microbiologist using a microscope, the *laboratory instrument* for modern statistics is the computer. Many software packages exist to aid a modern statistician in the conducting of an analysis. This text uses the **R Language and Environment for Statistical Computing** (?). Many other tools exist and they can generally be lumped into two categories: *point and click* interfaces, and *coding* languages. The *point and click* tools tend to be easier to use but may have limited features & methods available. The *coding* languages are substantially more versatile, powerful and provide solutions that are reproducible, but may have a steeper learning curve. We recap a few of the common tools below.

- **Excel** - Microsoft Excel is a common *point and click* spreadsheet tool that includes many statistical methods. In fact, it is possible you used Excel in your Introductory Statistics course.
- **SPSS** - Also known as the Statistical Package for the Social Sciences, SPSS looks similar to Excel but has many more statistical tools included. SPSS includes the ability to code in Python.
- **Minitab** - Another *point and click* statistical tool. It's appearance is similar to Excel but, similar to SPSS, includes most common statistical methods.
- **JMP** - Pronounced as "jump". This tool includes a graphical interface (much like Excel) but also provides the user the ability to code. It is commonly used in the design of experiments and some intro stat courses.
- **SAS** - arguable the first statistical software that was widely used. SAS is a coding language that has powerful data handling abilities and includes implementations for many statistical methods. There is a graphical version known as Enterprise Miner.
- **Python** - A high-level general-purpose programming language, Python has become very popular in the Data Science industry due to its ease of use, the large community of support and add-ons (called modules) that implement data analysis methods.

So why R?? - With so many tools available, you may ask, "why do we use R?" Simply put, R incorporates many of the features available in these other languages. Specifically,

- Similar to the other coding languages, R provides reproducible results!
- R is free!
- Installing a working version of R tends to be easier than Python.
- Similar to Python, R is fairly easy to learn.

- But unlike in Python, performing statistical analysis in R is a bit more straightforward.
- With Python, many Interactive Development Environments are popular, but RStudio is the standard for coding in R - this makes implementation in the classroom a bit easier since everyone uses the same tools.
- Similar to Python, R includes add-ons. In R we call them *packages* and we will make use of several.
 - Users can create their own packages to share with others!
- Similar to SAS, R has powerful data handling features and implementations for many statistical methods.

1.3 Data frames

A statistical dataset in R is known as a data frame, or `data.frame`. It is a rectangular array of information (essentially a table or matrix or spreadsheet). Rows of a data frame constitute different *observations* and the columns constitute different *variables* measured.

For example, imagine we record the name, class standing, height and age of all the students in a class: the first column will record each students' name, with the second column recording their standing (*e.g.*, junior, senior, etc...), the third column records the students' height and fourth column their age. Each row corresponds to a particular student (an observation) and each column a different property of that student (a variable).

1.3.1 Built-in data

R packages come bundled with many pre-entered datasets for exploration purposes. For example, here is the data frame `stackloss` from the `datasets` package (which loads automatically upon starting R):

```
stackloss
##      Air.Flow Water.Temp Acid.Conc. stack.loss
## 1         80         27         89         42
## 2         80         27         88         37
## 3         75         25         90         37
## 4         62         24         87         28
## 5         62         22         87         18
## 6         62         23         87         18
## 7         62         24         93         19
## 8         62         24         93         20
## 9         58         23         87         15
## 10        58         18         80         14
## 11        58         18         89         14
## 12        58         17         88         13
```

```
## 13      58      18      82      11
## 14      58      19      93      12
## 15      50      18      89      8
## 16      50      18      86      7
## 17      50      19      72      8
## 18      50      19      79      8
## 19      50      20      80      9
## 20      56      20      82      15
## 21      70      20      91      15
```

This data frame contains 21 observations on 4 different variables. Note the listing of the variable names, this is known as the *header*.

Another famous dataset is the Edgar Anderson's Iris data and is included in R:

```
head(iris, n=10)
```

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1         3.5         1.4         0.2   setosa
## 2           4.9         3.0         1.4         0.2   setosa
## 3           4.7         3.2         1.3         0.2   setosa
## 4           4.6         3.1         1.5         0.2   setosa
## 5           5.0         3.6         1.4         0.2   setosa
## 6           5.4         3.9         1.7         0.4   setosa
## 7           4.6         3.4         1.4         0.3   setosa
## 8           5.0         3.4         1.5         0.2   setosa
## 9           4.4         2.9         1.4         0.2   setosa
## 10          4.9         3.1         1.5         0.1   setosa
```

```
tail(iris, n=7)
```

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width  Species
## 144           6.8         3.2         5.9         2.3 virginica
## 145           6.7         3.3         5.7         2.5 virginica
## 146           6.7         3.0         5.2         2.3 virginica
## 147           6.3         2.5         5.0         1.9 virginica
## 148           6.5         3.0         5.2         2.0 virginica
## 149           6.2         3.4         5.4         2.3 virginica
## 150           5.9         3.0         5.1         1.8 virginica
```

Here the `head()` function prints the first few observations of the dataset, we specified the first 10 rows. The `tail()` functions prints out the last `n` rows (here specified to be 7). The size of the dataset is actually bigger and can be determined with the `dim` function,

```
dim(iris)
```

```
## [1] 150   5
```

There are 150 observations on 5 variables (`Sepal.Length`, `Sepal.Width`,

`Petal.Length`, `Petal.Width`, and `Species`).

Another useful function to quickly look at data is the `glimpse()` function in the R package `dplyr`, part of the `tidyverse` collection of packages (?). Here we apply the function to the built in dataset `esoph`, which is from a case-control study on esophageal cancer.

```
library(tidyverse)
glimpse(esoph)

## Rows: 88
## Columns: 5
## $ agegp      <ord> 25-34, 25-34, 25-34, 25-34, 25-34, 25-34, 25-34, 25-34, 25-3~
## $ alcgp      <ord> 0-39g/day, 0-39g/day, 0-39g/day, 0-39g/day, 40-79, 40-79, 40~
## $ tobgp      <ord> 0-9g/day, 10-19, 20-29, 30+, 0-9g/day, 10-19, 20-29, 30+, 0~
## $ ncases     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, ~
## $ ncontrols <dbl> 40, 10, 6, 5, 27, 7, 4, 7, 2, 1, 2, 1, 0, 1, 2, 60, 13, 7, 8~
```

There are 88 observations in this study with 5 recorded variables.

1.3.2 Types of Data

You'll notice in the above examples we see a mix of numeric and character values in the data frames. In R, every object has a type or *class*. To discover the type for a specific variable use the `class` function,

```
class(iris)
```

```
## [1] "data.frame"
```

We see that the class of the object `iris` is a `data.frame` consisting of 150 rows and 5 columns. You should also note from above that the first four columns are numeric while the last column appears to be characters. We can explore the class of each variable by specifying the name of the column of the dataset using the `$` notation. First, let's get the list of variables in the dataset using the `names()` function.

```
names(iris)
```

```
## [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

```
class(iris$Sepal.Length)
```

```
## [1] "numeric"
```

```
class(iris$Species)
```

```
## [1] "factor"
```

The class of `Sepal.Length` is numeric, which should not be too surprising given the values are numbers. The class of `Species` on the other hand is a

factor. In R, a factor is a **categorical variable!** Here it happens to be labeled with the scientific species name, but R treats it as a **category**.

Note: The **factor** class can be quite important when performing Statistics. Specifically, when categories are coded as numbers it can be problematic. Depending if a variable is a **factor** or **numeric**, it can cause statistical methods to perform differently.

1.3.3 Importing datasets into R

In practice you will often encounter datasets from other external sources. These are most commonly created or provided in formats that can be read in Excel. The best formats for importing dataset files into R is space-delimited (typically with the file extension `.txt`), tab-delimited text (either the file extension `.txt` or `.tsv`) or comma-separated values (file extension `.csv`). R can handle other formats, but these are the easiest with which to work.

There are multiple functions to import a data file in R. The tools we will outline here include many functions in the `readr` package (?), part of the `tidyverse`. We recommend reading through the Vignette by running the command `vignette("readr")` in the console.

1.3.3.1 `read_table()` – for Space delimited files

One common tool is the `read_table()` command. Below are two examples:

1. Read a file directly from the web. The below code imports the file `univadmissions.txt` (details about the data are below) from the textbook data repository into the current R session, and assigns the name `uadata` to the data frame object:

```
site <- "https://tjfisher19.github.io/introStatModeling/data/univadmissions.txt"
class(site)
```

```
## [1] "character"
```

```
options(readr.show_col_types = FALSE)
uadata <- read_table(site)
head(uadata)
```

```
## # A tibble: 6 x 5
##       id gpa.endyr1 hs.pct   act  year
##   <dbl>   <dbl>   <dbl> <dbl> <dbl>
## 1     1     0.98     61    20  1996
## 2     2     1.13     84    20  1996
## 3     3     1.25     74    19  1996
## 4     4     1.32     95    23  1996
## 5     5     1.48     77    28  1996
## 6     6     1.57     47    23  1996
```

A few notes on the above code. In the first line, we are creating a string of characters (that happens to be a web address) and *assigning* it to the R object `site` using the `<-` notation. You should note the **class** of the object `site` is **character**. The third line, `options(readr.show_col_types = FALSE)`, is *entirely optional* and tells the `readr` package to suppress some extra output from display. The key is we are applying the function `read_table()` with the file specified in the `site` variable. Lastly, the output of the function is being assigned to `uadata`, again using the `<-` notation. We will be using this same data again below.

2. Another option is to download the file to your computer, then read it. You could go to the URL above and save the data file to your default working directory on your computer. After saving it into a folder (or directory), you can read the file with code similar to:

```
uadata <- read_table("univadmissions.txt")
```

If you save the file to a directory on your computer other than the local directory (where your R code is stored), you must provide the path in the `read_table()` command:

```
uadata <- read_table("C:/My Documents/My STA363 Files/univadmissions.txt")
```

The function `read_table()` automatically converts a space delimited text file to a data frame in R.

1.3.3.2 read_csv() – for Comma Separated Value files

Similar to the above examples, reading in a Comma Separated Value (CSV) file is very similar. This is becoming the standard format for transferring files. An example of some **non-executed code** is provided below:

```
uadata_csv <- read_csv("c:/My Documents/My STA363 Files/univadmissions.csv")
```

Note: the file `univadmissions.csv` does not exist but this code has been included for example purposes.

1.3.3.3 load() – Loading an existing R data.frame

If data has already been input and processed into R as a **data.frame**, it can be saved in a format designed for R using the `save()` function and then can be reloaded using `load()`. For example, first we may save the University Admission data

```
save(uadata, file="uaDataInRformat.RData")
```

Then, sometime later, we may “load” that data into R

```
load("uaDataInRformat.RData")
```

1.4 Referencing data from inside a data frame

We saw above that we can access a specific variable within a dataset using the `$` notation. We can also access specific observations in the dataset based on rules we may wish to implement. As a working example we will consider the university admissions data we read in above.

Example: University admissions (originally in `?`).

The director of admissions at a state university wanted to determine how accurately students' grade point averages at the end of their freshman year could be predicted by entrance test scores and high school class rank. The academic years covered 1996 through 2000. This example uses the `univadmissions.txt` dataset in the repository. The variables are as follows:

- `id` – Identification number
- `gpa.endyr1` – Grade point average following freshman year
- `hs.pct` – High school class rank (as a percentile)
- `act` – ACT entrance exam score
- `year` – Calendar year the freshman entered university

This is the first dataset we input above in the `uadata` data frame.

The `tidyverse` package (`?`) is actually a wrapper for many R packages. One of the key packages we use for data processing is the `dplyr` package (`?`). It includes many functions that are similar to SQL (for those who know it). These tools tend to be very readable and easy to understand. First we will load the package `tidyverse` and then take a *glimpse* of the dataset:

```
library(tidyverse) # Load the necessary package
glimpse(uadata)    # Take a glimpse at the data
```

```
## Rows: 705
## Columns: 5
## $ id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
## $ gpa.endyr1 <dbl> 0.98, 1.13, 1.25, 1.32, 1.48, 1.57, 1.67, 1.73, 1.75, 1.76, ~
## $ hs.pct    <dbl> 61, 84, 74, 95, 77, 47, 67, 69, 83, 72, 83, 66, 76, 88, 46, ~
## $ act       <dbl> 20, 20, 19, 23, 28, 23, 28, 20, 22, 24, 27, 21, 23, 27, 21, ~
## $ year      <dbl> 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, ~
```

In the code above, you may note that everything to the right of a `#` is a *comment* – this is, text included in code that does function or execute as code. Comments are a useful tool to provide notes to use yourself about the code.

Key features when using the `tidyverse` is the pipe command, `|>` and *actions* on a `data.frame`. These *actions* are function calls and other sources may call them *verbs*, as you are performing some action on the data. We will introduce the function calls as needed.

Note: there are actually two different syntax for using the pipe in R, `|>` and

%>%. In the original version of this text, we used the latter %>% as the newer |> was not included in previous versions of R. At this point, |> is preferred but some examples may still use the older %>% notation.

As an example of some basic data processing, consider the case of extracting students with a GPA greater than 3.9:

- Take the `data.frame` `uadata` (from above) and pipe it (send it) to a function (or action), in this case the *filter* function.
- Then *filter* it based on some criteria, say `gpa.endyr1 > 3.9`.

```
uadata |>
  filter(gpa.endyr1 > 3.9)

## # A tibble: 29 x 5
##       id gpa.endyr1 hs.pct   act  year
##   <dbl>     <dbl> <dbl> <dbl> <dbl>
## 1  138       3.91    99    28  1996
## 2  139       3.92    93    28  1996
## 3  140        4     99    25  1996
## 4  141        4     92    28  1996
## 5  142        4     99    33  1996
## 6  266       3.92    85    15  1997
## 7  267       3.92    63    23  1997
## 8  268       3.92    98    23  1997
## 9  269       3.93    99    29  1997
## 10 270       3.96    99    29  1997
## # i 19 more rows
```

Another useful `dplyr` command for choosing specific variables is the `select` function. Below we select the ACT scores for students who have a first year GPA greater than 3.9 and with a graduating year of 1998.

```
uadata |>
  filter(gpa.endyr1 > 3.9,
         year==1998) |>
  select(act)

## # A tibble: 6 x 1
##       act
##   <dbl>
## 1    28
## 2    31
## 3    31
## 4    21
## 5    27
## 6    32
```

Note in the above, we take the `uadata` and pipe it to a `filter`, which selects

only those observations with `gpa.endyr1` greater than 3.9 and `year` equal (`==`) to 1998, then we pipe that subset of the data to the `select` function that returns the variable `act`.

IMPORTANT NOTE - R is an *evolving language* and as such occasionally has some wonky behavior. In fact, there are multiple versions of both the `filter` and `select` functions in R. If you ever use one and receive a strange error (comparison is possible only for atomic and list types or an unused argument error), first check for a typo. If the error still occurs it is likely the case that R is trying to use a different version of the `filter` or `select`. You can always force R to use the `dplyr` version by specifying the package as in the below example:

```
uadata |>
  dplyr::filter(gpa.endyr1 > 3.9,
               year==1998) |>
  dplyr::select(act)
```

1.5 Missing values and computer arithmetic in R

Sometimes data from an individual is lost or not available, these are known as missing values. R indicates such occurrences using the value `NA`. Missing values often need to be addressed because simply ignoring them when performing certain functions will result in no useful output.

As an example, we cannot find the mean of the elements of a data vector when some are set to `NA`. Consider the simple case of a vector of 6 values with one `NA`.

```
x <- c(14, 18, 12, NA, 22, 16)
class(x)
```

```
## [1] "numeric"
```

```
mean(x)
```

```
## [1] NA
```

Here, the object `x` is a *vector* of class `numeric`. We will not be working with vectors much in this class (although a column in a data frame can be considered a vector) but we do so here to demonstrate functionality.

Since one of the observations is missing, `NA`, the mean of the vector is also “not available” or `NA`.

This is not incorrect: logically speaking there is no way to know the mean of these observations when one of them is missing. However, in many cases we may still wish to calculate the mean of the non-missing observations.

Many R functions have a `na.rm=` logical argument, which is set to `FALSE`

by default. It basically asks if you want to skip, or remove, the missing values before applying the function. It can be set to `TRUE` in order to remove the NA terms. For example,

```
mean(x, na.rm=TRUE)    # mean of the non-missing elements
```

```
## [1] 16.4
```

The number system used by R has the following ways of accommodating results of calculations on missing values or extreme values:

- **NA: Not Available.** Any data value, numeric or not, can be NA. This is what you use for missing data. Always use NA for this purpose.
- **NaN: Not a Number.** This is a special value that only numeric variables can take. It is the result of an undefined operation such as 0/0.
- **Inf: Infinity.** Numeric variables can also take the values `-Inf` and `Inf`. These are produced by the low level arithmetic of all modern computers by operations such as 1/0. (You should not think of these as values as real infinities, but rather the result of the correct calculation if the computer could handle extremely large (or extremely small, near 0) numbers; that is, results that are larger than the largest numbers the computer can hold (about 10^{300})).

Scientific notation. This is a shorthand way of displaying very small or very large numbers. For example, if R displays `3e-8` as a value, it means $3 \times 10^{-8} = 0.00000003$. Similarly, `1.2e4` is $1.2 \times 10^4 = 12000$.

1.6 Exploratory Data Analysis (EDA)

The material presented in this section is typically called *descriptive statistics* in Intro Statistics books. This is an important step that should always be performed prior to a formal analysis. It looks simple but it is vital to conducting a meaningful analysis. EDA is comprised of:

- Numerical summaries
 - Means & medians (measures of centrality)
 - Standard deviations, range, Interquartile Range (IQR) (measures of spread)
 - Percentiles/quantiles & Five-number summaries (assessment of distributions)
 - Correlations (measures of association)
- Graphical summaries
 - One variable: boxplots, histograms, density plots, etc.
 - Two variables: scatterplots, side-by-side boxplots, overlaid density plots
 - Many variables: scatterplot matrices, interactive graphics

We look for outliers, data-entry errors, skewness or unusual distributions using EDA. We may ask ourselves: “are the data distributed as you expect?” Getting data into a form suitable for analysis by cleaning out mistakes and aberrations is often time consuming. It often takes more time than the data analysis itself! In this course, data will usually be ready to analyze but we will occasionally intermix messy data. You should realize that in practice it is rare to receive *clean* data.

1.6.1 Numeric Summaries

1.6.1.1 Measures of centrality and quantiles

Common descriptive measures of centrality and quantiles from a sample include

- Sample mean – typically denoted by $\bar{x}$, this is implemented in the function `mean()`
- Sample median – this is implemented in the function `median()`
- Quantiles – Quantiles, or sample percentiles, can be found using the `quantile()` function.
 - Three common quantiles are Q_1 (the sample 25th percentile, or first quartile), Q_2 (the 50th percentile, second quartile or *median*) and Q_3 (the sample 75th percentile or third quartile).
- Min and Max – the Minimum and Maximum values of a sample can be found using the `min()` or `max()` functions.

Each of these measures are covered in a typical introductory statistics course. We refer you to that material for more details.

1.6.1.2 Measuring variability

Earlier we defined statistics as the ‘study of variation’. The truth is that everything naturally varies:

- If you measure the same characteristic on two different individuals, you would likely get two different values (subject variability).
 - *e.g.*, the pulse rate of two individuals is likely to be different.
- If you measure the same characteristic twice on the same individual, you may get two different values (measurement error).
 - *e.g.*, if two different people manually measured someone’s pulse rate, you may get different results.
- If you measure the same characteristic on the same individual but at different times, you would potentially get two different values (temporal variability).
 - *e.g.*, measuring someone’s pulse rate while resting compared to while performing exercise sometime later.

Because almost everything varies, simply determining that a variable *varies* is not very interesting. The goal is to find a way to discriminate between variation

that is scientifically interesting and variation that reflects background fluctuations that occur naturally. This is what the science of statistics does!

Having some intuition about the amount of variation that we would expect to occur just by chance alone when nothing scientifically interesting is key. If we observe bigger differences than we would expect by chance alone, we may say the result is statistically significant. If instead we observe differences no larger than we would expect by chance alone, we say the result is not statistically significant.

Although often covered in introductory statistics courses, due to importance we review measuring the variance of a sample.

Variance

A *variance* is a numeric descriptor of the variation of a quantity. Variances will serve as the foundation for much of what we use in making statistical inferences in this course. In fact, a standard method of data analysis for many of the problems we will encounter is known as analysis of variance (shorthand ANOVA).

You were exposed to variance in your introductory statistics course. What is crucial now is that you understand what you are finding the variance of; *i.e.*, what is the “quantity” we referred to in the preceding paragraph? Consider the following example:

- *Example.* Suppose you collect a simple random sample of fasting blood glucose measurements from a population of n diabetic patients, where n is the standard notation for the size of a sample. The measurements, or observations, are denoted $x_1, x_2, \dots, x_n$. What are some of the different “variances” that one might encounter in this scenario?
 1. The n sampled measurements vary around their sample mean $\bar{x}$. This is called the sample variance (traditionally denoted s^2). The formula for s^2 is provided below. In R, we compute this with the function `var()`.
 2. All fasting blood glucose values in the population vary around the true mean of the population (typically denoted with μ , the lowercase Greek ‘mu’). This is called the population variance (usually denoted σ^2 where σ is the lowercase Greek ‘sigma’). Note that σ^2 cannot be calculated without a census of the entire population, so s^2 serves as an estimate of σ^2 .
 3. $\bar{x}$ is only a sample estimate of μ , so we should expect that it will vary depending on which n individual diabetic patients are sampled (*i.e.*, with a different sample we will calculate a different sample mean). This is called the variance of $\bar{x}$, and is denoted $\sigma_{\bar{x}}^2$. This variance, unlike the sample or population variance, describes how sample estimates of μ vary around μ itself. We will need to estimate $\sigma_{\bar{x}}^2$, since it cannot be determined without a full survey of the population. The

estimate of $\sigma_{\bar{x}}^2$ is denoted $s_{\bar{x}}^2$.

The variances cited above are each different measures. Even more measures of variability exist; for example, each different possible random sample of n patients will produce a different sample variance s^2 . Since these would all be estimates of the population variance σ^2 , we could even find the variance of the sample variances!

Some formulas:

$$\textbf{Sample Variance: } s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \text{ for sample size } n$$

$$\textbf{Estimated Variance of } \bar{x}: s_{\bar{x}}^2 = \frac{s^2}{n}$$

Three important notes:

1. The general form of any variance is a sum of squared deviations, divided by a quantity related to the number of independent elements in the sum known as the **degrees of freedom**. So in general,

$$\text{Variance} = \frac{\text{sum of square deviations}}{\text{degrees of freedom}} = \frac{SS}{df}$$

2. The formula for the population variance σ^2 is omitted since it is rarely used and requires the μ to be a known value. In practice, this can only be calculated with a population census.
3. When considering how a statistic (like $\bar{x}$) varies around the parameter it is estimating (μ), it makes sense that the more data you have, the better it should perform as an estimator. This is precisely why the variance of $\bar{x}$ is inversely related to the sample size n . **As n increases**, the variance of the estimate decreases; *i.e.*, the estimate will be closer to the thing it is estimating. This phenomenon is known as the law of large numbers and is typically discussed in Introductory Statistics courses.

Standard deviations and standard errors

While variances are the usual items of interest for statistical testing, variances are always in units squared (look at the equation for s^2 above). Often we will find that a description of variation in the original units of the quantity being studied is preferred, so we take their square roots and are back to the original units. The square root of a variance is called a **standard deviation** (SD). The square root of the variance of an estimate is called the **standard error** (SE) of the estimate:

Sample Standard Deviation: $s = \sqrt{s^2} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$, for sample size n

$$\text{Standard error of } \bar{x} : s_{\bar{x}} = \sqrt{s_{\bar{x}}^2} = \sqrt{\frac{s^2}{n}} = \frac{s}{\sqrt{n}}$$

1.6.2 Numeric Summaries in R

We recommend using `dplyr` for data handling and summary statistics. We can pipe (`|>`) our data frame into a function to calculate numeric summaries, this function is `summarize()`.

```
uadata |>
  pivot_longer(everything(), names_to="Variable", values_to="value") |>
  group_by(Variable) |>
  summarize(across(everything(),
                    list(Mean=mean, SD=sd, Min=min, Median=median, Max=max) ))

## # A tibble: 5 x 6
##   Variable    value_Mean value_SD value_Min value_Median value_Max
##   <chr>         <dbl>    <dbl>    <dbl>         <dbl>    <dbl>
## 1 act           24.5      4.01      13           25        35
## 2 gpa.endyr1     2.98     0.635     0.51         3.05        4
## 3 hs.pct        77.0     18.6       4            81       99
## 4 id           353     204.       1          353      705
## 5 year        1998     1.40    1996        1998     2000
```

For a brief moment ignore the code (we described below). Presented here are the mean, standard deviation, the minimum, median and maximum of each numeric variable in the University Admissions data frame. For example, the mean freshman year-end GPA over the period 1996-2000 was 2.98; the median over the same period was 3.05, suggesting the GPA distribution is slightly skewed left (not surprising). The range of the GPA distribution is from 0.51 to 4. The median ACT score for entering freshman was 25, and the range of ACT scores was from 13 to 35.

1.6.2.1 Coding Notes

In `tidyverse` there is package called `tidyr` (?) which is mainly comprised of two main functions: `pivot_longer()` and `pivot_wider()`. The purpose of the `pivot_longer()` function is to collect variables in a dataset and create a *tall* (or *longer*) dataset (this is typically known as going from wide-to-tall or wide-to-long format). We specify the variables (or columns) we want to pivot (here we pivot on everything, thus `everything()` in the code), the name of the new variable (here called `Variable`) storing the column names, and the variable name

that will store the values (here called `value`). The `pivot_wider()` function essentially does the opposite, turning a tall data frame into a wide format.

In `dplyr` resides the function `group_by()`. This function essentially does some *internal organization* of our data. Specifically, for each unique `Variable` name, it would group the data together. So, all the `gpa.endyr1` values will be considered a group, and separate from all the `hs.pct` and `act` groups. This function is particularly useful when coupled with `summarize()` (such as in the example above) because summary statistics will be calculated for each group.

The function `summarize()` tells R to calculate summary statistics across rows (or grouped rows) within a `data.frame`. In the above example, we compute a list of summary statistics (hence, `list(Mean=mean, SD=sd, Min=min, Median=median, Max=max)`) on all the non-grouped variables (`everything()`). We will see more examples below.

The code above introduces a few keys points to know:

- The code is spread across multiple lines with most lines ending with the piping operator `|>`. This is done for readability (and recommended by all users of R). When performing an analysis, you often will revisit the code at a later time. Readable code is paramount to reproducibility.
- The `tidyverse` structure of data processing is meant to be highly readable. Look back at the code and consider the following:
 - We get the university admissions data, and then send it to (`uadata |>`)
 - the function `pivot_longer()` that takes all columns, or variables (`everything()`) in the data set and stacks them on top of one another where a new variable called `Variable` designates the variable from the wide version of the data, and the new variable `Values` provides the corresponding value, and then send it to (`|>`)
 - a function that internally organizes the dataset, treating all `Values` with the same `Variable` value as a group (`group_by(Variable)`), and then send it to (`|>`)
 - a function that will calculate summary statistics (`summarize()`). This includes the mean, standard deviation, minimum, median and maximum of the `Values` for each `Variable` value since the data had been grouped.

Also note:

- Since we applied the command to the entire data frame (`everything()`), we also see numeric summaries of the variable `id`, even though it is meaningless to do. (note: just because a computer outputs it, that doesn't mean it's useful)
- If you look carefully, you'll note that `year` is treated as a number as well (*e.g.*, 1996, 1997, etc.). However, that doesn't necessarily mean it should be treated as a numeric variable! It is meaningless to find numeric summaries, such as means or medians, for the `year` variable because it

is qualitative (categorical) variable in the context of these data (again, just because a computer outputs it, that does not mean it is contextually correct).

These are common mistake to make. When R encounters a variable whose values are entered as numbers in a data frame, R will treat the variable as if it were a numeric variable (whether it is contextually or not). If R encounters a variable whose values are character strings, R will treat the variable as a categorical **factor**. But, in a case like `year` or `id` above, you have to explicitly tell R that the variable is really a **factor** since it was coded with numbers. To do this, we will `mutate` the variables `id` and `year` and turn them into factors.

```
uadata <- uadata |>
  mutate(id=factor(id),
         year=factor(year))
```

Here we tell R to take the `uadata` data, pipe it to the `mutate` function (which allows us to create and/or replace variables by changing/mutating them). We then assign the variables `id` and `year` as versions of themselves that are being treated as *factors* (or categories).

We can then check the type of each variable by *summarizing* over *all* the variables using the `class` function.

```
uadata |>
  summarize(across(everything(), class))
```

```
## # A tibble: 1 x 5
##   id      gpa.endyr1 hs.pct  act    year
##   <chr>   <chr>      <chr> <chr>  <chr>
## 1 factor numeric    numeric numeric factor
```

Now calculate the summary statistics for only numeric variables (*i.e.*, the use of the `select(where(is.numeric))` function).

```
uadata |>
  select(where(is.numeric)) |>
  pivot_longer(everything(), names_to="Variable", values_to="value") |>
  group_by(Variable) |>
  summarize(across(everything(),
                   list(Mean=mean, SD=sd, Min=min, Median=median, Max=max) ))
```

```
## # A tibble: 3 x 6
##   Variable  value_Mean value_SD value_Min value_Median value_Max
##   <chr>      <dbl>    <dbl>    <dbl>      <dbl>    <dbl>
## 1 act        24.5      4.01      13         25        35
## 2 gpa.endyr1  2.98      0.635     0.51       3.05       4
## 3 hs.pct     77.0      18.6      4          81       99
```

When a dataset includes categorical variables, the main numeric measure to

consider is the count (or proportions) of each category. To do this in R, we tell it to treat each category as a *group* and then count within the group. We will `group_by` the `year` and then within that year, count the number of observations (`summarize(Count=n())`). We also add a new variable (using `mutate`) that is the proportion.

```
uadata |>
  group_by(year) |>
  summarize(Count=n()) |>
  mutate(Prop = Count/sum(Count))
```

```
## # A tibble: 5 x 3
##   year Count Prop
##   <fct> <int> <dbl>
## 1 1996    142 0.201
## 2 1997    131 0.186
## 3 1998    154 0.218
## 4 1999    141 0.2
## 5 2000    137 0.194
```

Note the power of using the *tidyverse* tools (?). We are only *scratching the surface* of what can be done here. For more information, we recommend looking into the *Wrangling* chapters in the book *R for Data Science* (?).

1.6.3 Graphical Summaries

We begin a graphical summary with a matrix (or table) of univariate and bivariate plots using the `ggpairs()` function in the *GGally* package (?). The results can be seen in Figure ??.

```
library(GGally)
ggpairs(uadata, columns=2:5)
```

1.6.3.1 Coding note

Here we specify to only plot using the variables in columns 2, 3, 4 and 5 (the 2:5). We do this because the `id` variable in `uadata` is an identifier (column 1) and unique identifies each observations – it is not a variable of statistical interest – we do not want to try and plot it.

Note that by default the function provides a collection of univariate and bivariate plots (Figure ??). We recap each of the plots provided (further details on each plot type is provided below).

- On the diagonal of the matrix we see univariate plots – density plots for the numeric variables and a bargraph of frequencies for categorical variable ‘year’.
- In the upper right triangle of the matrix we see numeric measures for the amount of correlation between each of the numeric variables (Pearson

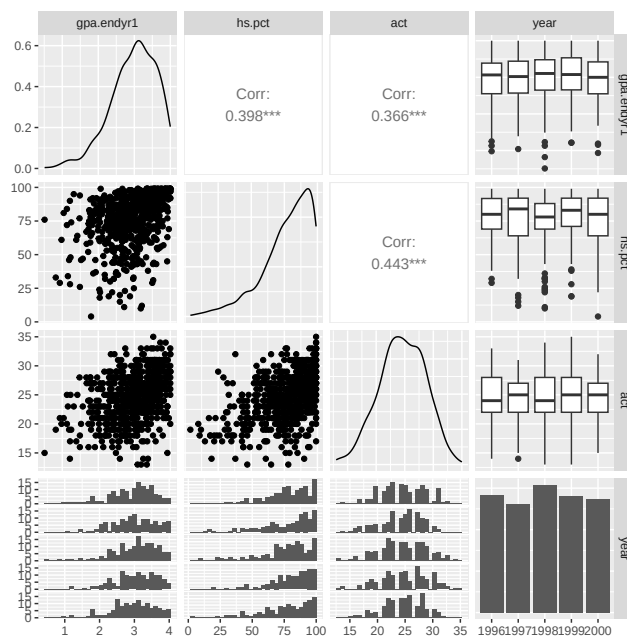


Figure 1.1: Pairs plot showing the relationship between the variables 'gpa.endyr1', 'hs.pct', 'act' and 'year'.

Correlation coefficients, typically denoted with r). In the case of a numeric variable matched with a categorical variable, side-by-side boxplots are provided.

- In the bottom left triangle of the matrix scatterplots are provided to show the relationship between numeric variables. We also see histograms for each of the numerical variable stratified by the categorical variable **year**. Now we will discuss each of the elements in more detail.

1.6.4 Distribution of Univariate Variables

The `ggpairs` function provides a look at the distributional shape of each variable using what is known as a *density plot* (this can be consider a more-modern, and arguably less subjective, histogram). The `ggpairs` function builds off a function known as `ggplot()` from the `ggplot2` package (?) which is included in `tidyverse` (?).

We can build density plots directly using the `geom_density()` function as seen in the code below. The resulting density plot is in Figure ??.

```
ggplot(uadata) +  
  geom_density(aes(x=gpa.endyr1))
```

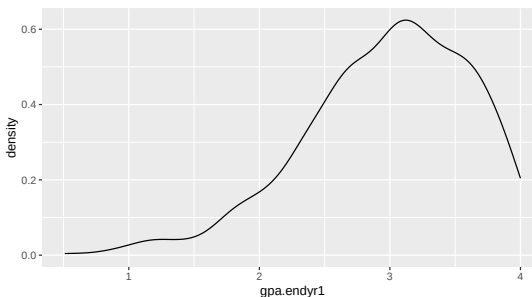


Figure 1.2: Density plot showing left-skewed distribution for the ‘gpa.endyr1’ variable

1.6.4.1 Coding Notes

Note the code is similar to the `|>` operator but with a `+`. The trick to working with plotting on the computer is to think in terms of *layers*. The first line `ggplot(uadata)` sets up a plot object based on the dataset `uadata`. The `+` says to add a layer, this layer is a density plot (`geom_density`) where the aesthetic values include an *x*-axis variable matching that of `gpa.endyr1`.

To generate even more complex distributional plots, first we need to perform some data processing. Above we saw `pivot_longer()`. The purpose of the `pivot_longer()` function is to collect variables in a dataset and create a *tall* (or *longer*, wide-to-long) dataset. We specify the variables (or columns) we want

to pivot (the `c(gpa.endyr1, hs.pct, act)` part in the below code), the name of the new variable (here called `var_names`) storing the column names, and the variable name that will store the values (here called `values`).

```
uadata.tall <- uadata |>
  pivot_longer(c(gpa.endyr1, hs.pct, act),
               names_to="var_names", values_to="values" )
head(uadata)
```

```
## # A tibble: 6 x 5
##   id   gpa.endyr1 hs.pct   act year
##   <fct>      <dbl> <dbl> <dbl> <fct>
## 1 1         0.98    61    20 1996
## 2 2         1.13    84    20 1996
## 3 3         1.25    74    19 1996
## 4 4         1.32    95    23 1996
## 5 5         1.48    77    28 1996
## 6 6         1.57    47    23 1996
```

```
head(uadata.tall)
```

```
## # A tibble: 6 x 4
##   id   year var_names values
##   <fct> <fct> <chr>      <dbl>
## 1 1     1996 gpa.endyr1  0.98
## 2 1     1996 hs.pct      61
## 3 1     1996 act         20
## 4 2     1996 gpa.endyr1  1.13
## 5 2     1996 hs.pct      84
## 6 2     1996 act         20
```

```
dim(uadata)
```

```
## [1] 705    5
```

```
dim(uadata.tall)
```

```
## [1] 2115    4
```

Take a minute and step through the above code to understand it. The data is now in a *tall*, or *long*, format. We can now utilize the `ggplot2` package for more advanced plotting. Here we want to draw a density plot for the `values` in the `tall`, or *long*, version of the dataset, called `uadata.tall`. But we want a different plot for each `var_names` variable (that is, `act`, `gpa.endyr1`, and `hs.pct`) so we add a `facet_grid()` layer. Figure ?? provides the result for the below code.

```
ggplot(uadata.tall) +
  geom_density(aes(x=values)) +
  facet_grid(.~var_names, scales="free_x") +
  theme_bw()
```

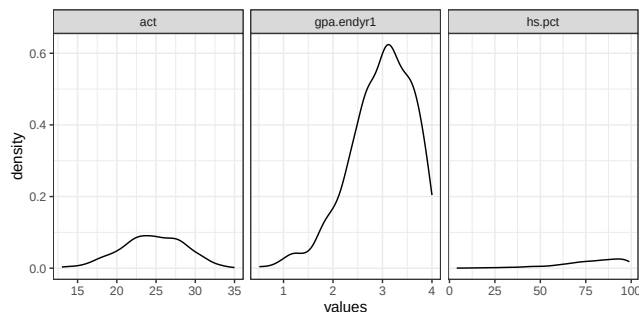


Figure 1.3: Side-by-side density plots showing the relative distributions of ‘act’, ‘gpa.endyr1’ and ‘hs.pct’.

1.6.4.2 Coding Notes

The `facet_grid` option tells the plot to draw each density plot for each observed value in `var_names`. The `facet` feature is similar to the `group_by` statement when calculating summary statistics. The `theme_bw()` specifies the plotting style (there are many others but we will typically use `theme_bw()`, `theme_minimal()` or the default behavior which corresponds to `theme_grey()`). The `free_x` option tells the plot that each *x*-axis is on a different scale. Feel free to modify the code to see how the plot changes!

We can see that GPA and high school percentile rankings are skewed left, and that ACT scores are reasonably symmetrically distributed.

Alternatively, we could build histograms to make the comparison.

```
ggplot(uadata.tall) +
  geom_histogram(aes(x=values), binwidth = 2.5) +
  facet_grid(var_names~., scales="free_y") +
  theme_bw()
```

Note the similarities and difference in Figures ?? and ??. Depending on whether we facet on the *y*-axis or *x*-axis we can get different appearances. Also note that the code for plotting a histogram is similar to that of a density plot (`geom_histogram` vs `geom_density`).

Bar Graphs

The best visual tool for simple counts is the Bar Graph (**note:** you should never use a pie chart, see here for an explanation). The below code generates a simple (and fairly boring) bar graph (Figure ??) displaying the number of observations for each `year` in the `uadata`.

```
ggplot(uadata) +
  geom_bar(aes(x=year) )
```

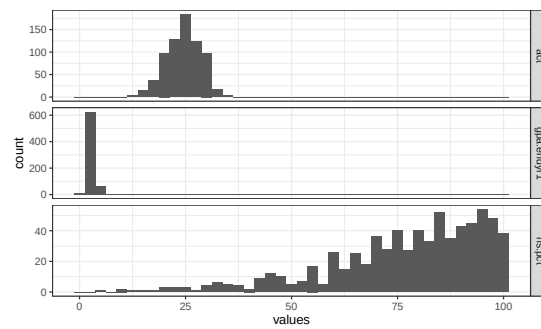


Figure 1.4: Stacked histogram showing the relative distributions of ‘act’, ‘gpa.endyr1’ and ‘hs.pct’.



Figure 1.5: Bar graph displaying the number of observations in the ‘uadata’ per ‘year’.

We see here that nothing too interesting is happening. A fairly uniform distribution of counts is seen across years.

1.6.5 Descriptive statistics and visualizations by levels of a factor variable

While the above summaries are informative, they overlook the fact that we have several years of data and that it may be more interesting to see how things might be changing year-to-year. This is why treating `year` as a factor is important, because it enables us to split out descriptions of the numeric variables by levels of the factor.

One of the simplest and best visualization tool for comparing distributions is the side-by-side boxplot, which displays information about quartiles (Q_1 , Q_2 and Q_3) and potential outliers. The below code generates Figure ??.

```
ggplot(uadata) +
  geom_boxplot(aes(x=act, y=year) ) +
  labs(title="ACT scores of Incoming Freshmen",
       x="Composite ACT Score",
       y="Year") +
  theme_classic()
```

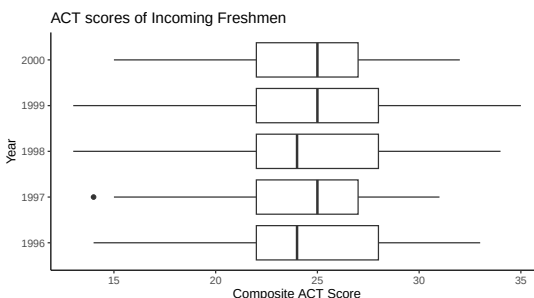


Figure 1.6: Side-by-side Box-whiskers plots showing the distribution of ACT scores in the ‘uadata’ per year.

The code is similar to the examples above except now we call the `geom_boxplot` function. Also note we are using `theme_classic()` which has some subtle differences compared to `theme_bw()` used above. Lastly, we include proper labels and titles on our plot here for a more professional appearance.

Next we construct side-by-side boxplots by year for each of the three numeric variables in the data frame. We utilize the taller or longer format of the admissions data in `uadata.tall`. Two versions of this plot are provided in Figure ?? and Figure ??.

```
# Code for Figure 1.6
ggplot(uadata.tall) +
  geom_boxplot(aes(x=values, y=year) ) +
  facet_grid(~var_names, scales="free_x") +
  theme_dark()
```

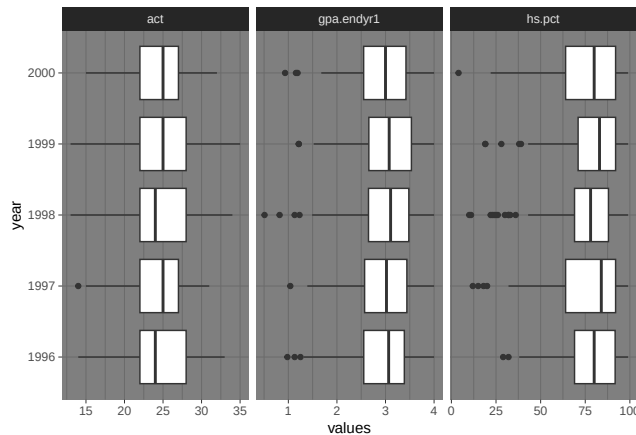


Figure 1.7: Side-by-side Box-whiskers plots showing the distributions of the variables ‘act’, ‘gpa.endyr1’ and ‘hs.pct’ for each year in the ‘uadata’ dataset.

Note another theme option, `theme_dark()` (obviously you will NOT want to use this theme if planning to print the document). A more aesthetically pleasing version of the plot (with proper labeling) is generated with the code below and can be seen in Figure ??.

```
# Code for Figure 1.7
ggplot(uadata.tall) +
  geom_boxplot(aes(x=values, y=year) ) +
  facet_grid(~var_names, scales="free_x") +
  labs(y="Year", x="", title="Distribution of ACT, GPA and HS Rank by Year") +
  theme_minimal()
```

This is a nice compact visual description of the changes in patterns of response of these three variables over time.

All three variables appear pretty stable over the years; the left skew in GPA and high school percentile rankings are still evident here (note the detection of some low outliers on these two variables).

We now revisit calculating summary statistics. Extensive grouped numeric summary statistics are available by using the `group_by()` and `summarize()` functions in `tidyverse`. Recall that the `group_by` function tells R to treat each value of the specified variable (in this case, called `var_names`) as a collection of similar options, then to perform the `summarize` function on each value in

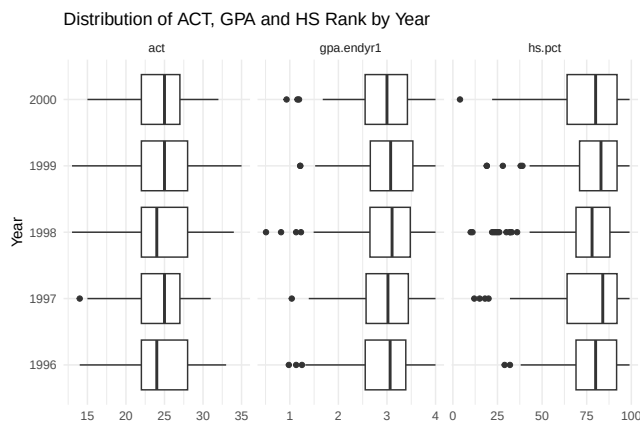


Figure 1.8: Side-by-side Box-whiskers plots showing the distributions of the variables ‘act’, ‘gpa.endyr1’ and ‘hs.pct’ for each year in the ‘uadata’ dataset.

var_names	Mean	StDev	Min	Q2	Median	Q3	Max
act	24.543	4.014	13.00	22.000	25.00	28.00	35
gpa.endyr1	2.977	0.635	0.51	2.609	3.05	3.47	4
hs.pct	76.953	18.634	4.00	68.000	81.00	92.00	99

the grouped variables. The below example is a more detailed version of the summary table from above.

```
library(knitr)

summary.table <- uadata.tall |>
  group_by(var_names) |>
  summarize(Mean=mean(values),
            StDev=sd(values),
            Min=min(values),
            Q2=quantile(values, prob=0.25),
            Median=median(values),
            Q3=quantile(values, prob=0.75),
            Max=max(values) )

kable(summary.table, digits=3) |>
  kable_styling(full_width=FALSE)
```

Here, we first pipe the `uadata.tall` into a `group_by()` function grouping by the `var_names` variable. Then we take that output and pipe it to the `summarize` function which allows us to calculate various summary statistics for each of the groups. In the above we calculate the mean and standard deviation of each

var_names	year	Mean	StDev
act	1996	24.690	3.885
act	1997	24.099	3.819
act	1998	24.506	4.277
act	1999	24.837	4.396
act	2000	24.555	3.609
gpa.endyr1	1996	2.929	0.653
gpa.endyr1	1997	2.974	0.634
gpa.endyr1	1998	3.000	0.654
gpa.endyr1	1999	3.026	0.601
gpa.endyr1	2000	2.955	0.632
hs.pct	1996	78.296	15.894
hs.pct	1997	76.641	20.197
hs.pct	1998	74.649	19.983
hs.pct	1999	79.454	17.045
hs.pct	2000	75.876	19.535

group, along with the 5-number summary. Lastly, we wrap the constructed table (its technically a data frame) in the `kable()` function from the `knitr` package (?) (where `kable` means ‘knitr’ ‘table’) for a nice display. We pipe that output to the `kable_styling()` function in the `kableExtra` package (?) for some additional formatting options (here, we condense the width of the table).

In the next example we group by both the `year` and the variables of interest.

```
summary.table2 <- uadata.tall |>
  group_by(var_names, year) |>
  summarize(Mean=mean(values),
            StDev=sd(values))
kable(summary.table2, digits=3) |>
  kable_styling(full_width=FALSE)
```

1.6.6 Descriptive statistics and visualizations for two numeric variables

For a pair of numeric variables the amount of linear association between the variables can be measured with the Pearson correlation. The Pearson correlation assesses the *strength* and *direction* of any linear relationship existing between two numeric variables. The correlation coefficients between the GPA, ACT and HS rank variables are provided Figure ??.

Recall from introductory statistics that the Pearson correlation r between two

variables x and y is given by the formula

$$r = \frac{1}{n-1} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s_x} \right) \left(\frac{y_i - \bar{y}}{s_y} \right).$$

r is a unitless measure with the property that $-1 \leq r \leq 1$.

Values of r near ± 1 indicate a near perfect (positive or negative) linear relationship between x and y . A frequently used rule of thumb is that $|r| > 0.80$ or so qualifies as a strong linear relationship. We can calculate the Pearson correlation with the `cor()` function in R. When applied to a `data.frame`, or `matrix`, it displays a correlation matrix (correlation of all variable pairs). In the below code we specify to only calculate the correlation based on the 3 numeric variables.

```
cor(uadata |> select(gpa.endyr1, hs.pct, act))
```

```
##           gpa.endyr1  hs.pct    act
## gpa.endyr1  1.000000 0.398479 0.365611
## hs.pct      0.398479 1.000000 0.442507
## act        0.365611 0.442507 1.000000
```

All pairwise correlations in this example are of modest magnitude ($r \approx 0.4$) (see Figure ?? or the above correlation matrix).

To visualize the relationship between numeric variables, we typically use a scatterplot. A scatterplot is a simple bivariate plot where dots, or points, are plotted at each (x, y) pair on the Cartesian plane. The code below uses `ggplot()` to build a scatterplot using `geom_point()`.

```
ggplot(uadata) +
  geom_point(aes(x=act, y=gpa.endyr1)) +
  labs(x="Composite ACT Score",
       y="Grade point average following freshman year") +
  theme_minimal()
```

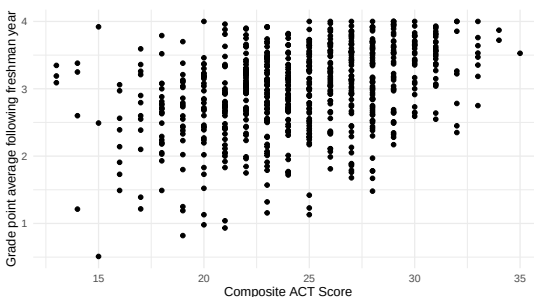


Figure 1.9: Scatterplot showing the relationship between the variables ‘act’ and ‘gpa.endyr1’ in the ‘uadata’ dataset.

You should note in Figure ?? the overall positive relationship (higher ACT scores tend to correspond to higher GPAs and corresponds to the Pearson correlation value of 0.366). We also note that due to the rounding of the ACT scores (always a whole number) the resulting plot has vertical striations.

We attempt to visually address the striations with *jittering* (adding some visual random perturbations) in Figure ?? with the code below using `geom_jitter()`. There, we specify no jittering on the vertical axis (thus, `height=0`) and let `ggplot2` use the default behavior horizontally.

```
ggplot(uadata) +
  geom_jitter(aes(x=act, y=gpa.endyr1), height=0 ) +
  labs(x="Composite ACT Score",
       y="Grade point average following freshman year") +
  theme_minimal()
```

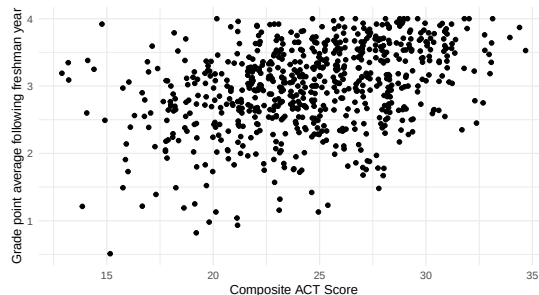


Figure 1.10: Jittered scatterplot showing the relationship between the variables ‘act’ and ‘gpa.endyr1’ in the ‘uadata’ dataset.

Note, with jittering we are only adding *randomness* to the **plot**, we are not changing the recorded values in the `data.frame`.

1.7 Sampling distributions: describing how a statistic varies

As described in the discussion on variability above, each of the numeric quantities calculated has some associated uncertainty. It is not enough to simply know the variance or standard deviation of an estimate. That only provides one measure of the inconsistency in an estimate’s value from sample to sample of a given size n .

If we wish to make a confident conclusion about an unknown population parameter using a sample statistic (*e.g.*, $\bar{x}$), we also need to know something about the general pattern of behavior that statistic will take if we had observed it over many, many different random samples. This long-run behavior pattern is

described via the probability distribution of the statistic, which is known as the **sampling distribution** of the statistic.

It is important to remember that we usually only collect one sample, so only one sample estimate will be available in practice. However, knowledge of the sampling distribution of our statistic or estimate is important because it will enable us to assess the reliability or confidence we can place in any generalizations we make about the population using our estimate.

Two useful sampling distributions: Normal and t

You should be very familiar with Normal distributions and t -distributions from your introductory statistics course. In particular, both serve as useful sampling distributions to describe the behavior of the sample mean $\bar{x}$ as an estimator for a population mean μ . We briefly review them here.

Normal distributions.

A random variable X that is normally distributed is one that has a symmetric bell-shaped density curve. A normal density curve has two parameters: its mean μ and its variance σ^2 . Notationally, we indicate that a random variable X is normal by writing $X \sim N(\mu, \sigma^2)$.

The normal distribution serves as a good sampling distribution model for the sample mean $\bar{x}$ because of the powerful Central Limit Theorem, which states that if n is sufficiently large, the sampling distribution of $\bar{x}$ will be approximately normal, regardless of the shape of the distribution of the original parent population from which you collect your sample (when making a few assumptions).

t distributions.

The formula given by

$$t_0 = \frac{\bar{x} - \mu}{SE_{\bar{x}}}$$

counts the number of standard error units between the true value of μ and its sample estimate $\bar{x}$; it can be thought of as the standardized distance between a sample mean estimate and the true mean value. If the original population was normally distributed, then the quantity t_0 will follow the Students' t -distribution with $n - 1$ degrees of freedom. The *degrees of freedom* is the only parameter of a t -distribution.

Notationally, we indicate this by writing $t_0 \sim t(\text{df})$ or $t_0 \sim t(\nu)$ where ν is a lowercase Greek 'nu' (pronounced 'new') and is the notation for degrees of freedom.

t -distributions look like the standard normal distribution, $N(0, 1)$: they are both centered at 0, are symmetric and bell-shaped (see below). The key difference is due to the fact that the quantity t includes two **sample summaries** in its construction: the sample mean $\bar{x}$, and $SE_{\bar{x}}$, which incorporates the sample standard deviation s . Because of this, t is more variable than $N(0, 1)$ (more room for noise). However, as n (and hence degrees of freedom) increases, s

becomes a better estimate of the true population standard deviation, so the distinction ultimately vanishes with larger samples. This can all be visualized in Figure ?? where the degrees of freedom increase from 2 to 10 and you see the t -distribution approaches the Normal bell curve.

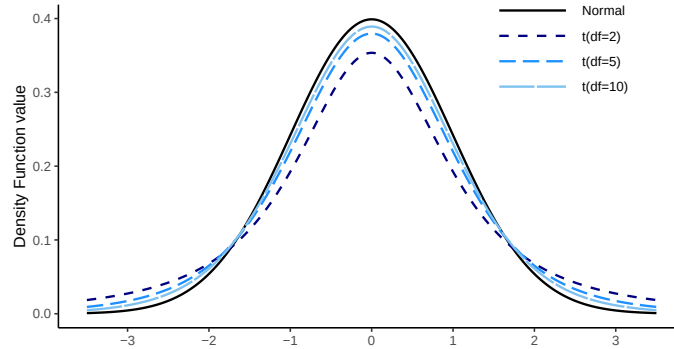


Figure 1.11: Overlaid distribution functions showing the relationship between a standard normal distribution and that of a t distribution with 3, 5 and 10 degrees of freedom. As the degrees of freedom increases, the t distribution becomes more *normal*

The t -distribution is usually the relevant sampling distribution when we use a sample mean $\bar{x}$ to make inferences about the corresponding population mean μ . Later in the course, the need to make more complicated inferences will require us to develop different sampling distribution models to facilitate the statistics involved. These will be presented as needed.

1.8 Two-sample inference

A key concept from your introductory statistics course that builds off sampling distribution theory is the comparison of the mean from two populations based on the sample means using a “two-sample t -test”. Consider testing

$$H_0 : \mu_A = \mu_B \quad \text{versus} \quad H_A : \mu_A < \neq > \mu_B$$

where the notation $< \neq >$ represents one of the common alternative hypotheses from your introductory statistics course (less than $<$, not equal to $\neq$, or greater than $>$) and, μ_A is the population mean from population A and μ_B is the population mean from population B . We statistically test this hypothesis via

$$t_0 = \frac{\bar{x}_A - \bar{x}_B}{SE_{\bar{x}_A - \bar{x}_B}}$$

where $SE_{\bar{x}_A - \bar{x}_B}$ is the standard error of the difference of sample means $\bar{x}_A$ and $\bar{x}_B$. It can be estimated in one of two ways (depending if we assume the

population variances are equal or not) and we reference your intro stat class for details (you may recall, pooled versus non-pooled variance). In R, we can perform this test, and construct the associated $(1 - \alpha)100\%$ confidence interval for $\mu_A - \mu_B$,

$$\bar{x}_A - \bar{x}_B \pm t_{\alpha/2} SE_{\bar{x}_A - \bar{x}_B}$$

using the function `t.test()`. Here $t_{\alpha/2}$ is the $1 - \alpha/2$ quantile from the t -distribution (the degrees of freedom depend on your assumption about equal variance).

Before jumping to an example, take a moment to look at the above null and alternative hypotheses. The null hypothesis states $\mu_A = \mu_B$, which effectively says there is no difference in the mean between the two groups. That is, whether the data came from group A or group B it does not provide any explanation about the variability in the data. However, the alternative says that there is a difference and that knowing the group from which the data came helps explain some of the observed variability.

Now for an example, consider comparing the ACT scores for incoming freshmen in 1996 to 2000 using the university admission data. Perhaps we have a theory that ACT scores were higher in 1996 than 2000. That is, we formally wish to test

$$H_0 : \mu_{1996} = \mu_{2000} \quad \text{versus} \quad \mu_{1996} > \mu_{2000}$$

First we need to select a subset of the data corresponding to those two years.

```
uadata.trim <- uadata |>
  filter(year %in% c(1996, 2000) )
```

Here we `filter()` the `uadata` so that only years in the vector `c(1996, 2000)` are included. Now that we have a proper subset of the data, we can compare the ACT scores between the two years and build a 98% confidence interval for the mean difference.

```
t.test(act ~ year, data=uadata.trim, conf.level=0.98,
       alternative="greater", var.equal=TRUE)
```

```
##
## Two Sample t-test
##
## data:  act by year
## t = 0.3013, df = 277, p-value = 0.382
## alternative hypothesis: true difference in means between group 1996 and group 2000 :
## 98 percent confidence interval:
## -0.791858      Inf
## sample estimates:
## mean in group 1996 mean in group 2000
##          24.6901          24.5547
```

We are 98% confident that the mean difference between 1996 ACT scores and 2000 ACT scores is in the interval $(-0.792, \infty)$. Further, we lack significant evidence (p -value ≈ 0.3817) to conclude that the average ACT score in 1996 is greater than that in 2000 for incoming freshmen (also note that the value 0 is included in the confidence interval).

1.8.0.1 Coding Notes

A few things about the code above.

- We use the notation `act ~ year`, this tells R that the `act` scores are a function of `year`. This is known as a **formula** and will be important throughout the course.
- We specify the data set to be used to perform the test (`data=uadata.trim`).
- We specify the confidence level (by default it is 0.95) with `conf.level=0.98` and the alternative hypothesis (by default it is a not-equal test) with `alternative=greater`.
- Lastly, we set `var.equal=TRUE` so the standard error (and degrees of freedom) are calculated under the assumption that the variance of the two populations is equal (pooled variance).

Chapter 2

Statistical Modeling and Designed Experiments

We begin our dive into Statistical Modeling by building from some introductory statistics material, namely two-sample inference. After introducing some crucial concepts, we expand two-sample inference into analysis of variance. The learning objective of this unit include:

- Distinguishing between observational studies and designed experiments.
- Understanding the basic structure of statistical model.
- Understanding that statistical inference can be formulated as the comparison of models.
- Performing a full analysis involving a one-factor experiment.

2.1 Observational Studies versus designed experiments

Before introducing some details on statistical models we need to step back and discuss the **source of our data**. Are the data a “sample of convenience”, or were they obtained via a designed experiment, or some random sampling scheme, or by some sort of survey? How the data were collected has a crucial impact on what conclusions can be meaningfully made.

It is important to recognize that there are two primary methods for obtaining data for analysis: **designed experiments** and **observational studies**. There is a third type of data collected through **survey sampling** methods that incorporates elements from both designed experiments and observational studies, but are excluded here. It is crucial to know the distinction between each type of data, as this results in a different approach to interpreting estimated models.

2.1.1 Observational Studies

In many situations a practitioner (perhaps you, or a client) simply collect measurements on predictor and response variables as they naturally occur, without intervention from the data collector. Such data is called **observational data**.

Interpreting models built on observational data can be more challenging. There are many opportunities for error and any conclusions will carry with them substantial unquantifiable uncertainty. Nevertheless, there are many important questions for which only observational data will be available. For example, how else can we study something like differences in prevalence of obesity, diabetes and other cardiovascular risk factors between different ethnic groups? Or the effect of socio-economic status on self esteem? It may be impossible to design experiments to investigate these questions, so we must make the attempt to build good models with observational data in spite of their shortcomings.

In observational studies, establishing causal connections between response and predictor variables is nearly impossible. In the limited scope of a single study, the best one can hope is to establish associations between predictor variables and response variables. But even this can be difficult due to the uncontrolled nature of observational data. Unmeasured and possibly unsuspected “lurking” variables may be the real cause of an observed relationship between Y and some predictor X .

In observational studies, it is important to adjust for the effects of possible confounding variables (more on that later). Unfortunately, one can never be sure that the all relevant confounding variables have been identified. As a result, one must take care in interpreting estimated models involving observational data.

For the next few chapters we will concentrate on analyses of data from experiments, but the topic of observational studies will be more thoroughly visited later.

2.1.2 Designed experiments

In a **designed experiment**, the researcher has control over the settings of the predictor variables. For example, suppose we wish to study several physical exercise regimens and how they impact calorie burn. In a designed experiment, we (the experimenter) assign the different exercise regimes to participants, thus controlling the settings of the predictor variables. Typically the values of the predictor variables are discrete (that is, a countably finite number of controlled values). Because of this, predictor variables of this type are known as **factors** and the specific values are often called the **treatments**. The **experimental units** (EUs) are the entities to which the treatment are applied; *e.g.*, the people we use for the study. We may control multiple predictor variables in a study, such as the amount of time spent exercising or the amount of carbohydrates consumed prior to exercising. Other predictors may not be controlled but can be measured, such as baseline metabolic variables and the participant’s age, so a

designed experiment can include some *observational* elements. Other variables, such as the temperature in the room or the type of exercise done, could be held fixed to control for potential environmental factors (we typically these **nuisance variables**).

Having control over the conditions in an experiment allows us to make stronger conclusions from the analysis. Another key feature from a designed experiment is that the model is chosen for us! We will see later in the class that with observational data we typically need to select a model, but with a designed experiment the model is predetermined based on the experiment. We discuss more on modeling later in this chapter.

2.2 Designed experiment vocabulary

2.2.1 What is an experiment?

In an experiment, a researcher manipulates one or more variables, while holding all other variables constant. The main advantage of well-designed experiments over observational studies is that we can establish cause and effect relationships between the predictors and response.

One of the most important things to keep in mind with analyzing designed experiments is that **the structure of the experiment dictates how the analysis may proceed**. There are a plethora of structural possibilities of which we will only scratch the surface here.

Consider, for example, these four scenarios:

1. Suppose a study is conducted on the effects a new drug has on patient blood pressure. At the start of the study the blood pressure of all participants is measured. After receiving the drug for two months the blood pressure of each patient is measured again. The analysis to determine the effect of the drug on blood pressure is the **Paired *t*-test** from your introductory statistics class. Here the *pairing* of each observation is the difference between the before and after blood pressure.
2. Consider a simple design with a single categorical input variable, or one factor, where each subject (experimental unit) is measured only under one of the factor levels. For example, suppose three different drugs are administered to 18 subjects. Each person is randomly assigned to receive only one of the three drugs. A response is measured after drug administration. The analysis we use for such data is known as a **one-way analysis of variance**.
3. We may have a design with two factors, where every level of the first factor appears with every level of the second factor. For example, suppose your data are from an experiment in which alertness levels of subjects were measured after they had been given one of three possible dosages of a

drug and whether or not they were allowed to nap for 20 minutes. Such an experimental design is known as a factorial design. Here, the two factors are drug dosage and the occurrence of a nap. The analysis we use for such data is known as a **two-way analysis of variance**.

4. Suppose we have a design with only one factor of interest (word type). Five subjects are asked to memorize a list of words. The words on this list are of three types: positive words, negative words and neutral words. The response variable is the number of words recalled by word type, with a goal of determining if the ability to recall words is affected by the word type. Note that even though there is only one factor of interest (word type) with three levels (negative, neutral and positive), this data structure differs greatly from a one-way analysis of variance mentioned above because each subject is observed under every factor level. Thus, there will be variability in the responses within subjects (how a individual varies between positive, negative and neutral words) as well as between subjects (how two different people vary). This fact will affect how we analyze the data. This is known as a **within-subjects analysis of variance**, or **repeated measures**.

We will cover each of these analyses in the next few chapters.

2.2.2 Analysis of variance

Analysis of variance (ANOVA) is an analytic tool in statistics to partition the variation in a response variable and attribute it to known sources in a model. In practice, it is essentially an extension of the more familiar two-sample t -test. As stated earlier, there are many different experimental structures possible when conducting an experiment and ANOVA can adapt to many of those structures. Some of the other widely used structures in practice are known as nested designs, split-plot designs, and repeated measures designs. It is worth reiterating that the structure of the experimental data will dictate how an analysis of variance is used to model the data. In each of the designs described above, some variant of an Analysis of Variance is used for analysis.

2.2.3 Elements of a designed experiment

All experiments have factors, a response variable, and experimental units. Here are some definitions and terms:

- **Factors:** A factor is an explanatory variable manipulated by the experimenter. Each factor has two or more levels (*i.e.*, different values of the factor). Combinations of factor levels form what are called **treatments**. The table below shows factors, factor levels, and treatments for a hypothetical experiment:

In this hypothetical experiment, the researcher is studying the possible effects of Vitamin C and Vitamin E on health. There are two factors: dosage of Vitamin C and dosage of Vitamin E. The Vitamin C factor has

Vitamin E	Vitamin C		
	0 mg	250 mg	500 mg
0 mg	Treatment 1	Treatment 2	Treatment 3
400 mg	Treatment 4	Treatment 5	Treatment 6

three levels: 0 mg per day, 250 mg per day, and 500 mg per day. The Vitamin E factor has 2 levels: 0 mg per day and 400 mg per day. The experiment has six treatments. Treatment 1 is 0 mg of E and 0 mg of C; Treatment 2 is 0 mg of E and 250 mg of C; and so on.

- **Response variable:** In the hypothetical experiment above, the researcher is looking at the effect of vitamins on health. The response variable in this experiment would be some measure of health (annual doctor bills, number of colds caught in a year, number of days hospitalized, etc.).
- **Experimental units:** The recipients of experimental treatments are called experimental units. The experimental units in an experiment could be anything - people, plants, animals, or even inanimate objects.

In the hypothetical experiment above, the experimental units would probably be people (or lab animals). But in an experiment to measure the tensile strength of string, the experimental units might be pieces of string. When the experimental units are people, they are often called participants; when the experimental units are animals, they are often called subjects.

Three characteristics of a well-designed experiment. While the elements above are common to all experiments, there are aspects to the manner in which the experiment is run which are critical to it being a “well-designed” experiment. A well-designed experiment includes design features that allow researchers to eliminate extraneous variables as an explanation for the observed relationship between the factor(s) and the response variable. Some of these features are listed below.

- **Control:** Control refers to steps taken to reduce the effects of extraneous variables (*i.e.*, variables other than the factor(s) and response). These extraneous variables are called lurking variables. Control involves making the experiment as similar as possible for experimental units in each treatment condition. Several strategies exist to help add some *control variables* to an experiment (all are closely related):
 - **Control group:** A control group is a baseline group that receives no treatment or a neutral treatment. To assess treatment effects, the experimenter compares results in the treatment group to results in the control group. In the above hypothetical example, the treatment of 0 mg of Vitamin C and 0 mg of Vitamin E likely acts as a control.
 - **Placebo:** Often, participants in an experiment respond differently

after they receive a treatment, even if the treatment is neutral. A neutral treatment that has no “real” effect on the dependent variable is called a placebo, and a participant’s positive response to a placebo is called the placebo effect. To control for the placebo effect, researchers often administer a neutral treatment (*i.e.*, a placebo) to the control group. The classic example is using a sugar pill in drug research. The drug is effective only if participants who receive the drug have better outcomes than participants who receive the sugar pill.

- **Baseline treatment:** another approach that is similar to a control or placebo group is the application of some sort of standard *accepted* treatment. For example, suppose we recruit 15 patients that have recently been diagnosed with a particular disease. We wish to study the effects of two new drugs, call them experimental drugs A and B. It would be unethical to assign some participants to a control (no treatment) or placebo (sugar pill) group, so we may compare the two new experimental drugs to an established treatment (this way all patients receive some sort of treatment for their disease). The established treatment acts as a *control group* or *placebo group* in this setting.
- **Blinding:** Of course, if participants know which group they have been assigned, it can bias the results. For example, if someone knows they are receiving a placebo, the placebo effect will be reduced or eliminated; and the placebo will not serve its intended control purpose. Blinding is the practice of not telling participants whether they are receiving a placebo or even what treatment they receive. In this way, participants in the control and treatment groups experience the placebo effect equally. Often, knowledge of which groups receive placebos is also kept from people who administer or evaluate the experiment. This practice is called double blinding. It prevents the experimenter from “spilling the beans” to participants through subtle cues; and it assures that the analyst’s evaluation is not tainted by awareness of actual treatment conditions.
- **Randomization:** Randomization refers to the practice of using chance methods (random number generation, etc...) to assign experimental units to treatments. In this way, the potential effects of lurking variables are distributed at chance levels (hopefully roughly evenly) across treatment conditions. For example, suppose we had 12 experimental units which we wanted to completely randomly allocate to three different treatments (call them A, B and C). We can accomplish this as follows in R:

```
ids <- 1:12
trt <- rep(c('A','B','C'), each=4)
randomized <- sample(ids, size=length(ids), replace=FALSE)
CRD <- data.frame(trt, randomized)
```

CRD

##	trt	randomized
## 1	A	5
## 2	A	6
## 3	A	8
## 4	A	1
## 5	B	3
## 6	B	2
## 7	B	10
## 8	B	11
## 9	C	9
## 10	C	4
## 11	C	7
## 12	C	12

Such a design is known as a **completely randomized design** (or CRD).

Randomization is arguably one of the most important pieces of any well-designed experiment. Failure to randomize can lead to confounded conclusions (see discussion below) and biased results!

Example: Consider a contrived, but simple, experiment where you might be comparing a new heart disease drug to a placebo. Suppose your sample consists of one-hundred adults each with diagnosed heart disease. Further, suppose the ages of the adults are uniformly distributed across the range 20 to 90. Out of convenience suppose the new drug is administered to the youngest fifty volunteers and the placebo to the oldest 50 volunteers.

The results of this experiment would not be trustworthy since there is the potential for the age of the patients to influence the results. However, if the drug/placebo assignment was randomly assigned to different patients of different ages and backgrounds, the randomization would help eliminate any age bias.

- **Replication:** Replication refers to the practice of assigning each treatment to many experimental units. The purpose of replication is as follows:
 - Estimate the noise in the system. Recall earlier in the text when we discussed sources of variation in data. If you were to apply precisely the same treatment to a set of different individuals, the response would naturally vary due to myriad reasons of little-to-no interest to the researcher. So, responses vary to some degree due to simple random chance fluctuations, or “noise.” For our purposes, this constitutes a single source of variation.
 - Now, apply different treatments to a set of individuals. The responses would still vary, but now there are two sources of variation: the random element described in the previous bullet point (because we still have different individuals), but now we also incorporate the differ-

ent treatments and how they may influence the response. So, one purpose of replication is **we can estimate the natural noise in the system, and can determine if the variability introduced by changing the treatment rises above the noise**. The more replication in an experiment, the better you can estimate the noise. However, this comes at an expense!

- Help wash out the effect of lurking variables. This was also mentioned as a reason for randomization. When coupled with randomization, replication provides the “opportunity” for your experiment to have a mix of all those lurking variable appear under each treatment, thus mitigating their effects from biasing your results.

2.2.3.1 Confounding

Confounding is a condition that occurs when the experimental controls do not allow the experimenter to reasonably eliminate plausible alternative explanations for an observed relationship between factors and the response. Needless to say, confounding renders experimental results to be seriously impaired, if not totally useless. Consider this example:

Example: A drug manufacturer tests a new cold medicine with 200 participants: 100 men and 100 women. The men receive the drug, and the women do not. At the end of the test period, the men report fewer colds.

What is the problem? This experiment implements no controls! As a result, many variables are confounded, and it is impossible to say whether the drug was effective. For example:

- Biological sex is confounded with drug use. Perhaps, men are less vulnerable to the particular cold virus circulating during the experiment, and the new medicine had no effect at all but less men happened to ill. Or perhaps the men experienced a placebo effect.

This experiment could be strengthened with a few controls that would mitigate the confounding effects:

- Women and men could be randomly assigned to treatments.
 - One treatment group could receive a placebo.
- Blinding could be implemented to reduce the influence of “expectation” on the part of participants with regard to the outcome of a treatment.

Then, after all this, if the treatment group (*i.e.*, the group getting the medicine) exhibits sufficiently fewer colds than the control group, it would be reasonable to conclude that the medicine was the reason for effectively preventing colds, and not some other lurking reason.

2.3 Statistical Analyses is Modeling

Think of a model airplane or a Lego kit of the Eiffel Tower – it is just a simplified representation of the real thing.

In many statistical problems we are interested in describing the relationship that may exist among several different measured variables. For example, your current GPA could be impacted by how much you study. However, there may be several contributing factors – your credit hour load, your ACT score, whether or not you have a part-time job, etc...

Of course, what makes your GPA is the result of a very highly complex combination of factors, some important and others not so important.

A **statistical model** is a mechanism to describe the structural relationship between some measured outcome (called the **response** variable) and one or more impacting variables (called **predictor** variables or **factors** or *features*, depending on the contextual circumstance) in a simplified mathematical way.

It is in this sense that you can think of a statistical model as a “simplification”: we know that the true relationship between response and predictor variables is very complex and detailed. The goal of the model is not to capture all the intricacies of the complexity but rather, the model only seeks to describe the essential features of any relationships that exist.

2.3.1 Example of a two-sample *t*-test as a model

It turns out, we have already fit a statistical model. Consider a variation of the two-sample *t*-test we performed in the previous chapter on the University Admissions data (?). For completeness we input the data again.

```
uadata <- read_table("https://tjfisher19.github.io/introStatModeling/data/univadmissions.txt")
```

We look to compare the ACT scores in 1996 and compare them to the ACT scores in 2000. Similar to code from Chapter 1, we filter to the years 1996 and 2000 and convert the year variable into a factor variable.

```
uadata.trim <- uadata |>
  filter(year %in% c(1996, 2000) ) |>
  mutate(year = factor(year) )

t.test(act ~ year, data=uadata.trim, conf.level=0.98,
       alternative="two.sided", var.equal=TRUE)
```

```
##
## Two Sample t-test
##
## data:  act by year
## t = 0.3013, df = 277, p-value = 0.763
```

```
## alternative hypothesis: true difference in means between group 1996 and group 2000
## 98 percent confidence interval:
##  -0.916072  1.186864
## sample estimates:
## mean in group 1996 mean in group 2000
##           24.6901           24.5547
```

Looking at the code you'll note here we are performing a `two.sided` or “not equal to” test (in chapter 1 we performed a `greater` than test, or “>” test). More importantly note the `act ~ year` notation – this is known as a **formula** in R. We are telling R that the ACT scores are a function of the student's incoming Year. This concept should be somewhat familiar as it is similar to the regression ideas you likely saw in your Intro Statistics course.

It turns out that nearly all statistical inference can be framed in terms of modeling. To see this we first must cover some *light math*.

Recall from Intro Statistics that a random variable Y is typically assumed to come from a Normal distribution with mean μ and standard deviation σ . Standard shorthand notation for this is to write

$$Y \sim N(\mu, \sigma).$$

You may also remember that if you subtract off the mean, μ , from the random variable Y it will just shift the distribution by μ units. For example

$$Y - \mu \sim N(0, \sigma)$$

Thus, we could reformulate this equation further with

$$Y = \mu + \varepsilon \tag{2.1}$$

where

$$\varepsilon \sim N(0, \sigma).$$

The equation (??) can be considered a *baseline* model. Here, we are simply stating that the random variable Y is a function of a mean μ and some random component ε , which happens to be from a Normal distribution with mean 0 and standard deviation σ . In words have the model

$$\mathbf{Data} = \mathbf{Systematic\ Structure} + \mathbf{Random\ Variation}$$

where in the baseline model the Systematic Structure is simply the constant μ .

Estimation

A quick note about estimation. In your Intro Stat class you worked with this model regularly. We would estimate μ with the sample mean $\bar{y}$ and estimate σ with the sample standard deviation s . One-sample hypothesis testing and confidence intervals are all formulated using this baseline model.

Two-sample t -test as a Model

Now consider the case of a two-sample t -test; for example, testing if ACT scores were different in 1996 compared to 2000. From a hypothesis testing perspective we are comparing:

$$H_0 : \mu_{1996} = \mu_{2000} \quad \text{vs.} \quad H_A : \mu_{1996} \neq \mu_{2000}$$

If H_0 were true, we have $\mu_{1996} = \mu_{2000} = \mu$ and the baseline model (??) is true. But, in the case of the alternative hypothesis, we have $\mu_{1996} \neq \mu_{2000}$ and thus our model is:

$$Y = \mu_j + \varepsilon, \quad \text{where } j = 1996 \text{ or } 2000$$

To further expand the model, let's create a new variable called τ that measures how the 2000 ACT scores deviate from the 1996 ACT scores. That is, consider a model for the i^{th} observation:

$$Y_i = \mu_{1996} + \tau + \varepsilon_i$$

Here the value μ_{1996} corresponds to the mean ACT score in 1996 and works as a baseline mean, τ measures how the year 2000 ACT scores **differ** from 1996 (increase? decrease? no change?) and ε_i is the underlying random part of the i^{th} observation Y_i . Thus, if Y_i is an observation from the year 2000, the structural component will be $\mu_{1996} + \tau$, whereas if the observation is from 1996, the structural component is simply μ_{1996} .

Another way to frame this model is to augment with a random variable X_i . If observation i is from 1996 then X_i takes on the value zero and if the observation is from 2000 then X_i takes on the value one. Variables of this structure are known as *indicator* or *dummy* variables. Let Y_i be the observed ACT score for student i and consider the following:

$$Y_i = \mu_{1996} + \delta \cdot X_i + \varepsilon_i. \quad (2.2)$$

Now μ_{1996} is the mean of 1996 ACT scores (when $X_i = 0$), δ is the effect the year 2000 has on mean ACT scores (*e.g.*, suppose mean ACT scores in 1996 were 27, but in 2000 they were 31, thus $\delta = 4$), thus $\mu_{1996} + \delta$ is the mean ACT score in the year 2000 (δ works much like the τ effect above). Lastly, ε_i is how observation i deviates from the structural component and we assume $\varepsilon_i \sim N(0, \sigma)$ for standard deviation σ for all $i = 1, \dots, n$.

When performing a two-sample t -test we are essentially testing $H_0 : \delta = 0$ versus $H_A : \delta \neq 0$. Note that if H_0 were true, we are back to the baseline model (??).

So to frame this another way, a two-sample t -test is a choice of

$$H_0 : \text{Baseline Model} \quad Y_i = \mu + \varepsilon_i$$

and

$$H_A : \text{More complex model} \quad Y_i = \mu + \delta \cdot X_i + \varepsilon_i.$$

To tie everything together note that in H_A we are essentially describing observation Y_i as a function of the variable X_i . Because of this we typically refer to Y_i as the **response** (or dependent) variable and X_i as a **predictor** (or independent) variable. This coincides with the R formula notation, `response ~ predictor`.

Back to the example

In this observational study, we compared the ACT scores of incoming freshmen in 1996 and compared them to ACT scores for the incoming class of 2000. With a p -value of 0.7634 on 277 degrees of freedom, we do not have evidence to suggest that the ACT scores for freshmen in 1996 differs from those in 2000.

2.4 One-Way ANOVA

When there is a single factor variable whose levels may only change between different experimental units, we can analyze the effect of the factor on the response using a one-way analysis of variance (one-way ANOVA) on the between-subjects factor. If we had a single factor with just two levels, you could still use a one-way ANOVA, but it would be equivalent to running an independent samples t -test (see above for an example). In this sense, one-way ANOVA is an extension of the two-sample t -test (think a k -sample test).

In this design, we observe random samples from k different populations. As a designed experiment, these populations may be defined on the basis of an administered treatment. The levels of the factor being manipulated by the researcher form the different treatments. Frequently, the data are obtained by collecting a random sample of individuals to participate in the study, and then randomly allocating a single treatment to each of the study participants. If so, then **the individual subjects (experimental units) receiving treatment j may be thought of as a random sample from the population of all individuals who could be administered treatment j .**

In an observation study, the different factor levels are observed when the data is collected. Again, if the data are collected as a random sample, the individual subjects in each of the k treatments can be considered a random sample from that sub-population.

2.4.1 Model Structure

The one-way data structure looks like the following:

Treatment 1	Treatment 2	...	Treatment k
Y_{11}	Y_{21}	...	Y_{k1}
Y_{12}	Y_{22}	...	Y_{k2}
Y_{13}	Y_{23}	...	Y_{k3}
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Y_{1n_1}	Y_{2n_2}	...	Y_{kn_k}

where Y_{ji} is the i^{th} observation within treatment j , for $j = 1, \dots, k$ and $i = 1, \dots, n_j$.

The model for such data has the form

$$Y_{ji} = \mu_j + \varepsilon_{ji} \quad (2.3)$$

where μ_j is the mean of the j^{th} treatment, $j = 1, \dots, k$, or more commonly,

$$Y_{ji} = \mu + \tau_j + \varepsilon_{ji}$$

where

- Y_{ji} is the i^{th} observation in the j^{th} treatment.
- μ is the overall mean of all the populations combined.
- τ_j is the deviation of the mean by the j^{th} treatment population from the overall mean μ .
- ε_{ji} is the random error term.

Similar to the two-sample t -test formulas above, this equation can also be written as

$$Y_i = \mu_1 + \delta_2 X_{2i} + \delta_3 X_{3i} + \dots + \delta_k X_{ki} + \varepsilon_i \quad (2.4)$$

where

- Y_i is the i^{th} observation.
- μ_1 is the mean of the first treatment.
- δ_j is the difference between the j^{th} treatment mean and the first treatment mean; *i.e.*, $\delta_j = \mu_j - \mu_1$.
- X_{ji} is an indicator variable, indicating if the i^{th} observation is in the j^{th} treatment; *i.e.*, $X_{ji} = 1$ when the i^{th} observation is in group j , otherwise $X_{ji} = 0$.

The usual test of interest in a one-way analysis of variance is to compare the population means. The null and alternative hypotheses are given by:

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_k$$

$$H_a : \text{at least two of the population means differ}$$

Expressed in terms of the τ -model parameters, we really want to test to see if there are no treatment mean deviations among the populations, so the above hypotheses can be equivalently expressed as follows:

$$H_0 : \tau_1 = \tau_2 = \dots = \tau_k = 0$$

$$H_a : \text{at least one } \tau_j \neq 0 \quad \text{for } j = 1, \dots, k$$

An alternative formulation is based on the coefficients on the function with indicator variables:

$$H_0 : \delta_2 = \delta_3 = \dots = \delta_k = 0$$

$$H_A : \text{at least one } \delta_j \neq 0 \quad \text{for } j = 2, \dots, k$$

With this last formulation, under H_0 all the δ_j terms equal zero, so the X_{ji} terms have no effect on the response, and the model reduces to the baseline model $Y = \mu + \varepsilon$ in (??).

In terms of modeling, we are testing:

$$H_0 : \text{Baseline Model: } Y_{ji} = \mu + \varepsilon_{ji}$$

$$H_a : \text{More Complex Model: } Y_{ji} = \mu + \tau_j + \varepsilon_{ji}, \quad \text{at least one } \tau_j \neq 0$$

and we note the more complicated model can be expressed a few different ways.

2.4.2 Testing Procedure

We test these equivalent hypotheses by partitioning the total variability in the response into two components: (1) variation between treatments, and (2) variation within treatments:

Variance in Response = Variance between Treatments + Variance within Treatments

Recall from section ?? that the variance is essentially a function of the sum of square deviations. In this sense, we decompose the sum of square deviations as follows:

$$SST = SSM + SSE$$

$$\sum_{j=1}^k \sum_{i=1}^{n_j} (y_{ji} - \bar{y})^2 = \sum_{j=1}^k n_j (\bar{y} - \bar{y}_j)^2 + \sum_{j=1}^k \sum_{i=1}^{n_j} (y_{ji} - \bar{y}_j)^2 \quad (2.5)$$

where $\bar{y}_j$ is the mean of the j^{th} group with sample size n_j and $\bar{y}$ is the mean of all the observations. Before describing these terms in some detail please note we will never **need these equations** to perform our analysis, but their structure can be helpful in explanation. Here,

- SST is the **total sum of squares** (or sum of squares total). From the equation, this is the same squared deviations in the calculation of s^2 . SST essentially measures the total variability in the response variable (without including any effects from the treatments).

- SSM is the **model sum of squares** (or the sum of squares explained by the model), which in a one-way ANOVA captures the variance between the treatments. Note that the equation compares the means of each treatment group to the overall mean, essentially measuring the variability among the k different means.
- SSE is often called the **sum of squares error** (a bit of a misnomer since no *error* has occurred) or residual squared error. It measures the unexplained variability, or the variability within the different treatments (look at the equation, it compares each observation to its treatment mean).

The variation between treatments (SSM), effectively is the variability that is explained by the structural components in the model. The latter partition (SSE) is essentially residual error, or unexplained variability. The derivation of this partitioning are outside the scope of this text, but we will *hint* at the ideas further in Chapter ???. The resulting method for this hypothesis is the ANOVA F -test which essentially compares SSM to SSE – if SSM is *large* relative to SSE , then the different treatments are explaining substantial variability in the response. If SSM is *small* relative to SSE , then the treatments are not explaining much variability. The details of this derivation are not important for our purposes, but the resulting method is known as the F -test.

The F -test works like other hypothesis test you may have seen (*e.g.*, the t -test or χ^2 goodness-of-fit test). The test statistic is a ratio of variance estimates and follows the Fisher-Snedecor distribution (or F -distribution for short). The details of this distribution are not important for this course and are covered in a Mathematical Statistics course. An example of a one-way ANOVA in R is provided below.

2.4.3 Example of One-Way ANOVA

Example: Drug effects on “knockdown” times (from the text ?).

Immobilization of free-ranging white-tailed deer by drugs allows researchers the opportunity to closely examine deer and gather valuable physiological information. Wildlife biologists tested the “knockdown” time (time, measured in minutes, from injection to immobilization) of three different immobilizing drugs. Thirty male white-tailed deer were randomly assigned to each of three treatments: Group A received 5mg of liquid succinylcholine chloride (SCC); group B received 8 mg of powdered SSC; and group C received 200 mg of phencyclidine hydrochloride.

Original Source: ?

The data appear in the R dataframe `deerKnockdown.RData` in our repository: (<https://tjfisher19.github.io/introStatModeling/data/deerKnockdown.RData>)

```
site <- "https://tjfisher19.github.io/introStatModeling/data/deerKnockdown.RData"
load(url(site))
glimpse(deer_knockdown_times)
```

Drug	Mean	SD	Min	Median	Max
A	10.3	5.438	4	10.5	23
B	9.0	3.300	5	8.5	16
C	4.9	1.792	3	4.5	8

```
## Rows: 30
## Columns: 2
## $ Drug      <chr> "A", "A", "A", "A", "A", "A", "A", "A", "A", "A", "B", ~
## $ Knockdown_time <dbl> 11, 5, 14, 7, 10, 7, 23, 4, 11, 11, 10, 7, 16, 7, 7, 5, ~
```

Note the subtle change in the code compared to `read_csv()` and `read_table()`. When loading an R data.frame from a website, we need to use the `url()` function to open and close the internet connection inside the `load()` function.

After inputting the data into R, we will take a quick look at some summary statistics. Using the methods of Chapter 1, we build a table of summary statistics by treatment group.

```
deer_knockdown_times |>
  group_by(Drug) |>
  summarize(Mean = mean(Knockdown_time),
            SD = sd(Knockdown_time),
            Min = min(Knockdown_time),
            Median = median(Knockdown_time),
            Max = max(Knockdown_time) ) |>
  kable(digits=3) |>
  kable_styling(full_width=FALSE)
```

We also provide a plot of the data – here we plot Box-Whiskers plots by Drug, while overlaying the original data using a jitter plot and including the sample mean of each group.

```
ggplot(deer_knockdown_times, aes(x=Knockdown_time, y=Drug)) +
  geom_boxplot() +
  geom_jitter(height=0.25) +
  stat_summary(fun="mean", geom="point", shape=23, fill="gray60", size=4) +
  labs(x="Time to Immobilization (min)", y="Drug Treatment" ) +
  theme_minimal()
```

Visually it appears the immobilization times under drug C seem to be noticeably lower on average than for the other two drugs (Drug C has a mean of 4.9 compared to 10.3 and 9.0 for A and B, respectively). Also, we may suspect the variance of immobilization times is different between the three drugs (the width of the box-plots and the reported standard deviations). That is, the distributions of immobilization times for drugs A and B are more disperse than for drug C.

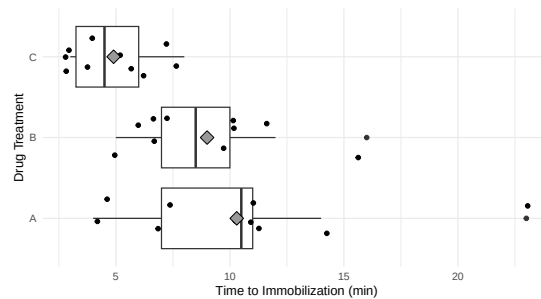


Figure 2.1: Box-whiskers plots, with data overlaid jittered data, and treatment means (gray diamond), showing the distribution of 'knockdown' times for each of the three drug treatments, A, B and C. Overall we see immobilization times is highest in treatment A, followed by B and C. We Also note that variability appears greatest in treatment A, followed by B and C.

To perform One-Way ANOVA we employ the `aov()` function (for Analysis Of Variance).

```
fit.drug <- aov(Knockdown_time ~ Drug, data=deer_knockdown_times)
```

That's it! The single line of code above fits the model described in (??) with $k = 3$ treatments corresponding to the three Drug types.

2.4.3.1 Coding Notes

The line of code above is fairly straightforward but R has done many things in that single line of code. The provided formula, `Knockdown_times ~ Drug`, and data declaration, `data=deer_knockdowntimes`, provide `aov()` all the information needed to fit the model. Later in the class we will see more complex formula statements but they all have the same general form: `response ~ predictor`.

The function call, `aov()`, estimates (or fits) the aforementioned linear model and calculates many associated parameters regarding that model. The `fit.drug <-` part of the code says to save that model in a new object that we call `fit.drug`. Much like the data types discussion in Section ??, the object `fit.drug` is a variable in R and has an associated type:

```
class(fit.drug)
```

```
## [1] "aov" "lm"
```

The object `fit.drug` has two associated classes: the `aov` type and `lm` type. This is because all ANOVA models are technically Linear Models (hence `lm`), so both object classes are assigned. Recall the derivation above where we write the response as a function of indicator variables in the form of a linear model, see (??). We also mentioned that the `aov()` calculates many parameters associated

with that model. The below code is only provided for explanatory purposes (it will never be needed to complete an analysis) but we can see most of the information that is stored in the `fit.drug` object with the `names()` function.

```
names(fit.drug)

## [1] "coefficients" "residuals"    "effects"      "rank"
## [5] "fitted.values" "assign"       "qr"          "df.residual"
## [9] "contrasts"    "xlevels"     "call"        "terms"
## [13] "model"
```

The object has 13 components of information stored in it! Some of these pieces of information are *technical* parameters that you will never need (e.g. `qr`), but others are part of a formal analysis and we will use them later. Although rarely needed for the analysis, it is worth noting that one piece of information is the model **coefficients**. Their exploration can be enlightening towards understanding the equation of the ANOVA model (??).

```
coefficients(fit.drug)

## (Intercept)      DrugB      DrugC
##          10.3         -1.3        -5.4
```

Here the **(Intercept)** has the value 10.3, which corresponds to the mean immobilization times for Drug A. The variable **DrugB** takes on the value -1.3, which is the estimated δ_j term from equation (??) for treatment B, and note that $10.3 + (-1.3) = 9.0$ which is the mean immobilization times for Drug B. Likewise, **DrugC** is the estimated value of δ_j in (??) for treatment C and $10.3 - 5.4 = 4.9$ is the mean immobilization time for Drug C.

Before proceeding with the analysis of the model output, we need to check the assumptions of our underlying model.

2.5 Model Assumptions & Assessment

Arguably the most important aspect in an analysis is the assumptions made with the chosen statistical model. In the field of statistics there are two types of assumptions made:

1. Assumptions about the systematic structure.
2. Assumptions about the random variation.

Most of the time, when referring to “assumptions” we are concerned about 2: Assumptions about the random variation. However, it is important to consider the assumptions on the structure.

2.5.1 Structural Assumptions

In all of our models/testing discussed above, we are assuming the response variable is a linear function on the predictor variable(s) (consider (??) and (??), both are linear equations). This assumption becomes more important later in the course when discussing linear regression. Here, since the model is looking at factors, or categorical treatments, as the inputs, the linear equation reduces to comparing the mean of each group.

From context, it is important to determine if this structure is appropriate.

2.5.2 Random Variation Assumptions

In this class, along with nearly all methods you learned in your introductory statistics course, we make the following assumptions about the underlying random variation (*i.e.*, stochastic part) of the response variable, ε :

- The ε_i terms are independent.
- The variance of ε_i is homogeneous; *i.e.*, $Var(\varepsilon_i) = \sigma^2$ is constant for all $i = 1, \dots, n$. This is sometimes known as being homoskedastic or having “constant variance”.
- The ε_i terms are Normally distributed.

Collectively, the three assumptions simply state

$$\varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2), \text{ for variance } \sigma^2$$

Important Note. In the above bullet points, the assumptions are listed in their order of importance. If you lack independence, *nearly all* statistical methods we cover in this text are invalid. The constant variance assumption is important but in many cases violations of the assumption can be addressed. Lastly, the Normality assumption tends to not be all that important except in small sample sizes and when the data are heavily skewed – but we can also address issues with Normality in many cases.

2.5.2.1 Independence

In general, independence (or lack thereof) is the result of the sampling scheme employed during data collection. If you collect your data according to a well designed experiment or a simple random sampling scheme with one observation per subject, there is usually no reason to suspect you will have a problem. Data collected sequentially in time (*e.g.*, daily high temperatures or repeated measures on the same subject) will exhibit a phenomenon known as autocorrelation (also called serial correlation) and is a major problem! Specialized methods are needed for this type of data. We will briefly cover the idea of repeated measures in Section ??, while a general treatment of autocorrelation is excluded from our work.

2.5.2.2 Constant Variance

To assess the constant variance assumption we typically consider two simple plots: the ‘Residuals vs Fitted’ plot and ‘Scale-Location’ plots as seen in Figure ?? (code to generate the plot is provided below).

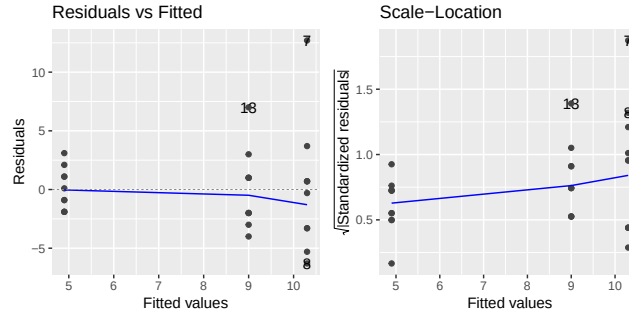


Figure 2.2: Residual diagnostic plots showing a ‘fanning’ effect in the residuals vs fitted and increasing linear trend in the Scale-Location plot, thus suggesting the variance increases with the mean.

Before reading these plots, we need to answer two questions: **What is a residual?** and **What is a fitted value?**

Consider the baseline model (??), $Y = \mu + \varepsilon$. We can estimate μ with $\bar{y}$, then we could *predict* each unobserved Y_i with $\hat{Y}_i = \bar{y}$. From there, we can approximate the random noise of the i^{th} observation ε_i with $e_i = Y_i - \hat{Y}_i = Y_i - \bar{y}$. The values $\hat{Y}_i$ are known as the **fitted**, or *predicted*, values and the e_i terms are the sample **residuals**.

In the case of the One-Way ANOVA model (??), we would calculate a different mean $\bar{y}_j$ for each of the k treatments and each Y_{ji} would be predicted by the appropriate $\bar{y}_j$ for $j = 1, \dots, k$. The sample residuals would be found by $e_{ji} = Y_{ji} - \hat{Y}_{ji} = Y_{ji} - \bar{y}_j$. Also recall that a fitted `aov` object in R already contains the residuals and fitted values.

How to read these plots?

A residuals versus fitted plot is provided in the left panel of Figure ?. If all is well, you should see fairly uniform (“constant”) spread of the points in the vertical direction and the scatter should be symmetric vertically around zero. The vertical spread here looks like it might be expanding as the fitted values increase (look at the dots from left-to-right, we see a ‘fanning’ effect where the vertical spread for fitted values around 5 is more narrow, than the spread at fitted values of 9, and the largest is at the fitted values near 10.5), suggesting that there may be non-constant error variance.

Note: In the case of ANOVA, you will also see *stacked* dots, or vertical strati-

fication in the Residuals vs Fitted plot. This is an artifact of the fitted values – all observations in the same treatment are predicted with the same value ($\bar{y}_j$), so you should expect to see this stratification. Also, when looking at ANOVA, we can generally *ignore* the plotted trend line (blue line in the left panel above). It will be more useful when assessing linear regression.

An attempt at refining the residuals vs fitted plot is given in the plot on the right in Figure ??, called the Scale-Location plot. The difference now is that instead of plotting the raw residuals e_{ij} on the vertical axis, R first standardizes them (so you can better check for extreme cases), takes their absolute value (to double the resolution in the plot) and then takes their square root (to remove skew that sometimes affects these plots). R then adds a trend line through the points: if the constant variance assumption is reasonable, this trend line should be roughly horizontal. Here, you can see it trending upward, so we likely have a variance that is increasing with the fitted values (*i.e.*, we not have constant variance).

Note: Using the absolute values of the residuals in the Scale-Location plot is often a good refinement. The reason being we typically do not care if a residual is positive or negative: all we are looking for is if the scatter is uniform across the plot. Thus, by looking at the magnitude of the residuals only (and not their direction), we are actually improving the resolution of the information that the plot contains in addressing constant variance. Also, like with the Residuals vs Fitted plot, we will see vertical stratifications since our fitted values are the sample mean within a treatment group.

2.5.2.3 Checking Normality

We typically make this assessment with a Normal quantile-quantile (Q-Q) plot.

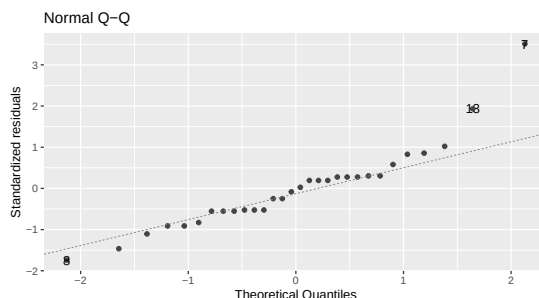


Figure 2.3: Normal Q-Q plot demonstrating the residuals may deviation from the Normality assumption.

In a normal Q-Q plot, the observed sample values (the “sample quantiles”) are plotted against the corresponding quantiles from a reference normal distribution (the “theoretical quantiles”). If the parent population is normally distributed, then the sample values should reasonably match up one-to-one with normal

reference quantiles, resulting in a plot of points that line up in a linear “45-degree” fashion.

A normal Q-Q plot of the residuals is presented in Figure ?? . A common *tool* to analyze these plots is sometimes known as the “fat pencil” test. Consider the thick pencils children are given when learning to write – lay that pencil on the plotted line, if all the points are covered by the pencil then the data is approximately normally distributed.

In Figure ?? there is some deviation from the line (observations 13 and 7). Coupled with the issues with variance outlined above, we may have evidence to suggest the normality assumption is not met.

Note: It is quite common for violations of constant variance and normality to go hand-in-hand. That is, if there are concerns about one, it is likely there will be concerns regarding the other.

2.5.3 Code to check assumptions

The code to generate the above plots can be achieved via the `autoplot` function when including the `ggfortify` library (?). The `autoplot` function on the fitted model provides 4 plots by default as seen in Figure ?? (note: we will discuss the Constant Leverage plot in Section ??).

```
library(ggfortify)
autoplot(fit.drug)
```

2.5.3.1 Coding Notes

You can think of the `autoplot()` function in `ggfortify` package as a *smart* function. That is, it makes some reasonable decisions for you. Specifically, you pass in the fitted ANOVA model (in this case `fit.drug`) and the `autoplot()` function determines the class of that model (here, the `aov` or `lm` type). Recognizing that model type, `autoplot` extracts the residuals and fitted values from the model, and generates the 4 plots in Figure ?? . With different object types, `autoplot()` will behave differently.

2.6 Transforming your response

In cases where we have violations of our underlying assumptions, a common remedy is to **transform the response variable**.

In our working example, we have evidence from the above residual diagnostic plots that the variance appears to be increasing with the response (determined by assessing the constant variance assumption) and that there may be some violations to normality. One way to address this issue is to transform the response variable. We will go into more details in a later chapter but common transformations include $Y^* = \log_{10}(Y)$, $Y^* = \sqrt{Y}$ and $Y^* = 1/Y$. Here we

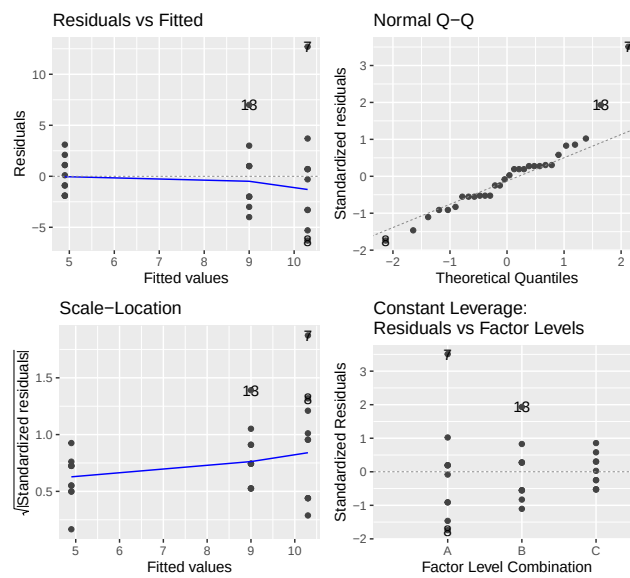


Figure 2.4: Residual plots for the One-Way ANOVA studying the effect of different drugs on immobilization times - Top-left a Residuals vs Fitted Plot; Top-right a Normal Q-Q plot of the residuals; Bottom-left the Scale-Location plot; and Bottom-right a Residuals vs Leverage plot.

will use a logarithmic as it is fairly common and more interpretable than most other transformations. To transform the response variable we use the `log10()` function.

Math Tangent: Note that the logarithm function is a monotone increasing function. So if $a > b$ then $\log(a) > \log(b)$. Thus, if an ANOVA model tells us that the mean of the log-response under the treatments differ, we can reasonably infer that the mean of the treatments differ (in the original units). Also, recall from an algebra course that the base of the logarithm does not affect the overall behavior of the function (you can easily convert from one base to another with the ‘change of base formula’). For this reason, we recommend you use a base-10 logarithm as it is commonly used in most scientific fields and is fairly interpretable (can interpret as orders of magnitude).

The following R code fits the one-way ANOVA model to the logarithm of knockdown times (transformed response) and provides the residuals plots in Figure ??.

```
fit.log.drug <- aov(log10(Knockdown_time) ~ Drug, data=deer_knockdown_times)
autoplot(fit.log.drug)
```

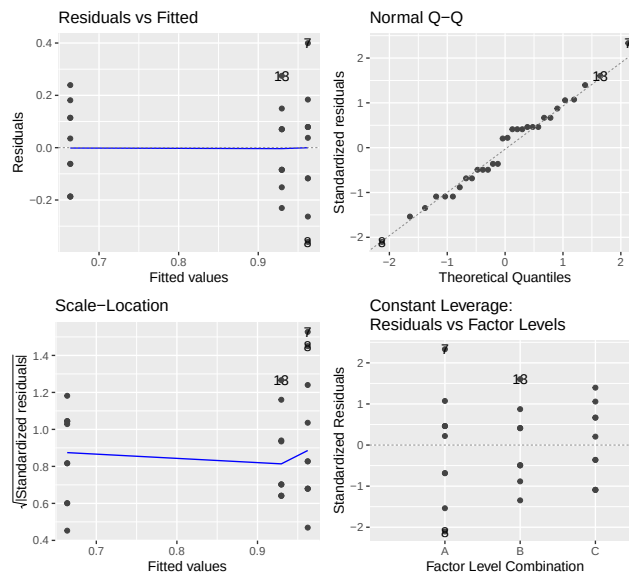


Figure 2.5: Residual diagnostic plots when the response variable is the logarithm of the immobilization time. Here the heteroskedasticity and non-normality in the original model has been improved, albeit not entirely mitigated, with some minor fanning in the residuals vs fitted plot.

Note the improvement in the Normal Q-Q plot – the observed quantiles closely match the theoretical quantiles. The Scale-Location plot shows that the issues

with a changing variance have largely been addressed as we see a nearly horizontal line. In the Residuals vs Fitted plot the violation in constant variance has improved although some indication the vertical spread is increasing is still present.

Overall, the logarithmic transformation has improved our model assumptions. We can try other transformations but a *perfect transformation* is unlikely to exist. When the variance increases with the fitted values, a logarithm on the response tends to be useful. Later in the text we will discuss other approaches to transforming the response, but for now we proceed with the logarithmic transformation.

2.6.1 A note on logarithmic transformations

Although logarithms tend to be somewhat easy to interpret, we do need to step through what is happening mathematically to properly interpret the results. Before doing so, recall some properties of logarithms from your Algebra course:

- $\log_{10}(1) = 0$,
- For an x such that $0 < a < 1$, $\log_{10}(a) < 0$,
- For an $a > 1$, $\log_{10}(a) > 0$,
- $\log_{10}(ab) = \log_{10}(a) + \log_{10}(b)$ and $\log_{10}(a/b) = \log_{10}(a) - \log_{10}(b)$,

and also recall the following power function properties:

- $10^0 = 1$,
- For an $a < 0$, then $0 < 10^a < 1$,
- For an $a > 0$, then $10^a > 1$,
- $10^{a+b} = 10^a \times 10^b$,

and of course the relationship between power functions and logarithms:

- $\log_{10}(10^a) = 10^{\log_{10}(b)} = a$.

Now, if you consider equation (??) with regard to our working example, Y is the knockdown time but we fit the ANOVA model on $\log_{10}(Y)$. So we have fit the following model:

$$\log_{10}(Y) = \mu_j + \varepsilon$$

where each μ_j corresponds to the mean of the logarithm of the response in treatment j . Using the indicator variable ideas (see (??) for the two-sample t -test derivation), the model can be expressed as

$$\log_{10}(Y_i) = \mu_A + \delta_B X_{Bi} + \delta_C X_{Ci} + \varepsilon_{ji}$$

where μ_A is the mean of the logarithm of knockdown times in group A. X_{Bi} takes on the value 1 if the i^{th} observation is in group B, otherwise it is zero. Likewise X_{Ci} for group C. The value δ_B measures how the mean of log-knockdown times in group B differs from A, and δ_C how group C differs from A.

If we were to “un”-log the above model to the original units by raising each side to the power of 10, we get:

$$10^{\log_{10}(Y)} = Y = 10^{\mu_j + \varepsilon} = 10^{\mu_j} \times 10^{\varepsilon}$$

and

$$10^{\log_{10}(Y_i)} = Y_i = 10^{\mu_A + \delta_B X_{Bi} + \delta_C X_{Ci} + \varepsilon_{ji}} = 10^{\mu_A} \times 10^{\delta_B X_{Bi}} \times 10^{\delta_C X_{Ci}} \times 10^{\varepsilon_i}$$

by properties of logarithms and power rules. So technically speaking, in terms of the original units, we have fit a **multiplicative model** because the effects of each treatment (the δ_j terms) are changing the response by multiplicative factors.

Interpretation of the multiplicative model is a bit more nuanced than that of a linear models since we are no longer simply adding or subtracting effects of different treatments.

Consider an observation from group A, μ_A is the mean of the base-10 logarithm knockdown times in group A. Now both $X_{Bi} = X_{Ci} = 0$ since the observation is in group A, and $10^0 = 1$. The expected value of ε is zero, so the three power terms to the right all reduce to one, and the knockdown times are modeled via $Y_i = 10^{\mu_A}$ where μ_A is on a log-scale so we get back to the original units since $10^{\log_{10}(a)} = a$.

However, suppose the observation is from group B, now $X_{Bi} = 1$ and $X_{Ci} = 0$. The term δ_B measures how the mean of the logarithm of knockdown times in group B differ from group A; *i.e.*, $\delta_B = \mu_B - \mu_A$ where μ_A and μ_B are the means of the logarithm of knockdown times in groups A and B, respectively. In terms of the original units, the model is $10^{\mu_A} \times 10^{\delta_B}$, which effectively multiplies the effect of treatment A by 10^{δ_B} . Now, in this case we can replace δ_B with $\mu_B - \mu_A$ and some algebra results in 10^{μ_B} , and since μ_B is the mean of the logarithm knockdown times in group B, we are in the original units.

2.7 ANOVA Testing in R

In the working example above, we fit a initial model, assessed its assumptions, transformed the response variable and fit the model again. Specifically, the following code was used in our model fitting for the deer immobilization times as a function of drug treatments:

```
# Input data from the course repository
site <- "https://tjfisher19.github.io/introStatModeling/data/deerKnockdown.RData"
load(url(site))

# Fit the model with the original response
fit.drug <- aov(Knockdown_time ~ Drug, data=deer_knockdown_times)
```

term	df	sumsq	meansq	statistic	p.value
Drug	2	0.536	0.268	8.219	0.002
Residuals	27	0.880	0.033	NA	NA

```
# Check model assumptions
autoplot(fit.drug)
# We determined a transformation is warranted due to issues
# with the constant variance and normality assumption

# Consider a log10 transformation on the response
# and asses the model assumptions
fit.log.drug <- aov(log10(Knockdown_time) ~ Drug, data=deer_knockdown_times)
autoplot(fit.log.drug)
# Model assumptions looked acceptable
```

Now that we have assessed that the model reasonably meets the underlying assumptions, we can proceed and perform the analysis. The test comparing the three drug population means (on a log scale) is provided via an ANOVA table. There are actually three different approaches to building this table in R. The `summary()` and `anova()` (excluded here, the result is nearly identical to `summary()`) commands both provide the table in R output:

```
summary(fit.log.drug)

##              Df Sum Sq Mean Sq F value Pr(>F)
## Drug          2  0.536   0.2679    8.22 0.0016 **
## Residuals    27  0.880   0.0326
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

An alternative approach that is useful when making reports is to use the `tidy()` function in the `broom` package (?) (installed along with the `tidyverse`). It creates a `data.frame` with the ANOVA table, then the `kable()` outputs an aesthetically pleasing table to a report:

```
library(broom)
drug.anova.df <- tidy(fit.log.drug)
kable(drug.anova.df, digits=3) |>
  kable_styling(full_width=FALSE)
```

2.7.1 Reading an ANOVA table

The ANOVA table includes quite a bit of information. The mathematical derivation and details on these components are not important for our purposes, but we do step through all the components of the ANOVA table:

- Degrees of freedom (df). Here, we see two different degrees of freedom are

reported.

- The df corresponding to the term **Drug** is the degrees of freedom associated with the different treatments. When there are k treatments, there will be $k - 1$ degrees of freedom.
- The df corresponding to **Residuals** are the degrees of freedom associated with the within treatment squared deviations. In the case of a One-way ANOVA, this will take on the value of $n - k$ where there are n total observations on k treatments.
- Sum of squared deviations (Sum Sq). These are the *SSM* and *SSE* terms from equation (??).
 - Here, 0.5357 is the *SSM* and 0.8800 is the *SSE*, we could calculate the *SST* via $SST = 0.5357 + 0.8800 = 1.4157$.
- Mean squared deviation (Mean Sq) - These are known as the mean squared deviations, and are calculated by dividing the sum of squared deviations by their corresponding degrees of freedom
 - Here, $0.5357/2 = 0.2679$ for the Mean Squared deviations of the model (sometimes labeled *MSM*) and $0.8800/27 = 0.0326$ for the mean squared error (often labeled *MSE*).
- The *F*-statistic is reported.
 - It is calculated as MSM/MSE , so $0.2679/0.0326 = 8.219$
- And the corresponding *p*-value; *i.e.*, the probability of observing a larger *F*-statistic by pure chance alone when the null hypothesis is true.
 - Here, the *p*-value is 0.00163.

Typically, when reporting the results of ANOVA, we report the *F*-statistic, the two degrees of freedom, and the corresponding *p*-value. Collectively, these four measures provide a reader with information to deduce the sample size, number of treatments and *effect* of the experiment.

In our working example, we would report using language similar to the following:
 > With an *F*-stat of 8.219 on 2 and 27 degrees of freedom (*p*-value = 0.00163), we can conclude that at least two of the drugs produce significantly different mean log-immobilization times.

2.7.2 Conclusions and Limitation of ANOVA

The results of a simple One-Way ANOVA are somewhat limited. The null hypothesis suggest the mean response is the same across all treatments, while the alternative tells us there *some* difference between mean responses. Unfortunately it provide no details on which treatments differ, or the magnitude of the differences.

Specifically, in regards to the ANOVA testing the logarithm of knockdown times, we found there is a difference among the three drug treatments; essentially ANOVA tells us that the null hypothesis $H_0 : \mu_A = \mu_B = \mu_C$ is rejected in favor of an alternative. Of course, what remains to be seen is which pair, or pairs, of drugs produce different log-mean immobilization times and the magnitude of

those differences. For that, we will need to look at techniques known as multiple comparison procedures.

2.8 Follow-up procedures – Multiple Comparisons

When there is no significant difference detected among treatment means, we can safely conclude that the treatment had no effect on the mean response. This effectively says that any differences in means were no larger than the type of change we might expect due to random chance. In such a case, the analysis is essentially over, except that we might want to assess the size of the observed treatment differences for descriptive purposes.

However, if significant differences are detected among the treatments (as in the knockdown times example above), the ANOVA F -test does not inform you where the differences lie. In such a situation, a follow-up procedure known as a multiple comparison procedure is necessary to seek out pairwise mean differences.

There are numerous multiple comparison methods in existence, and they all have certain advantages and disadvantages. Some are designed for specific follow-up comparisons and can be customizable (known as *contrasts*). Here we will discuss procedures for handling the following scenarios:

1. All possible pairwise comparisons.
2. All comparisons versus a control group.

So for example, if we had a one-way design with five treatment groups A , B , C , D and E , there would be 10 possible pairwise comparisons:

$$\begin{array}{llll} A \text{ vs. } B & B \text{ vs. } C & C \text{ vs. } D & D \text{ vs. } E \\ A \text{ vs. } C & B \text{ vs. } D & C \text{ vs. } E & \\ A \text{ vs. } D & B \text{ vs. } E & & \\ A \text{ vs. } E & & & \end{array}$$

whereas if (say) A was a control group and we were only interested in comparisons versus a control group, then there would be only 4 pairwise comparisons of interest:

$$\begin{array}{l} A \text{ vs. } B \\ A \text{ vs. } C \\ A \text{ vs. } D \\ A \text{ vs. } E \end{array}$$

Now, you may say to yourself, “So what is the big deal? I’ll just do 10 two-sample t -test!”

Well, if you find yourself in a multiple comparison situation, it is because you have declared that at least two groups differ based on the result of a single hypothesis test (the F -test). For that test, you used a specific criterion to make the decision (probably p -value < 0.05). You performed the analysis with an accepted false positive rate, known as the Type I error rate, or α (again, likely $\alpha = 0.05$).

But to tease out where the differences are, you now need to run multiple tests under the same “umbrella” or model. That gives rise to something known as the multiple testing problem. The problem is an increase in the false positive rate (the Type I error rate) that occurs when statistical tests are used repeatedly.

2.8.1 Multiple testing problem

If m independent comparisons are made, each using a Type I error rate of α , then the family-wise (overall) Type I error rate (FWER) for all comparisons considered simultaneously is given by

$$\alpha_{FWER} = 1 - (1 - \alpha)^m.$$

Illustration: Coin flips.

Suppose that we declare that a coin is biased if in 10 flips it landed heads at least 9 times. Indeed, if one assumes as a null hypothesis that the coin is fair, then it can be shown that the probability that a fair coin would come up heads at least 9 out of 10 times is 0.0107. This is relatively unlikely, and under usual statistical criteria such as “reject H_0 if p -value < 0.05 ”, one would declare that the null hypothesis should be rejected given this probability — *i.e.*, declare the coin is unfair.

A multiple testing problem arises if someone wanted to use this criteria (which is appropriate for testing the fairness of a *single* coin) to test the fairness of many coins. So, imagine you were to test 100 different coins using this method. For any particular coin (*i.e.*, some pre-selected coin), we would expect the likelihood of it flipping heads 9 or 10 times to be relatively small (in fact, it would be with probability 0.0107). However, seeing *some* coin (as in *any* of the 100 coins) behave that way, without concern for which coin, is more likely. In fact, the probability that all 100 fair coins are identified as fair by this criterion is $(1 - 0.0107)^{100} \approx 0.34$. This suggests, even if all 100 coins were in fact fair, the probably *some* coin is incorrectly flagged as unfair is approximately 66%. Therefore the application of a “single coin fairness criterion” to multiple comparisons would more likely than not **falsely identify** at least one fair coin as unfair!

The above illustration may seem contrived, but multiple testing is a reality and a natural follow-up procedure to ANOVA. If you conducted two hypothesis tests, each at $\alpha = 0.05$, the family-wise error rate is 0.0975. For three tests (*e.g.*, A vs B, A vs C and B vs C), the overall Type I error rate is 0.142625.

To address the problem, many multiple comparison procedures have been developed that help control the accumulation of the false positive rate. Some methods perform better than others, but common to all of them is making adjustments to p -values or CIs based upon how many comparisons are being made.

One common approach is known as the *Bonferroni* method, which simply adjusts the significance level (α) based on the number of test or confidence intervals being calculated. So for example, if you were making 6 comparisons and you wished to have a family-wide error rate of $\alpha_{FWER} = 0.05$, perform each of the comparisons at $\alpha = 0.05/6 = 0.00833$. This method is known as being conservative since it guarantees a family-wide error rate less than nominal level (note, $1 - (1 - 0.00833)^6 = 0.04897 < 0.05$).

Multiple comparisons in an ANOVA framework can be performed using the add-on R package `emmeans` (?). The package `emmeans` is quite powerful. We only demonstrate a few pieces of its functionality here. For those interested in a more in depth look, we suggest checking out the `emmeans` website. We will focus on two useful multicomparison methods: Tukey’s HSD and Dunnett’s.

2.8.2 Tukey’s HSD method

One of the more widely used methods is attributed to statistician John Tukey (creator of the Box-Whiskers plot and credited with creating the word “bit”). It is referred to as Tukey’s Honestly Significant Difference method (honestly!). Essentially, it performs all pairwise two-sample t -test on all the treatments but has the advantage of controlling the Type I error rate. It *adjust* the p -values of each two-sample t -test to account for the fact you are performing more than one test. This is similar to the Bonferroni method, but Tukey’s HSD is known to perform better.

First we load the `emmeans` package and then run our fitted model `fit.log.drug` through the `emmeans()` function. We provide it a formula to inform how we wish to perform the comparisons. In this case the formula is simple since we only have a single factor input `~ Drug`. Lastly, to do a “pairwise” comparison we use the `contrast()` function and tell it the type of comparison to implement.

```
library(emmeans)
drug.mc <- emmeans(fit.log.drug, ~ Drug)
contrast(drug.mc, "pairwise")
```

```
## contrast estimate      SE df t.ratio p.value
## A - B          0.0328 0.0807 27   0.406  0.9136
## A - C          0.2984 0.0807 27   3.696  0.0027
## B - C          0.2657 0.0807 27   3.291  0.0076
##
## Results are given on the log10 (not the response) scale.
## P value adjustment: tukey method for comparing a family of 3 estimates
```

The significant differences are what we expected based on our earlier EDA: drug C is significantly different in terms of log-mean immobilization time as compared to the other two drugs. This is determined from the Tukey-adjusted p -values of 0.0027 (when comparing drugs A and C) and 0.0076 (when comparing drugs B and C). B and A are not significantly different (p -value of 0.9136). The p -values in this output have been adjusted for performing 3 comparisons, so we can just compare them to the usual $\alpha = 0.05$ or whatever Type I error rate you use.

We can also obtain simultaneous 95% confidence intervals for the true mean differences due to each drug pairing using the `confint()` function on our `contrast` of an `emmeans` object:

```
confint(contrast(drug.mc, "pairwise"))
```

```
## contrast estimate      SE df lower.CL upper.CL
## A - B          0.0328 0.0807 27  -0.1674   0.233
## A - C          0.2984 0.0807 27   0.0983   0.499
## B - C          0.2657 0.0807 27   0.0655   0.466
##
## Results are given on the log10 (not the response) scale.
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
```

These may be interpreted as follows:

95% CI for $\mu_A - \mu_B = (-0.167, 0.233)$. This interval contains 0, so we cannot conclude that the true log-mean immobilization time significantly differ between drugs A and B.

95% CI for $\mu_A - \mu_C = (0.0983, 0.499)$. We can be 95% confident that drug A produces a true log-mean immobilization time that is between 0.0983 to 0.499 higher than drug C.

95% CI for $\mu_B - \mu_C = (0.066, 0.466)$. We can be 95% confident that drug B produces a true log-mean immobilization time that is between 0.066 and 0.466 higher than drug C.

We can also visualize these results via a plot:

```
plot(contrast(drug.mc, "pairwise"))
```

Here, we are interesting in comparing each plotted confident band to the value of 0. If zero is included in the interval, there is no statistical difference between those two treatments. If the interval does not include zero, we can conclude there is a statistical difference.

2.8.2.1 “Un”-transforming the response

You should note in some of the above output that `emmeans` provides a message that the “Results are given on the log10 (not the response) scale.” To put back

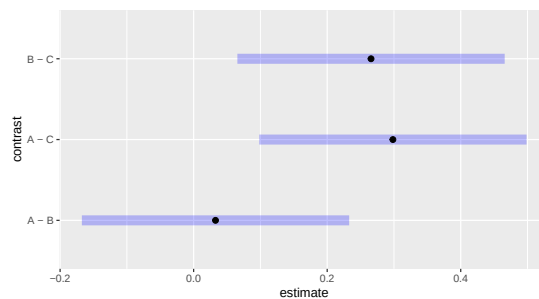


Figure 2.6: Tukey adjusted confidence interval plots comparing treatments ‘A’, ‘B’ and ‘C’ on a log-scale. Here, we see that treatments ‘B’ and ‘C’ are significantly different as well as treatments ‘A’ and ‘C’. We also note there is no significant difference between treatments ‘A’ and ‘B’ since the interval includes zero.

in the original units we could “un”-log the intervals by taking base-10 exponents (that is, raise 10 to the power of the value). But we need to take a moment to consider how to interpret the output.

To explain, suppose we are interested in comparing groups A vs B. Without a transformation, the Tukey HSD method effectively calculates the difference $\mu_A - \mu_B$ and this would lend itself to the two-sample t -test. But when the response has been transformed with a logarithm, the two mean components are on a logarithmic scale and we would essentially be calculating the difference $\log_{10}(\mu_A) - \log_{10}(\mu_B)$, which by properties of logarithms is equivalent to $\log_{10}(\mu_A/\mu_B)$, or the logarithm of the ratio of the two mean values! Thus if we “un”-log that difference, we are left with $10^{\log_{10}(\mu_A/\mu_B)} = \mu_A/\mu_B$ and we are comparing the ratio of the mean of group A to mean of group B. If the mean of the two groups were equal, we’d expect this ratio to take on a value near one. If the means differ, the ratio would be different than one.

The `emmeans` provides the functionality to put the response in the original units when a logarithmic transformation has been used. We specify the `type="response"` parameter in the `contrast()` function call as follows:

```
contrast(drug.mc, "pairwise", type="response")
```

```
## contrast ratio      SE df null t.ratio p.value
## A / B      1.08 0.200 27    1   0.406  0.9136
## A / C      1.99 0.370 27    1   3.696  0.0027
## B / C      1.84 0.343 27    1   3.291  0.0076
##
## P value adjustment: tukey method for comparing a family of 3 estimates
## Tests are performed on the log10 scale
```

You’ll note that the null value is 1, and the results are consistent with the

previous results – a difference exist between group C and groups A and B. We can also consider confidence intervals

```
confint(contrast(drug.mc, "pairwise", type="response"))
```

```
## contrast ratio SE df lower.CL upper.CL
## A / B 1.08 0.200 27 0.68 1.71
## A / C 1.99 0.370 27 1.25 3.15
## B / C 1.84 0.343 27 1.16 2.92
##
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
## Intervals are back-transformed from the log10 scale
```

and these intervals can also be visualized

```
plot(contrast(drug.mc, "pairwise", type="response"))
```

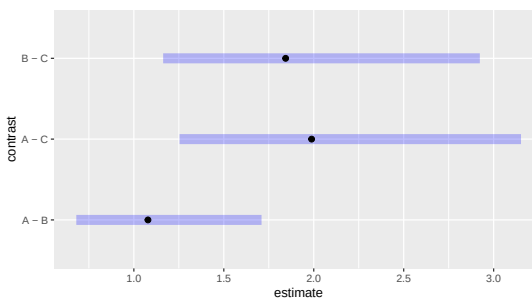


Figure 2.7: Tukey adjusted confidence interval plots comparing the ratio of treatments ‘A’, ‘B’ and ‘C’ since the original model was fit on a log-scale. Here, we see that treatments ‘B’ and ‘C’ are significantly different as well as treatments ‘A’ and ‘C’. We also note there is no significant difference between treatments ‘A’ and ‘B’ as the interval includes unity.

2.8.3 Dunnett multiple comparisons

The Dunnett method is applicable when one of your experimental groups serves as a natural *control* to which you want to compare the remaining groups. **Note** that there is no *control* group in our working example of immobilization times, but we include the code below for demonstration purposes where treatment level “A” is the reference (**ref**) control group.

In situations where a *control* group is present, it is advisable to use a method that only controls the family-wise error rate for the comparisons of interest, rather than over-controlling for all possible comparison such as when using a method like Tukey HSD. We still use the **emmeans** package but specify the comparisons are against a *control* group. The code is below:

```
contrast(drug.mc, "trt.vs.ctrl", ref="A")
```

```
## contrast estimate      SE df t.ratio p.value
## B - A      -0.0328 0.0807 27  -0.406  0.8750
## C - A      -0.2984 0.0807 27  -3.696  0.0019
##
## Results are given on the log10 (not the response) scale.
## P value adjustment: dunnett method for 2 tests
```

The two comparisons provided are A vs. B and A vs. C. The only significant difference is between A and C (p -value = 0.0019). As before, here are simultaneous CIs and plots:

```
confint(contrast(drug.mc, "trt.vs.ctrl"))
```

```
## contrast estimate      SE df lower.CL upper.CL
## B - A      -0.0328 0.0807 27  -0.222   0.157
## C - A      -0.2984 0.0807 27  -0.488  -0.109
##
## Results are given on the log10 (not the response) scale.
## Confidence level used: 0.95
## Conf-level adjustment: dunnett method for 2 estimates
```

```
plot(contrast(drug.mc, "trt.vs.ctrl"))
```

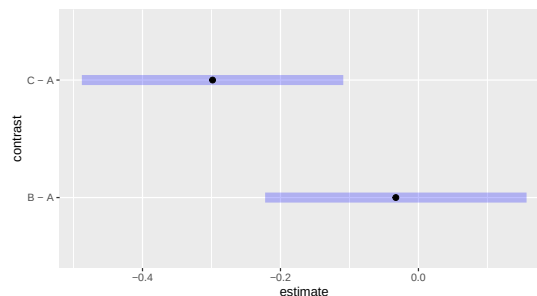


Figure 2.8: Dunnett adjusted confidence interval plots demonstrating that treatment ‘C’ is significantly different than control group ‘A’, while treatment ‘B’ is not significantly different than control group ‘A’.

It is interesting to compare the results you get via the two methods (Tukey vs Dunnett) for the same comparisons (*e.g.*, A vs. C). Can you explain what you see?

Note. If you do not specify a `ref=` in the `contrast()` function, group A will automatically be chosen as the reference group for Dunnett (the default behavior is to choose the first factor level [in this case the first alphabetically]). We recommend you always explicitly tell the code the reference groups. In the

below example we set the second (2) factor, B, to be the control with `ref="B"`.

```
contrast(drug.mc, "trt.vs.ctrl1", ref="B")
```

```
## contrast estimate      SE df t.ratio p.value
## A - B          0.0328 0.0807 27   0.406  0.8750
## C - B         -0.2657 0.0807 27  -3.291  0.0054
##
## Results are given on the log10 (not the response) scale.
## P value adjustment: dunnett method for 2 tests
```

Alternatively we can pass in a numeric reference (based on factor order)

```
contrast(drug.mc, "trt.vs.ctrl1", ref=2)
```

2.8.4 Coding notes about `emmeans`

As we mentioned above, the `emmeans` package [R-emmeans] includes more functionality than covered in this text. We do highlight two additional features from the `emmeans` package that may be useful.

For one, you can build *adjusted* confidence intervals for each of the three treatments. That is, rather than building a standard 95% confidence interval for each treatment (see Section @ref(#twoSampleInference)), the interval length can be adjusted since multiple comparisons are being made.

Below we build *Bonferroni adjusted* 95% confidence intervals for mean immobilization times of each treatment, after backtransforming from the logarithm.

```
confint(drug.mc, adjust="bonferroni", type="response")
```

```
## Drug response      SE df lower.CL upper.CL
## A          9.17 1.210 27     6.56    12.82
## B          8.50 1.120 27     6.08    11.89
## C          4.61 0.606 27     3.30     6.45
##
## Confidence level used: 0.95
## Conf-level adjustment: bonferroni method for 3 estimates
## Intervals are back-transformed from the log10 scale
```

We can also provide a plot of these intervals as seen in Figure ??.

```
plot(drug.mc, adjust="bonferroni", type="response")
```

Another feature that can be useful with `emmeans` functionality is the `tidy()` function from the `broom` package. However, we do recommend you use this function with some caution as it does not always work well and you can lose information. But, when it does work, the output is put into a data frame and can be processed with the `kable()` function for aesthetically pleasing tables. An

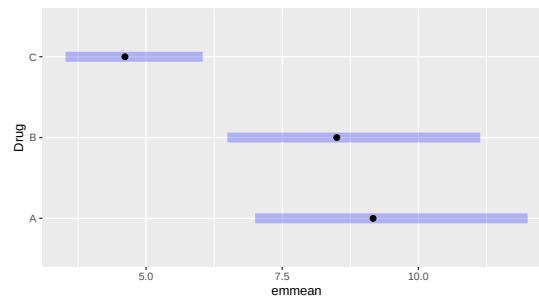


Figure 2.9: Bonferroni adjusted confidence intervals for the immobilization time of the three drug treatments.

Drug	response	std.error	conf.low	conf.high
A	9.169	1.205	6.556	12.825
B	8.503	1.118	6.079	11.893
C	4.612	0.606	3.297	6.451

example is provided below that uses the `tidy()` function, along with `select()` from `dplyr`, with `kable()` and `kable_styling()`.

```
tidy(drug.mc, conf.int=TRUE, type="response", adjust="bonferroni") |>
  select(Drug, response, std.error, conf.low, conf.high) |>
  kable(digits=3) |>
  kable_styling(full_width=FALSE)
```

2.9 Two-sample t -test as ANOVA

One-Way ANOVA was described as an extension of the two-sample t -test. It is worth noting that when you only have two treatments ($k = 2$), One-Way ANOVA and the two-sample t -test are equivalent. Consider the University Admissions dataset once again. For completeness, we include the code that inputs the data and filters the two years of study, 1996 and 2000.

```
uadata <- read_table("https://tjfisher19.github.io/introStatModeling/data/univadmissions.txt")
uadata.trim <- uadata |>
  filter(year %in% c(1996, 2000) )
```

We start with a plot to compare ACT scores across the two years of study.

```
ggplot(uadata.trim, aes(x=act, y=year)) +
  geom_boxplot() +
  stat_summary(geom="point", shape=23, fun="mean", size=3, fill="gray60") +
  labs(x="ACT Scores", y="Enrollment Year",
```

```
title="ACT Scores as a function of Enrollment Year") +
theme_minimal()
```

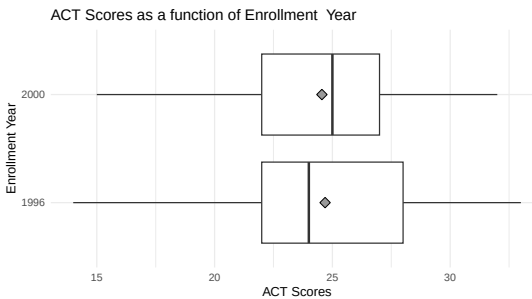


Figure 2.10: Box-whiskers plots, with overlaid means (gray diamond), showing the distribution of ACT scores for each of two university enrollment years, 1996 and 2000. Overall we see little difference in ACT scores between the two years.

In Figure ??, we see little-to-no difference in the mean ACT scores between 1996 and 2000. We can statistically test this via a One-Way ANOVA

```
uadata.anova <- aov(act ~ year, data=uadata.trim)
```

Although we didn't perform a residual analysis for the two-sample t -test, it makes the same assumptions as ANOVA – the underlying random part of the data should be independent, have constant variance and be normally distributed. We assess these assumptions via the residual plots described above.

```
autoplot(uadata.anova)
```

We get independence among the observations based on the data collection procedure. Here, it seems reasonable that each student will be independent of one another. We see no concerns regarding constant variance based on the Residuals vs Fitted plot and Scale-Location plot – the vertical spread appears to be about the same across the two groups. The Normal Q-Q plot also looks satisfactory with most points near that 45-degree line.

We proceed with a formal inference.

```
summary(uadata.anova)
```

```
##           Df Sum Sq Mean Sq F value Pr(>F)
## year      1      1.28    1.28    0.09  0.76
## Residuals 277    3900   14.08
```

With an F -stat of 0.091 on 1 and 277 degrees of freedom (p -value=0.763), we do not have evidence to suggest that the mean ACT scores differed in 1996 than 2000.

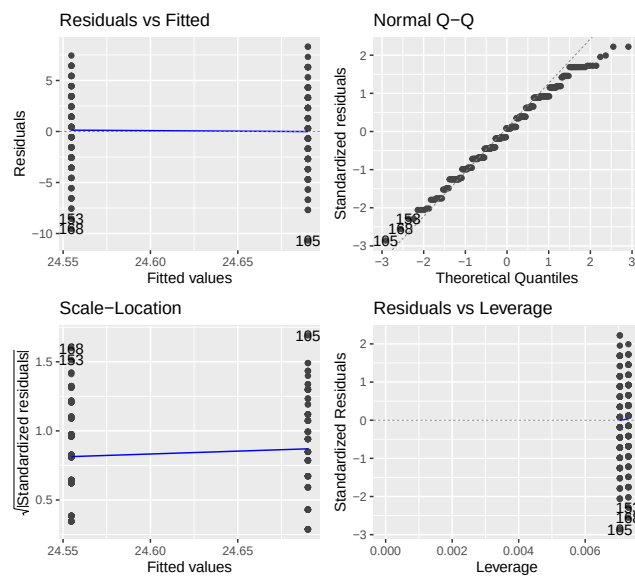


Figure 2.11: Residual plots for the One-Way ANOVA studying the effect of enrollment year on ACT scores - Top-left a Residuals vs Fitted Plot; Top-right a Normal Q-Q plot of the residuals; Bottom-left the Scale-Location plot; and Bottom-right a Residuals vs Leverage plot.

Note, this is the same conclusion as with the two-sample t -test. In fact, we get the exact same p -value using both methods:

```
t.test(act ~ year, data=uadata.trim, var.equal=TRUE)
```

```
##
## Two Sample t-test
##
## data: act by year
## t = 0.3013, df = 277, p-value = 0.763
## alternative hypothesis: true difference in means between group 1996 and group 2000
## 95 percent confidence interval:
## -0.749213 1.020005
## sample estimates:
## mean in group 1996 mean in group 2000
## 24.6901 24.5547
```

Further, note that $t_0^2 = 0.301^2 = 0.091 = F$ and $\sqrt{F} = \sqrt{0.091} = 0.301 = t_0$ after rounding! The two-sample t -test and One-way ANOVA with $k = 2$ are equivalent tests.

Follow-up multiple comparisons are not necessary here, for two reasons:

1. We found that the different treatments do not significantly influence the response variable. So any multiple comparisons would simply repeat that same finding (no difference in treatments).
2. We only have two treatments in this example. If we were to perform a follow-up procedure it would be the same two-sample t -test as above, and that is already equivalent to the One-way ANOVA analysis.

2.10 A Complete One-Way ANOVA Example

We conclude this chapter by looking at another example of a One-Way ANOVA. Here, the analysis has been somewhat streamlined as we are not introducing new techniques or modeling. We do take some time to incorporate some additional code.

Example: Growth Behavior of Spruce Tress (from the text ?).

A study was conducted looking at the bud strength of spruce seedlings under different conditions. Seeds that were deemed *healthy* and *diseased* were exposed to a solution of two different pH levels for two days, an acidic solution (pH of 5.5) and a neutral solution (pH of 7). Buds from the seedlings were then analyzed and a Bud Breaking Rating was recorded for each seedling (where a higher number indicates better *budding*). In the experiment, healthy seedlings exposed to a neutral solutions were considered a control compared to the other treatments. The control group is recorded with a 0 (zero), diseased seeds with

the neutral buffer are allocated to group 1, healthy seeds with the acidic buffer are group 2, and diseased seeds with the acid buffer are group 3.

The data are in the file `spruceTrees1way.csv` in the textbook repository.

Data Source: ?

We begin by inputting the data and looking at its structure.

```
site <- "https://tjfisher19.github.io/introStatModeling/data/spruceTrees1way.csv"
spruce_trees <- read_csv(site, show_col_types=FALSE)
glimpse(spruce_trees)

## Rows: 20
## Columns: 2
## $ BudBreaking <dbl> 0.8, 0.6, 0.8, 1.0, 0.8, 1.0, 1.0, 1.2, 1.4, 1.2, 1.0, 1.2~
## $ Treatment <dbl> 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0
```

Note that the different treatments have been recorded as a number. As discussed in the previous chapter, it is important for variables to be of the correct type for many functions in R. Specifically, if the `aoV()` function sees a numeric predictor variable, the analysis will be different than if categorical. So the first step will be to convert the numeric `Treatment` variable into a factor variable.

```
spruce_trees <- spruce_trees |>
  mutate(Treatment = factor(Treatment))
glimpse(spruce_trees)
```

```
## Rows: 20
## Columns: 2
## $ BudBreaking <dbl> 0.8, 0.6, 0.8, 1.0, 0.8, 1.0, 1.0, 1.2, 1.4, 1.2, 1.0, 1.2~
## $ Treatment <fct> 3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0
```

Now that the variable types are correct, we proceed with a short exploratory data analysis. Note that there are only 5 observations in each treatment, thus Box-Whiskers plots do not make much sense here (why summarize 5 observations with 5 ‘summary statistics’?). We build a short table and plot of the data with a mean added to the plot. To provide more meaning to our results, we create some copies of our factor variable and add contextual labels.

```
spruce_trees <- spruce_trees |>
  mutate(Trt_label = factor(Treatment,
    levels=c(0, 1, 2, 3),
    labels=c("Control",
      "Neutral & Diseased Seed",
      "Acid & Healthy Seed",
      "Acid & Diseased Seed") ),
    Trt_label_plot = factor(Treatment,
      levels=c(0, 1, 2, 3),
      labels=c("Control\n(Neutral & Healthy Seed)",
```

Treatment	Mean	SD
Control	1.28	0.110
Neutral & Diseased Seed	1.16	0.167
Acid & Healthy Seed	1.24	0.167
Acid & Diseased Seed	0.80	0.141

```
"Neutral & Diseased Seed",
"Acid & Healthy Seed",
"Acid & Diseased Seed") ) )
```

Using some of the tools from before, we look at a table of summary statistics by treatment

```
spruce_trees |>
  group_by(Trt_label) |>
  summarize(Mean = mean(BudBreaking),
            SD = sd(BudBreaking) ) |>
  kable(col.names = c("Treatment", "Mean", "SD"),
        digits=3) |>
  kable_styling(full_width=FALSE)
```

and a plot of our data in Figure ??.

```
ggplot(spruce_trees, aes(x=BudBreaking, y=Trt_label_plot)) +
  geom_jitter(height=0) +
  stat_summary(geom="point", shape=23, fun="mean", size=3, fill="gray60") +
  theme_minimal() +
  labs(y="", # no label is needed here, its implied
       x="Bud Breaking Rating")
```

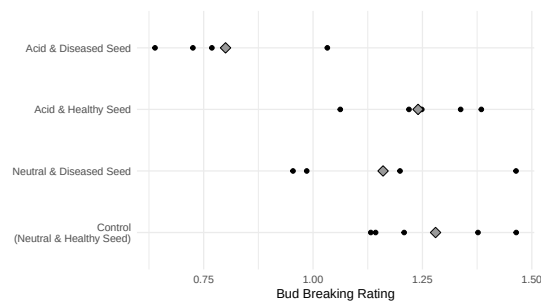


Figure 2.12: Jittered plot of the data, with overlaid treatment means (gray diamond), showing the distribution of Bud Breaking Ratings as a function of acid treatments on seedling health. Overall we see that diseased seeds treated in an acid solution appear to have lower Bud Breaking Ratings

Based on the summary statistic values and plot in Figure ??, it appears the Bud Breaking Rating for diseased seeds in the acid solution result in a lower Bud Breaking Rating. It is unclear if other two treatments (Neutral & Diseased seeds, and Acid with healthy seeds) differ from the control.

We fit our ANOVA model

```
spruce.anova <- aov(BudBreaking ~ Trt_label, data=spruce_trees)
```

and before conducting any inference we consider the assumptions by analyzing Figure ??.

```
autoplot(spruce.anova) +  
  theme_minimal()
```

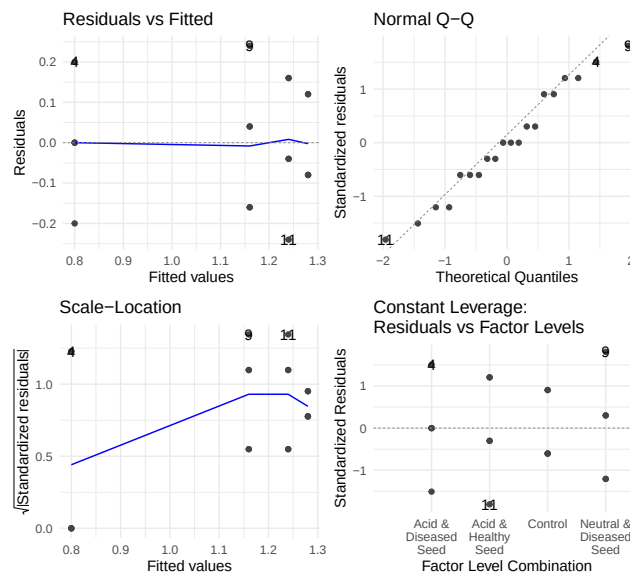


Figure 2.13: Residual plots for the One-Way ANOVA studying the four treatments on the Bud Breaking Rating of Spruce seedlings - Top-left a Residuals vs Fitted Plot; Top-right a Normal Q-Q plot of the residuals; Bottom-left the Scale-Location plot; and Bottom-right a Residuals vs Leverage plot.

- **Independence** can be assumed from the design of the experiment. Each Spruce tree seed should be reasonably independent.
- **Constant Variance** – The Residuals vs Fitted plot shows no systematic changing of the variance (the vertical spread of the points looks fairly uniform or random). The Scale-Location shows some evidence of an increase in variability, but coupled with the Residuals vs Fitted plot, there is nothing overly concerning.

- **Normality** – the Normal Q-Q plot shows the observed quantiles from the residuals largely follows that of the theoretical values. Thus Normality looks okay.

Overall, the model assumptions appear satisfactory, so we proceed with a formal inference.

```
summary(spruce.anova)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## Trt_label      3  0.720    0.240     10.9 0.00038 ***
## Residuals    16  0.352    0.022
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

With an F -stat of 10.9 on 3 and 16 degrees of freedom (p -value=0.00038), we have evidence to suggest that the mean Bud Breaking Rating of spruce seedlings varies by treatment.

As a follow-up, we conduct a multiple comparison analysis. We have a “control” in this experiment, thus use the Dunnett procedure.

```
spruce.mc <- emmeans(spruce.anova, ~Trt_label)
contrast(spruce.mc, "trt.vs.ctrl", ref=1)
```

```
## contrast              estimate      SE df t.ratio p.value
## Neutral & Diseased Seed - Control    -0.12 0.0938 16  -1.279  0.4589
## Acid & Healthy Seed - Control         -0.04 0.0938 16  -0.426  0.9254
## Acid & Diseased Seed - Control        -0.48 0.0938 16  -5.117  0.0003
##
## P value adjustment: dunnett method for 3 tests
```

Not too surprising given the results of Figure ??, the “Neutral & Diseased Seeds” and “Acid & Healthy Seed” Bud Breaking Ratings do not statistically differ from the control (p -values of 0.4589 and 0.9254, respectively). However, we have evidence that the diseased seeds that underwent an acid solution before planting do result in a different Bud Breaking Rating (p -value of 0.0003). We can also look at confidence intervals to get a sense of the effect size.

```
confint(contrast(spruce.mc, "trt.vs.ctrl", ref=1))
```

```
## contrast              estimate      SE df lower.CL upper.CL
## Neutral & Diseased Seed - Control    -0.12 0.0938 16  -0.365    0.125
## Acid & Healthy Seed - Control         -0.04 0.0938 16  -0.285    0.205
## Acid & Diseased Seed - Control        -0.48 0.0938 16  -0.725   -0.235
##
## Confidence level used: 0.95
## Conf-level adjustment: dunnett method for 3 estimates
```

and the confidence intervals can be plotted as well.

```
plot(contrast(spruce.mc, "trt.vs.ctrl", ref=1) ) +  
  theme_minimal()
```

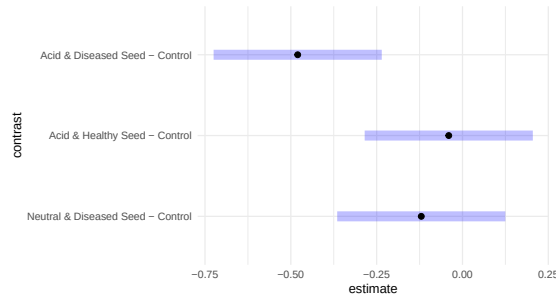


Figure 2.14: Dunnett adjusted confidence interval plots comparing the three treatments to the control group. Here, we see that the Diseased seeds with acid solution group is significantly different than the control, while the other two groups are not significantly different.

From Figure ?? and the report confidence intervals, the acid & diseased seeds result in reduction of approximately 0.235 to 0.725 on the Bud Breaking Rating. Also note that the other two intervals include zero, which is consistent with not being significantly different.

Overall Conclusions

Based on the analysis, we can conclude that a compared to a Control group of Healthy Seeds that underwent a Neutral Solution before planting, that Diseased Seeds that underwent an Acid solution before planting will result in a Bud Breaking Strength of 0.235 to 0.725 units less than that of a control group.

Chapter 3

Multiple Factor Designed Experiments

In the previous chapters we have seen how one variable (perhaps a factor with multiple treatments) can influence a response variable. It should not be much of a stretch to consider the case of multiple predictor variables influencing the response.

Example. Consider the admissions dataset with ACT scores as a response, besides the year of entry maybe a student's home neighborhood or state could also be a predictor?

In this chapter we will explore two key analyses for designed experiments – a Block design and Two-factor experiment. The learning objectives of this unit include:

- Understanding the structure of a multi-factor model.
- Understanding the rationale of a block design.
- Understanding the rationale and structure of a two-factor design.
- Performing a full analysis involving multiple predictor variables.

3.1 Blocking

Randomized Block Designs are a bit of an odd duck. The type of design itself is straightforward, but the analysis might seem a bit unusual.

The design is best described in terms of the agricultural field trials that gave birth to it. When conducting field trials to compare plant varieties, there is concern that some areas of a field may be more fertile than others (*e.g.*, soil nutrients, access to water, shade versus sun, etc...). For example, if one were comparing three plant varieties (call them A, B and C), it would not be a good

idea to use one variety in one location (say next to a creek), another variety in a different location, and the third variety way back in another part of the field near a forest because the effects of the varieties would be confounded with the natural fertility of the land. See Figure ?? as an example of a poorly designed experiment.

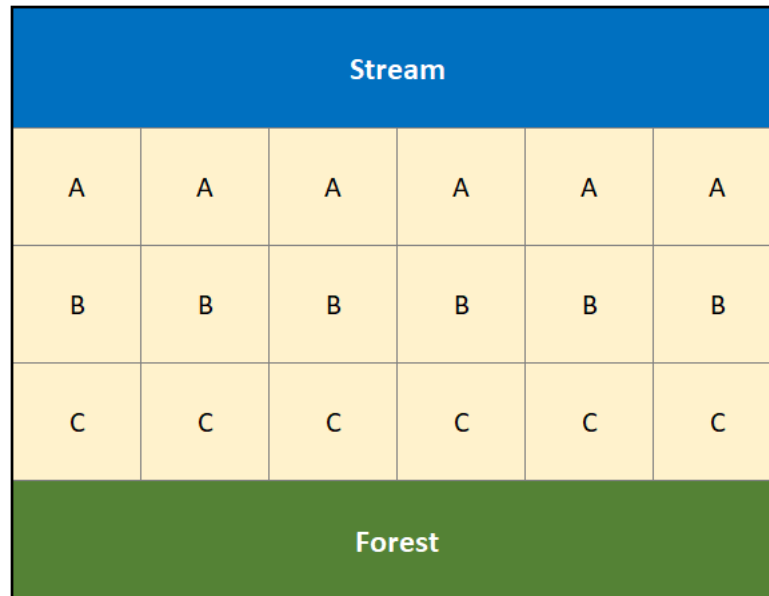


Figure 3.1: Diagram of a poorly designed experiment that is ignoring the potential confounding factors of a creek and forest.

The natural fertility (next to the creek, compared to next to the forest, compared to the middle) in this situation is a **confounding factor**. A confounding factor is something we are not necessarily interested in, but that none the less will have an impact on our assessment of the factor of interest (*e.g.*, plant variety).

We could attempt to mitigate the effect of this confounding factor through randomization. For example, suppose by random allocation the treatments are distributed as such as in Figure ??.

In Figure ?? the treatments have been randomly assigned to the different locations in an effort to mitigate the effects of the stream and forest on land fertility. However, by random chance we note that treatment “C” has 4 of its 6 replicates in proximity to the stream, with the remaining 2 next to the forest. A block design looks to address this limitation of randomization when there are suspected (known) confounding effects.

Stream					
A	C	C	B	C	C
A	B	B	A	B	B
C	C	B	A	A	A
Forest					

Figure 3.2: Randomized Design that attempts to account for the confounding factors of a creek and forest.

A **block** is defined to be “a relatively homogeneous set of experimental material.” This essentially means that we would like to be able to assess the treatment effects without undue influence from a contaminating or confounding factor, such as land fertility. One way to do this would be to apply all the different treatments (*e.g.*, the three plant varieties) to the same area of the field. Within one area (say next the creek in Figures ?? and ??), we should see consistency in natural fertility levels. Thus any differences observed in a measured response (such as crop yield) between plant varieties will not be due to variation in fertility levels, but rather due to the different plant varieties themselves.

A block design is one that attempts to reduce the noise in the measured response in order to “clarify” the effect due to the treatments under study.

A **randomized block design** in the field study would be conducted in the following way:

- The field is divided into blocks.
 - In our working example we use 3 blocks for the land next to the creek, next to the forest and that in the middle.
- Each block is divided into a number of units equal to, or more than, the number of treatments.
 - In the example we have 6 units in each block.
- Within each block, the treatments are assigned at random so that a different treatment is applied to each unit. When all treatments are assigned at least once in each block, it is known as a “Complete” Block Design. **The defining feature of the Randomized (Complete) Block Design is that each block sees each treatment at least once.**
 - We have randomly allocated 2 replicates of each treatment in each block for a complete and balanced design.

Figure ?? displays a randomized complete block design for the same experiment involving three plant varieties, “A”, “B” and “C”.

A good experimental design will be able to “parse out” the variability due to potential confounding factors, and thus give clarity to our assessment of the factor of interest. A block design can accomplish this task.

What can serve as a block? It is important to note that subjects (*i.e.*, experimental units) themselves may form blocks in a block design. It is also possible that different experimental units may collectively form a “block”. It depends on the circumstances. All that is needed is that a block, however, formed, creates a “relatively homogeneous set of experimental material.”

An example. You’ve already seen the concept of a block design! Consider the paired t -test included in Section ?. There, the individual vehicles are essentially treated as blocks. We observe two responses within each block (or vehicle).

Stream					
A	C	B	B	A	C
C	B	C	A	B	A
B	C	C	B	A	A
Forest					

Figure 3.3: Diagram of a Randomized Complete Block Design that accounts for the confounding factors of a creek and forest.

3.1.1 Data structure, model form and analysis of variance of a Randomized Block Design

Here is the data structure for a randomized block design (RBD) with one treatment replicate per block:

	Treatment 1	Treatment 2	...	Treatment k
Block 1	Y_{11}	Y_{21}	...	Y_{k1}
Block 2	Y_{12}	Y_{22}	...	Y_{k2}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
Block b	Y_{1b}	Y_{2b}	...	Y_{kb}

The model for such data has the form

$$Y_{ij} = \mu + \tau_i + \beta_j + \varepsilon_{ij}$$

where

- Y_{ij} is the observation for treatment i within block j .
- μ is the overall mean.
- τ_i is the effect of the i^{th} treatment on the mean response.
- β_j is the effect of the j^{th} block on the mean response.
- ε_{ij} is the random error term.

The usual test of interest is the same as in a one-way analysis of variance: to compare the population means due to the different treatments. The null and alternative hypotheses are given by:

$$H_0 : \tau_1 = \tau_2 = \dots = \tau_k = 0 \quad \text{versus} \quad H_a : \text{at least one } \tau_i \neq 0$$

We may also test for differences between blocks, but this is usually of secondary interest at best.

3.1.2 Block ANOVA Example

As a first example consider the following *classic* barley yield experiment.

Example. Barley Yields (from ?).

Data was collected on the yields of five varieties of barley grown at six different locations in 1931 and 1932. Of statistical interest is the mean difference in yield between the five different varieties.

Original Source: ?

Before proceeding with the data analysis, we first note the variables and factors of interests in this study.

- Barley yield - this is the response variable.

- Barley variety - the variety, or species, of barley. This is the factor variable of interest. It has 5 levels.
- Location and Year - The six different location and years serve as a block term in this model. We note that within each location and year, the environment should be fairly homogenous, so the location and year will serve as a blocking term.

The data is available in the file `fisherBarley.RData` on the class repository.

```
load(url("https://tjfisher19.github.io/introStatModeling/data/fisherBarley.RData"))
glimpse(fisherBarleyData)
```

```
## Rows: 60
## Columns: 5
## $ Yield      <dbl> 81.0, 80.7, 146.6, 100.4, 82.3, 103.1, 119.8, 98.9, 98.9, ~
## $ Variety    <chr> "Manchuria", "Manchuria", "Manchuria", "Manchuria", "Manc~
## $ LocationYear <chr> "L1-31", "L1-32", "L2-31", "L2-32", "L3-31", "L3-32", "L4~
## $ Location    <chr> "University Farm", "University Farm", "Waseca", "Waseca", ~
## $ Year        <dbl> 1931, 1932, 1931, 1932, 1931, 1932, 1931, 1932, 1931, 193~
```

We note the data includes a `LocationYear` variable that combines the information of both the `Location` and `Year`. We also note we have a single observation for each `Variety` and `LocationYear` – this we have a complete randomized block design with a single observation in each.

We first proceed with an inappropriate simple visualization of the data. We do this for demonstration and learning purposes.

```
p1 <- ggplot(fisherBarleyData) +
  geom_boxplot(aes(x=LocationYear, y=Yield) ) +
  labs(x="Location & Year", y="Yield (bushels)")
p2 <- ggplot(fisherBarleyData) +
  geom_boxplot(aes(x=Variety, y=Yield)) +
  labs(x="Barley Variety", y="Yield (bushels)")
grid.arrange(p1, p2, nrow=1)
```

First, a quick note about the above code. We build a simple boxplot of the yield by the block term and call it `p1`. Similarly we build a boxplot of yield by variety, called `p2`. We then use the `grid.arrange()` function in the `gridExtra` package (?) to put the two plots side-by-side. The results are in Figure ??.

Incorrect plots. We note that these plots are not necessarily appropriate for this data. For one, Boxplots display a 5 number summary – for each block term we only have five observations (one for each variety), so the figure on the left appears to summarize a sample of data when in fact we only have 5 observations – why not just display the raw data? Second, the figure on the right is ignoring the block structure – this is effectively equivalent to ignoring the *pairing* in a matched pairs design. Each environmental condition consisting of a location & year will have its own climate, rainfall, soil nutrients, etc... For

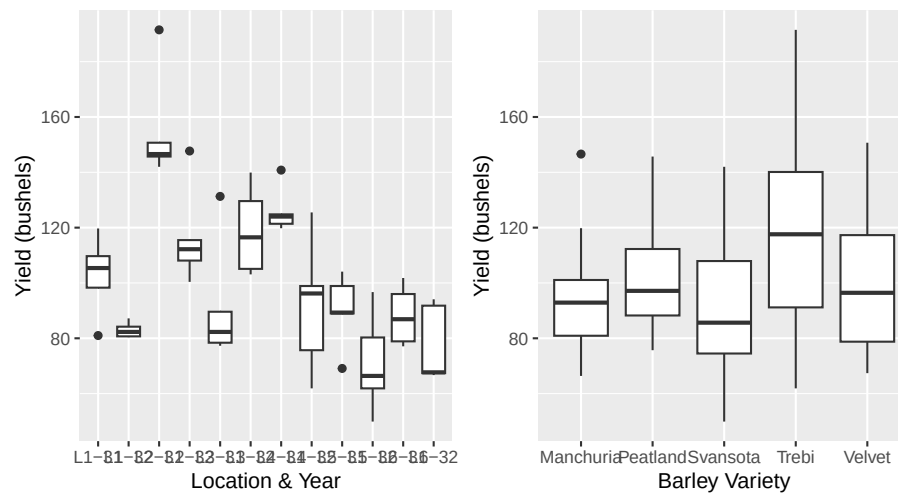


Figure 3.4: Boxplot distribution of the barley yeilds of ‘LocationYear’ and barley ‘Variety’.

example, it is possible a particular barley may grow well in one Location and year due to *optimal* precipitation and sunshine levels, but another variety may grow poorly there. Likewise, a different variety may prefer a different climate – this information cannot be deduced from this plot.

A better plot to consider is a profile plot similar to that used in the matched pairs design in Figure ?? from Section ?. Here, we explore the response at each treatment level within each block term.

```
ggplot(fisherBarleyData) +
  geom_line(aes(x=Variety, y=Yield,
                color=LocationYear, group=LocationYear) ) +
  labs(x="Barley Variety", y="Yield (bushels)", color="Location & Year") +
  theme_minimal()
```

This plot in Figure ?? is an improvement over the previous result but can be further tweaked for visual improvement. In particular, each line corresponds to one of the block terms (this is the common approach for these plots) but there are 12 different colors displayed, and it can be difficult to distinguish them. This can be considered *visual clutter*. Since the specific Location & Year combinations are not necessarily of interest in our study, we can *mute* their impact by plotting each line with the same color as in Figure ??.

```
ggplot(fisherBarleyData) +
  geom_line(aes(x=Variety, y=Yield,
                group=LocationYear), color="gray30" ) +
  labs(x="Barley Variety", y="Yield (bushels)") +
```

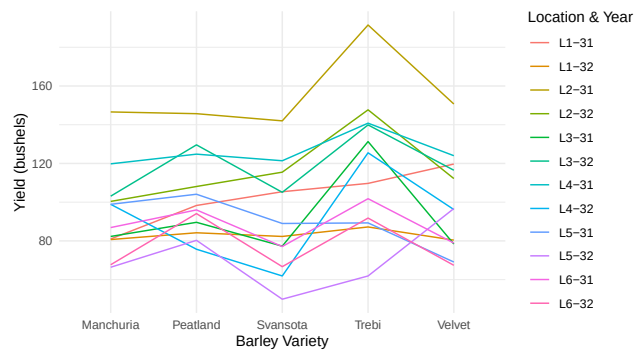


Figure 3.5: Profiles of each barley yield by variety profiling across each of the 12 Location and Year combinations.

```
theme_minimal()
```

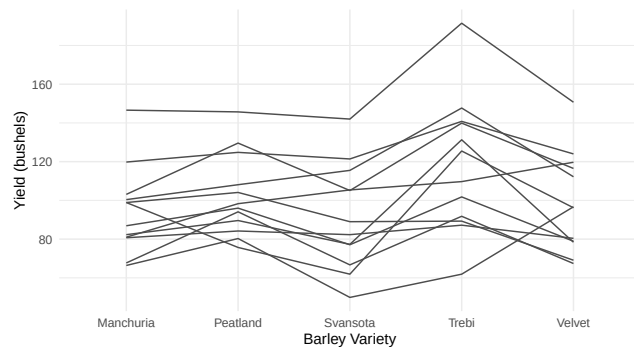


Figure 3.6: Profiles of each barley yield by variety profiling across each of the 12 Location and Year combinations.

From the profile plots in Figures ?? and ?? it is clear that certain Location & Years were more fruitful than others, in terms of barley yield. However, this is not the research question of interest - we are interested in determine how the different varieties influence yield. From the plots, this is not as clear. There is some evidence that the Trebi variety may result in more yield across all the blocks as well as maybe the Peatland variety.

To determine the effect variety has on yield we proceed with a formal analysis. Below is a formal analysis in a call of `aov()` – making sure to check underlying assumptions in Figure ??.

```
barley.fit <- aov(Yield ~ LocationYear + Variety, data=fisherBarleyData)
autoplot(barley.fit)
```

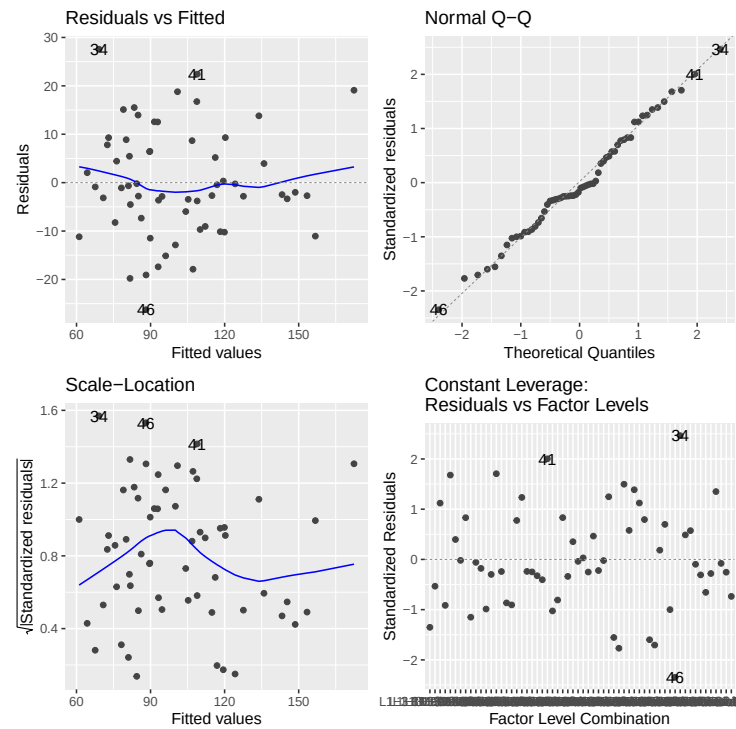


Figure 3.7: Residual diagnostic plots when the response variable is the barley yield as a function of location and year (block) and variety. Here, the residuals appear fairly homoskedastic based on the Residuals vs Fitted and Scale-Location plots. The normality assumption is satisfied based on the Normal Q-Q plot.

The assumptions look generally fine here (nothing too concerning about constant variance or normality). So, on to the analysis:

```
summary(barley.fit)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## LocationYear 11  31913     2901   17.00 2.1e-12 ***
## Variety       4   5310     1327    7.78 7.9e-05 ***
## Residuals    44   7509       171
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

We see that there is a significant difference in the yield between the five varieties ($F = 7.779$, $df_1 = 4$, $df_2 = 44$, $p\text{-value} = 0.0000785$)

You can also see how the total variation was partitioned into three components:

1. Within-LocationYear (*i.e.*, the block term) sum of squares (31913).
2. Between variety (*i.e.*, treatment) sum of squares (5310).
3. Residual sum of squares (7509).

We see that R also reports an F -statistic and associated p -value for the **LocationYear**. This is the block term and is not of concern to our analysis. So we **ignore** that F -statistic and p -value!

We can investigate the differences in yields between the five varieties using a multiple comparison procedure.

```
barley.mc <- emmeans(barley.fit, "Variety")
plot(contrast(barley.mc, "pairwise"))
```

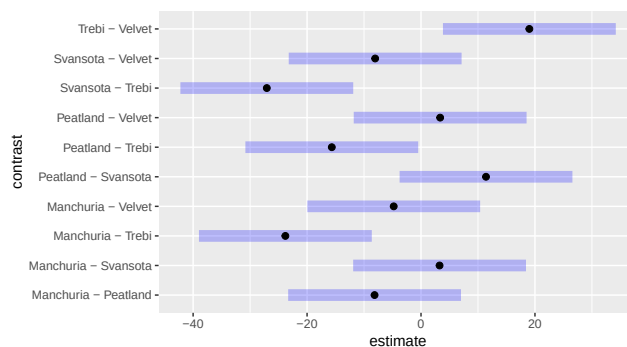


Figure 3.8: Tukey adjusted confidence interval plots comparing the five barley varieties (treatments). Here, we see that Trebti variety is statistically different than the other four varieties, while those four are all statistically similar (recall, we look for zero in the confidence interval of the differences).

We see that six of the ten Confidence Intervals in Figure ?? contain the value

zero, thus concluding there is not statistical difference between these varieties of barley. However, in four of the intervals zero is not included, and all four involve the ‘Trebi’ variety of barley - this we have statistical evidence to conclude that the Trebi variety results in a different yield than the other four varieties. Furthermore, based on the values of this intervals, it suggest that the Trebi variety will result in a greater yield than the other varieties ($\text{Trebi} - \text{Variety} > 0$ implies $\text{Trebi} > \text{Variety}$, and $\text{Variety} - \text{Trebi} < 0$ implies $\text{Trebi} > \text{Variety}$).

3.1.3 Another Block ANOVA Example

The previous Block designed had roots in agriculture, much like our original description. Here is a different example of block terms in an design.

Example. Weight loss study (published in ?).

Two reduction treatments and a control treatment were studied for their effects on weight change in obese women. The two reduction treatments involved were a self-induced weight reduction program and a therapist-controlled reduction program. Each of 10 subjects were assigned to the three treatment programs in a random order and measured for weight loss.

Original Source: ?

Subject	Control	Self-induced	Therapist
1	1.00	-2.25	-10.50
2	3.75	-6.00	-13.50
3	0.00	-2.00	0.75
4	-0.25	-1.50	-4.50
5	-2.25	-3.25	-6.00
6	-1.00	-1.50	4.00
7	-1.00	-10.75	-12.25
8	3.75	-0.75	-2.75
9	1.50	0.00	-6.75
10	0.50	-3.75	-7.00

There data are available in the R dataframe `wordrecall.RData` on the class repository (<https://tjfisher19.github.io/introStatModeling/data/>). Below is an analysis of this data.

```
weightloss <- read_tsv("https://tjfisher19.github.io/introStatModeling/data/weightloss.tsv")
glimpse(weightloss)
```

```
## Rows: 30
## Columns: 3
## $ Subject   <dbl> 1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 7, 7, ~
## $ Treatment <chr> "Control", "Self-induced", "Therapist", "Control", "Self-ind~
## $ WtChanges <dbl> 1.00, -2.25, -10.50, 3.75, -6.00, -13.50, 0.00, -2.00, 0.75, ~
```

Each subject gives three responses instead of one as in a usual one-way design.

In this manner, each subject (or person) forms a block (a person is a fairly homogeneous). The design is a randomized block design, because the experimenter randomly determines the order of treatments for each subject.

Before the formal analysis we build profile plots to explore the effects of the three treatments within each subject in Figure ??.

```
ggplot(weightloss, aes(x=Treatment, y=WtChanges)) +
  geom_line(aes(group=Subject), color="gray30" ) +
  stat_summary(fun="mean", geom="line", size=2, group=1, color="blue") +
  labs(x="Weight Loss Treatment", y="Weight Change") +
  theme_minimal()
```

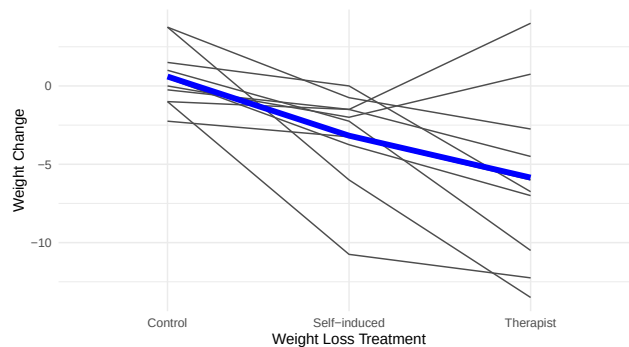


Figure 3.9: Profiles of each subject’s weight loss for the different treatments, with an overlaid ‘average’ highlighted in blue.

Overall, in Figure ?? the effect of the different treatments appears quite stark, with the Therapist and Self-induced treatments improving over the control. There is also quite a bit of subject-to-subject variability (*i.e.*, each person has their own profile!) and some subjects do not follow the “average” profile.

Here is how to do a formal analysis in a call of `aov()` – making sure to check underlying assumptions in Figure ??.

```
weightloss.fit <- aov(WtChanges ~ Subject + Treatment, data=weightloss)
autoplot(weightloss.fit)
```

There appears to be a minor violation in the constant variance assumption (there is some indication that fitted values away from zero are more variable?). But, this type of violation cannot be easily remedied by the standard techniques (a logarithm or square root transformation is not possible due to the negative values, and an reciprocal transformation is not possible due to a 0 observation). A single observation (observation 16) has a largest *z*-score value which deviates from the Normal line. We proceed with the analysis making note of some potential minor assumption violations.

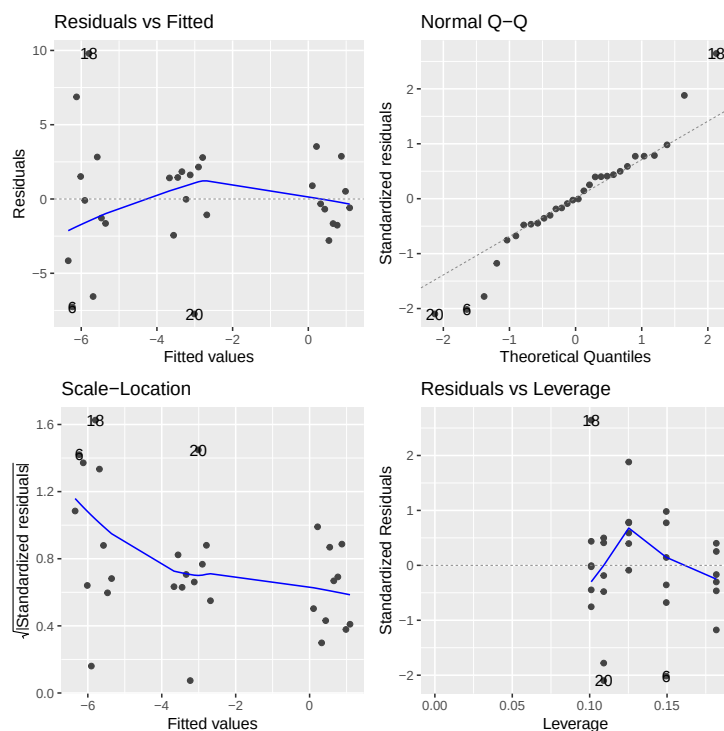


Figure 3.10: Residual diagnostic plots when the response variable is the weight change as a function of subject (block) and treatment. Here, the residuals appear to show some minor violations to the constant variance assumption as indicated by the Residuals vs Fitted and Scale-Location plots. Except for a single concerning observation, the normality assumption is satisfied based on the Normal Q-Q plot.

```
summary(weightloss.fit)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## Subject      1      3      3.0    0.19  0.663
## Treatment    2    210    105.0    6.88  0.004 **
## Residuals    26    397     15.3
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

We see that there is a significant difference in the mean weight change between the three treatments ($F = 6.878$, $df_1 = 2$, $df_2 = 26$, $p\text{-value} = 0.004$).

As a follow-up, we can investigate the differences in mean weight changes between the three treatments using a multiple comparison procedure and noting that there is a control treatment.

```
weightloss.mc <- emmeans(weightloss.fit, "Treatment")
plot(contrast(weightloss.mc, "trt.vs.ctrl") )
```

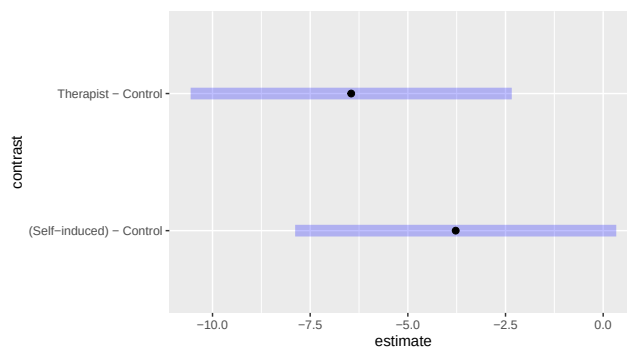


Figure 3.11: Dunnett adjusted confidence interval plots comparing the two weight-loss treatments against the control. Here, we see that Self-induced treatment is not statistically different than the control (zero is in the interval) while the Therapist based treatment results in significantly more weight loss than the control.

The confidence intervals comparing the therapist-based weight loss treatment to the control in Figure ?? does not contain the value zero, so we have evidence to suggest that the therapist based method results in a bigger weight change. However, the interval comparing the self-induced method to the control indicates there is not statistical difference (zero is included in the interval).

3.2 Two-factor Designs

A **factorial** structure in an experimental design is one in which there are two or more factors, and the levels of the factors are all observed in combination. For example, suppose you are testing for the effectiveness of three drugs (A, B, C) at three different dosages (5 mg, 10 mg, 25 mg). In a factorial design, you would observe a total of nine treatments, each of which consists of a combination of drug and dose (*e.g.*, treatment 1 is drug A administered at 5 mg, treatment 2 is drug A administered at 10 mg, etc). Note how quickly things will grow if you have more than two factors!

In this design, the data are usually obtained by collecting a random sample of individuals to participate in the study, and then randomly allocating a single treatment to each of the study participants as in a one-way design structure. The only difference now is that a “treatment” consists of a combination of different factors. If there are two factors, the data may be analyzed using a two-way ANOVA.

The two-way data structure with two factors **A** and **B** looks like the following:

	A₁	A₂	...	A_a
B₁	$Y_{111}, Y_{112}, \dots$	$Y_{211}, Y_{212}, \dots$	...	$Y_{a11}, Y_{a12}, \dots$
B₂	$Y_{121}, Y_{122}, \dots$	$Y_{221}, Y_{222}, \dots$	...	$Y_{a21}, Y_{a22}, \dots$
B₃	$Y_{131}, Y_{132}, \dots$	$Y_{231}, Y_{232}, \dots$	...	$Y_{a31}, Y_{a32}, \dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
B_b	$Y_{1b1}, Y_{1b2}, \dots$	$Y_{2b1}, Y_{2b2}, \dots$	...	$Y_{ab1}, Y_{ab2}, \dots$

Note there is replication (*i.e.*, multiple independent observations) in each treatment (combination of factors **A** and **B**).

The general model for such data has the form

$$Y_{ijk} = \mu + \alpha_i + \beta_j + \alpha\beta_{ij} + \varepsilon_{ijk}$$

where

- Y_{ijk} is the k^{th} observation in the i^{th} level of factor *A* and the j^{th} level of factor *B*.
- μ is the overall mean.
- α_i is the “main effect” of the i^{th} level of factor *A* on the mean response.
- β_j is the “main effect” of the j^{th} level of factor *B* on the mean response.
- $\alpha\beta_{ij}$ is the “interaction effect” of the i^{th} level of factor *A* and the j^{th} level of factor *B* on the mean response.
- ε_{ijk} is the random error term.

3.2.1 Analysis of a two-factor design

First recall an important feature of designed experiments – the analysis is determined by the experiment! In most cases of factorial designs, the primary interest is to determine if some interaction between the two factors influences the response. Therefore it is crucial to test the interaction term before assessing the effects of factor A alone on the response, because if A and B interact, then we cannot separate their effects. In other words, if the interaction between A and B significantly influences the response, then the effect that factor A has on the response changes depending on the level of factor B .

The usual testing strategy is as follows:

1. Fit a full interaction model.
2. Test for significant interaction between A and B :
 - a. If the interaction term is significant, you must look at comparisons in terms of treatment combination; *i.e.*, you cannot separate the effects of A and B .
 - b. If the interaction term is non-significant, you can look into deleting the interaction term from your model, fitting a reduced **main-effects model**. Then (and only then) may you look at the effects of A and B separately.

We consider two examples below.

3.2.2 Example with No Interaction

Example. Highway Signage (published in ?).

An experiment was conducted to determine the effects that the sign size and sign material have on *detection distance* (the distance at which drivers can first detect highway caution signs). Four drivers were randomly selected for each combination of sign size (small, medium, and large) and sign material (marked 1, 2, and 3). Each driver covered the same stretch of highway at a constant speed during the same time of day, and the detection distance (in feet) was determined for the driver's assigned caution sign.

Original Source: ?

The data appear in the csv file `highwaySigns.tsv` in the textbook repository.

First note the data is stored in a “Tab separated values” file, or TSV. This is similar to the more common, “Comma Separated values”, or CSV, format. The `readr` package includes a `read_tsv()` function that works just like `read_csv()` and `read_table()`.

```
highway_signs <- read_tsv("https://tjfisher19.github.io/introStatModeling/data/highwaySigns.tsv",
  glimpse(highway_signs)
```

```
## Rows: 36
```

```
## Columns: 3
## $ Distance <dbl> 2503, 1929, 1512, 2774, 2187, 2288, 3354, 2920, 2928, 2427, 2~
## $ Size      <chr> "Small", "Small", "Small", "Medium", "Medium", "Medium", "Lar~
## $ Material <dbl> 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3~
```

Take a look at the detection distances by treatment and note that we need to coerce the `Material` variable to be a factor variable since it is coded as a numer. We will also turn `Size` into a factor with a more logical order (recall that R will put the values in alphabetical order by default).

```
highway_signs <- highway_signs %>%
  mutate(Material = as.factor(Material),
         Size = factor(Size, levels=c("Small", "Medium", "Large")))
```

Now we wish to construct a plot of the data. We note that we have 4 observations per treatment (combination of size and material) so a boxplot makes no sense here. We attempt to plot the raw data, using color to distinguish the three sign materials. We also add some *jittering* and *dodging* to separate data points and materials.

```
ggplot(highway_signs) +
  geom_point(aes(x=Size, y=Distance, color=Material),
            position=position_jitterdodge() ) +
  labs(x="Sign Size", y="Detection Distance (ft)") +
  theme_bw()
```

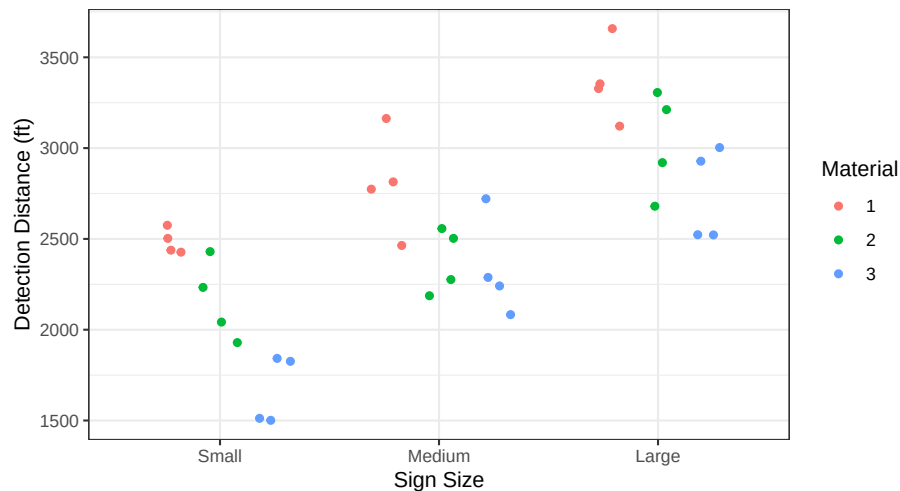


Figure 3.12: Boxplot distribution of the texture values as a function of sauce and fish type.

From Figure ??, there appears to be a size effect: Large signs have larger detection distances, followed by medium sized and then small. There may be

a material effect as material type 3 appears to have lower detection distances than type 1. Material type 2 is somewhere between types 1 and 3 in terms of detection distances.

We fit the interaction model in `aov()` by telling R to include an interaction term with the notation `factor1:factor2`. Below is the specific example for this study but first run a check on the residuals in Figure ??.

```
highway.fit <- aov(Distance ~ Size + Material + Size:Material, data=highway_signs)
autoplot(highway.fit)
```

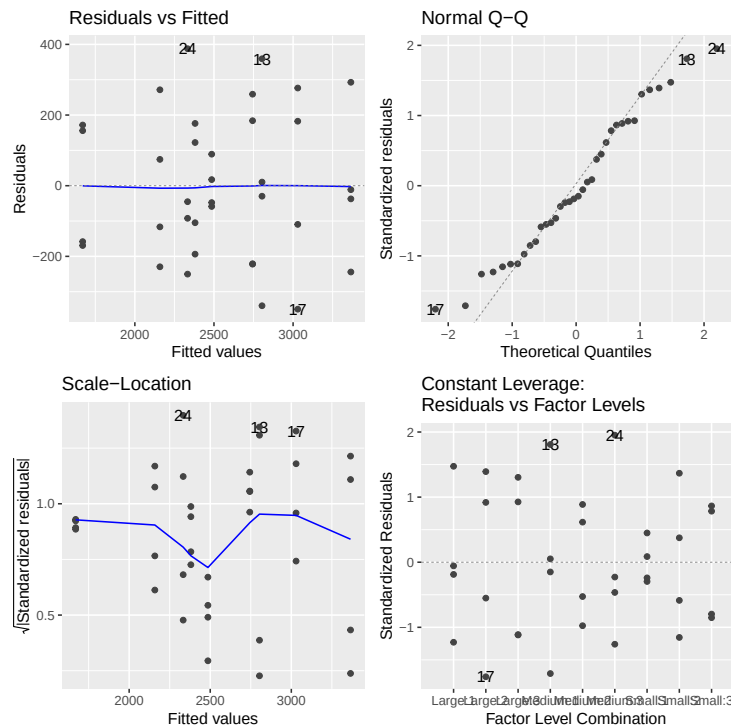


Figure 3.13: Residual diagnostic plots when the response variable is the detection distance as a function of sign size and sign material. Here, the residuals appear fairly homoskedastic based on the Residuals vs Fitted and Scale-Location plots. The normality assumption is satisfied based on the Normal Q-Q plot.

The assumptions look to be reasonably met. While there may be some minor issues with constant variance, it does not appear to be systemically related to the predicted (fitted) values, so transformations will not be helpful. Likewise, those few observations in the QQ-Plot are not overly concerning given their z -score values are in the range $(-2, 2)$.

At this point, we can also generate tables of response means after we fit the

interaction model. These consist of:

- The “grand mean” (overall mean, all data pooled together and estimate μ).
- Main effect means (*i.e.*, means for each factor level, taken one factor at a time).
- Interaction means (*i.e.*, treatment means for all factor combinations).

These means are estimates from the data and are purely descriptive (like the boxplots were), but informative nonetheless. We could calculate each one of the above using a series of `tidyverse` commands or using the R function `model.tables()`:

```
model.tables(highway.fit, "means")
```

```
## Tables of means
## Grand mean
##
## 2552.33
##
## Size
## Size
## Small Medium Large
## 2104.8 2505.9 3046.2
##
## Material
## Material
##      1      2      3
## 2884.9 2522.9 2249.2
##
## Size:Material
##      Material
## Size      1      2      3
## Small  2486 2158 1670
## Medium 2804 2381 2333
## Large  3365 3029 2744
```

We proceed to the hypothesis tests:

```
summary(highway.fit)
```

```
##           Df Sum Sq Mean Sq F value Pr(>F)
## Size       2 5356373 2678187   50.91 6.9e-10 ***
## Material   2 2440644 1220322   23.20 1.4e-06 ***
## Size:Material 4 217044   54261    1.03  0.41
## Residuals 27 1420315   52604
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The first test to inspect is the interaction test. The interaction is insignificant ($F = 1.031$, $df_1 = 4$, $df_2 = 27$, $p\text{-value} = 0.409$). Thus, we could conclude the effect that sign size type has on the mean detection distance does not depend on material type.

A visualization of the interaction effect may be obtained using an interactions plot. An interaction plot is basically a plot of treatment means, whereby the means for all treatments having a given fixed level of one of the factors are visually “connected”. We can build the interaction plot seen in Figure ?? using aesthetic options in `ggplot()` and with the `stat_summary` functions. Note both `color` and `group` are determined based on the `gender` variable.

```
ggplot(highway.fit,
       aes(x=Size, y=Distance, color=Material, group=Material)) +
  stat_summary(fun=mean, geom="point") +
  stat_summary(fun=mean, geom="line") +
  labs(x="Sign Size", y="Mean Detection Distance (ft)", color="Material")
```

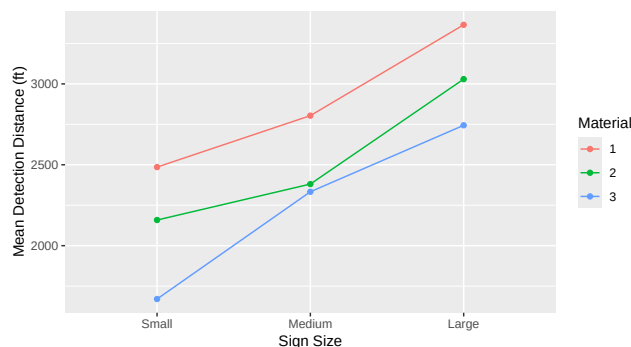


Figure 3.14: Interaction plot demonstrating that the gender and dosage factors do not interact.

There are three treatment mean traces, one for each material type. Each trace connects the mean detection distance of the sign size within each material.

The lack of significant interaction is evidenced by the parallelism of these traces: the material effect for small signs is depicted by the “gaps” between the traces on the left. Similarly, the material effect for the medium signs is depicted by the gaps at the middle points. Likewise, the material effect for the large signs is depicted by the gaps at the right points. Overall the three lines are reasonably parallel, and the interaction is not significant, this implies that the effect of material type within three sign sizes is about the same across all sign sizes.

In other words, the effect of material type does not depend on sign size.

The interaction term was non-significant. Visually, we may have support that

the two factors do not interact. We can consider a reduced model that eliminates the interaction term and includes only main effects:

```
highway.main.fit <- aov(Distance ~ Size + Material, data=highway_signs)
summary(highway.main.fit)
```

```
##              Df  Sum Sq Mean Sq F value    Pr(>F)
## Size           2 5356373 2678187    50.7 1.7e-10 ***
## Material       2 2440644 1220322    23.1 7.2e-07 ***
## Residuals     31 1637358    52818
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

From the results here (or with the interaction model), we see that the both material type and sign size appear to influence the detection distance of the fish (both p -values are smaller than typically-used significance levels).

At this point, the follow-up work would involve multiple comparisons among the levels of the significant main effect factors. Here, since there is no interaction, we can perform multiple comparisons within each of the main effects separately. First, we look at the influence of sign size.

```
twoway.size.mc <- emmeans(highway.main.fit, "Size")
confint(contrast(twoway.size.mc, "pairwise"))
```

```
## contrast      estimate    SE df lower.CL upper.CL
## Small - Medium    -401 93.8 31     -632     -170
## Small - Large     -941 93.8 31    -1172     -710
## Medium - Large    -540 93.8 31     -771     -309
##
## Results are averaged over the levels of: Material
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
```

We note that the adjusted 95% confidence do not include the value 0, thus all sign sizes result in different detection distances.

For the three different material types.

```
twoway.material.mc <- emmeans(highway.main.fit, "Material")
confint(contrast(twoway.material.mc, "pairwise"))
```

```
## contrast              estimate    SE df lower.CL upper.CL
## Material1 - Material2      362 93.8 31    131.1    593
## Material1 - Material3      636 93.8 31    404.8    867
## Material2 - Material3      274 93.8 31     42.8    505
##
## Results are averaged over the levels of: Size
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 3 estimates
```

Supplement	Dose		
	0.5 mg	1.0 mg	2.0 mg
Orange Juice	Treatment 1	Treatment 2	Treatment 3
Vitamin C	Treatment 4	Treatment 5	Treatment 6

Similar to the above, we note that all three confidence intervals do not include the value of 0, indicating that all three material types result in different mean detection distances.

3.2.3 Example with Interaction

Example. Tooth growth in guinea pigs (original from ?).

Consider an example that investigates the effects of ascorbic acid and delivery method on tooth growth in guinea pigs. Sixty guinea pigs are randomly assigned to receive one of three levels of ascorbic acid (0.5, 1.0 or 2.0 mg) via one of two delivery methods (orange juice or Vitamin C), under the restriction that each treatment combination has an equal number of guinea pigs. The response variable `len` is the length of odontoblasts (cells responsible for tooth growth). The two-way data structure here looks like this:

As alluded to before, there are ten replicate guinea pigs per treatment. The data appear in the R workspace `ToothGrowth` included in the `datasets` package (?). Here is the head of the dataframe:

```
data("ToothGrowth")
head(ToothGrowth)
```

```
##      len supp dose
## 1  4.2   VC  0.5
## 2 11.5   VC  0.5
## 3  7.3   VC  0.5
## 4  5.8   VC  0.5
## 5  6.4   VC  0.5
## 6 10.0   VC  0.5
```

Let us bypass generating numeric descriptive statistics for the moment, and instead jump to an interaction plot of the length response in Figure ??.

```
ggplot(ToothGrowth, aes(x=dose, y=len, group=supp, color=supp)) +
  stat_summary(fun=mean, geom="point") +
  stat_summary(fun=mean, geom="line") +
  labs(x="Dosage (mg per day)", y="Length of dontoblasts",
       color="Supplement")
```

There are a couple of things to note here:

1. There appears to be a substantial dose effect, in that higher doses of

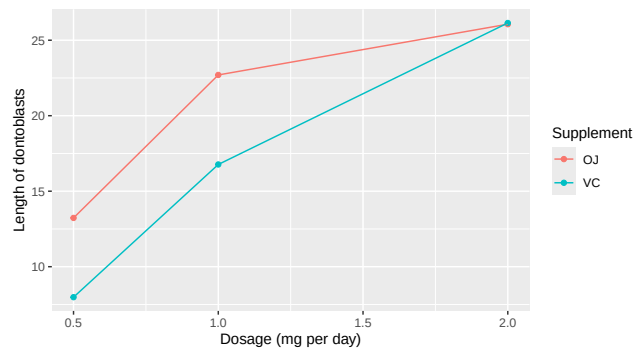


Figure 3.15: Interaction plot demonstrating that the dose and supplement may interact in predicting tooth length.

ascorbic acid generally result in higher mean length.

2. **However, there may be a substantial interaction effect between the type of supplement and dose in determining mean tooth length.** In particular, orange juice appears to be more effective than Vitamin C as a delivery method (longer tooth length), but only at lower dose levels. At the high dose level, there appears to be no difference between the supplements.

Once again, non-parallel traces may be the red flag. But, we need to run the ANOVA test for interaction to confirm if this is a significant effect. First, check assumptions in Figure ??.

```
tooth.fit <- aov(len ~ factor(dose) + supp + factor(dose):supp, data=ToothGrowth)
autoplot(tooth.fit)
```

All assumptions look fine here. So, proceed to the ANOVA tests:

```
summary(tooth.fit)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## factor(dose)   2   2426    1213    92.00 < 2e-16 ***
## supp           1    205     205    15.57 0.00023 ***
## factor(dose):supp  2    108      54     4.11 0.02186 *
## Residuals     54    712      13
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The test for interaction is significant ($F = 4.107$, $df_1 = 2$, $df_2 = 54$, $p\text{-value} = 0.0218$). Thus, we conclude the effect that supplement has on the mean tooth growth does depend on the dosage level. **We then ignore the main effects tests if the interaction is significant.**

Remember the strategy we introduced earlier:

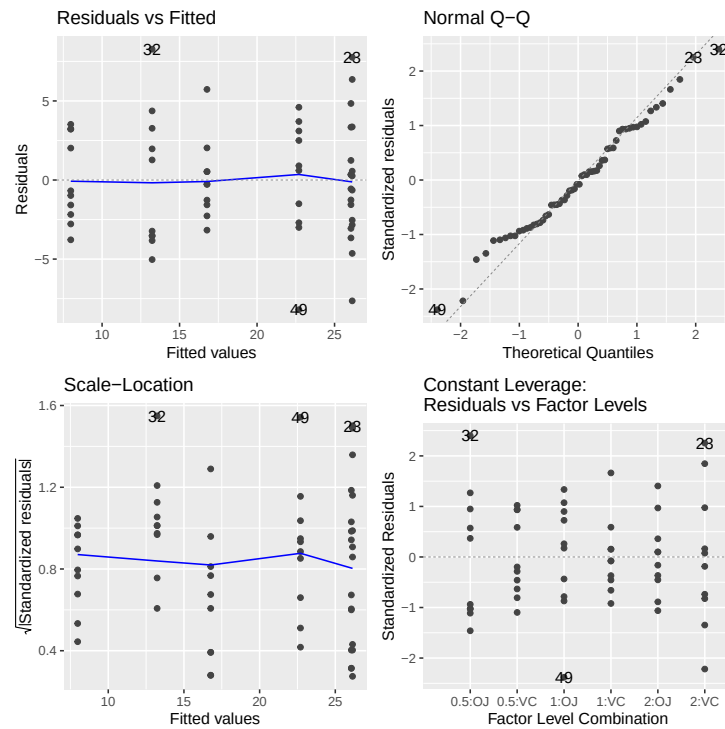


Figure 3.16: Residual diagnostic plots when the response variable is the tooth length as a function of supplement and dosage. Here, the residuals appear fairly homoskedastic based on the Residuals vs Fitted and Scale-Location plots. The normality assumption is satisfied based on the Normal Q-Q plot.

1. Fit a full interaction model.
2. Test for significant interaction between A and B :
 - a. **If interaction is significant, you must look at comparisons in terms of treatment combinations; *i.e.*, you cannot separate the effects of A and B .**
 - b. If interaction is non-significant, you may delete the interaction term from the model, fitting a reduced main-effects model. Then (and only then) may you look at the effects of A and B separately.

Performing multiple comparisons. In the case of a significant interaction effect in a two-factor ANOVA model, we typically follow up by performing multiple comparisons for the levels of one of the factors while holding the other factor fixed (the “conditioning” factor). So for example, in the present problem we could either

- Compare the supplements at each of the dose levels (3 tests total), or
- Compare the dose levels within each of the supplement types ($3 \times 2 = 6$ tests total).

Frequently in practice, the context of the problem will dictate to the researcher which way makes the most sense. We’ll provide code that performs both cases using the `emmeans` package.

```
tooth.mc1 <- emmeans(tooth.fit, pairwise ~ factor(dose) | supp)
tooth.mc2 <- emmeans(tooth.fit, pairwise ~ supp | factor(dose) )
tooth.mc1
```

```
## $emmeans
## supp = OJ:
##   dose emmean    SE df lower.CL upper.CL
##   0.5  13.23  1.15 54    10.93    15.5
##   1.0  22.70  1.15 54    20.40    25.0
##   2.0  26.06  1.15 54    23.76    28.4
##
## supp = VC:
##   dose emmean    SE df lower.CL upper.CL
##   0.5   7.98  1.15 54     5.68    10.3
##   1.0  16.77  1.15 54    14.47    19.1
##   2.0  26.14  1.15 54    23.84    28.4
##
## Confidence level used: 0.95
##
## $contrasts
## supp = OJ:
##   contrast      estimate    SE df t.ratio p.value
## dose0.5 - dose1    -9.47  1.62 54   -5.831 <.0001
## dose0.5 - dose2   -12.83  1.62 54   -7.900 <.0001
## dose1 - dose2     -3.36  1.62 54   -2.069  0.1060
```



```
##
## supp = VC:
## contrast      estimate    SE df t.ratio p.value
## dose0.5 - dose1    -8.79 1.62 54  -5.413  <.0001
## dose0.5 - dose2   -18.16 1.62 54 -11.182  <.0001
## dose1 - dose2      -9.37 1.62 54  -5.770  <.0001
##
## P value adjustment: tukey method for comparing a family of 3 estimates
tooth.mc2

## $emmeans
## dose = 0.5:
## supp emmean    SE df lower.CL upper.CL
## OJ    13.23 1.15 54    10.93    15.5
## VC     7.98 1.15 54     5.68    10.3
##
## dose = 1.0:
## supp emmean    SE df lower.CL upper.CL
## OJ    22.70 1.15 54    20.40    25.0
## VC    16.77 1.15 54    14.47    19.1
##
## dose = 2.0:
## supp emmean    SE df lower.CL upper.CL
## OJ    26.06 1.15 54    23.76    28.4
## VC    26.14 1.15 54    23.84    28.4
##
## Confidence level used: 0.95
##
## $contrasts
## dose = 0.5:
## contrast estimate    SE df t.ratio p.value
## OJ - VC      5.25 1.62 54   3.233  0.0021
##
## dose = 1.0:
## contrast estimate    SE df t.ratio p.value
## OJ - VC      5.93 1.62 54   3.651  0.0006
##
## dose = 2.0:
## contrast estimate    SE df t.ratio p.value
## OJ - VC     -0.08 1.62 54  -0.049  0.9609
```

Here we see the output includes the estimates and confidence intervals for the means (while holding one factor fixed) and also includes the contrasts (comparisons) of one factor while holding the second factor constant.

At a dose level of 0.5 mg, ascorbic acid delivery via orange juice produces signif-

icantly larger mean tooth growth than does delivery via Vitamin C. We determined this based on the the second set of output, where we see an adjusted p -value of 0.0021 comparing orange juice with Vitamin C. Similarly, at `dose=1.0`, there is still a difference but not at `dose=2.0`.

We can also use plotting features to graphically explore the factors while holding another factor constant.

```
plot(tooth.mc1)
```

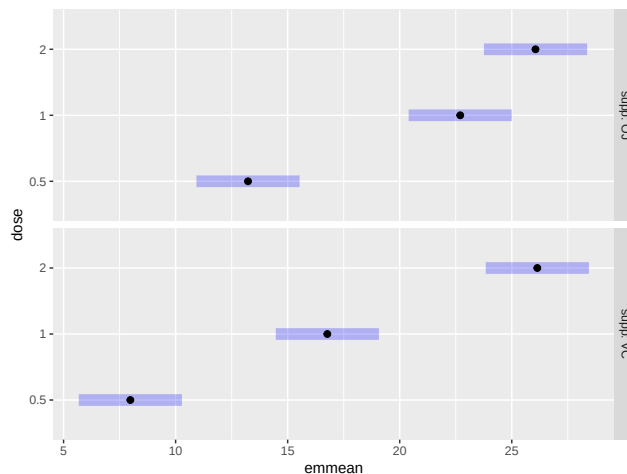


Figure 3.17: Graphical exploration of all treatments noting that the OJ supplement tends to result in higher tooth lengths than Vitamin C.

Here we explore the performance of the dosage amounts conditioning (essentially faceting) on the delivery type. Graphically we can see that the OJ method tends to have higher tooth lengths than Vitamin C for the low dosage levels of ascorbic acid.

Chapter 4

Advanced Designs

The previous chapters have established the groundwork of experimental design. We discussed the key elements of experimental units, factors, response variables, treatments, confounding influences and the purpose of replication and randomization. This former work just scratches the surface of the field of experimental design and the analyses associated with those designs. The objective of this chapter includes:

- Understanding the basic structure of factorial designs.
- Understanding the basic structure of within-subject designs.
- Building an appreciation for the complexity of experimental design.

4.1 Higher order factor models

In the previous two chapters we have seen a One-Way and Two-Way ANOVA. In the former, a single factor consisted of k -levels (and thus k -treatments) while in the latter the first factor may consist of k levels and the second factor j levels (and therefore $k \times j$ treatments). This idea can, of course, be expanded to even more terms, consider the following example.

4.1.1 Example of Three-factor design

Example. Marketing research contractors (?).

A marketing research consultant evaluated the effects of free schedule, scope of work and type of supervisory control on the quality of work performed under contract by independent marketing research agencies. The design of this experiment can be summarized as follows

Fee Level	Supervision			
	Local Supervisors		Traveling Supervisors	
	In house	Subcontracted	In house	Subcontracted
High	Treatment 1	Treatment 2	Treatment 3	Treatment 4
Average	Treatment 5	Treatment 6	Treatment 7	Treatment 8
Low	Treatment 9	Treatment 10	Treatment 11	Treatment 12

Factor	Factor Levels
Fee level	High
	Average
	Low
Scope	In house
	Subcontracted
Supervision	Local Supervisors
	Traveling Supervisors

The quality of work performed was measured by an index taking into account several characteristics of quality.

You should note there are **three factors** in this experiment, with 3, 2 and 2 levels for each factor. This means the number of treatments is $3 \times 2 \times 2 = 12$ and can be summarized in the following table.

Recall the main idea from two-factor experiments—the interaction terms are typically very important. In this three factor design we have the three main effects (Fee level, Scope and Supervision) but then we also have pairwise interactions between these variables and an interaction among all three. From a statistical modeling perspective, we are now essentially fitting the model

$$Y_{ijkm} = \mu + \alpha_i + \beta_j + \gamma_k + (\alpha\beta)_{ij} + (\alpha\gamma)_{ik} + (\beta\gamma)_{jk} + (\alpha\beta\gamma)_{ijk} + \varepsilon_{ijkm}$$

where

- Y_{ijkm} is the m^{th} observation in the i^{th} level of the first factor, the j^{th} level of the second factor and the k^{th} level of the third factor.
- μ is an overall mean.
- Fee level main effect α_i .
- Scope main effect β_j .
- Supervision main effect γ_k .
- Fee level and Scope interaction effect $(\alpha\beta)_{ij}$.
- Fee level and Supervision interaction effect $(\alpha\gamma)_{ik}$.
- Scope and Supervision interaction effect $(\beta\gamma)_{jk}$.
- Fee level, Scope and Supervision interaction effect $(\alpha\beta\gamma)_{ijk}$.
- ε_{ijkm} is the random error term.

As you can see this model has grown greatly compared to the two-way ANOVA examples from before.

What will happen if there are four factors?

In general, if an experiment consisted of n factors, we would potentially have $2^n - 1$ terms in our model. Thus with four factors you are looking at $2^4 - 1 = 16 - 1 = 15$ terms in your ANOVA decomposition. With five factors, 31 terms. This will quickly grow out of control.

Fortunately, for higher order factor problems there are statisticians available to help formulate a better design and potentially *optimize* some of the variables of study. We leave out the details of these topics in this introductory text but want to provide insight that these advanced designs and analyses can be done. Please consult a statistician before your experiment if you are planning a complex design!!

4.1.2 Three-factor ANOVA Example

Example continued. Below we complete the example involving the marketing research contractors.

First we read in the data and make sure R properly treats the factors.

```
data_site <- "https://tjfisher19.github.io/introStatModeling/data/marketingResearch.txt"
marketing <- read_table(data_site, col_types=cols() )

marketing <- marketing %>%
  mutate(Fee = as.factor(Fee),
         Scope = as.factor(Scope),
         Supervision = as.factor(Supervision))
```

We leave out EDA and residual analysis from this example for brevity. We fit the ANOVA model.

```
marketing.anova <- aov(Quality ~ Fee*Scope*Supervision, data=marketing)
```

Here we used the notation `Fee*Scope*Supervision` so R will fit a full factorial model (that is, all main effects and interaction terms). Now look at the ANOVA output.

```
summary(marketing.anova)
```

##	Df	Sum Sq	Mean Sq	F value	Pr(>F)
## Fee	2	10044	5022	679.34	< 2e-16 ***
## Scope	1	1834	1834	248.08	< 2e-16 ***
## Supervision	1	3832	3832	518.40	< 2e-16 ***
## Fee:Scope	2	2	1	0.11	0.90
## Fee:Supervision	2	1	0	0.05	0.95
## Scope:Supervision	1	575	575	77.75	1.6e-10 ***

```
## Fee:Scope:Supervision  2      4      2      0.27      0.77
## Residuals              36     266      7
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

As with the two-factor analysis, **always look at the most complex interaction term first**. Here we see that the three factor interaction term is not significant, thus we proceed to the pairwise interactions.

The interaction between Scope and Supervision is significant. Interactions involving the Fee level factor are not significant, thus we can interpret the Fee level main effect as being a significant influence on quality. We can also conclude that some combination of Scope and Supervision influence quality. We leave out any multiple comparisons analysis here as it gets very complex when three factors are involved.

4.2 Within-subject designs

In the previous two chapters, you have seen different experimental designs where a factor(s) levels are manipulated by the researcher in order to measure a response. In simpler design settings, the factors that are manipulated are of the “between-subjects” type. The one-way ANOVA example in chapter 2 on drug effects on alertness is illustrative of this: each participant (experimental unit) had exactly one drug randomly assigned to them. As such, there is only one observation per experimental unit in the design. Drug was a **between-subjects factor**.

Why is that important to note? Because, as you should realize by now, that assures that the observations in the data are all independent. It will be elaborated upon later in this course how crucial this assumption is to the validity of any statistical inferences we draw from the tests/CIs we derive. In fact, independence is more important to inference validity than either normality or constant variance assumptions.

Now, contrast this with the **paired t -test** (see section 2.4). An example was discussed that involved giving a sample of 20 people a drug treatment in order to study its effect on the Cholesterol of the patients. Think of that problem from a design of experiments point of view: what was/were the factor(s) that were manipulated by the researcher?

- It is not the drug treatment! Why? Because in the experiment, only one drug was studied: there was no “comparison” drug or placebo used to form a basis for comparison. As such, the drug treatment in that problem was not a variable at all: it was a constant.
- So what is left? The answer is TIME. The researcher measured Cholesterol at two time points on each patient (experimental unit): [1] just before the start of the drug treatment, and [2] 45 days later. The factor here is time,

and it has two levels. Time is a **within-subject factor**.

In a designed experiment that involves several measurements made on the same experimental unit over time, the design is called a **repeated measures design**.

The important thing to recognize here is that each patient provided two observations, not one. It stands to reason that *what a particular subject's Cholesterol is after 45 days will be related to their starting Cholesterol before the drug treatment*. That is the layman's way of saying that the measurements within a subject are **not independent** (*i.e.*, a patient is not independent of themselves – the observations are correlated). If in any analysis we ignored this basic truth, that analysis would be seriously flawed.

So how did we handle this before? A simple strategy which is common with paired data comparisons is to calculate each subject's difference score (yielding a single difference measurement per subject) and then performing a one-sample *t*-test on the differences. Each difference score is independent because each came from a different randomly chosen subject. This is what was done in the paired *t*-test analysis in section 2.4. Check back and remind yourself of this.

4.2.1 Blocks revisited: an approach to handling within-subjects factors

The notion and purpose of blocking in an experimental design was introduced in section 3.1. If you recall that witty definition of blocks as relatively homogeneously sets of experimental material, it may occur to you that subjects can serve as blocks. In fact, this was already stated in section 3.1. What this means for us now is that using a block design approach in analysis is 'a way' to handle correlated (non-independent) observations such as in the paired *t*-test problem above. We illustrate that here.

Example. Carbon Emissions (originally in ?).

The makers of the MAGNETIZER Engine Energizer System (EES) claim that it improves the gas mileage and reduces emissions in automobiles by using magnetic free energy to increase the amount of oxygen in the fuel for greater combustion efficiency. Test results for 14 vehicles that underwent testing is available in the file `carCOemissions.csv`. The data includes the emitted carbon monoxide (CO) levels, in parts per million, of each of the 14 vehicles tested before and after installation of the EES.

This is a classic before-after (Paired design) study. Below is some code to implement the test.

First, we quickly recap the Paired *t*-test result from Section REF

```
www <- "https://tjfisher19.github.io/introStatModeling/data/carCOemissions.csv"
vehicleCO <- read_csv(www, col_types = cols())
vehicleCO
```

```
## # A tibble: 14 x 3
##       id before after
##   <dbl> <dbl> <dbl>
## 1     1     1  1.6  0.15
## 2     2     2  0.3  0.2
## 3     3     3  3.8  2.8
## 4     4     4  6.2  3.6
## 5     5     5  3.6  1
## 6     6     6  1.5  0.5
## 7     7     7  2    1.6
## 8     8     8  2.6  1.6
## 9     9     9  0.15 0.06
## 10    10    10  0.06 0.16
## 11    11    11  0.6  0.35
## 12    12    12  0.03 0.01
## 13    13    13  0.1  0
## 14    14    14  0.19 0
```

and the paired t -test

```
t.test(vehicleCO$before, vehicleCO$after, paired=TRUE)

##
## Paired t-test
##
## data: vehicleCO$before and vehicleCO$after
## t = 3.146, df = 13, p-value = 0.00773
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  0.239443 1.289129
## sample estimates:
## mean difference
##      0.764286
```

Now, we will perform this same analysis using a Block ANOVA. The data is structure that 2 observations (the before and after) are on each row of the data frame. We will reshape this data into a tall, or long, format.

```
vehicleCO_tall <- vehicleCO %>%
  pivot_longer(cols=c(before, after),
               names_to="Time",
               values_to="Emitted_CO")
head(vehicleCO_tall)
```

```
## # A tibble: 6 x 3
##       id Time   Emitted_CO
##   <dbl> <chr>      <dbl>
## 1     1 1 before      1.6
```



```
## 2      1 after      0.15
## 3      2 before     0.3
## 4      2 after      0.2
## 5      3 before     3.8
## 6      3 after      2.8
```

Note the difference in the layout of the data here as compared to the original `vehicleCO` used in the paired t -test: each observation occupies a single row of the data file now. There is an identifier variable (`id`) that identifies which subject every observation belongs to. The variable `Time` refers to the time factor and `Emitted_CO` records the emitted Carbon Monoxide levels (ppm).

We will analyze this same data (addressing the same research question) by using a **block design approach**:

```
vehicleco.block.aov <- aov(Emitted_CO ~ factor(id) + Time, data=vehicleCO_tall)
summary(vehicleco.block.aov)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## factor(id)   13   56.7    4.36    10.6 7.2e-05 ***
## Time          1    4.1    4.09     9.9 0.0077 **
## Residuals    13    5.4    0.41
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Look at the F -test for the time factor (labeled `Time` here). This result is identical to the result of the paired t -test above ($F = 9.897$, $df_1 = 1$, $df_2 = 13$, p -value = 0.00773). Note, $\sqrt{9.897} = 3.146$, the value of the paired t -test statistic.

How does the block approach work here? You will note that the test of `Time` (the within subjects effect) uses the residual effect as the error term. Because the blocks are accounted for in the model (and here, the blocks constitutes the overall between-vehicle variability source), the only variation left over for the “residuals” is the variability due to the within-vehicle differences. In essence, there are **two components** to the random error now:

- Aggregated variability between vehicles (block effect)
 - Remaining unexplained (random) variability after accounting for aggregate vehicle-to-vehicle differences and aggregated time-to-time differences.
- We will call this leftover stuff “pure” error.

For a correct analysis, it is important that each factor of interest in the study be tested against the proper component of the error. Here, **we test a within-subjects effect against the random error component dealing with “pure” random error.**

Perhaps a **more general approach** to within-subjects analysis is to specify to R the fact that there are layers of random variability in the data introduced by the fact that we have more than one measurement per experimental unit. This

can be done using an `aov` model specification that explicitly cites these layers with an `Error` option. The model specification in R is generally

```
`aov(response ~ between-subjects factor structure + Error(EU variable/WS factor)`
```

In the present example, this would be as follows:

```
WS_aov <- aov(Emitted_CO ~ Time + Error(factor(id)/Time), data=vehicleCO_tall)
summary(WS_aov)
```

```
##
## Error: factor(id)
##           Df Sum Sq Mean Sq F value Pr(>F)
## Residuals 13   56.7    4.36
##
## Error: factor(id):Time
##           Df Sum Sq Mean Sq F value Pr(>F)
## Time       1   4.09    4.09    9.9 0.0077 **
## Residuals 13   5.37    0.41
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Again we see the same result. It turns out that for more complex studies this general approach is preferred.

4.2.2 A more involved repeated measures case study

If we can handle within-subjects factors using blocks, why not just stick with that all the time?

Well, the reason is that repeated measures designs can get complicated: multiple time measurement points (more than just two), the addition of between-subjects factors, etc. The following case study example illustrates this.

Example. Cholesterol study (from the University of Sheffield (?)).

A study tested whether cholesterol was reduced after using a certain brand of margarine as part of a low fat, low cholesterol diet. The subjects consumed on average 2.31g of the active ingredient, stanol ester, a day. This data set `cholesterolreduction.csv` contains information on 18 people using margarine to reduce cholesterol over three time points.

Variable Name	Variable	Data type
ID	Participant Number	
Before	Cholesterol before the diet (mmol/L)	Scale
After4weeks	Cholesterol after 4 weeks on diet (mmol/L)	Scale
After8weeks	Cholesterol after 8 weeks on diet (mmol/L)	Scale
Margarine	Margarine type A or B	Binary

```
"carCOemissions.csv"
chol <- read_csv("https://tjfisher19.github.io/introStatModeling/data/cholesterolreduction.csv",
glimpse(chol)

## Rows: 18
## Columns: 5
## $ ID          <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
## $ Before      <dbl> 6.42, 6.76, 6.56, 4.80, 8.43, 7.49, 8.05, 5.05, 5.77, 3.91~
## $ After4weeks <dbl> 5.83, 6.20, 5.83, 4.27, 7.71, 7.12, 7.25, 4.63, 5.31, 3.70~
## $ After8weeks <dbl> 5.75, 6.13, 5.71, 4.15, 7.67, 7.05, 7.10, 4.67, 5.33, 3.66~
## $ Margarine   <chr> "B", "A", "B", "A", "B", "A", "B", "A", "B", "A", "B", "B"~
```

There are two factors of interest here:

- Time points (3 levels within-subjects): before, after4weeks, after8weeks)
- Margarine type (2 level between-subjects): A or B

Note that the data is currently in wide form. We need to convert it to tall form:

```
chol_tall <- chol %>%
  pivot_longer(cols=Before:After8weeks,
    names_to="time.pt",
    values_to="chol.level")
head(chol_tall)

## # A tibble: 6 x 4
##       ID Margarine time.pt      chol.level
##   <dbl> <chr>      <chr>      <dbl>
## 1     1 B        Before         6.42
## 2     1 B    After4weeks         5.83
## 3     1 B    After8weeks         5.75
## 4     2 A        Before         6.76
## 5     2 A    After4weeks         6.2
## 6     2 A    After8weeks         6.13

chol_tall <- chol_tall %>%
  mutate(time.pt = factor(time.pt,
    levels=c("Before", "After4weeks", "After8weeks"),
    labels=c("Before", "After 4 weeks", "After 8 weeks")))
```

Note here we have changed the variable `time.pt` to be a factor and we order the factors levels so they are contextually sequential (rather than alphabetical order).

4.2.2.1 Exploratory Data Analysis

We are now ready for some EDA. A table of descriptive statistics by combinations of time and margarine group is our first cut:

```
chol.summary <- chol_tall %>%
  group_by(Margarine, time.pt) %>%
  summarise(Mean = mean(chol.level),
            SD = sd(chol.level),
            SE = sd(chol.level)/sqrt(length(chol.level)),
            n = length(chol.level))
kable(chol.summary)
```

Margarine	time.pt	Mean	SD	SE	n
A	Before	6.03556	1.363232	0.454411	9
A	After 4 weeks	5.55000	1.320672	0.440224	9
A	After 8 weeks	5.48889	1.307282	0.435761	9
B	Before	6.78000	0.919008	0.306336	9
B	After 4 weeks	6.13333	0.863713	0.287904	9
B	After 8 weeks	6.06889	0.825825	0.275275	9

We generally observe that cholesterol levels with margarine A appear lower than with margarine B. Also, there does appear to be some reduction in cholesterol level using both products: however, there also appears to be more unexplained variability (“noise”) in the cholesterol measurements under margarine A. We can further check this using some data visualizations. First let us try boxplots (not appropriate given the design) as seen in Figure ??.

```
ggplot(chol_tall) +
  geom_boxplot(aes(x=time.pt, y=chol.level)) +
  facet_wrap(~Margarine) +
  xlab("Time Point of Measurement") +
  ylab("Cholesterol level (mmol/L)")
```

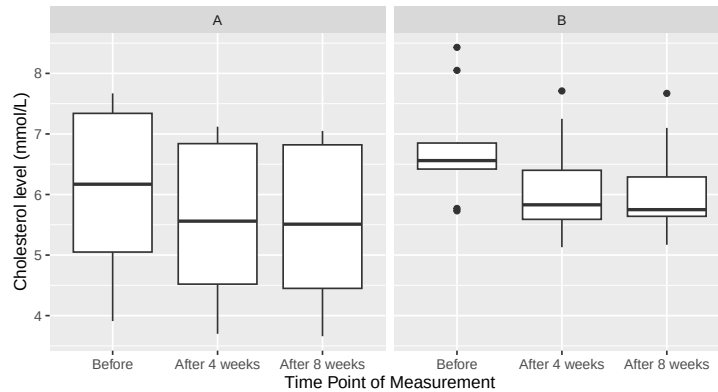


Figure 4.1: Boxplot distribution of the Cholesterol level as a function of Treatment and Time of measurement.

We see in Figure ?? that perhaps cholesterol levels are generally more consistent

under margarine B, and that it is just some outlier(s) that are the issue. However, this plot does not reveal a crucial piece of information: **the responses are correlated over time**. A better depiction would allow us to see how each individual subject's cholesterol changed over the span of the experiment as seen in Figure ??.

```
# Plot the individual-level cholesterol profiles
ggplot(chol_tall, aes(x=time.pt, y=chol.level, colour=Margarine, group=ID)) +
  geom_line() +
  geom_point(shape=21, fill="white") +
  facet_wrap(~Margarine) +
  xlab("Time Point of Measurement") +
  ylab("Cholesterol level (mmol/L)") +
  ggtitle("Subject-level cholesterol profiles by Margarine group")
```

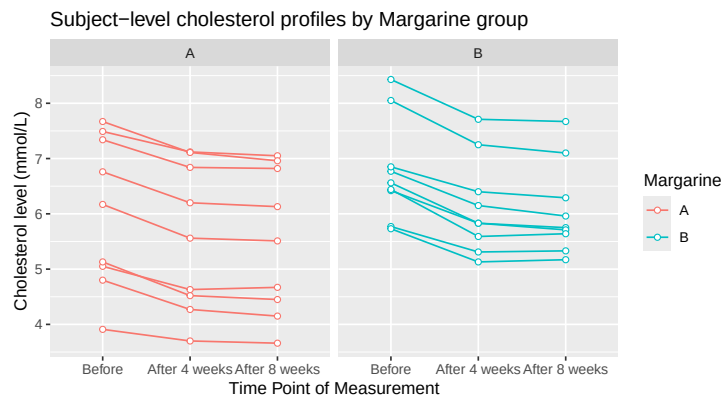


Figure 4.2: Profile plots of the Cholesterol level as a function of Treatment and Time of measurement.

In Figure ?? we see some things more clearly. The general pattern over time appears to be pretty consistent for every experimental unit (visually, each “line” in the plot). There are two individuals in margarine B whose cholesterol level seems unusually higher than the others: this might warrant further investigation. Now we provide an summary profile of the two treatments in ??.

```
# we include a position_dodge elements to avoid overlap
ggplot(chol.summary, aes(x=time.pt, y=Mean, colour=Margarine)) +
  geom_errorbar(aes(ymin=Mean-SE, ymax=Mean+SE), width=.1, position=position_dodge(0.3)) +
  geom_line(aes(group=Margarine), position=position_dodge(0.3)) +
  geom_point(position=position_dodge(0.3)) +
  xlab("Time Point of Measurement") +
  ylab("Cholesterol level (mmol/L)") +
  ggtitle("Mean Cholesterol level (with Standard Error bars)")
```

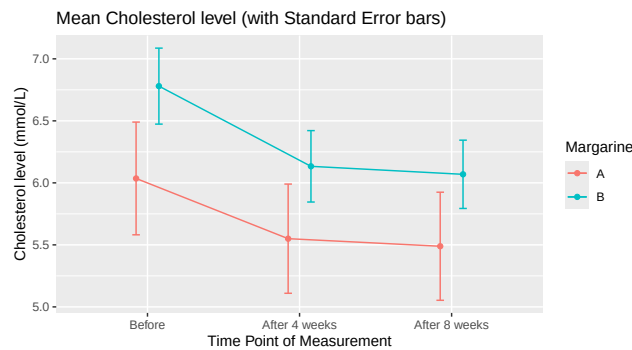


Figure 4.3: Average Profile of Cholesterol level by Treatment in Time.

In the summary version (??) we see that margarine A Cholesterol levels appear lower than margarine B. It is worth noting that the group assigned to margarine level A had lower cholesterol “before” the study began.

4.2.2.2 Preliminary inferential analysis

Of more concern to a fully correct analysis is addressing why there appears to be a difference between the margarine groups at the “Before” stage. At that point, no margarines had been administered, so if the subjects had been truly randomly assigned to the margarine groups (as they should be in a properly designed experiment!), we would have expected consistent cholesterol readings between the groups at that point. This can be formally tested as follows, with a simple independent samples *t*-test comparing the mean cholesterol levels of the A and B groups at the “Before” stage:

```
BeforeDataOnly <- chol_tall %>%
  filter(time.pt == "Before")
t.test(chol.level ~ Margarine, data=BeforeDataOnly, var.equal=TRUE)
```

```
##
## Two Sample t-test
##
## data: chol.level by Margarine
## t = -1.358, df = 16, p-value = 0.193
## alternative hypothesis: true difference in means between group A and group B is not
## 95 percent confidence interval:
## -1.906205 0.417316
## sample estimates:
## mean in group A mean in group B
## 6.03556 6.78000
```

The result is not statistically significant ($p\text{-value} = 0.1932$). That is actually

good news here! Although visually it appears different it is not statistically different.

4.2.2.3 A conditional repeated measures analysis

To get started, let's do a within-subjects (repeated measures) ANOVA on the Margarine A data only. The goal here would be to see if there is an effective reduction in mean cholesterol over the eight-week trial by using margarine A. Here goes: follow the code!

```
MargarineA <- chol_tall %>%
  filter(Margarine == "A")
RMaov_MargA <- aov(chol.level ~ time.pt + Error(factor(ID)/time.pt), data=MargarineA)
summary(RMaov_MargA)

##
## Error: factor(ID)
##           Df Sum Sq Mean Sq F value Pr(>F)
## Residuals  8  42.4      5.3
##
## Error: factor(ID):time.pt
##           Df Sum Sq Mean Sq F value  Pr(>F)
## time.pt    2  1.615   0.808    106 5.8e-10 ***
## Residuals 16  0.122   0.008
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The test of interest is labeled time.pt. It is significant ($F = 106.3$, $df_1 = 2$, $df_2 = 16$, $p\text{-value} = 5.7 \times 10^{-10}$). What this means is that, using Margarine A, there is significant evidence to conclude that the *true mean cholesterol level differs between at least two of the time points*. Of course, that result does not mean that the margarine is effective at reducing cholesterol levels. All it means is that the mean cholesterol level is changing at some point over the experiment. Since there are three time points, a multiple comparison procedure is warranted with the output below and in Figure ??.

```
MargA.mc <- emmeans(RMaov_MargA, ~ time.pt)

## Note: re-fitting model with sum-to-zero contrasts
contrast(MargA.mc, "pairwise")

## contrast                estimate      SE df t.ratio p.value
## Before - After 4 weeks      0.4856 0.0411 16  11.817  <.0001
## Before - After 8 weeks      0.5467 0.0411 16  13.304  <.0001
## After 4 weeks - After 8 weeks 0.0611 0.0411 16   1.487  0.3229
##
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```
plot(contrast(MargA.mc, "pairwise"))
```

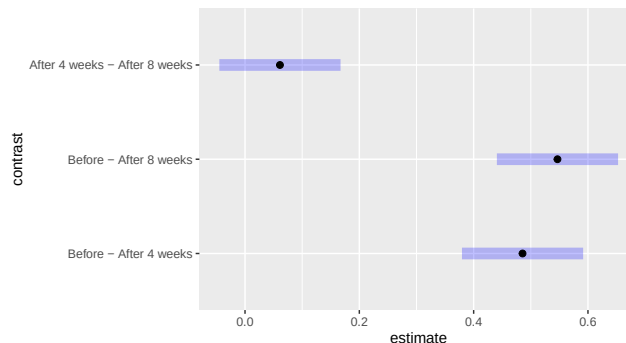


Figure 4.4: Comparing the effects of time within the Margarine A treatment group. We see there is no statistical difference between 4 and 8 weeks after treatment begins.

The significant reductions in mean cholesterol level under Margarine A occurs right after the start of the experiment. The mean cholesterol level is reduced by 0.485 (SE = 0.041) by week 4 (p -value < 0.0001); and by 0.546 (SE = 0.041) by week 8 (p -value < 0.0001). There is no appreciable change in mean cholesterol level between weeks 4 and 8 (p -value = 0.3229).

4.2.2.4 Full analysis: Formal Comparison of Margarines

It is always critically important to know the structure of the data, which here stems from the design of the experiment. In this case, we cannot just “compare the margarines” because there is a subject-level time profile under each margarine. Since the same time points are observed under each margarine group, the factors `margarine` and `time.pt` are crossed effects, which means we have a *factorial data structure* (see section 3.2). The “wrinkle” here is that one of the two factors is a between-subjects factor (Margarine) and the other is within-subjects (`time.pt`). But, we can handle this in `aov`:

```
RMaov_all <- aov(chol.level ~ Margarine + time.pt + Margarine:time.pt +
                 Error(factor(ID)/time.pt), data=chol_tall)
summary(RMaov_all)
```

```
##
## Error: factor(ID)
##           Df Sum Sq Mean Sq F value Pr(>F)
## Margarine  1    5.5    5.46    1.45  0.25
## Residuals 16   60.4    3.78
##
## Error: factor(ID):time.pt
```



```
##              Df Sum Sq Mean Sq F value Pr(>F)
## time.pt      2   4.32   2.160  259.49 <2e-16 ***
## Margarine:time.pt  2   0.08   0.040    4.78  0.015 *
## Residuals     32   0.27   0.008
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

As with the factorials models before, the first thing we must check is the significance of the interaction term. If this is significant, it means that the effect of time on the mean cholesterol is different between the two margarines. That is the case here ($F = 4.777$, $df_1 = 2$, $df_2 = 32$, $p\text{-value} = 0.0153$). As a result, we should do our multiple comparisons **by conditioning on one of the two factors**. Experimental context will dictate which of the two factors is the more reasonable to condition on. Here, I choose to condition on Margarine group, so I will get a time-based comparison of mean cholesterol level, but specific to each Margarine group:

```
RMall.mc <- emmeans(RMaov_all, ~ time.pt | Margarine)
```

```
## Note: re-fitting model with sum-to-zero contrasts
```

```
contrast(RMall.mc, "pairwise")
```

```
## Margarine = A:
```

```
## contrast                estimate      SE df t.ratio p.value
## Before - After 4 weeks      0.4856 0.043 32  11.290 <.0001
## Before - After 8 weeks      0.5467 0.043 32  12.711 <.0001
## After 4 weeks - After 8 weeks 0.0611 0.043 32   1.421  0.3424
##
```

```
## Margarine = B:
```

```
## contrast                estimate      SE df t.ratio p.value
## Before - After 4 weeks      0.6467 0.043 32  15.036 <.0001
## Before - After 8 weeks      0.7111 0.043 32  16.535 <.0001
## After 4 weeks - After 8 weeks 0.0644 0.043 32   1.498  0.3051
##
```

```
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```
plot(contrast(RMall.mc, "pairwise"))
```

Even though at first glance the time-based comparisons look about the same between the two margarines, the point estimates and plots reveal the story: the reductions in mean cholesterol following the initiation of the study are larger for margarine B (mean=0.7111 mmol/L, SE=0.043 by week 8) than for margarine A (mean=0.546 mmol/L, SE=0.043 by week 8).

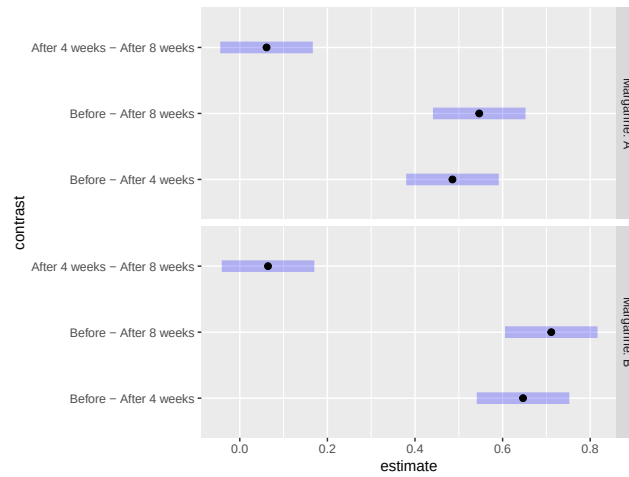


Figure 4.5: Graphical exploration of all treatments noting that there is no statistical difference between 4 and 8 weeks after the study has began regardless of Margarine treatment.

4.2.3 Further study

More thorough analysis of within-subjects and repeated measures designs are possible using advanced methods beyond the scope of this course. These methods include:

- processes that allow the analyst to estimate the degree of correlation between repeated measurements
- processes that allow one to model different correlation structures based upon experimental realities in the data collection process

For example, suppose we had a repeated measures experiment with subjects measured across four time points: Week 1, 2, 3 and 8. Because of the time spacing, we might expect that measurements between adjacent weeks to be more similar within a subject than weeks far removed in time. For example, it is reasonable to expect that results in week 2 are more strongly correlated to week 1 results than would be the correlation between weeks 3 and 1. Week 8's results might be so far removed from the first few weeks that the results there might be safely presumed to be nearly independent of earlier results. We could even be more restrictive, using an approach that assumes only adjacent time point results are correlated, but beyond that, assume independence.

Subtleties like this can be modeled and checked using advanced modeling techniques.

Chapter 5

Introduction to Multiple Regression

In the early chapters of our text we observed that everything varies. In a statistical model, the goal is to identify structure in the variation that the data possess. This means that we must partition the variation in the data into (1) a systematic component, and (2) a non-systematic, or random, component. After this unit, you should be able to

- Recognize the structure of a multiple regression model.
- Fit a regression model in R.
- Interpret the coefficients in a regression model.
- Distinguish between the model constructed in design of experiments (ANOVA) and that in regression.

In ANOVA testing, the systematic component is comprised of measured factors of research interest that may or may not relate to the response variable. The random component is usually a catch-all for everything else: if the model is built well, there should be no systematic information left in the random component: rather, it should only contain random fluctuations due to the natural inherent variability in the measurements.

Suppose instead of factors (categorical inputs) we have measured predictor variables. For example, consider the admissions data from Chapter 1 of this text; there may be a systematic tendency to see higher freshman GPAs from students with higher ACT scores. Both variables are measured numeric values (compared to pre-determined treatments). This leads to the idea of regression modeling.

5.1 Regression Model

The general form of a multiple linear regression (MLR) model is

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

where Y is the response variable, X_j is the j^{th} predictor variable with coefficient β_j and ε is the unexplained random error.

Note that if $\beta_1 = \beta_2 = \dots = \beta_k = 0$ we reduce to our baseline model

$$Y = \beta_0 + \varepsilon = \mu + \varepsilon$$

we saw earlier in the class. The multiple regression model is a generalization of a k factor ANOVA model but instead of categorical inputs, we have numeric, or quantitative, inputs.

Simple Linear Regression. A special case of the above model occurs when $k = 1$ and the model reduces to

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

This is known as the simple linear regression (SLR) model (covered in your Intro Statistics course). It is very rare that a practicing Statistician will ever fit a SLR but we will utilize it to explain some key concepts.

It should be noted that multiple regression is in general a $k + 1$ dimensional problem, so it will usually not be feasible to graphically visualize a fitted model like we can with SLR (which was 2-dimensional). Not to worry though, as we can quantitatively explain what is going on in higher dimensions. In the following sections we will utilize SLR to help visualize important statistical concepts.

The **goals of linear regression** are:

- Formal assessment of the impact of the predictor variables on the response.
- Prediction of future responses.

These two goals are fundamentally different and may require different techniques to build a model. We outline the fundamental concepts and statistical methods over the next six chapters.

5.2 Fitting a regression model

Regression models are typically estimated through the method of least squares. For the sake of visualizing the concept of least squares, we will consider a SLR example.

5.2.1 Simple Linear Regression Example

Example. Muscle Mass with Age (originally in ?).

A person's muscle mass is expected to decrease with age. To explore this relationship in women, a nutritionist randomly selected 15 women from each 10-year

age group beginning with age 40 and ending with age 79. The data reside in the file `musclemass.txt`. The variables in the dataset of interest are `mass` and `age`.

First input the data and take a look at the observations.

```
site <- "https://tjfisher19.github.io/introStatModeling/data/musclemass.txt"
muscle <- read_table(site, col_type=cols())
glimpse(muscle)

## Rows: 60
## Columns: 2
## $ mass <dbl> 106, 106, 97, 113, 96, 119, 92, 112, 92, 102, 107, 107, 102, 115, ~
## $ age <dbl> 43, 41, 47, 46, 45, 41, 47, 41, 48, 48, 42, 47, 43, 44, 42, 55, 5~
```

Figure ?? below displays the data.

```
ggplot(muscle) +
  geom_point(aes(x=age, y=mass)) +
  labs(title="Muscle Mass vs Age",
       x="Age (years)", "Muscle Mass (kg)") +
  theme_minimal()
```

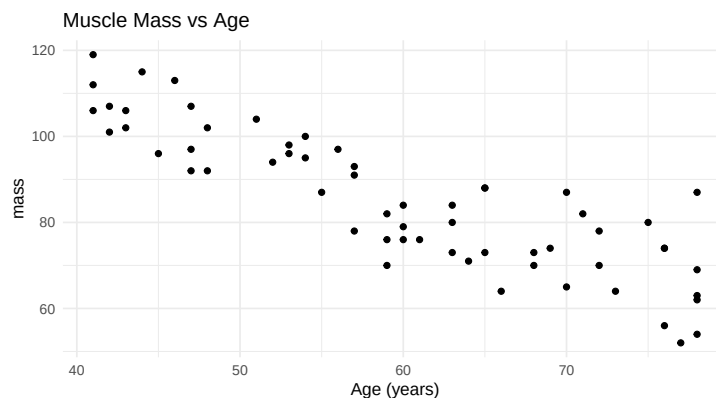


Figure 5.1: Scatterplot showing the relationship between age and muscle mass in a selection of women aged 40 to 79.

We can clearly see the negative trend one would expect: as age increases, muscle mass tends to decrease. You should also notice that it decreases in a roughly linear fashion, so it makes sense to fit a simple linear regression model to this data.

The systematic component of a simple linear regression model passes a straight line through the data in an attempt to explain the linear trend (see Figure ??). We can see that such a line effectively explains the trend, but it does not explain

it perfectly since the line does not touch all the observed values. The random fluctuations around the trend line are what the ε terms account for in the model.

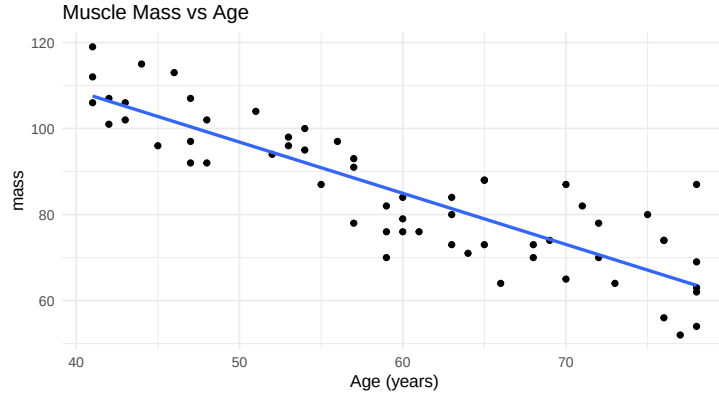


Figure 5.2: Scatterplot showing the relationship between age and muscle mass in a selection of women aged 40 to 79 with a fitted overlaid simple linear regression line.

The next goal is to somehow find the best fitting line for this data. There are an infinite number of possible straight line models of the form $Y = \beta_0 + \beta_1 X + \varepsilon$ that we could fit to a data set, depending on the values of the slope β_1 and y-intercept β_0 of the line. Given a scatterplot, how do we determine which slope and y-intercept produces the “best fitting” line for a given data set? Well, first we need to define what we mean by “best”.

Our criterion for finding the best fitting line is rooted in the residuals that the line would produce. In the two-dimensional simple linear regression case, it is easy to visualize this. When a straight line is “fit” to a data set, the fitted (or “predicted”) values for each observation fall on the fitted line (see Figure ??). However, the actual observed values randomly scatter around the line. The vertical discrepancies between the observed and predicted values are the residuals we spoke of earlier. We can visualize this by zooming into the plot.

It makes some logical sense to use a criterion that somehow collectively minimizes these residuals, since the best fitting line should be the one that most closely passes through the observed data. We need to estimate β_0 and β_1 for this “best” line.

Also note in the Figure ?? that any reasonable candidate model must pass through the data, producing both positive residuals (actual response values $>$ fitted response values) and negative residuals (actual response values $<$ fitted response values). When we collectively assess the residuals, we do not want the positive ones to cancel out or offset the negative ones, so our criterion will be to minimize the sum of squared residuals (thus all are positive). This brings us

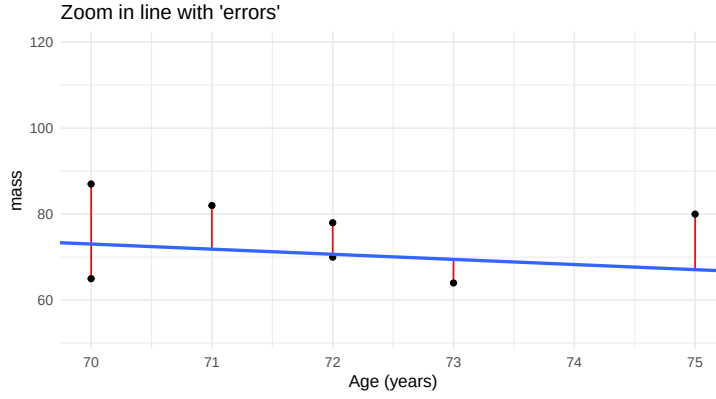


Figure 5.3: Zoomed in view of fitted regression line modeling the relationship of age and mass muscle mass, highlighting the residuals of the fit.

to what is known as the method of least squares (LS), which is outlined below in the context of simple linear regression.

Method of Least Squares

We propose to “fit” the model $Y = \beta_0 + \beta_1 X + \varepsilon$ to a data set of n pairs: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. The goal is to optimally estimate β_0 and β_1 for the given data.

Denote the estimated values of β_0 and β_1 by b_0 and b_1 , respectively. Note that it is also common to denote the estimated values as $\hat{\beta}_0$ and $\hat{\beta}_1$.

The fitted values of Y are found via the linear equation $\hat{Y} = b_0 + b_1 X$ (or $\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X$). In terms of each individual (x_i, y_i) sample observation, the fitted and observed values are found as follows:

Fitted (predicted) values	Observed (actual) values
$\hat{y}_1 = b_0 + b_1 x_1$	$y_1 = b_0 + b_1 x_1 + e_1$
$\hat{y}_2 = b_0 + b_1 x_2$	$y_2 = b_0 + b_1 x_2 + e_2$
$\vdots$	$\vdots$
$\hat{y}_n = b_0 + b_1 x_n$	$y_n = b_0 + b_1 x_n + e_n$

The difference between each corresponding observed and predicted value is the sample residual for that observation:

$$\begin{aligned}
 e_1 &= y_1 - \hat{y}_1 \\
 e_2 &= y_2 - \hat{y}_2 \\
 &\vdots \\
 e_n &= y_n - \hat{y}_n
 \end{aligned}$$

or in general, $e_i = y_i - \hat{y}_i$. The method of least squares determines b_0 and b_1 so that

$$\text{Residual sum of squares (RSS)} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (b_0 + b_1 x_i))^2$$

is a minimum. In other words, any other method of estimating the y -intercept and slope, β_0 and β_1 , respectively, will produce a larger RSS value than the method of least squares.

Minimizing RSS is a calculus exercise, so we will skip the details here. The resulting line is the “best-fitting” straight line model we could possibly obtain for the data.

- b_0 and b_1 are called the least squares estimates of β_0 and β_1 .
- The line given by $\hat{Y} = b_0 + b_1 X$ is called the simple linear regression equation.

We use R to fit such models and estimate β_0 and β_1 using the `lm()` function (`lm`=linear model). No hand calculations required! Linear models are fit using the R function `lm()`, and the basic format for a formula is given by `response ~ predictor`.

The `~` (“tilde”) here is read as “is modeled as a linear function of” and is used to separate the response variable from the predictor variable(s). For simple linear regression, the form is `lm(y ~ x)`. In other words, `lm(y ~ x)` fits the regression model $Y = \beta_0 + \beta_1 x + \varepsilon$.

The y -intercept is always included, unless you specify otherwise. `lm()` creates a model object containing essential information about the fit that we can extract with other R functions. We illustrate via an example involving the muscle mass dataset.

```
muscle.fit <- lm(mass ~ age, data=muscle)
glimpse(muscle.fit)
```

```
## List of 12
## $ coefficients : Named num [1:2] 156.35 -1.19
##   .. attr(*, "names")= chr [1:2] "(Intercept)" "age"
## $ residuals    : Named num [1:60] 0.823 -1.557 -3.417 11.393 -6.797 ...
##   .. attr(*, "names")= chr [1:60] "1" "2" "3" "4" ...
## $ effects      : Named num [1:60] -658.15 -107.83 -3.34 11.48 -6.69 ...
##   .. attr(*, "names")= chr [1:60] "(Intercept)" "age" "" "" ...
## $ rank         : int 2
## $ fitted.values: Named num [1:60] 105 108 100 102 103 ...
##   .. attr(*, "names")= chr [1:60] "1" "2" "3" "4" ...
## $ assign       : int [1:2] 0 1
## $ qr           :List of 5
```



```
## ..$ qr      : num [1:60, 1:2] -7.746 0.129 0.129 0.129 0.129 ...
## ..$ attr(*, "dimnames")=List of 2
## ..$ attr(*, "assign")= int [1:2] 0 1
## ..$ qraux: num [1:2] 1.13 1.19
## ..$ pivot: int [1:2] 1 2
## ..$ tol   : num 1e-07
## ..$ rank  : int 2
## ..$ attr(*, "class")= chr "qr"
## $ df.residual : int 58
## $ xlevels     : Named list()
## $ call        : language lm(formula = mass ~ age, data = muscle)
## $ terms       :Classes 'terms', 'formula' language mass ~ age
## ..$ attr(*, "variables")= language list(mass, age)
## ..$ attr(*, "factors")= int [1:2, 1] 0 1
## ..$ attr(*, "dimnames")=List of 2
## ..$ attr(*, "term.labels")= chr "age"
## ..$ attr(*, "order")= int 1
## ..$ attr(*, "intercept")= int 1
## ..$ attr(*, "response")= int 1
## ..$ attr(*, ".Environment")=<environment: R_GlobalEnv>
## ..$ attr(*, "predvars")= language list(mass, age)
## ..$ attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
## ..$ attr(*, "names")= chr [1:2] "mass" "age"
## $ model       :'data.frame': 60 obs. of 2 variables:
## ..$ mass: num [1:60] 106 106 97 113 96 119 92 112 92 102 ...
## ..$ age : num [1:60] 43 41 47 46 45 41 47 41 48 48 ...
## ..$ attr(*, "terms")=Classes 'terms', 'formula' language mass ~ age
## ..$ attr(*, "variables")= language list(mass, age)
## ..$ attr(*, "factors")= int [1:2, 1] 0 1
## ..$ attr(*, "dimnames")=List of 2
## ..$ attr(*, "term.labels")= chr "age"
## ..$ attr(*, "order")= int 1
## ..$ attr(*, "intercept")= int 1
## ..$ attr(*, "response")= int 1
## ..$ attr(*, ".Environment")=<environment: R_GlobalEnv>
## ..$ attr(*, "predvars")= language list(mass, age)
## ..$ attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
## ..$ attr(*, "names")= chr [1:2] "mass" "age"
## - attr(*, "class")= chr "lm"
```

You'll note from the `glimpse()` function there are many attributes inside the `lm` object. We will utilize many of these in our exploration of regression models in the coming chapters.

5.2.2 Multiple Regression Example

To fit a multiple regression, the code is essentially the same. Consider another example.

Example: Property appraisals (example from ?).

Suppose a property appraiser wants to model the relationship between the sale price (`saleprice`) of a residential property and the following three predictor variables:

- `landvalue` - Appraised land value of the property (in \$)
- `impvalue` - Appraised value of improvements to the property (in \$)
- `area` - Area of living space on the property (in sq ft)

The data are in the R workspace `appraisal.RData` in our repository. Lets use R to fit what is known as a “main effects” multiple regression model. The form of this model is given by:

$$\text{Sale Price} = \beta_0 + \beta_1(\text{Land Area}) + \beta_2(\text{Improvement Value}) + \beta_3(\text{Area}) + \varepsilon$$

Note the model is just an extension of the simple linear regression model but with three predictor variables. Similar to our study of ANOVA modeling we follow a standard pattern for analysis:

1. Describe the data both numerically and graphically.
2. Fit a model.
3. Once satisfied with the fit, check the regression assumptions.
4. Once the assumptions check out, use the model for inference and prediction.

Before embarking on addressing each part, it might be instructive, just once (since this is your first multiple regression encounter), to actually look at the raw data to see its form:

```
load(url("https://tjfisher19.github.io/introStatModeling/data/appraisal.RData"))
kable(appraisal)
```

saleprice	landvalue	impvalue	area
68900	5960	44967	1873
48500	9000	27860	928
55500	9500	31439	1126
62000	10000	39592	1265
116500	18000	72827	2214
45000	8500	27317	912
38000	8000	29856	899
83000	23000	47752	1803
59000	8100	39117	1204
47500	9000	29349	1725
40500	7300	40166	1080
40000	8000	31679	1529
97000	20000	58150	2455
45500	8000	23454	1151
40900	8000	20897	1173
80000	10500	56248	1960
56000	4000	20859	1344
37000	4500	22610	988
50000	3400	35948	1076
22400	1500	5779	962

We see there are four variables ($k + 1 = 4$) and 20 observations ($n = 20$). Each row contains a different member of the sample (in this case, a different property). Notice the one property with the relatively high selling price as compared to the others.

Pairwise scatterplots are given below to visualize the bivariate associations. Here we use the `ggpairs()` function in the add-on package `GGally` (?). Pairwise scatterplots provide a means to visually explore all $k + 1$ dimensions of a dataset, but note that as k and n (the sample size) increase, these plots can get very “busy”.

```
ggpairs(appraisal, columns=c(2:4,1) )
```

Note that using the `columns=c(2:4,1)` option, we have reordered the variables for the plot (generally, it is best to have the response variable last so it appears on the y -axis in the scatterplots). In the bottom left of the matrix of plots in Figure ?? we have pairwise scatterplots. At first glance, it seems as though each of the three predictors positively relates to sales price. However, we also get to see the plots of predictors against themselves. This can be highly informative and will be of some importance to us later on. There appear to be positive associations between all the predictors (not surprisingly given the context). It is also instructive to note that three specific individual properties with high appraised land values seem to be the catalyst for these apparent associations.

In the upper right corner of Figure ?? are the Pearson correlation coefficients

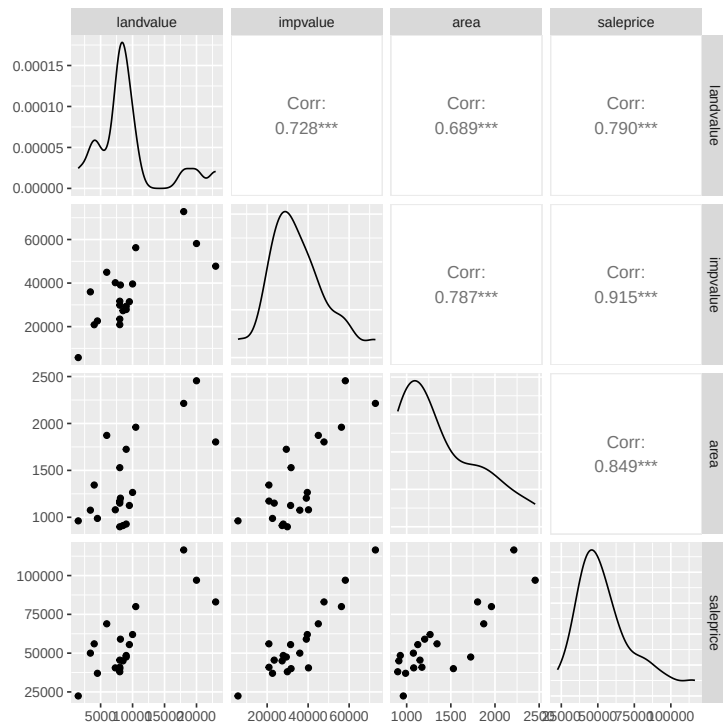


Figure 5.4: Pairs plot showing the relationship between the variables ‘landvalue’, ‘impvalue’, ‘area’ and ‘saleprice’.

(which measures the amount of linear relationship between the two variables) and along the diagonal are density plots of each variable (providing some information about each variables shape).

We now fit the “main effects” MLR model for predicting $Y = \text{saleprice}$ from the three predictors. Initially, we might be interested in seeing how individual characteristic(s) impact sales price.

```
appraisal.fit <- lm(saleprice ~ landvalue + impvalue + area, data=appraisal)
summary(appraisal.fit)
```

```
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025   2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1384.197   5744.100    0.24  0.8126
## landvalue      0.818     0.512    1.60  0.1294
## impvalue       0.819     0.211    3.89  0.0013 **
## area          13.605     6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF, p-value: 3.87e-08
```

Here are some observations from this regression model fit:

Parameter estimates. The fitted model, where $Y = \text{sale price}$, is:

$$\hat{Y} = 1384.197 + 0.818(\text{Land value}) + 0.819(\text{Improvement value}) + 13.605(\text{Area})$$

That is, the least squares estimates of the four β -parameters in the model are $b_0 = 1384.2$, $b_1 = 0.817$, $b_2 = 0.819$, and $b_3 = 13.605$. We will discuss their interpretation later.

Residual standard error. In regression, the error variance σ^2 is a measure of the variability of all possible population response Y -values around their corresponding predicted values as obtained by the true population regression equation. It is called “error” variance because it deals with the difference between

true values vs. model-predicted values, and hence can be thought of as measuring the “error” one would incur by making predictions using the model.

Since a variance is always a sum of squares divided by degrees of freedom (SS/df), we can estimate σ^2 for a simple linear regression model using the following:

$$S_\varepsilon^2 = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - p} = \frac{\text{RSS}}{n - p}.$$

The degrees of freedom for this error variance estimate is $n - p$ where p is the number of parameters estimated in the regression model, here $p = k + 1 = 3 + 1 = 4$. So, we had to “spend” $p = k + 1$ degrees of freedom from the n available degrees of freedom in order to estimate $\beta_0, \beta_1, \dots, \beta_k$. (Hopefully you can see that you cannot fit a model with n or more parameters to a sample of size n ; you will have “spent” all your available degrees of freedom, and hence you could not estimate σ^2 .)

We will usually just denote the estimated error variance using S^2 instead of S_ε^2 . R provides the residual standard error in the `summary()` output from the linear model fit, which is the square root of the estimated error variance, and thus has the advantage of being in the original units of the response variable Y . Here, $s = 7915$ which is our estimate for σ . Applying an Empirical Rule type argument (remember from Intro Statistics!), we could say that approximately 95% of this model’s predicted sale prices would fall within $\pm 2(7915) = \pm \$15,830$ of the actual sales prices. The error degrees of freedom are $n - p = 20 - 4 = 16$.

Interpretation. Since this is essentially the standard deviation of the residuals, we could interpret the value of S as essentially the average residual size; *i.e.*, the average size of the prediction errors produced by the regression model. In the present context, this translates to stating that the regression model produces predicted sale prices that are, on average, \$7,915 dollars off from the actual measured sale price values.

As you can see, such a measure is valuable in helping us determine how well a model performs (*i.e.*, smaller residual error $\rightarrow$ better fitting model). So, S^2 and the standard errors are important ingredients in the development of inference in regression.

It should also be noted that at this point we are not even sure if this is a good model, and if not, how we might make it better. So later on, we will discuss the “art” of model building.

5.3 Interpreting β -parameter estimates in MLR

Suppose we fit a model to obtain the multiple linear regression equation:

$$\hat{Y} = b_0 + b_1 X_1 + b_2 X_2 + \dots + b_k X_k$$

What does b_1 mean? In multiple regression involving simultaneous assessments of many predictors, interpretation can become problematic. In certain cases, a b_i coefficient might represent some real physical constant, but oftentimes the statistical model is just a convenience for representing a more complex reality, so the real meaning of a particular b_i may not be obvious.

At this point in our trek through statistical model, it is important to remember that there are two methods for obtaining data for analysis: designed experiments and observational studies. It is important to recall the distinction because each type of data results in a different approach to interpreting the β -parameter estimates in a multiple linear regression model.

5.3.1 Designed experiments

In a designed experiment, the researcher has control over the settings of the predictor variables $X_1, X_2, \dots, X_k$. For example, suppose we wish to study several physical exercise regimens and how they impact calorie burn. The experimental units (EUs) are the people we use for the study. We can control some of the predictors such as the amount of time spent exercising or the amount of carbohydrate consumed prior to exercising. Some other predictors might not be controlled but can be measured, such as baseline metabolic variables. Other variables, such as the temperature in the room or the type of exercise done, could be held fixed.

Having control over the conditions in an experiment allows us to make stronger conclusions from the analysis. One important property of a well-designed experiment is called orthogonality. Orthogonality is useful because it allows us to easily interpret the effect one predictor has on the response without regard to any others. For example, orthogonality would permit us to examine the effect of increasing $X_1 = \text{time spent exercising}$ on $Y = \text{calorie burn}$, without any concern for $X_2 = \text{carbohydrate consumption}$. This can only occur in a situation where the predictor settings are judiciously chosen and assigned by the experimenter. Let us look at an example.

5.3.2 Example of Orthogonal Design

Example. Cleaning experiment (from ?).

An experiment was performed to measure the effects of three predictors on the ability of a cleaning solution to remove oil from cloth. The data are in the R workspace `cleaningexp.RData`. Here are some details:

Response:

- `pct.removed` - Percentage of the oil stain removed

Predictors:

- `soap.conc` - Concentration of soap, in % by weight

- `lauric.acid` - Percentage of lauric acid in the solution
- `citric.acid` - Percentage of citric acid in the solution

Soap concentration was controlled at two levels (15% and 25%), lauric acid at four levels (10%, 20%, 30%, 40%), and citric acid at three levels (10%, 12%, 14%). Each possible combination of the three predictors was tested on five separate stained cloths, for a total of $5 \times 2 \times 4 \times 3 = 120$ measurements. We want to illustrate the effect of orthogonality on the β -parameter estimates.

```
load(url("https://tjfisher19.github.io/introStatModeling/data/cleaningexp.RData"))
glimpse(cleaningexp)
```

```
## Rows: 120
## Columns: 5
## $ soap.conc    <int> 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15~
## $ lauric.acid   <int> 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10~
## $ citric.acid   <int> 10, 10, 10, 10, 10, 12, 12, 12, 12, 12, 14, 14, 14, 14, 14~
## $ rep          <int> 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5~
## $ pct.removed   <dbl> 20.5, 14.1, 19.1, 20.8, 18.7, 21.2, 22.7, 20.3, 23.9, 20.1~
```

Since we have many numeric levels and we are interested in the numeric association (as compared to the categorical association in ANOVA), we fit a linear model to all three predictors:

```
summary(lm(pct.removed ~ soap.conc + lauric.acid + citric.acid, data=cleaningexp))
```

```
##
## Call:
## lm(formula = pct.removed ~ soap.conc + lauric.acid + citric.acid,
##     data = cleaningexp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.084 -1.995  0.045  2.078  8.787
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -49.6392     2.4169  -20.5   <2e-16 ***
## soap.conc    2.1987     0.0554   39.6   <2e-16 ***
## lauric.acid   0.8629     0.0248   34.8   <2e-16 ***
## citric.acid   2.5956     0.1698   15.3   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.04 on 116 degrees of freedom
## Multiple R-squared:  0.963, Adjusted R-squared:  0.962
## F-statistic: 1.01e+03 on 3 and 116 DF, p-value: <2e-16
```


Take note of the model's β -parameter estimates and their SEs.

Now for illustration only, let's drop `soap.conc` from the model:

```
summary(lm(pct.removed ~ lauric.acid + citric.acid, data=cleaningexp))

##
## Call:
## lm(formula = pct.removed ~ lauric.acid + citric.acid, data = cleaningexp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.15  -11.04   -1.78   11.45   19.78
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -5.6658     8.1577  -0.69   0.4887
## lauric.acid   0.8629     0.0942   9.16 2.1e-15 ***
## citric.acid   2.5956     0.6449   4.02  0.0001 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11.5 on 117 degrees of freedom
## Multiple R-squared:  0.461, Adjusted R-squared:  0.452
## F-statistic: 50.1 on 2 and 117 DF,  p-value: <2e-16
```

Notice that the coefficient estimates do not change regardless of what other predictors are in the model (this would hold true if we dropped `lauric.acid` or `citric.acid` from the model as well - try it!). This is what orthogonality ensures. This means that we are safe in assessing the size of the impact of, say, soap concentration on the ability to remove the oil stain, without worrying about how other variables might impact the relationship.

So in a designed experiment with orthogonality properties, we can interpret the value of b_1 unconditionally as follows:

A one-unit increase in x_1 will cause a change of size b_1 in the mean response.

Side note. When we deleted the predictors one at a time, we effectively were taking that predictor's explanatory contribution to the response and dumping it into the error component of the model. Here, since each predictor was significant (p -value < 0.0001), this removal caused the residual standard error to increase substantially, which subsequently made the SEs of the coefficients, t -statistics and p -values change. We want to make it clear that **in practice it is not recommended to remove significant effects from the model** — it was only done here to demonstrate that orthogonality ensures that the model's β -parameter estimates are unchanged regardless of what other predictors are included. However, the results of tests/CIs for those coefficients may change

depending on what is included in the model (if you remove an insignificant predictor, the residual SE will change only slightly and hence have negligible impact on SEs of the coefficients, t -statistics and p -values).

5.3.3 Observational studies

In most regression settings, you simply collect measurements on predictor and response variables as they naturally occur, without intervention from the data collector. Such data is called observational data.

Interpreting models built on observational data can be challenging. There are many opportunities for error and any conclusions will carry with them substantial unquantifiable uncertainty. Nevertheless, there are many important questions for which only observational data will ever be available. For example, how else would we study something like differences in prevalence of obesity, diabetes and other cardiovascular risk factors between different ethnic groups? Or the effect of socio-economic status on self esteem? It is impossible to design experiments to investigate these since we cannot control variables (or it would be grossly unethical to do so), so we must make the attempt to build good models with observational data in spite of their shortcomings.

In observational studies, establishing causal connections between response and predictor variables is nearly impossible. In the limited scope of a single study, the best one can hope for is to establish associations between predictor variables and response variables. But even this can be difficult due to the uncontrolled nature of observational data. **Why?** It is because unmeasured and possibly unsuspected “lurking” variables may be the real cause of an observed relationship between response Y and some predictor X_i . Recall the earlier example where we observed a positive correlation between the heights and mathematical abilities of school children? It turned out that this relationship was really driven by a lurking variable – the age of the child. In this case, the variables height and age are said to be confounded, because for the purpose of predicting math ability in children, they basically measure the same predictive attribute.

In observational studies, it is important to adjust for the effects of possible confounding variables. Unfortunately, one can never be sure that the all relevant confounding variables have been identified. As a result, one must take care in interpreting β -parameter estimates from regression analyses involving observational data.

Here is probably the best way of interpreting a β -parameter estimate (say b_1) when dealing with observational data:

b_1 measures the effect of a one-unit increase in x_1 on the mean response when all the other (specified) predictors are held constant.

Even this, however, is not perfect. Often in practice, one predictor cannot be changed without changing other predictors. For example, competing species of ground cover in a botanical field study are often negatively correlated, so

increasing the amount of cover of one species will likely mean the lowering of cover of the other. In health studies, it is unrealistic to presume that an increase of 1% body fat in an individual would not correlate to changes in other physical characteristics too (*e.g.*, waist circumference).

Furthermore, this interpretation requires the specification of the other variables – so changing which other variables are included in the model may change the interpretation of b_1 . Here's an illustration:

5.3.4 Example of observational regression interpretation

Example: Property appraisals (from ?).

We earlier fit a full “main effects” model predicting $Y = \text{salesprice}$ from three predictor variables dealing with property appraisals. This is an observational study, since the predictor values are not set by design by the researcher. Here's a brief recap of the fitted model:

```
summary(appraisal.fit)

##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025   2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1384.197    5744.100    0.24  0.8126
## landvalue      0.818      0.512    1.60  0.1294
## impvalue       0.819      0.211    3.89  0.0013 **
## area          13.605      6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF, p-value: 3.87e-08
```

Interpretation. We see $b_3 = 13.605$, this value may be best interpreted in context by either of the following:

- “For each additional square foot of living space on a property, we estimate an increase of \$13.61 in the mean selling price, holding appraised land and improvement values fixed.”
- “Each additional square foot of living space on a property results in an

average increase of \$13.61 in the mean selling price, after adjusting for the appraised value of the land and improvements.”

In Summary: When a predictor’s effect on the response variable is assessed in a model that contains other predictor variables, that predictor’s effect is said to be adjusted for the other predictors.

Now, suppose we delete `landvalue` (an insignificant predictor, p -value > 0.05 ; we will cover this in more detail in the next chapter). How is the model affected?

```
summary(lm(saleprice ~ impvalue + area, data=appraisal))
```

```
##
## Call:
## lm(formula = saleprice ~ impvalue + area, data = appraisal)
##
## Residuals:
```

##	Min	1Q	Median	3Q	Max
##	-15832	-5200	1260	4642	13836

```
##
## Coefficients:
```

##		Estimate	Std. Error	t value	Pr(> t)
##	(Intercept)	-10.191	5931.634	0.00	0.99865
##	impvalue	0.959	0.200	4.79	0.00017 ***
##	area	16.492	6.599	2.50	0.02299 *

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8270 on 17 degrees of freedom
## Multiple R-squared:  0.881, Adjusted R-squared:  0.867
## F-statistic: 63 on 2 and 17 DF, p-value: 1.37e-08
```

Notice that the values of both b_2 and b_3 changed after the predictor X_1 (`landvalue`) was deleted from the model. This means that our estimate of the effect that one predictor has on the response depends on what other predictors are in the model (compare this with the example of orthogonality from earlier).

5.4 Why multiple regression?

We briefly tangent with a discussion of why multiple linear regression is superior to running “separate” simple linear regressions.

Because complex natural phenomena are typically impacted by many characteristics, it would be naive in most circumstances to think that just one variable serves as an adequate explanation for an outcome. Instead, we consider the simultaneous impact of potential predictors of interest on the response. Useful models reflect this fact.

The “one-predictor-at-a-time approach” can be quite bad. Suppose you are considering three potential predictors X_1 , X_2 , X_3 on a response Y . You might be tempted to fit three separate simple linear regression models to each predictor:

$$\begin{aligned} Y &= \beta_0 + \beta_1 X_1 + \varepsilon \\ Y &= \beta_0 + \beta_2 X_2 + \varepsilon \\ Y &= \beta_0 + \beta_3 X_3 + \varepsilon \end{aligned}$$

As we shall see, this approach to regression is fundamentally flawed and is to be avoided at all costs. The problem is that if you fit “too simple” a model you will not account for the “collective” impact of multiple predictors of interest, you may then fail to detect significant relationships, or even come to completely wrong conclusions. Below we provide two examples of the dangers of simple linear regression.

To fully understand these examples, you will need to review ???. For now we highlight the individual regression t -test and residual error to demonstrate the limitations of simple linear regression.

5.4.1 Example of Spurious Correlation

Here is an example to illustrate potential dangers of using a simple linear regression.

Example. Punting Ability in American Football (published in ?)

In the game of American Football, a *punter* will kick a ball to the opposing team as far as possible towards the opponent’s end zone, attempting to maximize the distance the receiving team must advance the ball in order to score. One key feature of a *good punt* is the “hang time,” the amount of time the ball ‘hangs’ in the air before being caught by the punt returner. An experiment was conducted where each of 13 punters kicked the ball 10 times and the experimenter recorded their average hang time and distances, along with other measures of strength and flexibility.

- Punter_id - an identifier for each punter
- Hang_time - The average hangtime
- Distance -
- RLS - right leg strength (pounds)
- LLS - left leg strength (pounds)
- RHF - right hamstring muscle flexibility (degrees)
- LHF - left hamstring muscle flexibility (degrees)
- Power - Overall leg strength (foot-pounds)

Original Source: ?

The data is available in the file `puntingData.csv`.

```

site <- "https://tjfisher19.github.io/introStatModeling/data/puntingData.csv"
punting <- read_csv(site, col_types=cols() )
glimpse(punting)

## Rows: 13
## Columns: 8
## $ Punter_id <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13
## $ Hang_time <dbl> 4.75, 4.07, 4.04, 4.18, 4.35, 4.16, 4.43, 3.20, 3.02, 3.64, ~
## $ Distance <dbl> 162.50, 144.00, 147.50, 163.50, 192.00, 171.75, 162.00, 104.~
## $ RLS <dbl> 170, 140, 180, 160, 170, 150, 170, 110, 120, 130, 120, 140, ~
## $ LLS <dbl> 170, 130, 170, 160, 150, 150, 180, 110, 110, 120, 140, 130, ~
## $ RHF <dbl> 106, 92, 93, 103, 104, 101, 108, 86, 90, 85, 89, 92, 95
## $ LHF <dbl> 106, 93, 78, 93, 93, 87, 106, 92, 86, 80, 83, 94, 95
## $ Power <dbl> 240.57, 195.49, 152.99, 197.09, 266.56, 260.56, 219.25, 132.~

```

First note that this data was collected through an experiment, but the data is **observational** – the experimenter did not control any of the variables of interest.

For our example, we will consider RLS, LLS and Power as potential predictor variables for hang time. Let us look at the scatterplot matrix of the data.

```
ggpairs(punting, columns=c(4,5,8,2) )
```

Based on Figure ?? it appears that all three variables, RLS, LLS and Power, are all moderately or strongly correlated with the hang time. Suppose we fit three simple linear regression models to the data.

```

slr_rls <- lm(Hang_time ~ RLS, data=punting)
slr_lls <- lm(Hang_time ~ LLS, data=punting)
slr_power <- lm(Hang_time ~ Power, data=punting)

```

Look at the summary output of each of the three models.

```

summary(slr_rls)

##
## Call:
## lm(formula = Hang_time ~ RLS, data = punting)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.458 -0.183  0.035  0.198  0.431
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.28437    0.53569    2.40  0.03538 *
## RLS          0.01785    0.00359    4.98  0.00042 ***

```

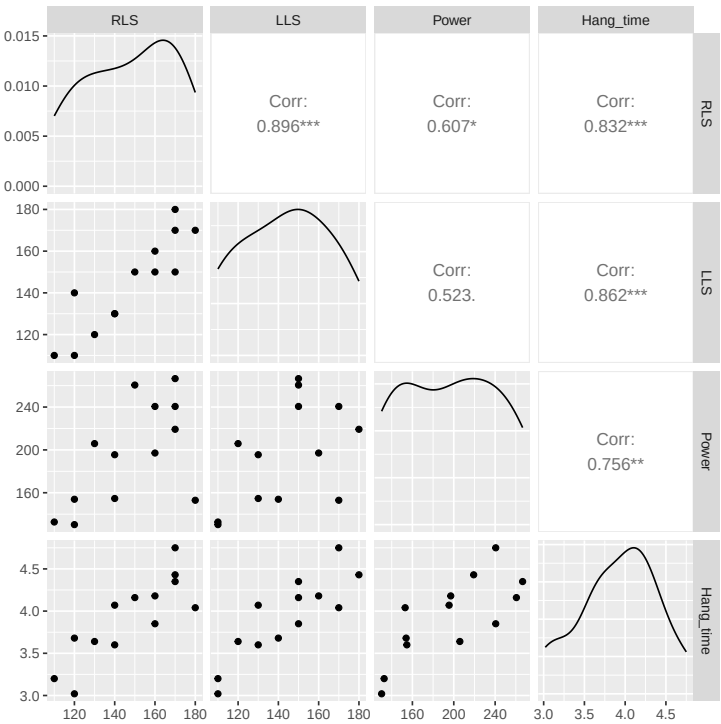


Figure 5.5: Pairs plot showing the relationship between right leg strength, left leg strength, power and hang time.

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.283 on 11 degrees of freedom
## Multiple R-squared:  0.692, Adjusted R-squared:  0.664
## F-statistic: 24.8 on 1 and 11 DF,  p-value: 0.000419
```

```
summary(slr_lls)
```

```
##
## Call:
## lm(formula = Hang_time ~ LLS, data = punting)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.3616 -0.1701 -0.0662  0.1576  0.4038
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.27628     0.47390   2.69  0.02091 *
## LLS          0.01838     0.00326   5.65  0.00015 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.259 on 11 degrees of freedom
## Multiple R-squared:  0.743, Adjusted R-squared:  0.72
## F-statistic: 31.9 on 1 and 11 DF,  p-value: 0.00015
```

```
summary(slr_power)
```

```
##
## Call:
## lm(formula = Hang_time ~ Power, data = punting)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.4138 -0.2583  0.0003  0.2523  0.4862
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.40448     0.40679   5.91  0.0001 ***
## Power        0.00773     0.00202   3.83  0.0028 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.334 on 11 degrees of freedom
## Multiple R-squared:  0.571, Adjusted R-squared:  0.532
```



```
## F-statistic: 14.7 on 1 and 11 DF, p-value: 0.0028
```

Although we have not formally covered inference on regression, we might conclude from these results that each of the predictor variables influences hang time (see the very small p -value associated with the predictor variables). The scatterplots (Figure ??) seems to support this as well.

However, it would be erroneous to conclude that this relationship implies causation (a caution wisely applied to all observational data, as we will discuss later). The sample of punters span a range of abilities, and each might utilize their left or right leg more in the act of kicking. So a logical question to ask is “how do all these variables, collectively, influence hang time?”

We fit a multiple regression predicting hang time from all three predictor variables.

```
mlr_punting <- lm(Hang_time ~ RLS + LLS + Power, data=punting)
summary(mlr_punting)
```

```
##
## Call:
## lm(formula = Hang_time ~ RLS + LLS + Power, data = punting)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-0.3463	-0.0832	0.0044	0.0404	0.3412

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.103924	0.388267	2.84	0.019 *
RLS	0.000232	0.006195	0.04	0.971
LLS	0.013521	0.005744	2.35	0.043 *
Power	0.004270	0.001540	2.77	0.022 *

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.202 on 9 degrees of freedom
## Multiple R-squared:  0.871, Adjusted R-squared:  0.828
## F-statistic: 20.3 on 3 and 9 DF, p-value: 0.000241
```

Compare this result to the simple regression models. When all three variables are included, the variable RLS appears to be unimportant (p -value = 0.9723). The earlier result occurred because both hang time and RLS are highly correlated with LLS – so the omission of LLS led us to detect what is known as a **spurious association** between RLS and hang time. To phrase another way, LLS and RLS appear to be providing much of the same, or redundant, information to the model. We may also consider the addition of LLS and Power to be *confounding* in regards to the variable RLS.

Model	Residual.Standard.Error
SLR: RLS	0.283211
SLR: LLS	0.258640
SLR: Power	0.334334
MLR: RLS+LLS+Power	0.202488

Further, we can see how the multiple regression model improves on the accuracy of the predicted values of the regression model. Consider the table of residual standard errors below.

Note that the multiple regression model has arguably the ‘best’ (smallest) residual standard error of the four models considered – even though it includes the variable RLS which appears to not be important when LLS and Power are included.

The addition of a multiple predictors into the model also alters the nature of the question being asked. The simple linear regression asks: “Does RLS influence hang time?” The multiple linear regression, however, asks a more useful question: “Does RLS influence hang time once any differences due to LHS and Power are considered?” The answer is ‘not really’. It also asks: “Does LLS influence hang time once any differences due to RLS and Power are accounted for?” The answer is ‘yes’.

5.4.2 Example of Confounding in Regression

In the previous section, ??, we saw how a combination of variables can change the interpretation of a variable in the presence of others. In Section ?? we discussed using *blocks* as a way to control for potential confounding factors, or nuisance variables. By modeling this extra (in some cases *known*) variability, we can more accurately determine if the variables of interest influence the response variable. In the below example we demonstrate that regression can be used in a similar way as a block design to control for nuisance variation.

Example. Bleaching Pulp (from ?)

The production of paper requires the pupl used in the manufacturing process to be whitened by bleaching in a chemical reaction. The bleaching agents are usually chlorine dioxide (ClO_2) and hydrogen peroxide (H_2O_2). The amounts of these two chemical used in the process effects the whiteness of the pulp, and ultimately that of the paper.

Original source: ?

The file `bleachingPulp.txt` on the textbook site contains the result of an experiment where 20 combinations of chlorine dioxide and hydrogen peroxide were varied and the *brightness* or *whiteness* of the paper was determined (smaller the number, the brighter the paper). The file is a tabbed separated value format

```

site <- "https://tjfisher19.github.io/introStatModeling/data/bleachingPulp.txt"
pulp <- read_tsv(site, col_type=cols())
glimpse(pulp)

## Rows: 20
## Columns: 3
## $ C102      <dbl> 2.06, 2.06, 2.06, 2.06, 2.06, 3.52, 3.52, 3.52, 3.52, 3.52, ~
## $ H2O2      <dbl> 0.0, 0.2, 0.4, 0.6, 0.8, 0.0, 0.2, 0.4, 0.6, 0.8, 0.0, 0.2, ~
## $ Whiteness <dbl> 10.55, 9.59, 8.98, 8.46, 7.73, 6.16, 5.55, 5.06, 4.79, 4.74, ~

```

Suppose a paper scientist wished to know the effect hydrogen peroxide has on the brightening process. The scientist fits a simple linear regression modeling the whiteness as a function of the amount of hydrogen peroxide.

```

slr_h2o2 <- lm(Whiteness ~ H2O2, data=pulp)
summary(slr_h2o2)

##
## Call:
## lm(formula = Whiteness ~ H2O2, data = pulp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.733 -1.756 -0.755  1.179  4.596
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      5.95        1.01   5.91 1.3e-05 ***
## H2O2             -2.06        2.05  -1.00   0.33
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.6 on 18 degrees of freedom
## Multiple R-squared:  0.0527, Adjusted R-squared:  7.12e-05
## F-statistic:      1 on 1 and 18 DF,  p-value: 0.33

```

From the summary output it appears that the amount of hydrogen peroxide does not influence the brightness of the paper. Further, we note the residual standard error is 2.59922 which is not all that different than the standard deviation of the `Whiteness` variable, 2.59931.

However, the above analysis is **incorrect** because we do not account for the potential effect of the chlorine dioxide (a **confounding** variable) – the experiment used both. When we fit a multiple regression model we get a different result.

```

mlr_pulp <- lm(Whiteness ~ H2O2 + C102, data=pulp)
summary(mlr_pulp)

```

```
##
## Call:
## lm(formula = Whiteness ~ H2O2 + ClO2, data = pulp)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.1019 -0.7244 -0.0817  0.6817  1.5107
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   11.939     0.626   19.08 6.4e-13 ***
## H2O2          -2.056     0.712   -2.89  0.01 *
## ClO2          -1.407     0.122  -11.53 1.9e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.901 on 17 degrees of freedom
## Multiple R-squared:  0.893, Adjusted R-squared:  0.88
## F-statistic: 70.6 on 2 and 17 DF, p-value: 5.83e-09
```

Based on the multiple regression model, it appears hydrogen peroxide does predict the brightness of the pulp. Also note the decrease in the residual standard error.

For reference, the residual standard error for the simple linear regression with just chlorine dioxide is 1.06879. So it is apparent that both predictor variables provide a better mode than either simple linear regression approach.

5.5 Concluding the Multiple Regression Model

Hopefully the examples outlined in this chapter will give you an appreciation for why we must strive to find a good model for our data, and **not** *fit once* and “hope for the best”. In observational studies, many different models may be fit to the same data, but that does not mean they are all good! Findings and results can vary based on which model we choose: precision of our predictions, interpretations of the parameter estimates, etc., so we must use caution and wisdom in our choices. In the following chapters we will cover methods that will help us in our model building.

As statistician George Box once said (?):

“All models are wrong but some are useful.”

Chapter 6

Inference regarding Multiple Regression

We now start the discussion of using the least squares simple linear regression model for the purpose of statistical inference about the parent population from which the sample was drawn. Remember that by “inference,” we mean finding relevant confidence intervals or running hypothesis tests about various aspects of the modeled relationship. After this unit, you should be able to

- Statistically determine if a model significantly predicts a response variable.
- Test each predictor variable’s ability to significantly predict the response and measure its impact with a confidence interval.
- Determine the amount of variation in the response variable explained by the model.

6.1 Assumption checking

Any time we run a hypothesis test or build a confidence interval in the context of a regression analysis, the validity of the findings depends on several assumptions being met. These assumptions need to be checked **in advance** using regression diagnostics. Think of diagnostics as preventative medicine for your data.

The five collective assumptions for a standard regression models are as follows:

- **Linearity assumption:** We assume that the structural part of the linear regression model is correctly specified.
- **Error assumptions:**
 1. The errors are independent (the independence assumption)
 2. The errors have homogeneous variance (the constant variance assumption)
 3. The errors are normally distributed (the normality assumption)

- **Unusual observations:** Occasionally, a few observations may not fit the model well. These have the potential to dramatically alter the results and may cause violations to the assumptions above. So we should check for them and investigate their validity.

You'll note these assumptions are nearly identical to those we discussed in experimental design (see ??). We will diagnose the assumptions in a similar fashion.

Recall from earlier that the general form of a model is

$$\text{Data} = \text{Systematic Structure} + \text{Random Variation}$$

Since modeling seeks to identify structure in the data, the goal of a good model is to capture most (if not all) of the structure in the first partition, leaving nothing but random noise in the “leftovers”. *All of our assumptions deal, in one way or another, with studying the random variation component via inspection of the residuals from a fitted model.* The motivation is that if all things are satisfactory, there should be no structure, pattern or systematic behavior remaining in the errors.

Diagnostic techniques can be either graphical or numerical, but **we will focus on graphical diagnostics of the assumptions**. *The important thing to keep in mind when you fit a regression model is that the first model you try might prove to be inadequate. Regression diagnostics often suggest improvements or remedies, which means that model building is an iterative and interactive process.* It is quite common to repeat diagnostics on a succession of models fit to the same data.

This section is only meant to provide you with the means to run a cursory check of the assumptions in a rudimentary linear regression problem. We will deal with specific issues and remedies for addressing assumption violations in more detail later in the text.

6.1.1 Example of Assumption Checking in SLR

Example. Muscle Mass with Age (originally in ?).

Consider the age and muscle mass dataset from the previous chapter (?). Before using the muscle mass regression model to perform tests/CIs about the mass vs. age relationship, check the regression assumptions. This can be achieved by using the `autoplot()` function on the fitted `lm` object and results in Figure ??.

```
site <- "https://tjfisher19.github.io/introStatModeling/data/musclemass.txt"
muscle <- read_table(site, col_type=cols())
muscle.fit <- lm(mass ~ age, data=muscle)

autoplot(muscle.fit)
```

Linearity assumption

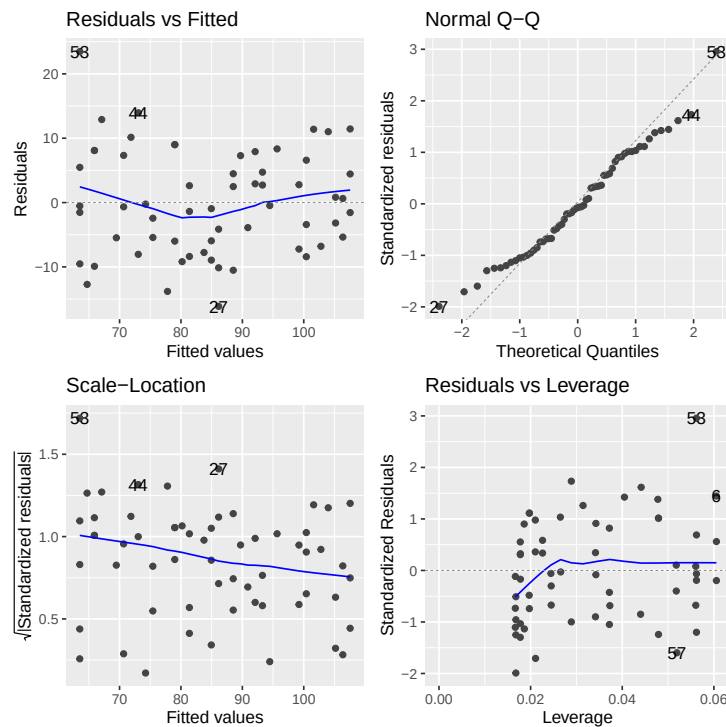


Figure 6.1: Residual plots for the simple linear regression modelling muscle mass as a function of age - Top-left a Residuals vs Fitted Plot; Top-right a Normal Q-Q plot of the residuals; Bottom-left the Scale-Location plot; and Bottom-right a Residuals vs Leverage plot.

We have fit a straight-line model to the data. If the actual trend that relates muscle mass to age is linear, then this way of defining the structural part of the model should adequately explain all the systemic trends in the data. If it is not adequate, then we would still be observing trend in the residuals.

To check this assumption, we return to the Residuals vs Fitted plot. Specifically, the smoothed trend line in this plot (the blue line). If the linearity assumption is met, the residuals should bounce randomly around the $e_i = 0$ line, resulting in a roughly flat (horizontal) smoother. Generally, look for clear systematic patterns in that smoother. The subtle curvature in this line here suggests that we **might** need to fit a model that accommodates curvature in the response/predictor relationship. That is, the linearity assumption here seems to be **mildly** violated.

Error Assumptions

The three assumptions regarding the random noise component (or errors) are the same as those discussed in Section ??.

Independence is determined based on the design of our experiment or data collection procedure – here we must rely on the scientist who conducted this study and assume the sampled individuals were chosen at random and are representative of the target population (adult women).

Constant variance is assessed based on the other three plots – we should see no systematic patterns in the residuals. Except for observation 56 (which may be an outlier), we note no visible pattern to the residuals in any of the plots. So we have no real concern about a violation in the constant variance assumption.

Lastly, the QQ-Plot assesses normality of the ε terms – we wish for the empirical quantiles to closely match the theoretical, resulting in a fairly straight 45-degree line. Here we note some slight deviation from the 45-degree line but nothing overly concerning.

Warning: When first studying and practicing statistics, many practitioners will fixate on a few single observations when exploring these plots. Don't let your eyes fool you and don't fixate on a few points that appear just a *little* different. A little different is typically okay – when they are greatly different, this can be a concern.

Unusual observations

In regression, there are two basic classifications of unusual observations:

1. **Outliers.** These are relatively isolated observations that are poorly predicted by the fitted model; i.e., observations that are “extreme” in the Ys. Outliers can inflate the standard errors of the residuals, resulting in the potential masking of truly significant effects and confidence intervals that are too wide. Outliers are determined by looking at each point's standardized residual value, denoted r_i , and is considered high if $r_i > |3|$.

2. **High-leverage points.** These are observations whose predictor values are far from the center of the predictor space; i.e., observations that are unusually “extreme” in X . High-leverage points have the potential to exert greater influence on the estimation of the β -coefficients in our model. Leverage is measured by something known as a hat value, denoted h_i , and is considered high if $h_i > 2p/n$.

The plot labeled Residuals vs Leverage (in the lower right hand corner of the diagnostic plot quartet) provides a nice visualization of both the standardized residuals and hat values. Potentially suspicious points are flagged in R by their observation number. Observation 53 appears to be a potential outlier with a value close to 3. Since none of the leverage values appear to be above $2p/n = 4/60 = 0.0667$, we do not have any high leverage points.

We will discuss these concepts in more detail later when we learn about Cook’s Distance.

6.2 Overall F -test for model signifance

Before proceeding to investigating the effects of the individual predictors one at a time, we usually perform a “whole-model” test to first confirm that there is merit (i.e. “utility”) to the model. This is in essence testing the following hypotheses:

H_0 : none of the predictor variables (X_i ’s) are useful in determining the response.
versus

H_a : at least one of the predictor variables is useful in predicting the response.

Since the general form of the model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$, these hypotheses can be equivalently written in terms of the model’s β -parameters as follows:

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_k = 0 \quad \text{vs.} \quad H_a : \text{at least one } \beta_i \neq 0$$

One way of thinking about testing this null hypothesis is to think about fitting two models to the data, one using all the predictors (the “full” model), and the other using none of the predictors (a *reduced* “null” model), and seeing if there is a significant difference between them. Thinking this way, the hypotheses may be rewritten again as:

$$H_0 : \text{the model is } Y = \beta_0 + \varepsilon \quad \text{vs.} \quad H_a : \text{the model is } Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

We now construct a test statistic that compares the residual variance of the two competing models. We fit both the “full” H_a model and the reduced “null”

H_0 model and obtain their respective residual sums of squares values. It is a mathematical fact that $RSS_{H_a} \leq RSS_{H_0}$ (thought exercise: why?). If the difference $RSS_{H_0} - RSS_{H_a}$ is small in magnitude, then the fit of the reduced “null” H_0 model is almost as good as the “full” H_a model, so we would prefer the reduced model on the basis of simplicity.

On the other hand, if the difference is large, then the superior fit of the full H_a model would be preferred. If we scale this difference by how much error variation was in the full model in the first place, then this suggests that something like

$$\frac{\text{Reduction in error variance, } H_0 \text{ model} \rightarrow H_a \text{ model}}{\text{Error variance in } H_a \text{ model}} \leftrightarrow \frac{RSS_{H_0} - RSS_{H_a}}{RSS_{H_a}}$$

would potentially be a good test statistic.

In practice, we will use what is known as **ANOVA F-statistic** to compare the numerator variance to the denominator variance:

$$F = \frac{(RSS_{H_0} - RSS_{H_a}) / (\text{error } df_{H_0} - \text{error } df_{H_a})}{RSS_{H_a} / \text{error } df_{H_a}}$$

Under the usual regression assumptions, this F -ratio follows a sampling distribution known as an F -distribution. The F -distribution has two parameters: the degrees of freedom for the numerator, and the degrees of freedom for the denominator. **Yes, it is the same underlying test as that in ANOVA from experimental design!**

F -statistics are the standard method for comparing two variances. Here’s what could happen:

- If F is too large, we reject H_0 in favor of H_a and conclude that at least one of the predictor variables is useful in determining the response (*i.e.*, at least one $\beta_i \neq 0$).
- If F is small, we fail to reject H_0 in favor of H_a and conclude that there is insignificant evidence to conclude that at least one of the predictor variables is useful in determining the response.

The “whole-model” F -test for model utility is generally assessed first when fitting a model.

6.2.1 Example of model overall F -test

Example: Property appraisals (example from ?).

Recall the property appraisal dataset from the previous chapter, section ??.

```
load(url("https://tjfisher19.github.io/introStatModeling/data/appraisal.RData"))

appraisal.fit <- lm(saleprice ~ landvalue + impvalue + area, data=appraisal)
summary(appraisal.fit)

##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025   2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1384.197    5744.100   0.24  0.8126
## landvalue     0.818      0.512    1.60  0.1294
## impvalue     0.819      0.211    3.89  0.0013 **
## area        13.605      6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF, p-value: 3.87e-08
```

The F -statistic on 3 and 16 degrees of freedom is 46.72, with p -value < 0.0001 . We reject H_0 and conclude at least one predictor is useful for predicting sales price.

Two things to be aware of:

1. What if we fail to reject H_0 ? Does that mean that none of the predictors matter? No – A failure to reject the null hypothesis is not the end of the game — you may just have insufficient data to detect any real effects, which is why we must be careful to say “fail to reject” the null rather than “accept” the null. It would be a mistake to conclude that no real relationships exist. We may have misspecified the structural form of the model or there may be unusual observations obscuring a real effect.
2. What if we reject H_0 ? By the same token, when H_0 is rejected, this does not mean that we have found the best model. All we can say is that at least one of the predictors is useful. We do not know whether all the predictors are required to predict the response, or just some of them. Other predictors might also be added and may be helpful. Either way, the omnibus F -test is just the beginning of an inferential analysis, not the end. What follows next are the more detailed assessments of the

individual predictors.

6.3 Individual parameter inference

After running a whole-model F -test and determining that the fitted model has utility (*i.e.*, if we rejected $H_0 : \beta_1 = \beta_2 = \dots = \beta_k = 0$), we may proceed to running tests or finding confidence intervals for the individual β_i parameters. These inferences may be used to

- Determine which predictors are significant predictors of the mean response.
- Estimate the size of the partial effect of each predictor on the response.
- “Streamline” the model through the removal of insignificant predictors.

The last point above will serve as our first taste of model building, *i.e.*, the “arts and crafts” aspect of tweaking a model so that it best explains the observed patterns and trends in the data. There will be much more on this later, but for now we can at least see a bit of it in action.

The calculated values of b_i are just point estimates of the true values β_i for the population relationship. β_i is an unknown parameter, so we use our estimate b_i to build confidence intervals or run hypothesis tests about β_i (much like in Intro stat when you used $\bar{x}$ as a proxy for μ). While β_0 is usually of limited interest (and only in specific circumstances), the remaining β_i terms are critical parameters because it measures the “true” linear rate of change in Y as X_i is increased by one unit.

6.3.1 Parameter t -tests

The usual parameter test of interest in regression deals with the slope on term X_i :

$$H_0 : \beta_i = 0 \quad \text{versus} \quad H_a : \beta_i \neq 0$$

This null hypothesis basically states that X_i and Y have no linear relationship. Rejection of this null hypothesis offers statistical confirmation that there is a significant linear relationship between X_i and Y . The test statistic for this is a t -statistic, and is given by

$$t = \frac{\text{point estimate} - \text{hypothesized value}}{\text{standard error of point estimate}} = \frac{b_i - 0}{SE_{b_i}}$$

and is provided in the `summary()` output in R.

6.3.2 Example of individual parameter t -test

Example: Property appraisals (example from ?).

Consider individual test from the Property Appraisal data.

```
summary(appraisal.fit)
```

```
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025   2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1384.197    5744.100    0.24  0.8126
## landvalue      0.818      0.512    1.60  0.1294
## impvalue       0.819      0.211    3.89  0.0013 **
## area          13.605      6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF, p-value: 3.87e-08
```

$H_0 : \beta_1 = 0$ (appraised land value has no effect on sales price, after adjusting for appraised value of improvements and area of living space)

$H_a : \beta_1 \neq 0$ (appraised land value has an effect on sales price, after adjusting for appraised value of improvements and area of living space)

The test statistic is $t = 1.599$ (with $df=16$), and p -value = 0.1294. We fail to reject H_0 and conclude that appraised land value has no significant effect on sales price, after adjusting for appraised value of improvements and area of living space.

$H_0 : \beta_2 = 0$ (appraised value of improvements has no effect on sales price, after adjusting for appraised land value and area of living space)

$H_a : \beta_2 \neq 0$ (appraised value of improvements has an effect on sales price, after adjusting for appraised land value and area of living space)

The test statistic is $t = 3.889$ (with $df=16$), and $p\text{-value} = 0.0013$. We reject H_0 and conclude that appraised value of improvements has a significant effect on sales price, after adjusting for appraised land value and area of living space.

$H_0 : \beta_3 = 0$ (area of living space has no effect on sales price, after adjusting for appraised value of land and improvements)

$H_a : \beta_3 \neq 0$ (area of living space has an effect on sales price, after adjusting for appraised value of land and improvements)

The test statistic is $t = 2.071$ (with $df=16$), and $p\text{-value} = 0.0549$. We reject H_0 and conclude that the area of living space has a marginally significant effect on sales price, after adjusting for appraised value of improvements and area of living space.

Summary: The appraised value of improvements is the most significant predictor of sales price. Area of living space is marginally significant and appraised land value is not significant.

6.3.3 Model trimming

What about insignificant predictors? At this point, one might wonder about the need for retaining appraised land value (`landvalue`) as a predictor due to its insignificance. However, we saw in section ?? that multiple regression is generally better.

We may delete insignificant predictors one at a time to see the effect on the quality of the fitted model. Remember: a simple model that does essentially as well as a more complex model is preferred.

So if deleting an insignificant variable is not a detriment to the model, its removal can just justified.

So, here's what happens by deleting `landvalue` as a predictor (we actually did this in the last chapter to see the effect on the β parameters):

```
summary(lm(saleprice ~ impvalue + area, data=appraisal))

##
## Call:
## lm(formula = saleprice ~ impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15832  -5200   1260   4642  13836
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -10.191    5931.634    0.00  0.99865
## impvalue       0.959      0.200    4.79  0.00017 ***
```

```
## area          16.492          6.599          2.50  0.02299 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8270 on 17 degrees of freedom
## Multiple R-squared:  0.881, Adjusted R-squared:  0.867
## F-statistic:   63 on 2 and 17 DF, p-value: 1.37e-08
```

The quality of the fit of this model is comparable, albeit a bit worse, to the model containing all three predictors. Consider:

- The residual SE has increased from $s = 7915$ to $s = 8269$. In other words, the quality of `saleprice` predictions suffers somewhat by deleting `landvalue` as a predictor, even though it is statistically insignificant. For this reason, we may wish to retain it.

This is actually a wise method of determining the “value” of a predictor to the model. Another (and mathematically equivalent) way is to look at the adjusted R^2 values for the two models. We will cover that aspect shortly.

6.3.4 Confidence Intervals

As stated before, confidence intervals are usually more informative than tests. The form of these intervals is no different than for t-based CIs you’ve seen previously:

$$b_i + t_{0.025} \times SE_{b_i}$$

The point estimates and their SEs were provided in the R output from `summary()`.

6.3.5 Example of individual parameter confidence intervals

Example: Property appraisals (example from ?).

After fitting the linear model, getting 95% CIs for β_i is a snap with the `confint()` function:

```
confint(appraisal.fit)
```

```
##              2.5 %      97.5 %
## (Intercept) -1.07928e+04 13561.14546
## landvalue   -2.66591e-01  1.90219
## impvalue     3.72807e-01  1.26608
## area        -3.21529e-01  27.53168
```

We focus only on interpretation of the significant predictors:

- After adjusting for appraised land value and area of living space of a given property, we can be 95% confident that each additional dollar of appraised value of improvements is worth, on average, an additional \$0.37 to \$1.26 to the sale price.
- We could make a statement about the CI for the adjusted effect of area of living space (95% CI for area is essentially (0, 27.53)). This interval skirts the value of 0 because of the marginal significance of the partial effect. So, we could say that after adjusting for appraised land and improvements values of a given property, we can be 95% confident that each square foot of living space is worth, on average, no more than \$27.53 to the sale price.

6.4 Confidence and prediction bands

If we are satisfied with the quality of the fit of a multiple linear regression model (*i.e.*, if we feel the residual SE is low enough to facilitate decent predictions), we may use the model to generate CIs or PIs for the response variable Y given some settings for the predictor variables $X_1, X_2, \dots, X_k$. The only caveat to keep in mind when finding such CIs/PIs is the issue of extrapolation outside the region of the observed values of the predictors (called the predictor space). You should stay inside the observed joint region of the predictors when making such predictions.

6.4.1 Simple Linear Regression Confident and Prediction bands

We begin with a look at our simple linear regression example, the muscle mass dataset, for visualization and explanation.

One of the great uses of regression models is to generate useful predictions of the response variable Y given some setting for the predictor variable X .

Given a hypothetical value of X (call it x_0), what is the predicted response value? For example, what is the predicted mean muscle mass for a 65 year old woman? Easy:

$$\hat{Y} = 156.35 - 1.19(65) = 79.0$$

However, as good statisticians we know that the above result is not sufficient. Remember: it is only an estimate of $\mu_{Y|X=65}$, the true mean muscle mass of women age 65. We would like to construct confidence limits around this estimate to get an idea of its precision.

There are actually two kinds of “prediction” that can be made using $\hat{Y}$ as our point estimate. The distinction is between

- prediction of the mean response value at a given X value
- prediction of a future individual response at a given X value

For example,

- We could obtain an interval estimate for the true mean muscle mass of all 65 year old women; or,
- We could obtain an interval estimate for the muscle mass of an unobserved 65 year old woman.

Why would these be different? It is because if we are estimating the true mean response at a given X value, the only contributor to the uncertainty of the prediction is the uncertainty in the fitted line (*i.e.*, uncertainty due to the slope and intercept estimates).

On the other hand, if we are estimating a future individual response value at a given X value, the contributors to the uncertainty of the prediction are both the uncertainty in the line and the variation in the individual points about the line. Thus, the SE for the prediction of a single future response value will be larger than the SE for predicting a mean response value.

To help distinguish between the two kinds of intervals, we will call an interval estimate for the true mean response a confidence interval (CI) (also known as a confidence band) for the mean response, and an interval estimate for an individual future response value a prediction interval (PI) (also known as prediction bands) for an individual response. Which one is more appropriate depends on context, but confidence intervals are used more frequently in practice.

The `predict()` extractor function in R will generate CIs and PIs from a linear model object according to your specifications for values of X .

6.4.2 Example of confidence and prediction intervals

Example. Muscle Mass with Age (originally in ?).

```
muscle.fit <- lm(mass~age,data=muscle)
predict(muscle.fit, newdata=data.frame(age=65), int="conf")
```

```
##      fit      lwr      upr
## 1 78.9969 76.6987 81.295
```

```
predict(muscle.fit, newdata=data.frame(age=65), int="pred")
```

```
##      fit      lwr      upr
## 1 78.9969 62.4758 95.5179
```

Note that the fitted value (labeled `fit`) is the same for both (78.996). However, the 95% PI for muscle mass of an unobserved 65 year old woman is 62.5 to 95.5, whereas the 95% CI for the mean muscle mass of all 65 year old women is 76.7 to 81.3. Thus, we can formally interpret the intervals as follows:

- We can be 95% confident that the mean muscle mass for the entire population of women aged 65 years is between 76.7 to 81.3.

- We can be 95% confident that the muscle mass for a single random selected female aged 65 will be between 62.5 to 95.5.

If we had deleted `newdata=data.frame(age=65)` from the `predict()` function call, we would get PIs (or CIs) for every age value that occurred in the original data set. However, since the values of age appear in random order, it might be a bit nicer to first sort them, and then generate the intervals:

```
muscle.trim <- muscle %>%
  distinct(age) %>%
  arrange(age)
predict(muscle.fit, newdata=muscle.trim, int="pred")
```

##		fit	lwr	upr
## 1		107.5567	90.7083	124.4052
## 2		106.3668	89.5541	123.1794
## 3		105.1768	88.3980	121.9555
## 4		103.9868	87.2401	120.7334
## 5		102.7968	86.0803	119.5133
## 6		101.6068	84.9185	118.2950
## 7		100.4168	83.7549	117.0787
## 8		99.2268	82.5893	115.8642
## 9		95.6568	79.0811	112.2325
## 10		94.4668	77.9078	111.0258
## 11		93.2768	76.7325	109.8211
## 12		92.0868	75.5553	108.6183
## 13		90.8968	74.3761	107.4175
## 14		89.7068	73.1950	106.2186
## 15		88.5168	72.0119	105.0218
## 16		86.1368	69.6397	102.6339
## 17		84.9468	68.4507	101.4430
## 18		83.7568	67.2597	100.2540
## 19		81.3768	64.8717	97.8820
## 20		80.1869	63.6748	96.6989
## 21		78.9969	62.4758	95.5179
## 22		77.8069	61.2750	94.3388
## 23		75.4269	58.8673	91.9864
## 24		74.2369	57.6606	90.8132
## 25		73.0469	56.4519	89.6419
## 26		71.8569	55.2412	88.4725
## 27		70.6669	54.0287	87.3051
## 28		69.4769	52.8142	86.1396
## 29		67.0969	50.3794	83.8144
## 30		65.9069	49.1592	82.6546
## 31		64.7169	47.9371	81.4967
## 32		63.5269	46.7131	80.3407

So, for example, we can be 95% confident that the muscle mass of an unobserved 52 year old woman will be between 77.9 to 111.0. (Note that these are PIs for future observations, not the current observations!)

Since we have a two-dimensional problem here, it is instructive to graphically take a look at the confidence limits and prediction limits across the entire span of the predictor space (called confidence bands and prediction bands). The display of the confidence and prediction bands for the muscle mass model fit appears below.

Two things you should notice:

1. The prediction bands are wider than the confidence bands (as we have already established).
2. The width of the confidence bands (and prediction bands) is not uniform over the entire predictor space. It is subtle in this particular example, but a close look will reveal that as you move away from the center of the data, the intervals get wider.

The below code will plot the fitted line with the confidence and prediction bands as seen in Figure ??.

```
muscle.bands <- muscle.trim %>%
  mutate(ci.lo = predict(muscle.fit, newdata=muscle.trim, int="conf")[,2],
         ci.hi = predict(muscle.fit, newdata=muscle.trim, int="conf")[,3],
         pi.lo = predict(muscle.fit, newdata=muscle.trim, int="pred")[,2],
         pi.hi = predict(muscle.fit, newdata=muscle.trim, int="pred")[,3] )
muscle.with.fit <- muscle %>%
  mutate(Fitted=fitted(muscle.fit))

ggplot() +
  geom_ribbon(data=muscle.bands, aes(x=age, ymin=pi.lo, ymax=pi.hi), fill="gray80") +
  geom_ribbon(data=muscle.bands, aes(x=age, ymin=ci.lo, ymax=ci.hi), fill="gray60") +
  geom_line(data=muscle.with.fit, aes(x=age, y=Fitted), size=1.25, color="blue") +
  geom_point(data=muscle.with.fit, aes(x=age, y=mass) ) +
  theme_minimal()
```

Notes about the code. We are using the layering features in `ggplot()`. First we plot the prediction interval as a `geom_ribbon` type with a lighter colored gray. Then the confidence band (since we know it will be narrower) on the next layer. Then the fitted line and original data points.

6.4.3 Extrapolation warning!

It might be tempting to use your model to make predictions outside the observed range of your predictor variable (*e.g.*, what is the predicted muscle mass of a 90 year old woman?). The best advice: DON'T!

Regression isn't some magic pill that will get you reliable predictions of what's

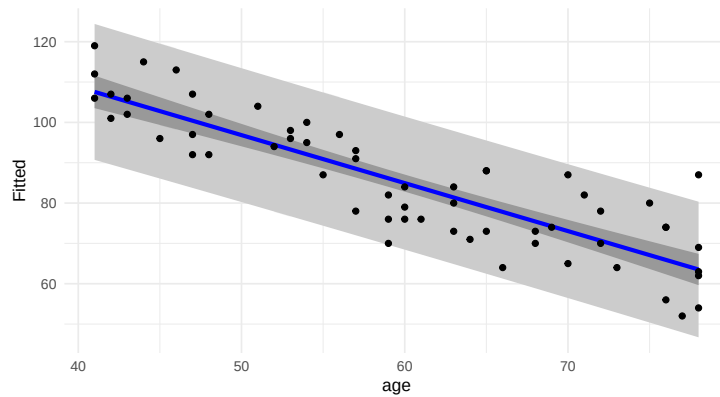


Figure 6.2: Scatterplot with regression line (blue line), confidence bands (dark gray) and prediction bands (light gray).

going on out in some region where you have no information. You may have a model that fits the observed data well, but the model may be completely different when you move outside this range. For instance, perhaps muscle mass reaches a point of stabilization near age 80, resulting in a plot that levels off. If you used the model we fitted above to make a prediction for a geriatric female above age 80, you may vastly underpredict her true muscle mass. A relationship that looks linear on a small scale might be completely different on a larger scale.

That is why when you conduct your own investigations, you should collect data from the entire region of predictor values of research interest.

In summary, don't extrapolate because:

- confidence and prediction intervals for the response get wider the farther away from the center of the data you move; and
- the structural form of the relationship may not be the same as you move away from the observed predictor space.

The two plots in Figure ?? below illustrate these points. Suppose you only observe the data in the narrow range around $X = 0$ as shown in the plot on the left. If you fit a line to these data, you get the observed trend (and prediction bands). However, if the true nature of the relationship outside this narrow range were radically different (and you do not know this because you collected no data outside your “narrow” range), then extrapolating beyond your limited view of the relationship may produce wildly inaccurate predictions (see the right side plot).

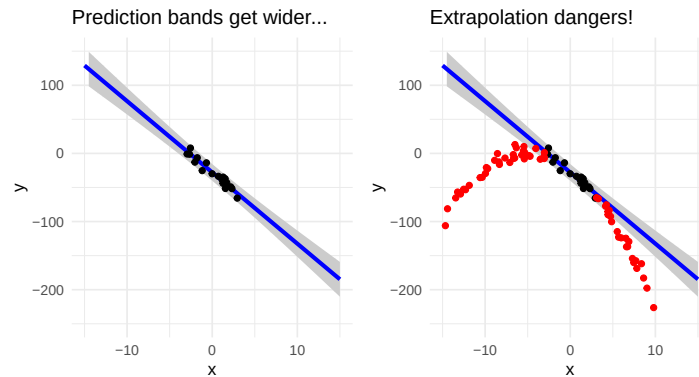


Figure 6.3: Scatterplot with regression line demonstrating the fanning of prediction bands (left) and the dangers of extrapolation (right).

6.4.4 Confidence and Prediction Bands in Multiple Regression

We can do the same sort of analysis with a multiple regression dataset, unfortunately it is not as easy to plot.

Example: Property appraisals (example from ?).

```
x.pred <- data.frame(landvalue=11000, impvalue=43500, area=2000)
predict(appraisal.fit, x.pred, int="pred")
```

```
##      fit      lwr      upr
## 1 73235.9 54922.7 91549.1
```

We are 95% confident that this property will sell for between \$54,922 and \$91,549.

6.5 Goodness-of-fit

We seek a descriptive measure of how well the fitted model explains the observed response values. One way of doing this is to again consider the general model form

$$\text{Data} = \text{SystematicStructure} + \text{RandomVariation}$$

and thinking of partitioning the variation in the response values data (*i.e.*, the y -values) into the two corresponding components:

- Variation explained by the structural relationship between y and the predictor(s);

- Variation left unexplained by the model (*i.e.*, variation due to “error”).

We use sums of squares as the basis of the partitioning of variability. The idea is that if the model explains the response variation well, then there should be relatively little error variation left over, that is, RSS should be relatively small. We start as follows:

- TSS is the total sum of squares, and here serves as our measure of variability of response values around their mean. This requires no knowledge of the predictors.
- RSS is the residual sum of squares (or deviance), and here serves as our measure of variability of response values around their model predictions. This obviously requires knowledge of the predictors.

Formulaically, $TSS = \sum_{i=1}^n (y_i - \bar{y})^2$ and $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$.

Figure ?? below is a graphic illustrating how one may think of “sizing” these two quantities. The picture shows a simulated data example where there is a clearly strong linear trend. You can see that once you account for the linear trend element (which explains a lot of what you see in the plot), the relative size of the leftover residual variation is low, *i.e.* RSS is the much smaller partition of the total sums of squares.

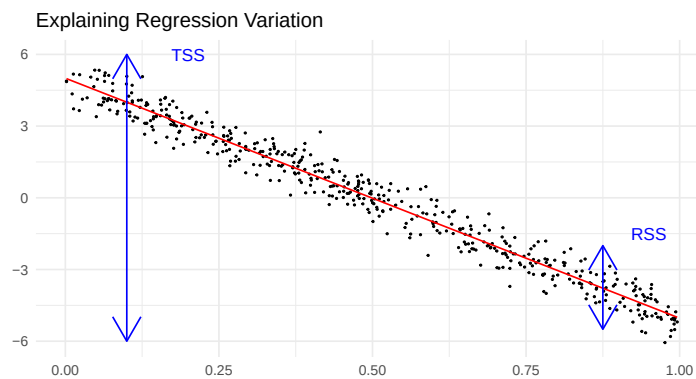


Figure 6.4: Simulated data with regression line demonstrating the decomposition of variation in the data, TSS vs. RSS.

The difference between TSS and RSS is the sum of squares due to the model (let’s call it $SS(model)$). One can show that

$$TSS = SS(model) + RSS$$

6.5.1 Coefficient of determination

A basic descriptor of the “goodness of fit” for our model is the given by the ratio of the model sums of squares to the total sums of squares, commonly referred to as R^2 :

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS}$$

This is valuable because it has a nice clean interpretation as the proportion of total variation in the response that is explained by the model. Since it is a proportion, it is always true that $0 \leq R^2 \leq 1$. In R, it is provided in the `summary()` function for a linear model object, labeled **Multiple R-squared**. It is also known as the coefficient of determination.

Adjusted R^2 , or R^2_{adj} , takes into account the degrees of freedom for both total variation and error variation, and is generally preferable to R^2 :

$$R^2_{adj} = 1 - \frac{RSS/df_{error}}{TSS/df_{total}} = 1 - \frac{RSS/(n-p)}{TSS/(n-1)} = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

R^2_{adj} is not technically a proportion like R^2 , but it is more useful than R^2 when comparing different models for the same data. It should also be said that, although informative, these measures do not give any direct indication of how well the regression equation will predict when applied to a new data set. Like R^2 , R^2_{adj} is supplied with the `summary()` function and is labeled **Adjusted R-squared**. We show the output below for the appraisal model.

```
summary(appraisal.fit)
```

```
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025   2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1384.197   5744.100    0.24  0.8126
## landvalue     0.818     0.512    1.60  0.1294
## impvalue     0.819     0.211    3.89  0.0013 **
## area        13.605     6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF,  p-value: 3.87e-08
```

Using Adjusted R^2 since it is preferred in multiple regression, we see that a combination of appraised land value, appraised value of improvements and area explains about 87.8% of the variation in sale price.

6.5.2 Akaike's Information Criterion

At this point, we will introduce what might at first seem like a completely off-the-wall measure of model fit attributable to Japanese statistician Hirotugu Akaike, known as the Akaike Information Criterion, or AIC (?). The AIC is derived from information theory methods developed in the 1970s, and provides a powerful tool for the comparison of different models with respect to the quality of fit.

For models based on normal theory and using least squares estimation, the AIC is defined as follows:

$$AIC = n \log(RSS/n) + 2p$$

where n = sample size, RSS is the model residual sum of squares, and p = number of model parameters (i.e. number of β parameters in the model). In R we can calculate the AIC value for a fit using the `AIC()` function.

```
AIC(appraisal.fit)
```

```
## [1] 421.356
```

It should be noted that, in itself, the value of AIC for a given data set and model has no meaning. Its utility comes when we compare AIC values of different candidate models for the same data (covered in chapter ??). Later on, as regression models become more complex with the inclusion of potentially many predictor variables, we will revisit AIC to help us in the selection of the “best” model among many choices.

A similar measure is known as the Bayesian Schwarz Information Criteria (?), or BIC. It is calculated with the formula

$$BIC = n \log(RSS/n) + \log(n)p$$

and is calculated in R with the `BIC()` function.

```
BIC(appraisal.fit)
```

```
## [1] 426.334
```


Chapter 7

More on multiple linear regression

7.1 Model comparison – Reduced F -tests

Occasionally, we might be interested in more specialized tests of a multiple regression model's β parameters. For example, consider the property appraisals data (again). The regression model we are using is

$$\text{Sale price} = \beta_0 + \beta_1(\text{Land value}) + \beta_2(\text{Improvements value}) + \beta_3(\text{Area}) + \varepsilon$$

Think about questions like these:

- Do the two appraisals collectively have a significant effect on mean sale price?
- Is the effect of land value appraisal different from the effect of improvements value appraisal on the mean sale price?

To address questions like these, we first need to translate them into equivalent expressions in terms of the model's β parameters. Doing so will give rise to hypotheses we can test using the principle of ANOVA that was introduced earlier.

Logic. To formulate an approach to conducting the test, let's work on the first question above: "Do the two appraisals collectively have a significant effect on mean sale price?" To get started, you need to think about what the null and alternative hypotheses would be to test such a statement. They would look like this:

- H_0 : The two appraisals collectively have no effect on mean sale price
- H_a : At least one of the two appraisals has an effect on mean sale price

Now, translate what this means in terms of the model's β parameters:

- H_0 : $\beta_1 = \beta_2 = 0$
- H_a : at least one of β_1 or β_2 is not 0

How we proceeded with tests like those in the previous chapters where we essentially compared two models: one involving a “full” model and the other involving a “reduced” null model, and checking if there is a significant difference between them. The transition from the full model to the reduced model is to impose the null hypothesis on the full model. Thinking this way, the hypotheses above may be rewritten as:

- H_0 : the model is Sale price = $\beta_0 + \beta_3(\text{Area}) + \varepsilon$
- H_a : the model is Sale price = $\beta_0 + \beta_1(\text{Land value}) + \beta_2(\text{Improvement value}) + \beta_3(\text{Area}) + \varepsilon$

The R function `anova()` will do the computation of the ANOVA F -statistic and its associated p -value for this test, once you feed it the fitted full and reduced R model objects. The form of the R function is `anova(reducedmodel, fullmodel)`

So, the test of H_0 : The two appraisals collectively have no effect on mean sale price (i.e. $\beta_1 = \beta_2 = 0$) is conducted as follows (recall `appraisal.fit` is the full model):

```
reduced.appraisal.fit <- lm(saleprice ~ area, data=appraisal)
anova(reduced.appraisal.fit, appraisal.fit)
```

```
## Analysis of Variance Table
##
## Model 1: saleprice ~ area
## Model 2: saleprice ~ landvalue + impvalue + area
##   Res.Df      RSS Df Sum of Sq    F    Pr(>F)
## 1      18 2.732e+09
## 2      16 1.002e+09  2 1.729e+09 13.8 0.000329 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The F -statistic value is 13.802, with 2 and 16 degrees of freedom. The p -value is 0.0003, so we reject H_0 . There is sufficient evidence to conclude that the appraisals collectively have a significant effect on mean sale price.

Before doing one more example, here is a summary of the steps involved:

1. Fit a full model, and save it as an R model object.
2. Impose your null hypothesis H_0 onto the full model to create the reduced model.
3. Fit the reduced model, and save it as an R model object.
4. Run the ANOVA F -test by issuing `anova(reducedmodel, fullmodel)`.

We now perform another example to demonstrate further functionality. *Is the effect of land value appraisal different from the effect of improvements value appraisal on the mean sale price?*

The null and alternative hypotheses for testing this can be expressed as follows:

- H_0 : Land value appraisal and improvements value appraisal have the same effect
- H_a : Land value appraisal and improvements value appraisal have different effects

In terms of the model's β parameters, this null hypothesis is just stating that β_1 and β_2 are equal (but not necessarily 0). So, the hypotheses may be rewritten as

$$H_0 : \beta_1 = \beta_2 \quad \text{versus} \quad H_a : \beta_1 \neq \beta_2$$

Let's denote the common value of β_1 and β_2 under the null hypothesis using the symbol γ . Then it is crucial to see that we can write the hypotheses in terms of full and reduced “null” models as follows:

- H_0 : the model is Sale price = $\beta_0 + \gamma(\text{Land}) + \gamma(\text{Improv}) + \beta_3(\text{Area}) + \varepsilon$
- H_a : the model is Sale price = $\beta_0 + \beta_1(\text{Land}) + \beta_2(\text{Improv}) + \beta_3(\text{Area}) + \varepsilon$

or equivalently,

- H_0 : the model is Sale price = $\beta_0 + \gamma(\text{Land} + \text{Improv}) + \beta_3(\text{Area}) + \varepsilon$
- H_a : the model is Sale price = $\beta_0 + \beta_1(\text{Land}) + \beta_2(\text{Improv}) + \beta_3(\text{Area}) + \varepsilon$

Note in this last notation the null hypothesis essentially is performing a multiple regression on two predictor variables: Area and a new variable Land + Improv. We can perform this test in one of two ways.

Method 1 - Creating a new variable

```
appraisal <- appraisal %>%
  mutate(LandImprov = landvalue + impvalue)
red.model1 <- lm(saleprice ~ LandImprov + area, data=appraisal)
anova(red.model1, appraisal.fit)
```

```
## Analysis of Variance Table
##
## Model 1: saleprice ~ LandImprov + area
## Model 2: saleprice ~ landvalue + impvalue + area
##   Res.Df      RSS Df Sum of Sq  F Pr(>F)
## 1      17 1.002e+09
## 2      16 1.002e+09  1      426.8  0  0.998
```

Method 2 - Calculation “on the fly”

The `I()` function in R can be used to create a single new predictor for the null model by calculating the sum of the `landvalue` and `impvalue` variables “on the fly”. It is important to note this variable is not saved for future use. Here is the analysis:

```
red.model2 <- lm(saleprice ~ I(landvalue + impvalue) + area, data=appraisal)
anova(red.model2, appraisal.fit)
```

```
## Analysis of Variance Table
##
## Model 1: saleprice ~ I(landvalue + impvalue) + area
## Model 2: saleprice ~ landvalue + impvalue + area
##   Res.Df      RSS Df Sum of Sq  F Pr(>F)
## 1      17 1.002e+09
## 2      16 1.002e+09  1    426.8  0  0.998
```

In both cases we see the F -statistic value is essentially 0, with 1 and 16 degrees of freedom. The p -value is 0.998, so we fail to reject H_0 . There is insufficient evidence to conclude that land value appraisal and improvements value appraisal have different effects on the mean sale price of a property.

Lastly, note the two fitted reduced models are the same

```
red.model1
```

```
##
## Call:
## lm(formula = saleprice ~ LandImprov + area, data = appraisal)
##
## Coefficients:
## (Intercept)  LandImprov      area
##   1386.277      0.819    13.605
```

```
red.model2
```

```
##
## Call:
## lm(formula = saleprice ~ I(landvalue + impvalue) + area, data = appraisal)
##
## Coefficients:
##              (Intercept)  I(landvalue + impvalue)              area
##              1386.277              0.819              13.605
```

Also note the fitted full model

```
appraisal.fit
```

```
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
```

```
##
## Coefficients:
## (Intercept)    landvalue    impvalue      area
##      1384.197         0.818         0.819      13.605
```

The coefficients on `landvalue` and `impvalue` are nearly identical (differ in the third decimal place). It should not be too surprising we failed to reject the null hypothesis that they were statistically the same.

7.2 Categorical Predictor Variables

Up to now, all of our regression examples have included predictors that have been numerically valued variables; i.e., they have been quantitative variables. Predictors that are qualitative in nature (e.g., gender, ethnic group, eye color) are usually called categorical predictors or factors. How can predictors such as these be incorporated into a regression analysis?

To use qualitative predictors in a regression model, we need to recode their values. The way to do this is to create one or more dummy variables. Dummy variables are variables that only take on the value 0 or 1, indicating the absence (0) or presence (1) of a particular qualitative characteristic.

7.2.1 A qualitative predictor with two levels

Suppose we are interested in the effect of a fixed dosage of two different medications on reducing blood glucose level in men. One of these meds is an experimental drug (let's call it drug A), and the other is a placebo (drug P). For a two-level variable such as this, one dummy variable is all that is required to distinguish between the two drugs. We could define our dummy variable as follows:

$$X = \begin{cases} 0 & \text{if drug } P \\ 1 & \text{if drug } A \end{cases}$$

X is now a “numerically valued” (0 or 1) variable that simply serves to indicate that drug A was the administered drug. In such a coding, the placebo will serve as a reference (or “baseline”) group to which the drug A group will be compared. We will see this when we construct models using X .

How does the regression model look? First, we fit a linear model in the usual way using X as the predictor. The model form is given by

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Writing out the model explicitly for both drug groups will prove enlightening. Since X is basically an on/off switch indicating if A is the administered drug,

the model parameters and their interpretations break down as follows:

Drug	X	Model for mean response
Placebo	0	$\mu_{placebo} = \beta_0 + \beta_1(0) = \beta_0$
A	1	$\mu_A = \beta_0 + \beta_1(1) = \beta_0 + \beta_1$

Key items to notice:

- The structural part of the model under the placebo is just β_0 . So, β_0 represents $\mu_{placebo}$, the mean response for Y under the placebo.
- The structural part of the model under drug A is $\beta_0 + \beta_1$. So, $\beta_0 + \beta_1$ represents μ_A , the mean response for Y under drug A .
- Now, put these two facts together. What does β_1 alone estimate? It should quickly become clear that β_1 represents the change in the mean response for Y from *placebo* to drug A . In other words, β_1 is the key parameter of interest in the model, since it is an estimate of $\mu_A - \mu_{placebo}$. So, the usual regression t -test of $H_0 : \beta_1 = 0$ is really a test of $H_0 : \mu_A - \mu_{placebo} = 0$, or equivalently, $H_0 : \mu_A = \mu_{placebo}$ (essentially the two-sample t -test). Let's consider an example.

Example. Previous research indicates that children borne by diabetic mothers may suffer from obesity, high blood pressure and glucose intolerance. Independent random samples of adolescent offspring of diabetic and non-diabetic mothers were evaluated for potential differences in vital measurements, including blood pressure. The data are in the file `diabeticoffspring.txt` in our data repository. Use this data to (a) test to see if there is a difference in mean systolic BP between diabetic and non-diabetic offspring, and if so (b) estimate the true mean difference using a 95% CI. This is essentially an intro statistics problem.

We begin by obtaining the data.

```
www <- "http://www.users.miamioh.edu/hughesmr/sta363/diabeticoffspring.txt"
diabetes <- read.table(www, header=TRUE)
head(diabetes)
```

```
## sys.bp mother
## 1 127 diabetic
## 2 137 diabetic
## 3 112 diabetic
## 4 108 diabetic
## 5 123 diabetic
## 6 120 diabetic
```

Now we perform a two-sample t -test.

```
t.test(sys.bp ~ mother, var.equal=TRUE, data=diabetes)
```

```
##
```

```
## Two Sample t-test
##
## data: sys.bp by mother
## t = 4.559, df = 177, p-value = 9.58e-06
## alternative hypothesis: true difference in means between group diabetic and group nondiabetic
## 95 percent confidence interval:
##  4.54379 11.48121
## sample estimates:
##      mean in group diabetic mean in group nondiabetic
##                118.000                109.987
```

Now, let's re-do this as a regression problem using a quantitative predictor. If the assumptions underlying the independent samples *t*-test are met, then the assumptions underlying a regression approach to the problem are also met (we will check these momentarily).

Note that the variable `mother` is a character variable in the data frame, R automatically treats it as a qualitative variable and creates a dummy variable for you.

```
class(diabetes$mother)
```

```
## [1] "character"
```

So we can perform regression as before, and R will handle the `factor` variable correctly.

```
diabetes.fit <- lm(sys.bp ~ mother, data=diabetes)
summary(diabetes.fit)
```

```
##
## Call:
## lm(formula = sys.bp ~ mother, data = diabetes)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -33.00  -6.99  -0.99   8.00  32.01
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      118.00      1.18  100.42 < 2e-16 ***
## mothernondiabetic    -8.01      1.76   -4.56 9.6e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11.7 on 177 degrees of freedom
## Multiple R-squared:  0.105, Adjusted R-squared:  0.1
## F-statistic: 20.8 on 1 and 177 DF, p-value: 9.58e-06
```

Note that the independent samples t -test and the regression t -test for β_1 produce identical p -values. That's because they both test the exact same hypothesis. Also note that the estimate for β_1 in the regression model is $b_1 = -8.013$, which is what you get when you calculate the difference the two group's mean estimates ($109.9875 - 118.0000$).

You should note that R has labeled the predictor variable `mothernondiabetic`. This is your indication as to the coding scheme used by R for the qualitative predictor. In this case, R created a dummy variable as

$$X = \begin{cases} 0 & \text{if mother is } \textit{diabetic} \\ 1 & \text{if mother is } \textit{not diabetic} \end{cases}$$

and so we can interpret the meaning of the β parameters in the model $Y = \beta_0 + \beta_1 X + \varepsilon$:

Mother	X	Model for mean response
<i>Diabetic</i>	0	$\mu_{diabetic} = \beta_0 + \beta_1(0) = \beta_0$
<i>Not diabetic</i>	1	$\mu_{nondiabetic} = \beta_0 + \beta_1(1) = \beta_0 + \beta_1$

- β_0 is the true mean systolic blood pressure of diabetic mothers.
- $\beta_0 + \beta_1$ is the true mean systolic blood pressure of nondiabetic mothers.
- Therefore, $\beta_1 = (\beta_0 + \beta_1) - \beta_0$ is the true mean change in systolic blood pressure from diabetic to nondiabetic mothers.

Confidence intervals for the β parameters in this regression model should be interpreted accordingly, and are found the same as before using `confint()`:

```
confint(diabetes.fit)
```

```
##                2.5 %    97.5 %
## (Intercept)    115.6811 120.31892
## mothernondiabetic -11.4812  -4.54379
```

So, our interpretations are as follows:

- $b_0 = 118.0$ is our estimate of β_0 , the true mean systolic blood pressure of diabetic mothers. The 95% CI for β_0 is (115.68, 120.32). Thus, we can be 95% confident that the true mean systolic blood pressure of diabetic mothers is between 115.68 to 120.32.
- $b_1 = -8.013$ is our estimate of β_1 , the true mean change in systolic blood pressure from diabetic to nondiabetic mothers. The 95% CI for β_1 is (-11.48, -4.54). Thus, we can be 95% confident that the true mean systolic blood pressure in nondiabetic mothers is between 4.54 to 11.48 lower than in diabetic mothers.

If you look up at the original independent samples t -test output, you'll see this same confidence interval expressed in terms of $\mu_{diabetic} - \mu_{nondiabetic}$.

Important. When we find confidence intervals for the β parameters in a regression model with a qualitative predictor, it is imperative to know the interpretation of each of the β parameters in the context of the problem. In other words, you need to know both of

- The form of the model used (in the last example, this was $Y = \beta_0 + \beta_1 X + \varepsilon$)
- The manner of coding used for the qualitative variable (here, $X = 1$ is a non-diabetic mother).

7.2.2 A qualitative predictor with more than two levels

When using a qualitative predictor with more than two levels, we must use a coding scheme (i.e., a method of constructing dummy variables) that adequately distinguishes between all the levels of the qualitative predictor. Basically, let this be your guide:

In general, if a factor has k levels, we will require $k-1$ dummy variables to sufficiently code the factor for regression.

For example, consider a study of the efficacy of four drugs (A , B , C and a placebo P) on glucose reduction. Coding up these 4 drugs in the regression will require $4 - 1 = 3$ dummy variables. We have flexibility in how we could define these three dummies, but here is probably the most meaningful way given the context of the experiment:

$$I_A = \begin{cases} 1 & \text{if drug } A \\ 0 & \text{otherwise} \end{cases}, \quad I_B = \begin{cases} 1 & \text{if drug } B \\ 0 & \text{otherwise} \end{cases}, \quad I_C = \begin{cases} 1 & \text{if drug } C \\ 0 & \text{otherwise} \end{cases}$$

I_A serves as an indicator that drug A was administered, I_B for drug B , and I_C for drug C . The model may be written as

$$Y = \beta_0 + \beta_1(I_A) + \beta_2(I_B) + \beta_3(I_C) + \varepsilon$$

So how do we indicate when the placebo P is the administered drug? That's easy: it's when all three dummy variables are 0 (i.e. "switched off"). In this coding, the placebo P serves as the reference level to which drugs A , B and C will be compared. You can see this by constructing the model using I_A , I_B and I_C :

Drug	I_A	I_B	I_C	Model for mean response
A	1	0	0	$\mu_A = \beta_0 + \beta_1(1) + \beta_2(0) + \beta_3(0) = \beta_0 + \beta_1$
B	0	1	0	$\mu_B = \beta_0 + \beta_1(0) + \beta_2(1) + \beta_3(0) = \beta_0 + \beta_2$
C	0	0	1	$\mu_C = \beta_0 + \beta_1(0) + \beta_2(0) + \beta_3(1) = \beta_0 + \beta_3$
Placebo	0	0	0	$\mu_P = \beta_0 + \beta_1(0) + \beta_2(0) + \beta_3(0) = \beta_0$

From this coding stems the physical interpretation of the model's β coefficients:

- β_0 is the true mean glucose reduction under the placebo.
- β_1 is the true mean change in glucose reduction from placebo to drug *A*.
- β_2 is the true mean change in glucose reduction from placebo to drug *B*.
- β_3 is the true mean change in glucose reduction from placebo to drug *C*.

Important note. The above dummy variable coding scheme is the most meaningful way to code drug for this analysis because the placebo is the natural choice for a “reference” drug. However, there are many ways to create a valid coding, and we must know which one R uses in order to correctly interpret the model.

Whole model *F*-test. We’ve seen that the whole model ANOVA *F*-test for a multiple linear regression model given by $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$ is a test of the hypotheses

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_k = 0 \quad \text{versus} \quad H_a : \text{At least one } \beta_i \neq 0$$

So when we have a regression model containing one qualitative predictor with more than two levels, what does the whole-model *F*-test really test? It all stems back to the interpretation of the model’s β coefficients based on the coding scheme used.

Consider the previous four drug example. The whole model *F*-test would be a test of the hypothesis $H_0 : \beta_1 = \beta_2 = \beta_3 = 0$. If you look at the aforementioned table, it is not too difficult to see that

- $\beta_1 = (\beta_0 + \beta_1) - \beta_0 = \mu_A - \mu_P$
- $\beta_2 = (\beta_0 + \beta_2) - \beta_0 = \mu_B - \mu_P$
- $\beta_3 = (\beta_0 + \beta_3) - \beta_0 = \mu_C - \mu_P$

So, the following must be interchangeable:

$$H_0 : \beta_1 = \beta_2 = \beta_3 = 0 \quad \longleftrightarrow \quad H_0 : \mu_A - \mu_P = \mu_B - \mu_P = \mu_C - \mu_P = 0$$

If we just add μ_P to each piece of the hypothesis as expressed on the right, we see that the null hypothesis for the whole model *F*-test here is actually a test of

$$H_0 : \mu_A = \mu_B = \mu_C = \mu_P$$

In other words, we now have a way to test for the simultaneous equivalence of multiple population means! This is a powerful tool for facilitating population comparisons that we will draw on frequently.

Note We have essentially derived a One-Way ANOVA test using multiple regression! Mathematically it can be shown that One-Way ANOVA (in fact, most of experimental design) can be expressed a multiple regression problem.

Example: Posttraumatic stress disorder in rape victims. This example is based on a study in the Journal of Counseling and Clinical Psychology. The subjects

were 45 rape victims who were randomly assigned to one of four groups. The four groups were:

1. Stress Inoculation Therapy (SIT) in which subjects were taught a variety of coping skills
2. Prolonged Exposure (PE) in which subjects went over the rape in their mind repeatedly for seven sessions
3. Supportive Counseling (SC) which was a standard therapy control group
4. Waiting List (WL) – a baseline control group.

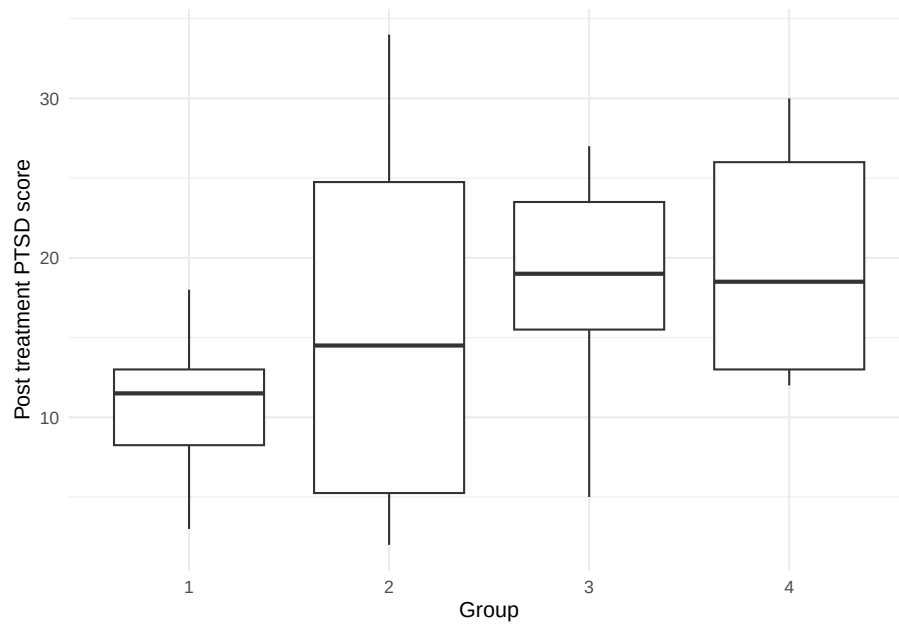
In the actual study, pre- and post-treatment measures were taken on a number of variables. For our purposes we will only look at post-treatment data on PTSD severity, which was the total number of symptoms endorsed by the subject. The goal is to compare the effects of the treatments with respect to post-treatment PTSD severity. The data appear in the R workspace `ptsd.RData` in our repository.

```
load("ptsd.RData")
head(ptsd)
```

```
##   ID Group Score
## 1  1     1     3
## 2  2     1    13
## 3  3     1    13
## 4  4     1     8
## 5  5     1    11
## 6  6     1     9
```

As always, first look at the data. Since the predictor is qualitative (although coded as numeric), side-by-side boxplots are in order. It is also a good idea to coerce the numeric codes for the variable `Group` to be recognized by R as qualitative values rather than quantitative numbers. This can be done via the `as.factor()` command:

```
ptsd <- ptsd %>%
  mutate(Group=as.factor(Group))
ggplot(ptsd) +
  geom_boxplot(aes(x=Group, y=Score)) +
  ylab("Post treatment PTSD score") +
  theme_minimal()
```

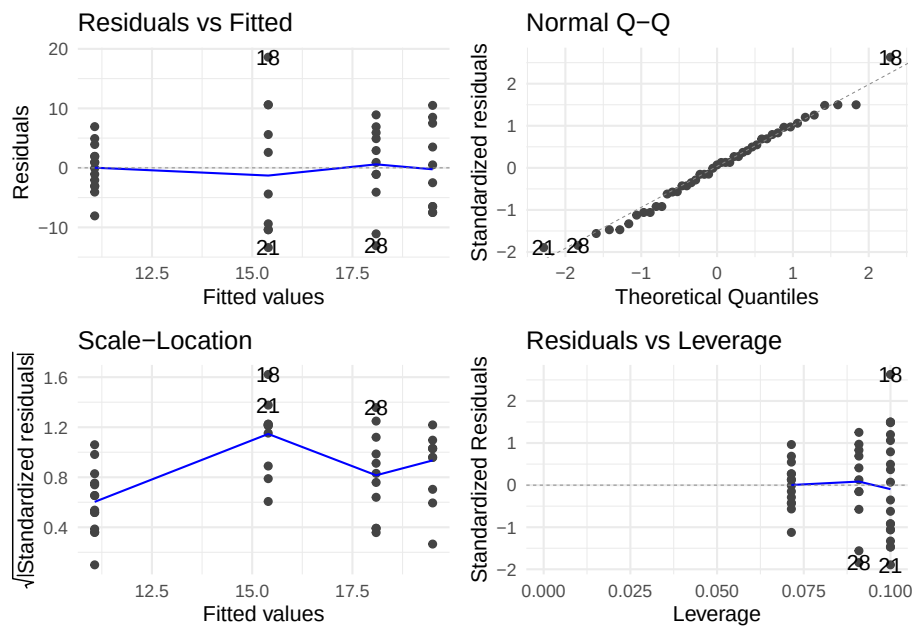


There is a lot of variability in the PTSD score in Group 2 (Prolonged Exposure treatment). At first glance, it appears as though SIT therapy may be producing a lower average number of post-treatment symptoms.

The dramatic difference in variability patterns in the response from treatment to treatment may result in a violation of the constant variance assumption in the upcoming regression analysis.

Let's run a regression using Group as our qualitative predictor of PTSD score. As always, first check the regression assumptions before any inference:

```
ptsd.fit <- lm(Score ~ Group, data=ptsd)
autoplot(ptsd.fit) + theme_minimal()
```



Note how these plots look different when dealing with a qualitative predictor.

Normality appears to be OK. No outliers appear to be present. Constant variance appears to have some potential violations (see the wobbly Scale-Location plot with a peak in the group with a fitted value of about 15.5). Nonetheless, we will proceed for now with hypothesis tests as an illustration, even though we know the results may be suspect.

```
summary(ptsd.fit)
```

```
##
## Call:
## lm(formula = Score ~ Group, data = ptsd)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.40   -4.40    0.50    4.93   18.60
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    11.07      1.99     5.56 1.8e-06 ***
## Group2         4.33      3.09     1.40 0.1683
## Group3         7.02      3.00     2.34 0.0244 *
## Group4         8.43      3.09     2.73 0.0093 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 7.46 on 41 degrees of freedom
## Multiple R-squared:  0.182, Adjusted R-squared:  0.122
## F-statistic: 3.05 on 3 and 41 DF,  p-value: 0.0394
```

Note that R by default used a dummy variable coding that sets Group=1 as the reference level for Group. The default coding in R is to order the factor levels (either alphabetically or numerically, whichever applies) and choose the first level as the reference level. Here, the ordered Group levels in the data frame are 1, 2, 3 and 4: thus, 1 was chosen as the reference level.

For convenience, you can change the reference level. The command to do so is called `relevel()` in R. In this example, Group=4 (the Waiting List baseline control group) is the most natural choice for a reference group to which we may compare the other treatments for PTSD. So, I change the Group reference level to 4 below and re-fit the model. Take note of what changes and what does not change from the first coding scheme:

```
ptsd <- ptsd %>%
  mutate(Group=relevel(Group, ref="4"))
ptsd.fit <- lm(Score ~ Group, data=ptsd)
summary(ptsd.fit)

##
## Call:
## lm(formula = Score ~ Group, data = ptsd)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.40  -4.40   0.50   4.93  18.60
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    19.50      2.36    8.27 2.8e-10 ***
## Group1         -8.43      3.09   -2.73  0.0093 **
## Group2         -4.10      3.33   -1.23  0.2258
## Group3         -1.41      3.26   -0.43  0.6676
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.46 on 41 degrees of freedom
## Multiple R-squared:  0.182, Adjusted R-squared:  0.122
## F-statistic: 3.05 on 3 and 41 DF,  p-value: 0.0394
```

We can investigate the effects of different therapies using elements from the output above as follows:

- The whole-model F-test for $H_0 : \mu_1 = \mu_2 = \mu_3 = \mu_4$ is significant ($F(3, 41)$)

$= 3.046$, $p\text{-value} = 0.0394$), so we can conclude that there is a significant difference in the true mean post-treatment PTSD severity scores between at least two of the treatments. [Note how the F -test is invariant to the dummy variable scheme used.]

- The different therapies applied account for only 12.2% of the total variation in the observed PTSD severity scores. [R_a^2 is also invariant to the dummy variable scheme used.]
- The estimated mean PTSD scores for the four treatment groups are:
 - $b_0 = 19.5$ for the Waiting List (WL) control group,
 - $b_0 + b_1 = 19.5 - 8.429 = 11.071$ for the SIT therapy,
 - $b_0 + b_2 = 19.5 - 4.100 = 15.4$ for the PE therapy,
 - $b_0 + b_3 = 19.5 - 1.409 = 18.091$ for the SE therapy.
- In comparing therapies to the WL control therapy, the only significant difference in mean PTSD score occurs with Group 1 (SIT). The test of $H_0 : \beta_1 = 0$ here is equivalent to a test of $H_0 : \mu_{SIT} - \mu_{WL} = 0$. There is sufficient evidence here to conclude that the true mean PTSD score is significantly lower in the SIT therapy than under the WL therapy ($t = -2.731$, $df = 41$, $p\text{-value} = 0.00928$).

A 95% confidence interval for β_1 (and hence $\mu_{SIT} - \mu_{WL}$) can be found thus:

```
confint(ptsd.fit)
```

```
##              2.5 %   97.5 %
## (Intercept) 14.73889 24.26111
## Group1      -14.66232 -2.19482
## Group2      -10.83322  2.63322
## Group3       -7.98751  5.16932
```

We can be 95% confident that the SIT therapy will produce, on average, between 2.19 to 14.66 fewer PTSD symptoms than will the WL therapy. [Note that this CI is the only one among the CIs for β_1 , β_2 , and β_3 that does not contain 0.]

7.3 Bridging Regression and Designed Experiments – ANCOVA

Analysis of Covariance (or ANCOVA) refers to regression problems that have a mixture of both quantitative and qualitative predictors.

As an illustration, let's return to the experiment where we are interested in the effect of a fixed dosage of two different medications on reducing blood glucose level in men. One of these meds is an experimental drug (drug A), and the other is a placebo (drug P). Let's create a dummy variable D for distinguishing the drug groups as follows:

$$D = \begin{cases} 0 & \text{if drug } P \\ 1 & \text{if drug } A \end{cases}$$

Suppose that we know that the body weight of an individual might influence the effectiveness of the drug at the administered dose. For example, the effectiveness of the placebo might not be impacted by body weight, but it may be the case that for people taking the experimental drug, the more you weigh the less effective the drug is. If this is the case, then perhaps we should not model glucose reduction using only D as our predictor.

Analysis of covariance is a technique that can be used to build a model to predict glucose reduction as a function of both body weight (a quantitative predictor, sometimes called a covariate) and type of drug (a qualitative predictor, or factor). ANCOVA adjusts the group comparison for weight differences and then estimates the effectiveness of the drugs.

ANCOVA is a powerful analytical tool because:

- it is tantamount to simultaneously fitting different regression models to the different groups and then formally comparing the models.
- it can also be used when you have more than two groups and/or more than one covariate.

For now, we'll look at the simple case of two groups and one covariate.

Let Y = glucose reduction and X = body weight. A variety of different linear models may be considered here, offered below from simple to complex:

1. Same regression line for both groups. We fit the model $Y = \beta_0 + \beta_1 X + \varepsilon$. Nothing in the model references which drug is administered, so both drug groups are effectively “pooled” together. This model imposes structure on the data that says that the effect of body weight on glucose reduction is identical regardless of drug – obviously, this may be too simplistic and unrealistic.
2. Separate regression lines for each group, but with equal slopes. We fit the model $Y = \beta_0 + \beta_1 X + \beta_2 D + \varepsilon$. This is known as an additive model involving weight and drug type. This model imposes structure on the data that says that the effect that the different drugs have glucose reduction is the same regardless of weight. In other words, this model allows for a difference in drug effect to be modeled, but restricts that effect to be the same regardless of body weight.

Writing out the model explicitly for both drug groups will prove enlightening. The model parameters and their interpretations break down as follows:

Drug	D	Model
placebo	0	$Y = \beta_0 + \beta_1 X + \beta_2(0) + \varepsilon = \beta_0 + \beta_1 X + \varepsilon$
A	1	$Y = \beta_0 + \beta_1 X + \beta_2(1) + \varepsilon = (\beta_0 + \beta_2) + \beta_1 X + \varepsilon$

You can see that the slope of the model for either drug is the same ($= \beta_1$), so the model imposes that body weight has the same effect on glucose

reduction regardless of drug. However, the intercept terms differ between the models for the two drugs: it is β_0 for placebo, but $\beta_0 + \beta_2$ for drug A . Since the difference between these two intercepts is β_2 , its interpretation is that it represents the constant shift in glucose reduction when changing from placebo to drug A . In short, β_2 measures the effect of drug regardless of body weight, and β_1 measures the effect of body weight regardless of drug. Of course, if the effect of drug truly depends on body weight, then we can't separate their influence on the response; i.e., their effects are not additive. Which leads us to....

3. Separate regression lines for each group, with different slopes. Recall that earlier we said that the effectiveness of the placebo might not be impacted by body weight, but it may be the case that for people taking the experimental drug that the more you weight the less effective the drug is. If so, the effective difference between the drugs depends on body weight: perhaps for lower weight people the drugs differ greatly in effectiveness, but as weight increases the difference in effectiveness vanishes. In such a case, we say that drug and weight interact to determine the response. Here's a formal definition:

Two variables X_1 and X_2 are said to **interact** if the effect that X_1 has on the response Y depends on the value of X_2 .

To fit an interaction model, we include multiplicative terms between the covariate(s) and factor(s). The model is

$$Y = \beta_0 + \beta_1 X + \beta_2 D + \beta_3 (D \cdot X) + \varepsilon$$

This is called an interaction model involving weight and drug type. This model imposes structure on the data that says that the effect that the different drugs have on glucose reduction may vary depending on weight. In other words, this model allows for a difference in drug effect to be modeled, and that the size of that drug effect may vary depending on body weight. Here is how this model breaks out explicitly for each drug:

Drug	D	Model
placebo	0	$Y = \beta_0 + \beta_1 X + \beta_2(0) + \beta_3(0 \cdot X) + \varepsilon$ $= \beta_0 + \beta_1 X + \varepsilon$
A	1	$Y = \beta_0 + \beta_1 X + \beta_2(1) + \beta_3(1 \cdot X) + \varepsilon$ $= (\beta_0 + \beta_2) + (\beta_1 + \beta_3)X + \varepsilon$

The models for each drug differ in both y-intercept and slope, so we are basically fitting different regression lines between glucose reduction and weight for each drug. Of specific interest is how the slopes differ between the models for the two drugs: it is β_1 for placebo and $\beta_1 + \beta_3$ for drug A . Since the difference between these two slopes is β_3 , its interpretation is that it represents the difference in the

rates of change in glucose reduction for a one-unit increase in weight between placebo to drug *A*. In short, β_3 measures the difference in the effect of weight between the two drugs. If $\beta_3 \neq 0$, then the two groups have different slopes and thus the size of the drug effect on glucose reduction depends on body weight.

Since the unequal slopes model is most flexible, we usually use it as our initial model and then simplify the model if warranted. The next example illustrates this using R.

7.3.1 An ANCOVA example with a two-level factor

Example: Glucose Reduction. We conduct an experiment to study the efficacy of a fixed dosage of two different medications (a new experimental drug *A*, and a placebo *P*) on reducing blood glucose level in men. Body weight may also have an effect on a drug's impact on glucose levels (measured in mg/dl), so it is included as a covariate for the analysis. The data are in the R workspace `drug2.RData` in our data repository.

```
load("drug2.RData")
head(drug2)
```

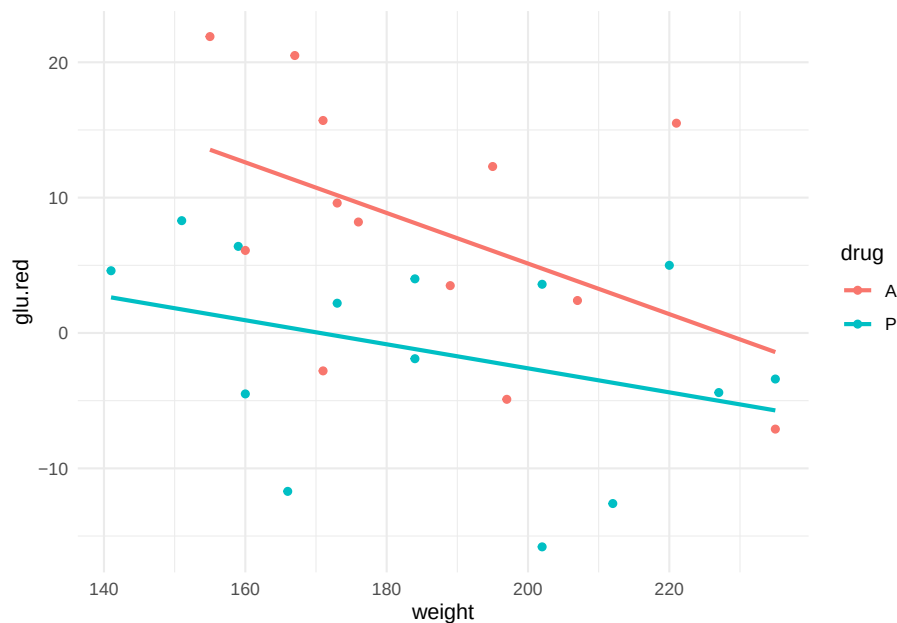
```
##   drug weight glu.red
## 1    P   160   -4.5
## 2    P   235   -3.4
## 3    P   202  -15.8
## 4    P   173    2.2
## 5    P   166  -11.7
## 6    P   220    5.0
```

First, let's look at the data and generate a scatterplot of `glu.red` by `weight`. Because we now have a trend line relating glucose reduction to weight for each drug group, we can use an enhanced scatterplot by adding on another aesthetic layer (`color`).

```
ggplot(drug2) +
  geom_point(aes(x=weight, y=glu.red, color=drug) ) +
  geom_smooth(aes(x=weight, y=glu.red, color=drug), se=FALSE, method="lm") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

7.3. BRIDGING REGRESSION AND DESIGNED EXPERIMENTS – ANCOVA203

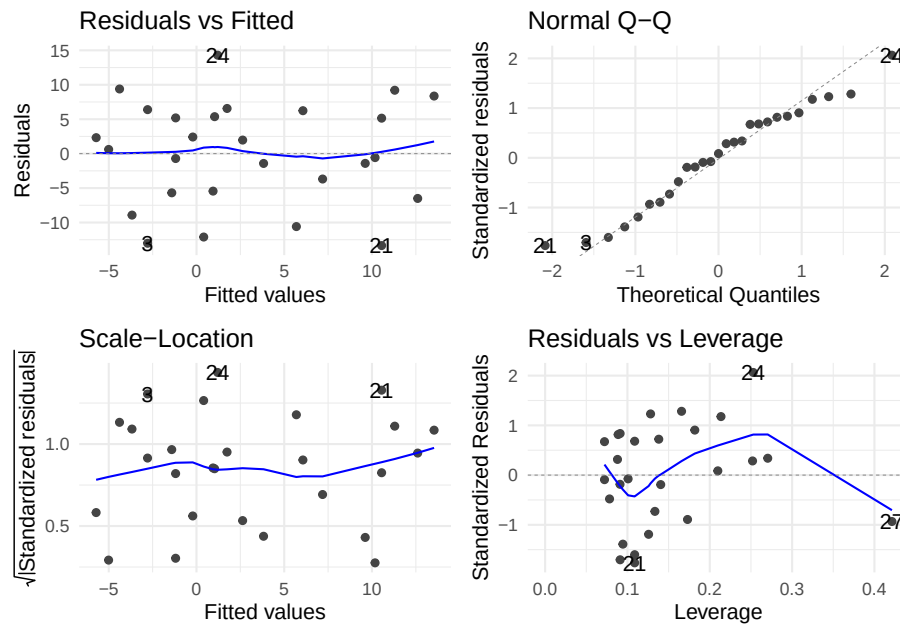


We get to see the scatter of points, plus both a linear trend line for both drug groups. Here are some preliminary observations:

- It appears as though, over this weight range, that there is higher general glucose reduction in experimental drug group (A).
- There appears to be some effect of weight on glucose reduction (generally lower glucose reduction measurements in heavier men; slopes are negative).
- There also appears to be some evidence that the effect of weight on glucose reduction is more pronounced under the experimental drug as opposed to the placebo (steeper slope for drug A).

Because of the third bullet point above, we should fit the interaction model first. I do so below, after first checking assumptions:

```
drug.fit <- lm(glu.red ~ weight + drug + weight:drug, data=drug2)
autoplot(drug.fit) + theme_minimal()
```



The assumptions of constant variance, normality and linearity look good. We proceed to investigate the model:

```
summary(drug.fit)
```

```
##
## Call:
## lm(formula = glu.red ~ weight + drug + weight:drug, data = drug2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.350  -5.566   0.608   5.802  14.294
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  42.5076    17.9473   2.37  0.027 *
## weight       -0.1869     0.0958  -1.95  0.063 .
## drugP        -27.3532    22.7834  -1.20  0.242
## weight:drugP  0.0981     0.1212   0.81  0.427
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8.01 on 23 degrees of freedom
## Multiple R-squared:  0.381, Adjusted R-squared:  0.3
## F-statistic: 4.71 on 3 and 23 DF, p-value: 0.0105
```

First notice that R automatically created a dummy variable for the two-level drug factor using the following coding:

$$D = \begin{cases} 0 & \text{if drug } A \\ 1 & \text{if drug } P \end{cases}$$

The whole-model F -test is significant ($F(3, 23) = 4.712$, p -value = 0.01047), verifying that the model does have utility in explaining glucose reduction in men. So, we proceed to investigate the effects of weight and drug type.

The first terms you should always check in an interaction model are the interaction terms! Here, we proceed to first check for “equal slopes” by testing the interaction term to see if we can simplify the model. There is only one dummy variable constructed since drug has two levels, so we can just look at the single interaction term `weight:drugP`. We test $H_0 : \beta_{\text{weight.drug } P} = 0$ to check for equal slopes. The test statistic is $t = 0.809$ with a p -value of 0.4267. Since this is not significant, we can delete the interaction term without hurting the explanatory power of the model. So, an equal slopes (i.e. additive) model is warranted.

We now drop the interaction term, which reduces the model to a main effects model:

```
drug.fit.reduced <- lm(glu.red ~ weight + drug, data=drug2)
summary(drug.fit.reduced)

##
## Call:
## lm(formula = glu.red ~ weight + drug, data = drug2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -12.877  -5.674   0.282   5.459  12.145
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  31.1166    11.0526   2.82  0.0096 **
## weight       -0.1256     0.0583  -2.16  0.0413 *
## drugP        -9.0871     3.0639  -2.97  0.0067 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.95 on 24 degrees of freedom
## Multiple R-squared:  0.363, Adjusted R-squared:  0.31
## F-statistic: 6.84 on 2 and 24 DF, p-value: 0.00446
```

The weight-adjusted effect of drug on glucose reduction is highly significant ($t = -2.966$, $p = 0.0067$). Weight is marginally significant ($t = -2.156$, $p = 0.0413$).

Also note that the model SE has been improved from 8.011 to 7.953. The model cannot be further reduced without sacrifice of explanatory power for the response.

The fitted model, where Y = glucose reduction, is

$$\hat{Y} = 31.116 - 0.126 \times \text{weight} - 9.087 \times D$$

where $D = 1$ if the placebo is administered, and 0 otherwise. Using a breakdown like we first introduced above, we can write out the separate regression lines for each drug:

Drug	D	Model
<i>placebo</i>	1	$31.116 - 0.013 \times \text{weight} - 9.087(1)$ $= 22.029 - 0.013 \times \text{weight}$
<i>A</i>	0	$31.116 - 0.013 \times \text{weight} - 9.087(0)$ $= 31.116 - 0.013 \times \text{weight}$

We can use `confint()` to find CIs for the β -parameters in an ANCOVA model since ANCOVA is just a special case of the usual multiple linear regression model. Here is the result from R, with interpretations provided:

```
confint(drug.fit.reduced)
```

```
##                2.5 %       97.5 %
## (Intercept)  8.305129 53.92798704
## weight      -0.245841 -0.00539219
## drugP       -15.410611 -2.76351199
```

- The 95% CI for β_{weight} is given by $(-0.2458, -0.0054)$. We can be 95% confident that each additional pound of weight lowers the true mean glucose reduction by between 0.0054 mg/dl to 0.2458 mg/dl, regardless of drug. [Note: These are very small changes when expressed in terms of weight change per pound, so it might be more useful to re-express the change in terms of a 10 lb weight increase: the same CI then becomes $(10 \times (-0.0054), 10 \times (-0.2458)) = (-0.054, -2.458)$ mg/dl drop in glucose reduction for every 10 lb increase in weight, regardless of drug.]
- The 95% CI for β_D is given by $(-15.41, -2.76)$. Recall that β_D measures the change in mean glucose reduction when changing from drug *A* to the *placebo*. Thus, we can be 95% confident that drug *A*'s efficacy is between 2.76 mg/dl to 15.41 mg/dl higher with the placebo. This finding is valid regardless of weight since we are adjusting for weight in the equal slopes model.

R^2 is not great for this model, but none the less we may use the model for making predictions. Here are a few model predictions using R:

```
p.A.150 <- predict(drug.fit.reduced, newdata=data.frame(weight=150, drug="A"),int="conf")
p.A.150
```

```
##      fit      lwr      upr
## 1 12.2741 5.99873 18.5494
```

```
p.P.150 <- predict(drug.fit.reduced, newdata=data.frame(weight=150, drug="P"),int="conf")
p.P.150
```

```
##      fit      lwr      upr
## 1 3.18701 -3.04853 9.42255
```

```
p.A.225 <- predict(drug.fit.reduced, newdata=data.frame(weight=225, drug="A"),int="conf")
p.A.225
```

```
##      fit      lwr      upr
## 1 2.85283 -3.68925 9.39491
```

```
p.P.225 <- predict(drug.fit.reduced, newdata=data.frame(weight=225, drug="P"),int="conf")
p.P.225
```

```
##      fit      lwr      upr
## 1 -6.23423 -12.5805 0.112083
```

The first interval may be interpreted as follows: we can be 95% confident that the true mean reduction in glucose due to drug A for all men weighing 150 lb to be between 5.99 mg/dl to 18.55 mg/dl. For men of the same weight taking the placebo, the corresponding interval is -3.05 mg/dl to 9.39 mg/dl.

In short, the usual procedure for an ANCOVA model is:

1. Start by fitting a full interaction (“different slopes”) model.
2. Test the interaction term(s) first.
 - If the interaction term(s) is/are insignificant, delete them and fit an “equal slopes” model. Proceed to step 3.
 - If the interaction term(s) is are significant, the model cannot be further simplified. You must interpret the effect of the covariate by level of the qualitative predictor.
3. If using an equal slopes model, you may investigate the overall effects of the qualitative predictor and the covariate independently.

7.3.2 ANCOVA with a multi-level factor

Example: As an example, we revisit the data from the previous section, only now augment the experiment to study the efficacy of four drugs (called *A*, *B*, *C* and a placebo *P*) on glucose reduction (measured in mg/dl). Weight is still our covariate, and its effect may be different depending on the drug administered.

Coding up 4 drugs in the regression requires $4 - 1 = 3$ dummy variables, as first cited above. We have flexibility in how we could define these three dummy

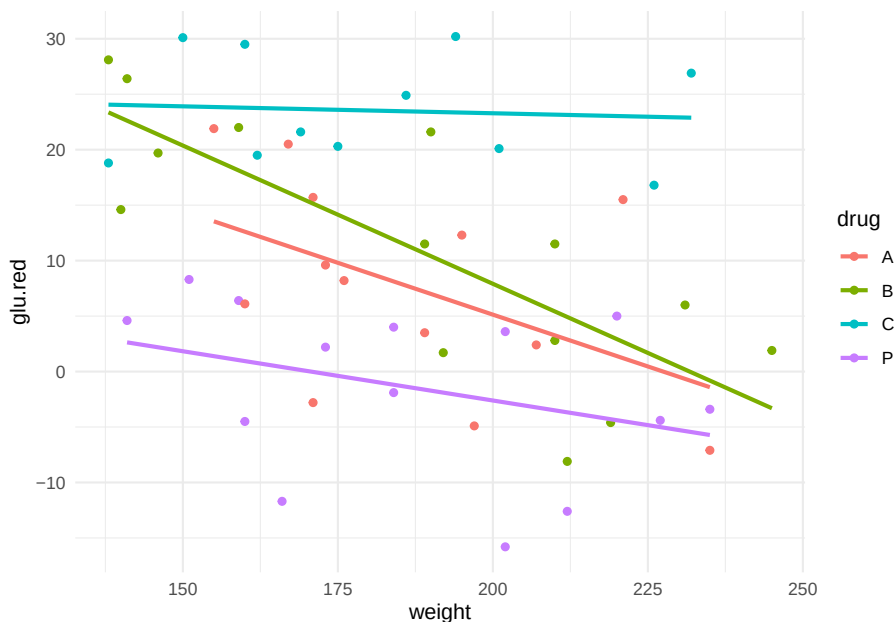
variables, but here is probably the most meaningful way given the context of the drug experiment:

$$D_A = \begin{cases} 1 & \text{if drug } A \\ 0 & \text{otherwise} \end{cases}, \quad D_B = \begin{cases} 1 & \text{if drug } B \\ 0 & \text{otherwise} \end{cases}, \quad D_C = \begin{cases} 1 & \text{if drug } C \\ 0 & \text{otherwise} \end{cases}$$

The above dummy variable coding scheme is the most meaningful way to code drug for this analysis because the placebo is the natural choice for a “reference” drug.

The data are in the R workspace `drug4.RData` and let’s look at an advanced scatterplot

```
## `geom_smooth()` using formula = 'y ~ x'
```



Preliminary assessment: It appears that drugs *A* and *B* perform very similarly across individuals of varying weights but lose their effectiveness in heavier individuals. However, weight does not appear to have as much of an impact on the effectiveness of drug *C* (fairly shallow slope); moreover, glucose reduction for *C* is generally quite high regardless of weight. The placebo *P* appears to be largely ineffective (the responses tend to fluctuate around zero), but it is interesting that there appears to be some weight effect in the placebo (can you think of an explanation for this?).

Step 1. We start with the full interaction (“different slopes”) model. We also choose to set the placebo as the reference level. Checking residual assumptions

are left as an exercise to the interested reader.

```
drug4 <- drug4 %>%
  mutate(drug=relevel(drug, ref="P"))
m1 <- lm(glu.red ~ weight + drug + drug:weight, data=drug4)
summary(m1)
```

```
##
## Call:
## lm(formula = glu.red ~ weight + drug + drug:weight, data = drug4)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
##	-13.350	-5.304	0.729	5.456	14.294

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
## (Intercept)	15.1544	12.8630	1.18	0.245
## weight	-0.0888	0.0680	-1.31	0.198
## drugA	27.3532	20.8811	1.31	0.197
## drugB	42.6055	16.7041	2.55	0.014 *
## drugC	10.6264	19.1505	0.55	0.582
## weight:drugA	-0.0981	0.1111	-0.88	0.382
## weight:drugB	-0.1604	0.0881	-1.82	0.075 .
## weight:drugC	0.0763	0.1030	0.74	0.463

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.34 on 44 degrees of freedom
## Multiple R-squared:  0.692, Adjusted R-squared:  0.643
## F-statistic: 14.1 on 7 and 44 DF, p-value: 1.98e-09
```

The whole-model F -test is significant. Let's check to see if the model can be simplified. The three interaction terms in the model (`weight.drugA`, `weight.drugB` and `weight.drugC`) assess differences in slope between the reference drug P versus A , B and C respectively. The null hypothesis for testing if the slopes are equal is:

$$H_0 : \beta_{\text{weight.drugA}} = \beta_{\text{weight.drugB}} = \beta_{\text{weight.drugC}} = 0$$

If this hypothesis is true, then the four slopes are the same; i.e. parallel. To see if we can simplify to an “equal slopes” model, test these three parameters using an ANOVA F -test:

```
m2 <- lm(glu.red ~ weight + drug, data=drug4)
anova(m2, m1)
```

```
## Analysis of Variance Table
##
## Model 1: glu.red ~ weight + drug
## Model 2: glu.red ~ weight + drug + drug:weight
##   Res.Df  RSS Df Sum of Sq    F Pr(>F)
## 1      47 2765
## 2      44 2372  3      393.3 2.432 0.0777 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The F -test for the null hypothesis of parallel slopes is marginally significant ($F = 2.43$, $\text{dfnum} = 3$, $\text{dfden} = 44$, $p\text{-value} = 0.0777$). There is *marginal* evidence of an interaction between drug and weight: the effect of drug on glucose reduction depends marginally on the weight of the person.

Should we simplify to the “equal slopes” model (model m2 above)? There are a couple of things you can consider to help you decide:

- From the parameter estimates above, it appears that the largest difference between drugs with respect to how weight affects the reduction in glucose is between the placebo and drug B . The slope estimate for the placebo is -0.08882 ; for drug B , the slope estimate is $-0.08882 + (-0.16043) = -0.249$. This means that each additional pound of weight lowers the mean glucose reduction by 0.249 mg/dl if taking drug B . This change is marginally significant (see `weight:drugB`: $t = -1.822$, $p\text{-value} = 0.0753$), and can be estimated with a CI for $\beta_{\text{weight:drugB}}$:

```
confint(m1)
```

```
##              2.5 %      97.5 %
## (Intercept) -10.769269 41.0780042
## weight      -0.225934  0.0482880
## drugA       -14.729867 69.4362663
## drugB        8.940645 76.2702598
## drugC       -27.968812 49.2215998
## weight:drugA -0.321899  0.1257777
## weight:drugB -0.337924  0.0170668
## weight:drugC -0.131266  0.2839360
```

The 95% confidence interval for the true mean difference in slopes between the placebo and B is $(-0.3379, 0.0170)$ mg/dl per pound. This CI contains 0, suggesting no significant slope difference.

- You could check the change in the residual SE or R_{adj}^2 values by using the “equal slopes” model:

```
summary(m2)
```

```
##
## Call:
```

7.3. BRIDGING REGRESSION AND DESIGNED EXPERIMENTS – ANCOVA211

```
## lm(formula = glu.red ~ weight + drug, data = drug4)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.46  -5.04   0.35   5.38  13.01
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  26.6557     7.0688   3.77 0.00045 ***
## weight      -0.1504     0.0362  -4.15 0.00014 ***
## drugA         9.0639     2.9546   3.07 0.00357 **
## drugB        12.5859     2.8992   4.34 7.5e-05 ***
## drugC        24.1076     3.0974   7.78 5.4e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.67 on 47 degrees of freedom
## Multiple R-squared:  0.641, Adjusted R-squared:  0.61
## F-statistic: 20.9 on 4 and 47 DF,  p-value: 5.81e-10
```

R_{adj}^2 dropped from 0.6424 to 0.6099 by simplifying to the “equal slopes” model. This detriment to the quality of fit suggests there is validity in retaining the “different slopes” model.

Chapter 8

Model Building Considerations

We now spend some time on the intricacies of regression modeling.

8.1 Regression assumptions revisited

Any time we run a hypothesis test or build a confidence or prediction interval in the context of a regression analysis or ANOVA, the validity of the findings depends on the assumptions being met. Here we remind you of the assumptions:

Error assumptions:

1. The errors are independent (the independence assumption)
2. The errors have homogeneous variance (the constant variance assumption)
3. The errors are normally distributed (the normality assumption)

Linearity assumption:

- In the context of regression, We assume that the structural part of the linear regression model is correctly specified.

Unusual observations:

- Occasionally, a few observations may not fit the model well. These have the potential to dramatically alter the results, so we should check for them and investigate their validity.

We have already introduced graphical ways to check these assumptions using a process we call regression diagnostics. Think of diagnostics as preventative medicine for your data.

Our goal now is to investigate ways to address assumption violations so that we may trust the results of statistical tests and confidence intervals. We can tell you that the typical approach to remedy assumption violations usually involves some form of data transformation or a specifying a new (better) model.

8.2 Violations of the independence assumption

Arguably the most vital of all assumptions necessary to validate tests and confidence intervals in the assumption that the errors are independent (i.e. uncorrelated). Even mild violations can greatly tarnish statistical inferences.

In general, checking for independence is not easy. However, if you collect your data according to a simple random sampling scheme with one observation per subject, there is usually no reason to suspect you will have a problem. In such a case, simple random sampling of individuals ensures independence of the information contributed to the study by each individual. In the realm of experimental design, this is typically handled through the design. The only circumstances where you will need to address the issue are outlined below.

Correlated (non-independent) errors can arise as a result of:

8.2.1 Collecting data that are temporal or spatial in nature

This means that the data are such that you can meaningfully think about the “proximity” of observations to each other. This can happen by thinking in terms of time (e.g., measurements taken more closely together in time might tend to be more similar) or space (e.g., measurements located more geographically close to each other might tend to look more similar). Here are two examples:

Example: Corn yields. Suppose you are interested in studying the effect of annual rainfall on a crop yield (in bushels per acre). For a particular plot of land, you have the following measurements:

- **year** Year studied (1970-2007)
- **yield** Yield of corn (in bushels/acre)
- **rain** Annual rainfall (in inches)

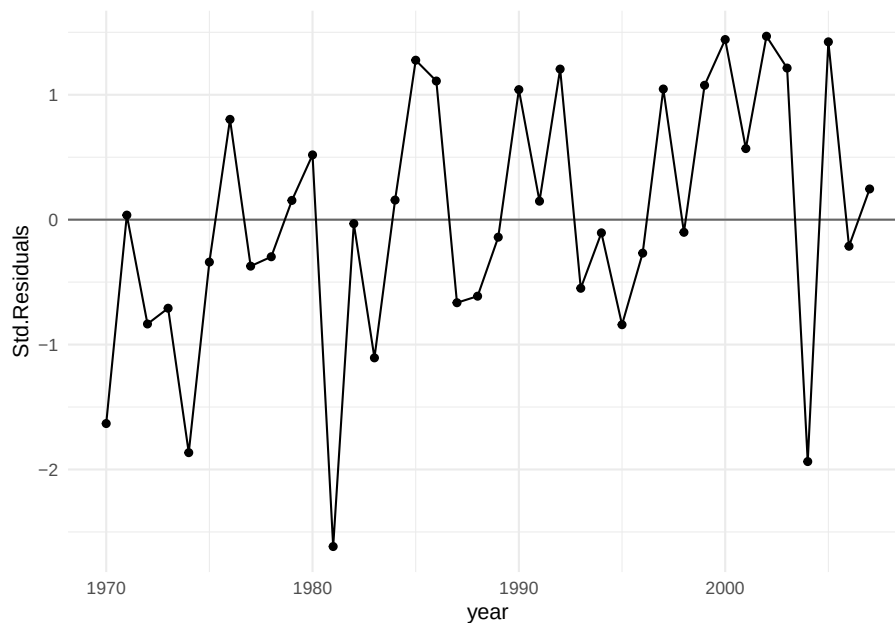
Here, we are repeatedly measuring the same plot of land over time. It might be reasonable to suspect that the amount of rainfall in years immediately preceding a given year might have some impact on the yield in that given year (e.g., recovery from years of drought). If so, the observations within this plot might not be independent.

If the data are obtained in a time sequence, a residuals vs. order plot (or index plot) helps to see if there is any correlation between the error terms that are near each other in the sequence. **This plot is only appropriate if you know the order in which the data were collected!** Highlight this, circle this,

do whatever it takes to remember it — it is a very common mistake made by people new to regression diagnostics.

Below is R code for the corn yield data. This code fits a simple linear regression model where the yield is a function of the annual rainfall amount, and generates a plot of the standardized (mean 0, standard deviation of 1) residuals vs `year` (which corresponds to the order of the data).

```
site <- "http://www.users.miamioh.edu/hughesmr/sta363/cornyield.txt"
site <- "docs/data/cornyield.txt"
corn <- read_table(site)
corn.fit <- lm(yield ~ rain, data=corn)
corn <- corn %>%
  mutate(Std.Residuals = rstandard(corn.fit))
ggplot(corn) +
  geom_abline(intercept=0, slope=0, color="gray40") +
  geom_line(aes(x=year, y=Std.Residuals)) +
  geom_point(aes(x=year, y=Std.Residuals)) +
  theme_minimal()
```



In general, residuals exhibit random fluctuation around the $e_i = 0$ line across time suggests that there is no correlation. If there was correlation in the residuals, we would see longer consecutive runs of residuals either above or below the $e_i = 0$ line. Unless these effects are strong, they can be difficult to spot. Nothing is obviously wrong here.

Example: Ground cover. Suppose we are engaged in a field study in a

mountainous region. We take measurements at various locations on a particular mountain in order to study how rainfall, elevation and soil pH impact the amount of cover (per square meter) of a certain species of moss.

Why might non-independence be a concern in this example? Think about two measurements taken at geographically close locations: they might tend to look more similar than those made between measurement locations far apart, even if the predictors are all similar in value. In other words, proximity might produce similarity. That violates independence.

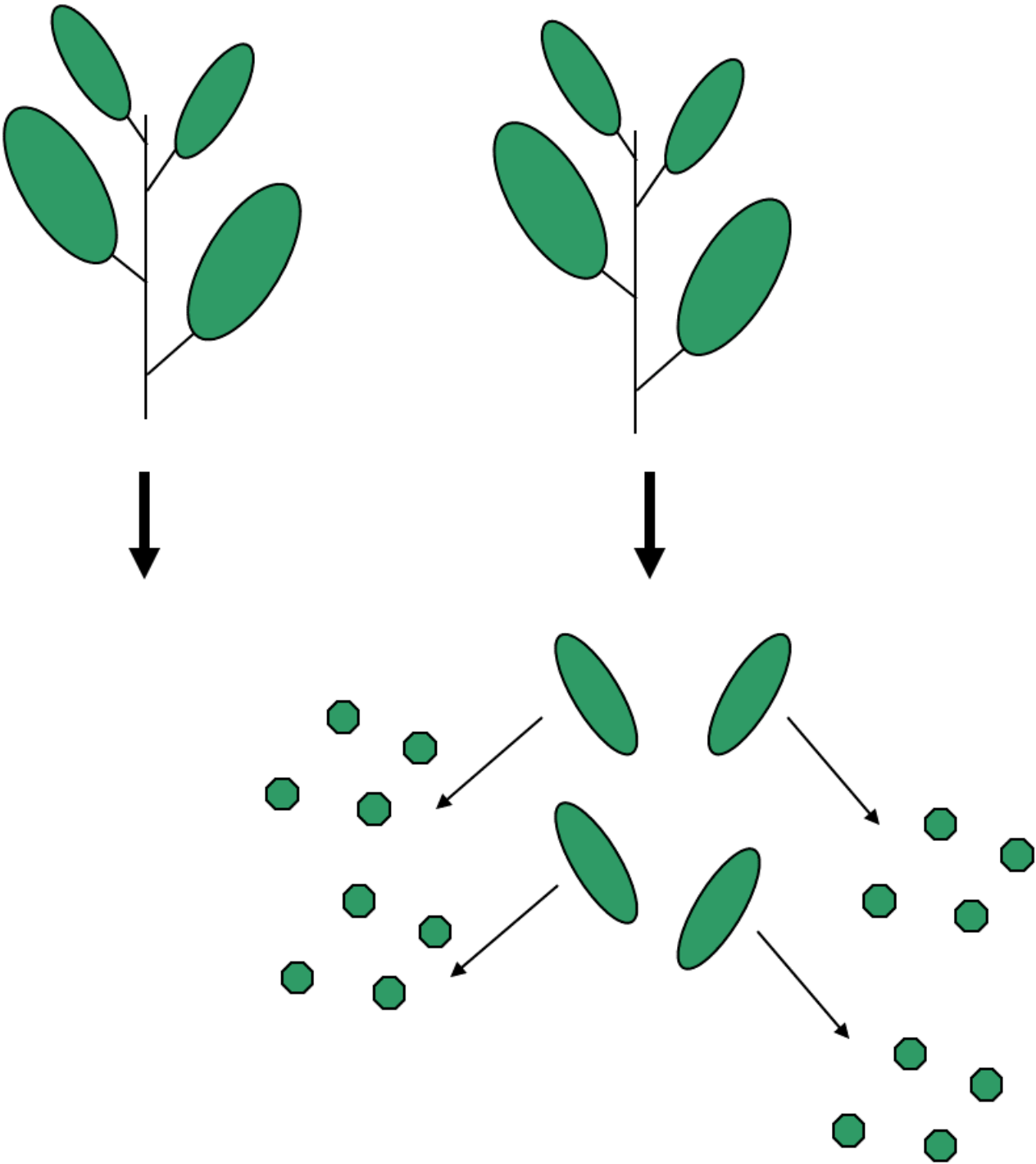
8.2.2 Pseudoreplication

In the most basic form of a designed experiment, each separate subject (EU) contributes one independent piece of information about the response to the study. But, common to all designed experiments is the notion of replication: this means we assign the same experimental conditions to several independent subjects (EUs), the idea being that we want to know how much natural variation in the response is present without changing the conditions of the experiment in order to estimate the random error component in a model. Two or more independent EUs receiving the same treatment are called replicates.

Pseudoreplication is actually a form of spatial correlation introduced above in the ground cover example, but here it is purposefully built into the sampling plan of a designed experiment. Here's an example:

Example: Calcium content of leaves. In a study to analyze the calcium content in leaves, three plants are chosen, from which four leaves were sampled. Then from each of these twelve leaves, four discs were cut and processed.

An illustration of the data collection process appears below



This dataset will have 48 values of calcium concentration from the leaf discs. But to treat these values as 48 independent pieces of information would not be correct! Why?

Discs from the same leaves are likely to be closer in value than those from different leaves, and those on the same plant will be more similar to each other than those from different plants. There are natural clusters or hierarchies within the data, and this fact needs to be reflected in the way it is analyzed.

In truth, this example has only three replicates (the number of plants), not 48. The measurements within a plant are not real replicates, but rather pseudoreplicates.

8.2.3 What if we have non-independent errors?

Your first step is to think about the context to determine if the data are likely to have the problem. The solution to the non-independence problem always necessitates a change in how the data are analyzed.

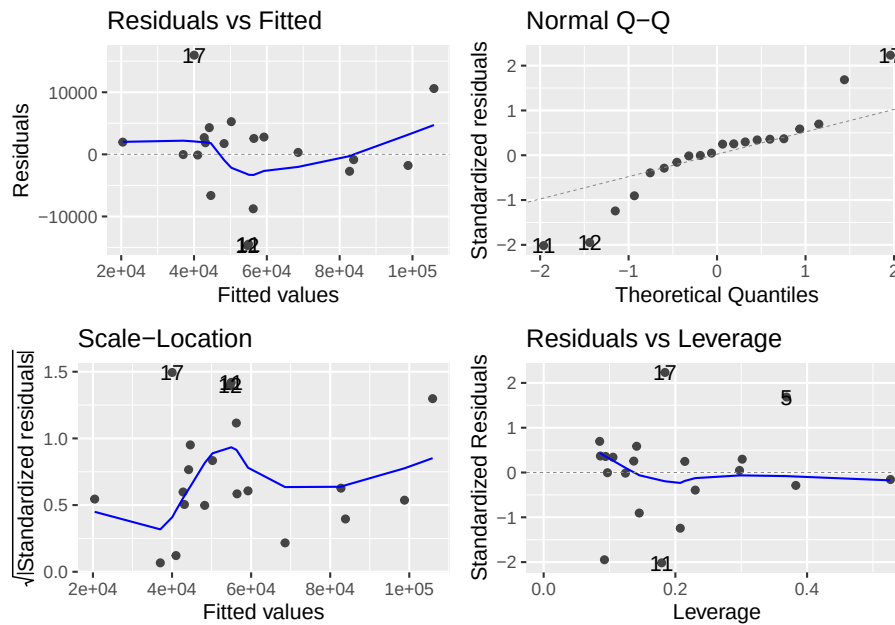
1. If you have a designed experiment that has pseudoreplication or temporal characteristics built-in, you will need to account for this in how you construct your model. Examples of this sort of structure are covered in ??.
2. If you have observational data that has pseudoreplication or spatial or temporal characteristics, more advanced modeling techniques are required that are beyond the scope of this course (e.g. times series models, random coefficients models, etc.).

8.3 Constant Variance Violations

For hypothesis tests, confidence interval or prediction interval results to be correct, we must assume that the variance of the errors is constant; i.e., it does not change across varying values of the predictors or the response. Constant error variance is sometimes referred to as homoscedasticity. This is usually a more crucial assumption than normality in validating model inferences but violations of the two are often connected.

The basic plot to investigate the constant variance assumption is a Residuals vs Fitted plot; i.e., a plot of e_i versus $\hat{y}_i$. If all is well, you should see fairly uniform (“constant”) variability in the vertical direction and the scatter should be symmetric vertically around zero. This plot is the first of the four-plot diagnostic summary in R (upper-left corner):

```
appraisal.fit <- lm(saleprice ~ landvalue + impvalue + area, data=appraisal)
autoplot(appraisal.fit)
```



R adds a smoother to help you determine trend, which looks non-existent here (which is good). However, don't put too much credence in the smoother if the sample size is small. The thing you should notice is what a violation looks like. Typically, a violation appears as a systematic fanning or funneling of points on these plots. This can occur in either direction: left to right, or right to left. Our experience has been that students learning regression diagnostics for the first time tend to over-interpret these plots, looking at every twist and turn as something potentially troublesome. You will especially want to be careful about putting too much weight on residual versus fitted plots based on small datasets. Sometimes datasets are just too small to make plot interpretation worthwhile. Don't worry! With a little practice, you will learn how to read these plots.

A refinement of the residuals vs fitted plot is the Scale-Location plot (bottom-left). The difference is that instead of plotting the raw residuals e_i on the vertical axis, R first standardizes them (so you can better check for extreme cases), takes their absolute value (to double the resolution in the plot) and then takes their square root (to remove skew that sometimes affects these plots). A smoother trending upward or downward from left to right indicates a constant variance problem. Again, only check the smoother if n is large.

What if we have non-constant error variance? There are a few strategies to address the problem, which we outline here:

1. Try a power, or Box-Cox, transformation on Y . Sometimes problems of non-normality and non-constant variance go hand-in-hand, so treating one problem frequently cures the other. However, be aware that this is not

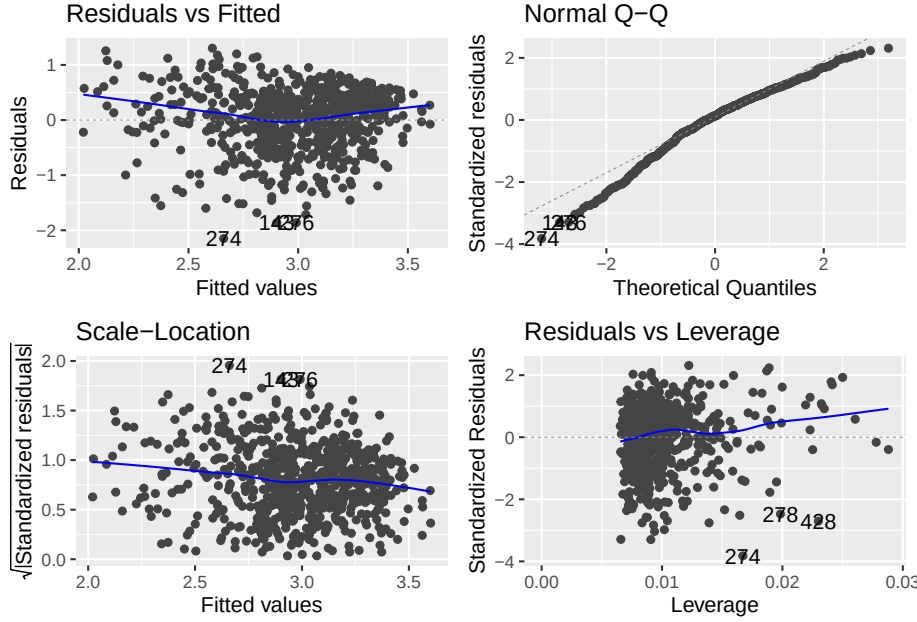
always the case. Common “on-the-fly” first choices for transformations are $\sqrt{Y}$ and $\log(Y)$. The log transformation would be more appropriate for more severe cases.

2. Weighted least squares (WLS) regression is a technique that fits a regression model not by minimizing $RSS = \sum e_i^2$ as in ordinary regression, but rather by minimizing a weighted residual sum of squares $WRSS = \sum w_i e_i^2$, where a “weight” w_i is attached to observation i . w_i is chosen in such a way so that it is inversely proportional to the variance of the error ε_i at that point. We leave out the details to WLS here but it is easily implemented in R by using the `weights` option in `lm()` and the topic is covered in more advanced regression courses..

3. Use a Generalized Linear Model (GLM). For certain response variables, there may be a natural connection between the value of the response and the variance of the response. Two common examples of this are when your Y variable is a count or a proportion. In these cases, you should instead consider fitting a form of a generalized linear model. These types of models allow flexibility in choice of an error distribution (it doesn’t necessarily need to be normal) and error variance (doesn’t necessarily need to be constant). We will see a bit more on this later in the course; see ?? and ??.

Example: University admissions. A director of admissions at a state university wanted to determine how accurately students’ grade point averages at the end of their freshman year could be predicted by entrance test scores and high school class rank. We want to develop a model to predict `gpa.endyr1` from `hs.pct` (High school class rank as a percentile), `act` (ACT entrance exam score) and `year` (Calendar year the freshman entered university). The variable `year` will be treated as a categorical factor.

```
uadata.fit <- lm(gpa.endyr1 ~ hs.pct + act + factor(year), data=uadata)
autoplot(uadata.fit)
```



We see some substantial non-constant variance (the residuals vs fitted plot shows clearly systematic fanning; the scale-location plot shows a downward trend in the adjusted residuals as the fitted values increase) as well as some non-normality. Even though n is large here and hence the non-normality isn't a serious issue (more on that below).

8.3.1 Box-Cox Power Transformations

Power transformation, also known as Box-Cox transformations, are typically implemented to treat issues with Normality. However, it is often the case that violations of the constant variance assumption are accompanied with violations of normality. A transformation of the response variable Y can often address both violations.

The Box-Cox transformation method is a popular way to determine a normalizing data transformation for Y . It is designed for strictly positive responses ($Y > 0$) and determines the transformation to find the best fit to the data. In the Box-Cox method, instead of fitting the original Y as the response, we determine a transformed version $t_\lambda(Y)$, where

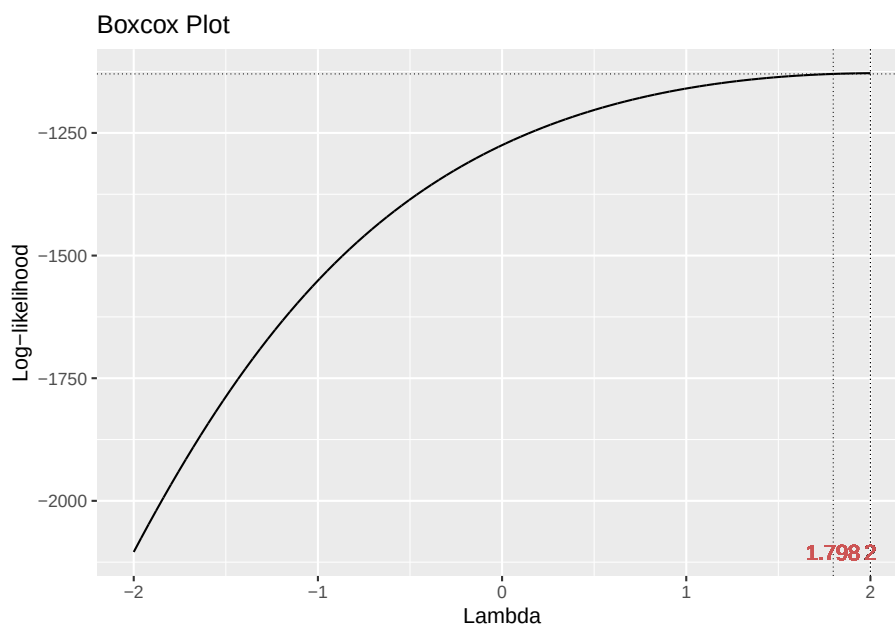
$$t_\lambda(Y) = \begin{cases} \frac{Y^\lambda - 1}{\lambda} & \text{for } \lambda \neq 0 \\ \log(Y) & \text{for } \lambda = 0 \end{cases}$$

From the data, the “power” λ is estimated. Once optimally determined, we fit the linear model using the transformed response:

$$t_{\lambda}(Y) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

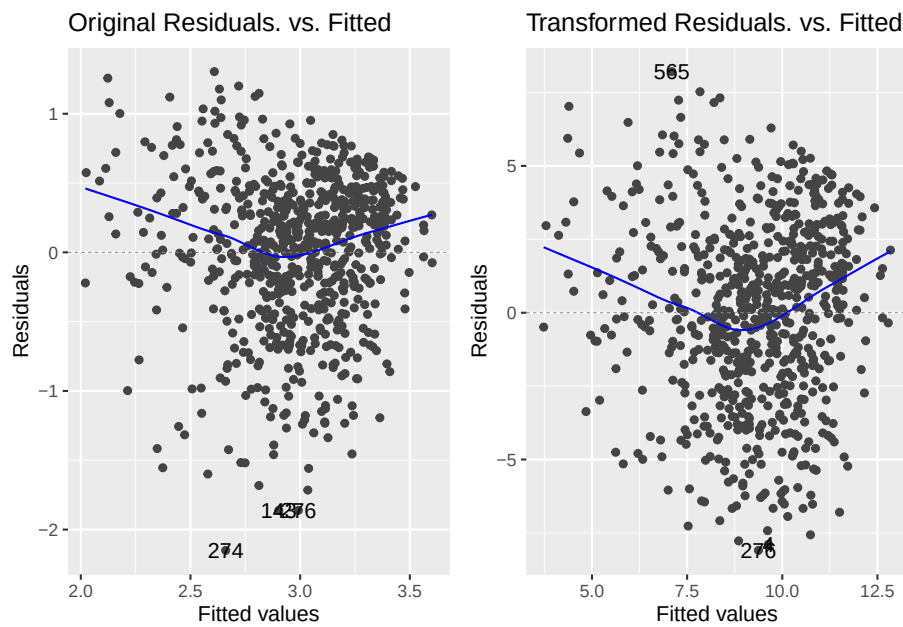
The `gg_boxcox()` function in the `lindia` package will produce a plot of the profile likelihood against the transformation parameter λ . We typically look for the λ value that includes the maximum (or peak) of the plotted curve. Let's see what a Box-Cox normalizing transformation does for the error variance problem for the GPA predictor problem.

```
gg_boxcox(uadata.fit)
```

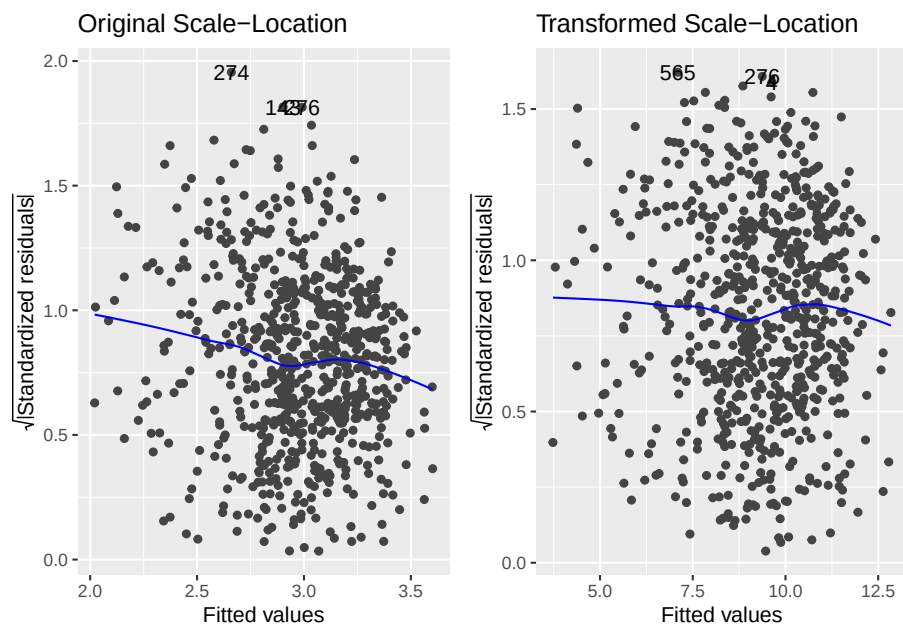


We typically choose a convenient λ value near the peak. Here, using $\lambda = 2$ (i.e., squaring the response `gpa.endyr1`) will be the best normalizing transformation.

How does the transformation improve our error variance problem? Below is a direct comparison of the residuals vs fitted plots for the original response (model `m`, on the left) and the transformed response (model `m2`, on the right):



The strong left skew in the residuals around 0 is gone, and the overall pattern looks much more uniform as a result of the transformation. So, a normalizing transformation helped cure a non-constant variance problem. The scale-location plots show the “cure” even better:



Thus, using Y^2 as our response (instead of Y) effectively treats the problem.

8.4 Normality violations

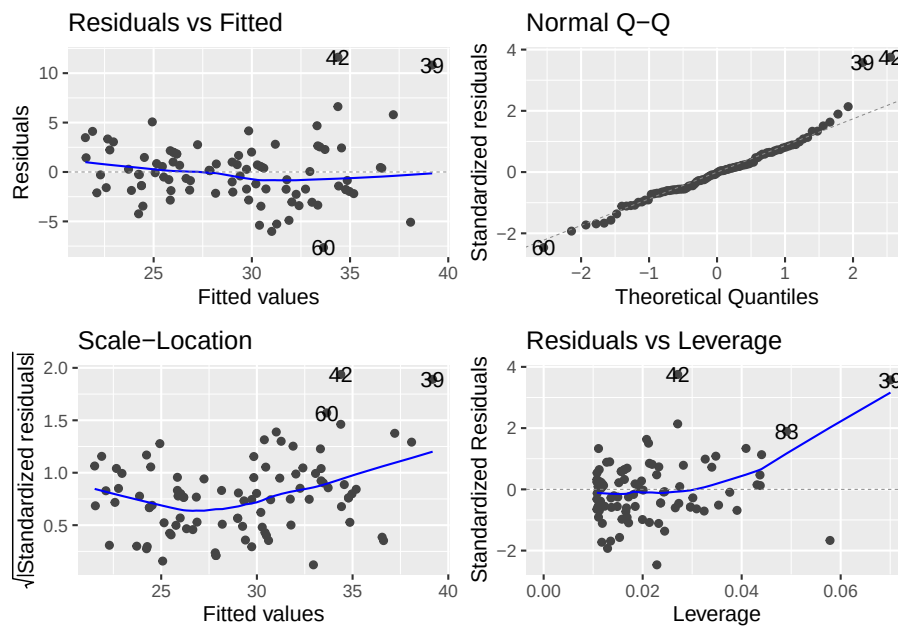
What if we have non-normal errors?

- A treatment for non-normality comes usually in the form of a *power transformation* applied to the response variable Y .
- You can actually get away with some mild violation of the normality assumption in many cases. The reason is because of the Central Limit Theorem: if you have a large sample, the CLT will help bail you out when you have normality violations. If your sample is small, though, the normality assumption is pretty important.

Example: 1993 Passenger Cars. Cars were selected at random from among 1993 passenger models that were listed in both the Consumer Reports issue and the PACE Buying Guide. Pickup trucks and Sport/Utility vehicles were eliminated due to incomplete information in the Consumer Reports source. Duplicate models (e.g., Dodge Shadow and Plymouth Sundance) were listed at most once. That data reside in the `Cars93` data frame in the `MASS` package. After loading the package, details about all the variables in the data frame may be obtained by typing `?Cars93`.

Let's look at a simple linear regression to predict highway mileage (MPG.highway) from the weight of the car (Weight). I will look at the normal Q-Q plot for the residuals, too:

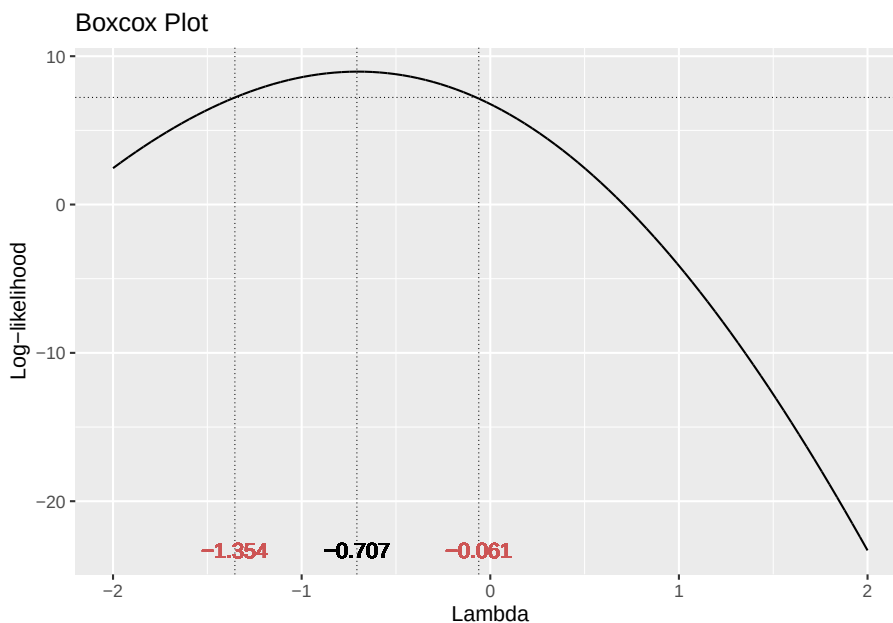
```
library(MASS)
data(Cars93)
m1 <- lm(MPG.highway ~ Weight, data=Cars93)
autoplot(m1)
```

Normality is typically assessed via a Normal Q-Q plot (upper right corner). There is a noticeable skew in the points at the top of the plot, suggesting a skew in the residuals. Since normally distributed data should be symmetric, this suggests a violation of the normality assumption. As aforementioned, violations of normality typically are typically linked with violations of constant variance and we see that here with variance issues (looks like the variance increases with the Fitted values)

We can consider a Box-Cox transformation

```
gg_boxcox(m1)
```



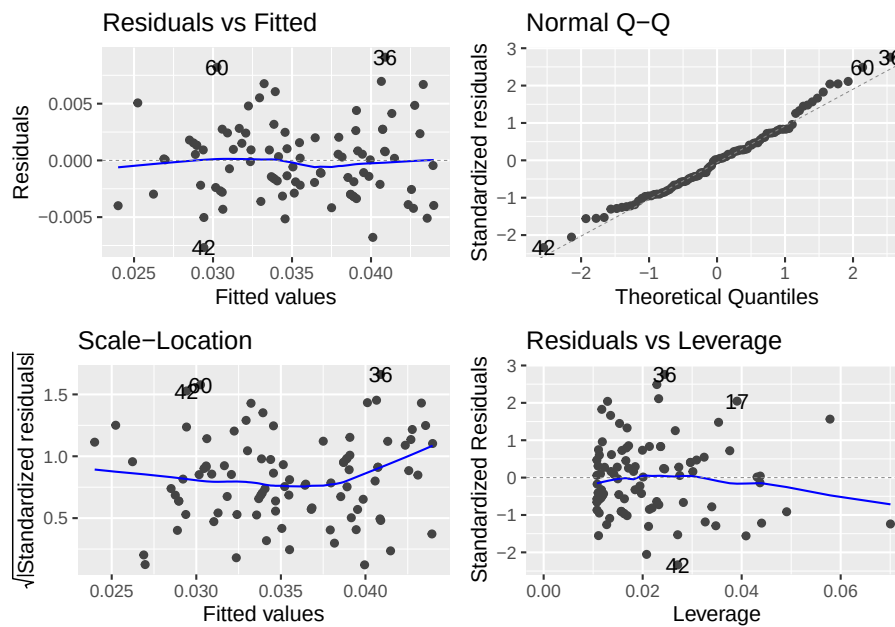
The optimal choice of power transformation is near $\lambda = -0.75$. The confidence interval for λ runs from about -1.4 to 0 . For ease of interpretation, we typically use a convenient value of λ nearest to the optimum. So in this case, I would choose $\lambda = -1$ for my optimal transformation. In other words, instead of fitting Y as the response, use

$$t_{\lambda}(Y) = \frac{Y^{-1} - 1}{-1} = \frac{1}{Y}$$

.

Since $Y = \text{MPG.highway}$, we instead use the reciprocal $1/\text{MPG.highway}$ as our response variable. This should help normalize the residuals and hence validate any test or confidence or prediction intervals. Let's see:

```
Cars93 <- Cars93 %>%
  mutate(Rec.MPG.highway = 1/MPG.highway)
t.m1 <- lm(Rec.MPG.highway ~ Weight, data=Cars93)
autoplot(t.m1)
```



This looks much better than the normal Q-Q plot for our original R model `m1`, which used `MPG.highway` as the response.

Two notes:

1. Regression coefficients will need to be interpreted with respect to the transformed scale. There is no straightforward way of “untransforming” them to values that can be interpreted in the original scale. Also, you cannot directly compare regression coefficients between models where the response transformation is different. Difficulties of this type may dissuade one from transforming the response.
2. Even if you transform the response, you will want to express model predictions back in the original scale. This is simply a matter of “untransforming” by using the inverse function of the original transformation. For example, suppose I use model `trans.m1` to obtain a 95% PI for the true highway mileage for a car weighing 3000 lb:

```
car.pred <- predict(t.m1, newdata=data.frame(Weight=3000),int="pred")
car.pred
```

```
##          fit      lwr      upr
## 1 0.0348106 0.0281462 0.041475
```

The PI (0.028, 0.0414) is in terms of `1/MPG.highway`. Note that this is actually equivalent to “gallons per mile” here! To untransform, reciprocate:

```
car.pred.mpg <- 1/car.pred
car.pred.mpg
```

```
##          fit      lwr      upr
## 1 28.7269 35.5287 24.1109
```

We are 95% confident that the true highway mileage for an unobserved individual car weighing 3000 lb is somewhere between 24.11 to 35.53 miles per gallon. Note: The lower and upper prediction limits in R are switched because of the reciprocal transformation we used.

8.5 Violations of the linearity assumption

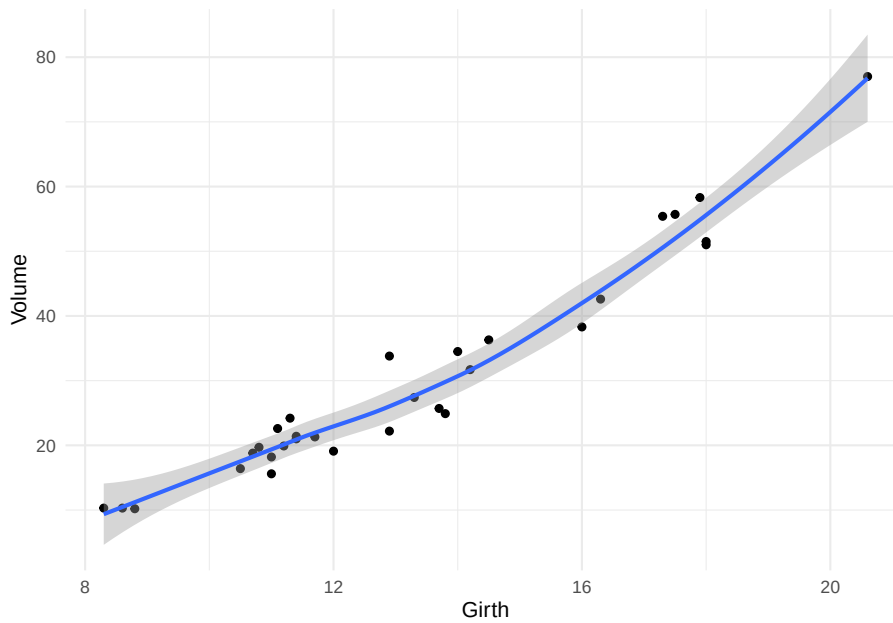
When fitting a linear regression model, it is assumed that the structural part of the model has been correctly specified. If so, this ensures that the mean value of the residuals e_i is always 0, which is desirable since we do not want the model to have any systematic tendency to overpredict or underpredict Y .

Example: Tree volume. This data set provides measurements of the girth, height and volume of timber in 31 felled black cherry trees. Girth is the diameter of the tree (in inches) measured at 4 feet 6 inches above the ground. The data are in the data frame `trees` from the `datasets` package in the R base installation. For now, we will only consider a model to predict volume from girth, ignoring height. (Question: why might we want to do this?)

First, let's look at a plot of Volume against Girth. I add a scatterplot smoother (called a LOESS, or LOWESS, curve) to make a preliminary assessment of the relationship:

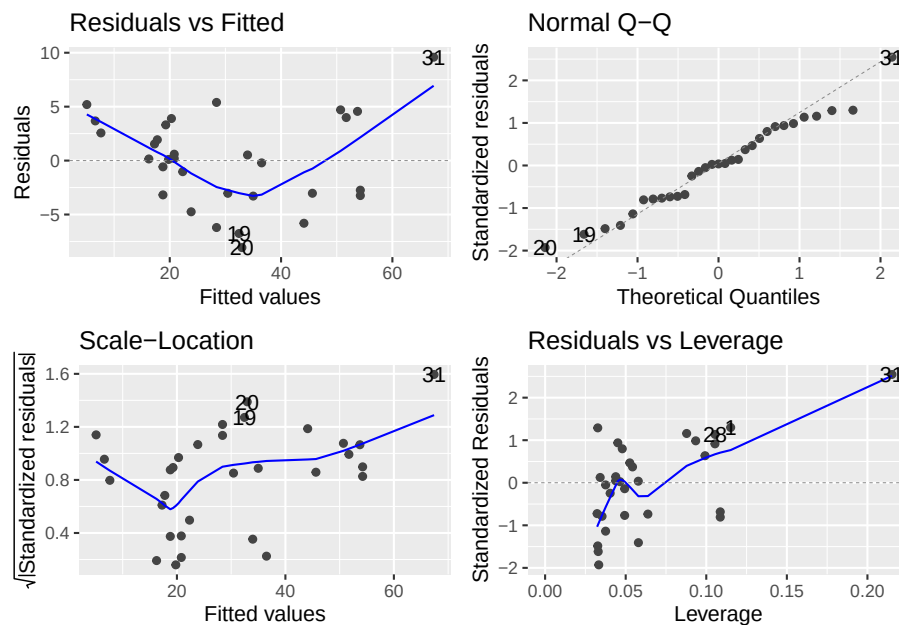
```
data(trees)
ggplot(trees) +
  geom_point(aes(x=Girth, y=Volume)) +
  geom_smooth(aes(x=Girth, y=Volume), method="loess") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



It's pretty clear that the relationship is non-linear: the rate of increase in volume per inch of girth is higher for larger trees. If we fit the model $\text{VOLUME} = \beta_0 + \beta_1 \text{GIRTH} + \varepsilon$ to these data, we are erroneously forcing the same rate of increase in volume regardless of tree size, since the rate parameter β_1 is the same regardless of girth. Let's see the effect of doing this on the residual diagnostics:

```
tree.fit <- lm(Volume ~ Girth, data=trees)      # fits an incorrect straight-line model
autoplot(tree.fit)
```



Consider the Residuals vs Fitted plot (upper-left). If the linearity assumption were being met, the residuals should bounce randomly around the $e_i = 0$ line, but there is an obvious trend in the residuals, clearly depicted by the smoother curve: model predicted values that are low (<20) or high (>50) tend to be underpredicted, that is, their residuals e_i are positive; and model predicted values of modest magnitude (between 20 and 50) tend to be overpredicted (their residuals e_i are negative).

Addressing linearity violations involves making adjustments to how the predictors enter the model. This usually means that we try applying transformations to the predictor(s). Common choices are to fit a polynomial model in X , taking logs, etc. Trial and error can reveal good choices of transformation.

In the tree example, the smoother suggest that a quadratic model (the curve we see in the above plot) might perform better:

$$VOLUME = \beta_0 + \beta_1 GIRTH + \beta_2 GIRTH^2 + \varepsilon$$

We fit this model in R:

```
tree.fit2 <- lm(Volume ~ Girth + I(Girth^2), data=trees)
summary(tree.fit2)
```

```
##
```

```
## Call:
```

```
## lm(formula = Volume ~ Girth + I(Girth^2), data = trees)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.489 -2.429 -0.372  2.076  7.645
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  10.7863    11.2228   0.96  0.34473
## Girth        -2.0921     1.6473  -1.27  0.21453
## I(Girth^2)    0.2545     0.0582   4.38  0.00015 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.33 on 28 degrees of freedom
## Multiple R-squared:  0.962, Adjusted R-squared:  0.959
## F-statistic: 350 on 2 and 28 DF, p-value: <2e-16
```

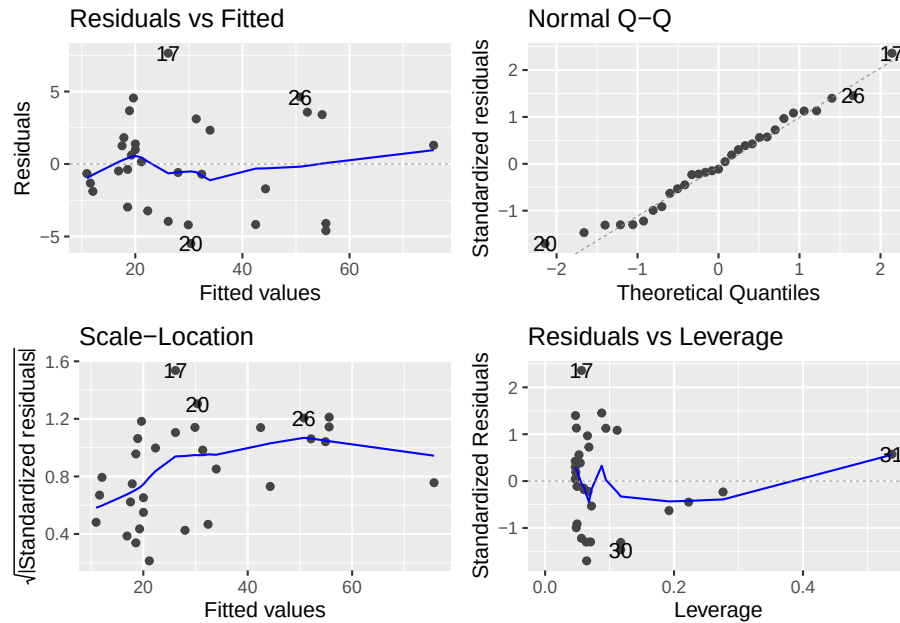
Note the p-value of the quadratic term (0.000152). This means that curvature in the girth/volume relationship is statistically significant. We could have equivalently investigated this by using `anova()` to compare the straight-line model (`tree.fit`) to the quadratic model (`tree.fit2`):

```
anova(tree.fit, tree.fit2)
```

```
## Analysis of Variance Table
##
## Model 1: Volume ~ Girth
## Model 2: Volume ~ Girth + I(Girth^2)
##   Res.Df  RSS Df Sum of Sq    F Pr(>F)
## 1      29 524.3
## 2      28 311.4  1      212.9 19.15 0.000152 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Now consider the residuals from the quadratic fit

```
autoplot(tree.fit2)
```



The trend we saw in the residuals for the straight-line model is now gone. So, the linearity assumption has been satisfied here by fitting a quadratic model.

Also note the adjusted R-squared value increases with the quadratic model (from 0.9331 to 0.9588).

8.6 Detecting and dealing with unusual observations

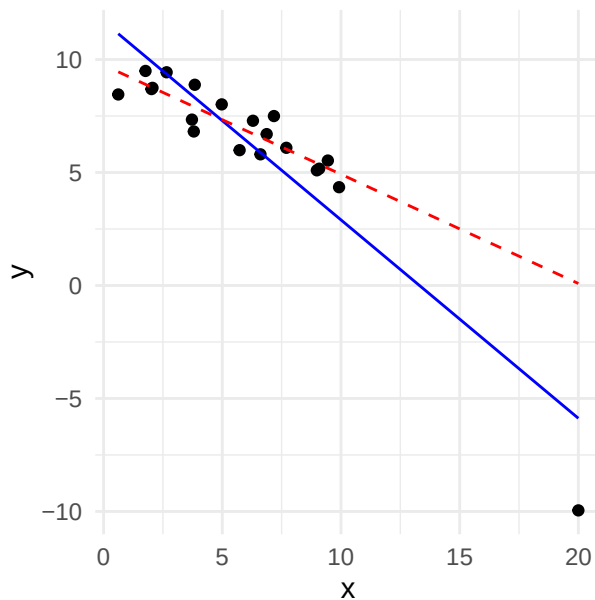
Sometimes we will encounter unusual observations in our data. It is important to have mechanisms for identifying such observations because they can wreak havoc on certain aspects of an analysis. Once identified, we need a strategy for dealing with them. In regression, there are three classifications of unusual observations:

1. **High-leverage points.** These are observations whose predictor values are far from the center of the predictor space; i.e., observations that are “extreme” in the X s. This is not bad per se, but high-leverage points have the potential to exert greater influence on the estimation of the β -coefficients in our regression model.
2. **Outliers.** These are relatively isolated observations that are poorly predicted by the fitted model; i.e., observations that are “extreme” in the Y s. Outliers can inflate the standard errors of the residuals, resulting in the potential masking of truly significant effects on the response and

prediction intervals that are too wide.

3. **Influential observations.** These are observations that are both outlying and have high leverage. These are called *influential* because the β -parameter estimates can change dramatically depending on whether or not they are included in the analysis.

Example. As an illustration, see the scatterplot below. The point in the lower right-hand corner likely qualifies as an influential observation. Why?



The blue solid line is the least squares regression line for the data when the lower right-hand corner point is included. If it is excluded, however, the regression line is given by the red dashed line, which has a much shallower slope. That means that this point is highly *influential* on the fit.

The point in question is isolated in the X direction (i.e. isolated from other values of the predictor X , with a value of about 20 compared to all other between 0 and 10), so it has high leverage. If you look in the Y (response) dimension, the point is also a potential outlier (value near -10 compare to others in the range of 5 to 10), since it falls relatively far from the solid line.

We now introduce ways to measure these attributes (leverage, outlyingness, influence) for each observation. We will have R compute them, and then we will build plots out of them.

We leave out a lot of technical detail and derivation here, and instead focusing on the usage of these measures. Derivations may be found in more advanced texts on regression.

Leverage. You have a data set with n observations ($i = 1, 2, \dots, n$) fit with a model with p β -parameters. The leverage for observation i is measured using a quantity called a hat value h_i (sometimes, it is just called the leverage). The hat value h_i is a measure of the distance of the i^{th} observation from the centroid of the predictor values. The larger h_i is, the farther removed the observation is from the “geographical center” of the predictor space.

In R, the h_i values may be obtained by `hatvalues()`

Rule of thumb: An observation has noteworthy leverage if $h_i > 2p/n$

Outliers. Residuals are the natural way to determine how “far off” a model prediction is from a corresponding observed value. However, now we need to use the residuals to determine how poorly predicted a particular observation may be. To do so, we need to scale the residuals somehow. There are two essential flavors of residual:

1. Raw residuals: $e_i = y_i - \hat{y}_i$, in R: `residuals()`

These are the usual residuals you get directly from fitting the model. They are in the units of the response variable Y .

2. Standardized residuals: $r_i = \frac{e_i - \bar{e}}{SE_e}$, in R: `rstandard()`

These are the residuals after scaling them by their standard error. Standardizing puts the residuals into units of standard deviation (like z -scores from Intro Stats). You may then use an Empirical Rule-type argument to identify unusually large residuals.

Rule of thumb: An observation is a potential outlier if $|r_i| > 3$.

Influential observations. There are actually several ways to measure how an observation might influence the fit of a model (e.g., change in β -coefficient estimates for each observation based on whether or not it is included or excluded from the analysis). The most commonly used measure of influence combines measures of leverage and outlyingness, and is known as Cook’s Distance. It is given by

$$D_i = \frac{1}{p} r_i^2 \frac{h_i}{1 - h_i}$$

In R, the D_i values may be obtained using `cooks.distance()`

Rule of thumb: An observation may be influential if $D_i > 4/df_{\text{error}}$.

Recall that in R, the `autoplot()` function when applied to a linear model object created with `lm()` generates a set of four diagnostic plots. The plot labeled Residuals vs Leverage provides a nice visualization of the standardized residuals, the hat values, and contours corresponding to values of the Cook’s Distance influence measure. Let’s look at a new example.

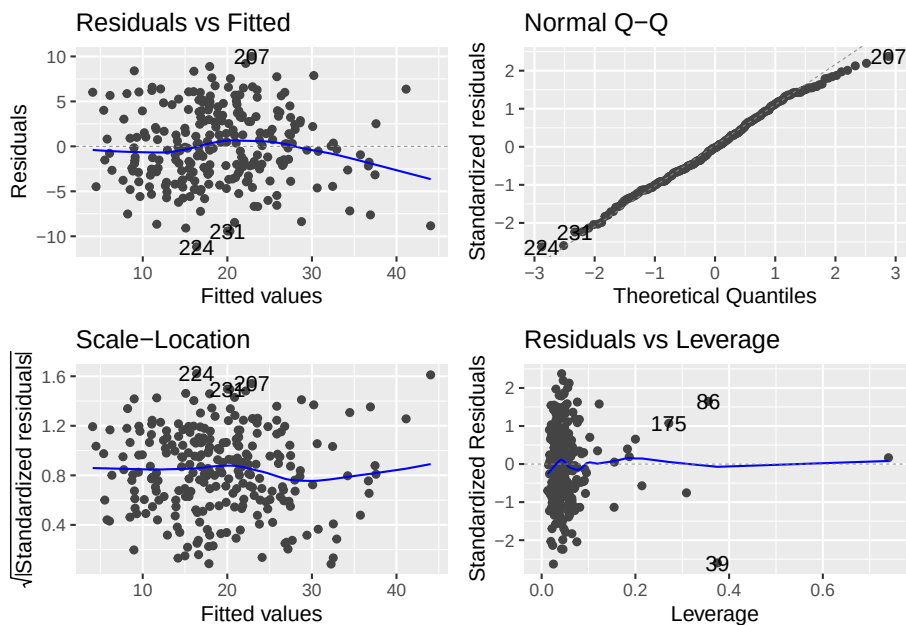
Example: Estimating body fat percentage. The R workspace `bodyfat.RData` in our repository gives the percentage of body fat determined by the method of underwater weighing, in addition to numerous body circumference measurements, for a random sample of 252 men. The goal of the study is to develop a good predictive model for estimating percentage of body fat using body circumference measurements only.

We fit a main effects model using `bodyfat.pct` as the response and the remaining 13 variables as predictors, and request the diagnostic plots for the model fit:

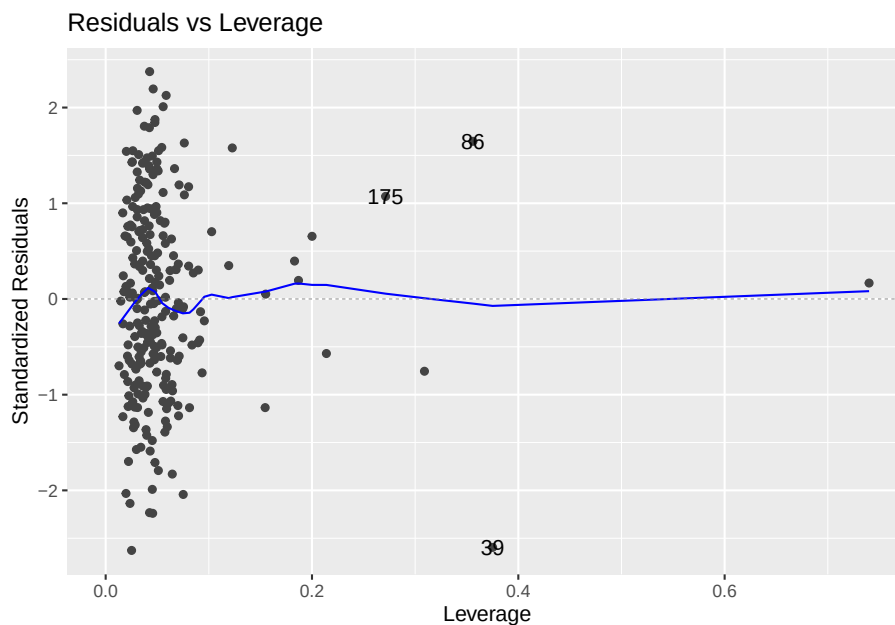
```
site <- "http://www.users.miamioh.edu/hughesmr/sta363/bodyfat.txt"
bodyfat <- read.table(site, header=TRUE)
bodyfat.fit <- lm(bodyfat.pct ~ ., data=bodyfat)
```

The notation `bodyfat.pct ~ .` tells R to treat the variable `bodyfat.pct` as the response and all other variables are predictors.

```
autoplot(bodyfat.fit)
```



Look at the plot in the bottom-left corner, zoomed in below.



There are no observations with $|standardized\ residual| > 3$. Also, there are points with noticeably high leverages. In this example, a high leverage point is any observation with

$$h_i > \frac{2p}{n} = \frac{2(14)}{252} = 0.111$$

The trace through the center of the plot is a smoother, but here it closely corresponds to where $r_i = 0$ (that's where Cook's Distance $D_i = 0$; i.e., where an observation exerts negligible influence on the model if retained or removed). Three influential points are identified by R on the plot: observations numbered 39, 86, and 175 in the data frame.

A strategy for dealing with unusual observations

You should certainly have a good idea now that identifying and handling outliers and influential data points is a somewhat subjective business. It is for this reason that analysts should use the measures described above only as a way of screening their data for potentially influential observations. With this in mind, here is a recommended strategy for dealing with unusual observations:

- Even though it is an extreme value, if an unusual observation can be understood to have been produced by essentially the same sort of physical or biological process as the rest of the data, and if such extreme values are expected to eventually occur again, then such an unusual observation indicates something important and interesting about the process you are investigating, and it should be kept in the data.

- If an unusual observation can be explained to have been produced under fundamentally different conditions from the rest of the data (or by a fundamentally different process), such an unusual observation can be removed from the data if your goal is to investigate only the process that produced the rest of the data. In essence, this observation is outside your intended population of study.
- An unusual observation might indicate a mistake in the data (like a typo, or a measuring error), in which case it should be corrected if possible or else removed from the data before fitting a regression model (and the reason for the mistake should be investigated).

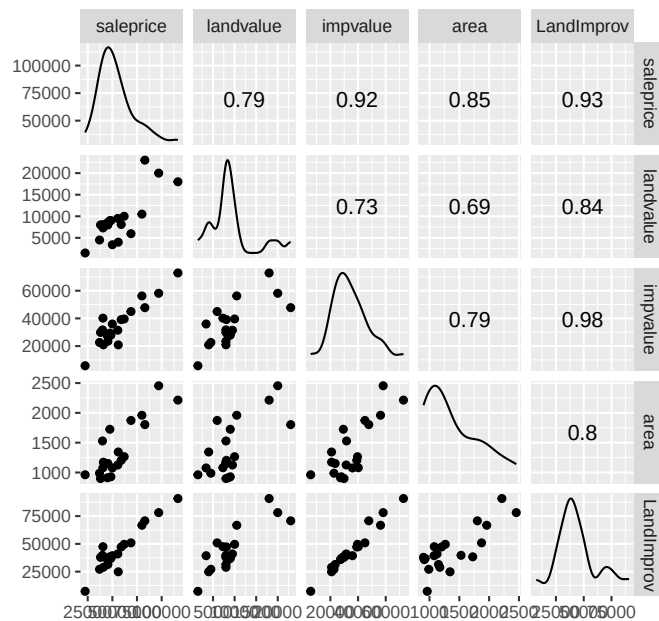
8.7 Multicollinearity

Multicollinearity (abbreviated MC) is the condition where two or more of the predictors in a regression model are highly correlated among themselves. It is a condition in the X s, and has nothing to do with the response variable Y . MC is always present in non-orthogonal data (i.e., observational studies). The issue is to what degree it is present.

Why is MC a potential concern? Very strong MC can have a severe impact on your ability to assess certain aspects of a regression analysis. Let's see via an example.

Example: Housing Appraisal. Consider the housing appraisal data we've used multiple times. We have already established that the two values (improved appraised value and land value appear connected).

```
ggscatmat(appraisal)
```



We see that all the predictor variables appear to be linearly connected (fairly strong correlations). We have moderately strong correlation between all three predictors and all three appear to be linearly connected to the response variable. Recall the full regression model.

```
summary(appraisal.fit)
```

```
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025    2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1384.197   5744.100    0.24  0.8126
## landvalue      0.818     0.512    1.60  0.1294
## impvalue      0.819     0.211    3.89  0.0013 **
## area          13.605     6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
```

F-statistic: 46.7 on 3 and 16 DF, p-value: 3.87e-08

The land value and area do not show up as significant but visually both appear important (and both have correlation greater than 0.7).

Causes of multicollinearity

- Including a predictor that is computed from other predictors in the model (e.g., family income = husband income + wife income, and the regression includes all three income measures).
- Including the same or almost the same variable twice (e.g., height in feet and height in inches); or more commonly, using two different predictors that operationalize the same underlying concept (e.g., land value and area?)
- Improper use of dummy variables (e.g., failure to exclude one category).
- The above all imply some sort of error on the researcher's part. But, it may just be that predictors really and truly are highly correlated.

Consequences of multicollinearity

- In the presence of severe MC, we cannot make reliable assessments of the contribution of individual predictors to the response. High MC greatly inflates the SE_b values (the standard errors for the model's parameter estimates). As a result, confidence intervals for the β -coefficients are too wide and t -statistics are too small. Moreover, β -coefficient estimates tend to be very unstable from one sample to the next.
- “**Whole-model**” assessments are **unaffected** by MC. These include CIs/Pis for the response Y , ANOVA F -tests for the whole model or joint significance of multiple predictors, R^2 , etc...

Variance Inflation Factors (VIFs). High MC greatly inflates the standard errors for the model's parameter estimates $b_1, b_2, \dots$. The degree of inflation is measured by the VIF, or variance inflation factor, which is directly related to the R^2 notion in the above paragraph. To calculate the VIF for b_i , you run an “auxiliary regression” of X_i on the remaining predictors. The VIF for b_i is then given by

$$VIF_i = \frac{1}{1 - R_i^2}$$

where R_i^2 is the R^2 value from modeling X_i on the remaining predictors. If R_i^2 is large, say > 0.90 , then $VIF_i > 10$. VIFs are the most effective way to detect Multicollinearity. Generally speaking, $VIF_i > 10$ is considered a major issue and $5 < VIF_i < 10$ is of moderate concern. VIF values under the value of 5 are not considered worrisome.

The function `vif()` in the add-on package `car` computes VIFs for a fitted model. Based on our preliminar view there is some concern

```
library(car)
vif(appraisal.fit)

## landvalue  impvalue    area
##   2.29985    3.17126    2.83913
```

Here, we see there is no major concerns about MC based on VIF values.

Warning signs and checks for severe multicollinearity

- None of the t -statistics for the tests of individual β -coefficients is statistically significant, yet the whole-model F -statistic is significant.
- Check to see if your β -coefficient estimates that are wildly different in sign or magnitude than common sense would dictate.
- Check the VIFs (variance inflation factors). Any $VIF > 10$ indicates severe MC.
- You can examine the Pearson correlations between predictors and look for large values (e.g. 0.70 or higher), but the problem with this is that one predictor may be some linear combination of several other predictors, and yet not be highly correlated with any one of them. That is why VIFs are the preferred method of detecting MC.
- Check to see how stable coefficients are when different samples are used. For example, you might randomly divide your sample in two. If coefficients differ dramatically when fitting the same model to different subsets of your data, MC may be a problem.
- Try a slightly different model using the same data. See if seemingly innocuous changes (e.g., dropping a predictor, or using a different operationalization of a predictor) produce big shifts. In particular, as predictors are dropped, look for changes in the signs of effects (e.g., switches from positive to negative) that seem theoretically strange.

Dealing with multicollinearity

- Make sure you have not made any flagrant errors; e.g., improper use of computed or dummy variables.
- Use a variable selection procedure to try to break up the MC (the next chapter deals with this).
- Try some predictor amputation. If a certain predictor has a high VIF, try deleting it and then re-assess the reduced model. The only caveat in doing this is that if the predictor truly belongs in the model for practical reasons, this can lead to specification error and bias in estimation of β -parameters.
- In some cases, centering your predictor variables at 0 can reduce or eliminate MC. By “centering,” we mean shifting each predictor’s values so that they have a mean of 0.
- Perform a more complex model fit, such as *Principle Components Regression* can alleviate any concerns about MC, but it makes interpretation of the model difficult.

- Perform joint hypothesis tests rather than t -tests or CIs for individual parameters. For example, suppose you fit a model containing seven predictors $X_1, X_2, \dots, X_7$. You determine that X_2, X_3 and X_5 are highly correlated. Instead of doing three t -tests for each of these individual coefficients, perform an ANOVA F -test of the null hypothesis $H_0 : \beta_2 = \beta_3 = \beta_5 = 0$. Such tests are not impacted by MC.

Finally, remember that multicollinearity has no appreciable effect on predictions of the response variable Y . So if the only intended purpose for your model is response prediction via CIs/PIs for Y , then taking drastic steps to curb any MC among the predictor variables is a bit like performing major surgery to treat a cold.

8.8 Scale changes

Rescaling a predictor X_i means applying a linear transformation of the form

$$\frac{X_i - c_1}{c_2}$$

where c_1 and c_2 are constants ($c_2 \neq 0$), and then fitting a model. One particular important scaling is to *standardize* the predictor variables, namely the i^{th} observation of predictor j , or the datum X_{ij} is scaled via

$$\frac{X_{ij} - \bar{X}_j}{S_{X_j}}$$

where $\bar{X}_j$ is the mean of all observed values of predictor variable X_j and S_{X_j} is the standard deviation of all observed values of predictor variable X_j . Scaling can have several advantages in certain situations:

1. A change of units via rescaling might aid interpretation.
2. Transforming all quantitative predictors to the same (or similar) scale results in parameter estimates $b_1, b_2, \dots$ that can be meaningfully compared to each other.
3. In extreme cases, numerical stability in computing can be enhanced when all the predictors are put on a similar scale.
4. Scaling can sometimes help with issues of multicollinearity.
5. Transforming all predictors to the same scale (standardizing) is necessary in some variable selection procedures (the next chapter).

Rescaling X_j will leave t -tests, F -tests, R^2 , R_{adj}^2 and the residual standard error unchanged. However, the parameter estimate b_j will be rescaled to $c_2 b_j$, and any CI for β_j will likewise be affected.

Example Property appraisals. Let's revisit the appraisal dataset we continue to utilize. Recall the response variable is the sale price (**saleprice**) with the following predictor variables.

- **landvalue** - Appraised land value of the property (in \$)
- **impvalue** - Appraised value of improvements to the property (in \$)
- **area** - Area of living space on the property (in sq ft)

Note two of the variables are in the units of dollars and the other is in square feet (the units are completely different). The computer treats every value as a number (it does not care about units) but given the difference in scale and units, this can lead to one variable appearing more important than another.

First let's recall the fitted model:

```
summary(appraisal.fit)

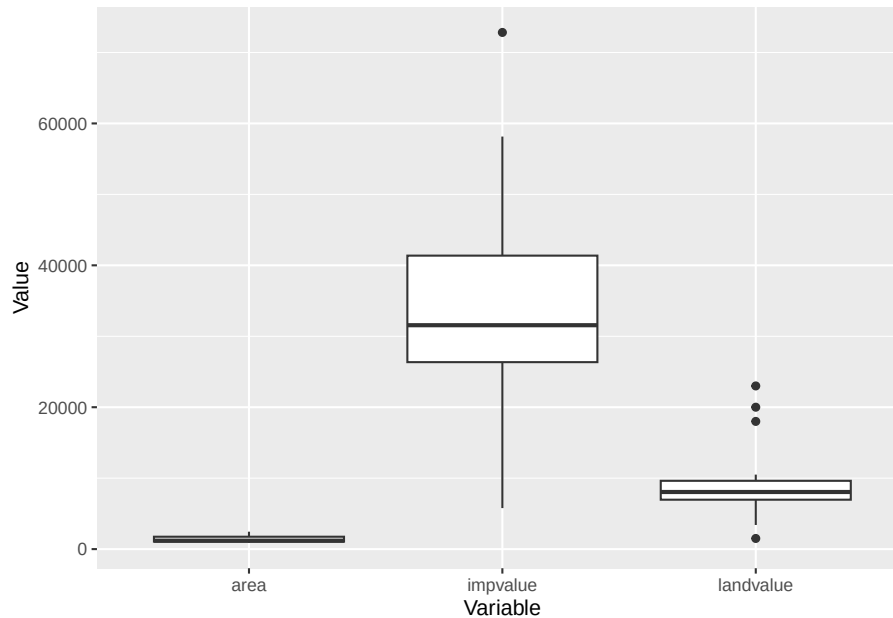
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025    2717   15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1384.197    5744.100    0.24  0.8126
## landvalue      0.818      0.512    1.60  0.1294
## impvalue       0.819      0.211    3.89  0.0013 **
## area          13.605      6.569    2.07  0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF,  p-value: 3.87e-08
```

Those who know much about houses know that the size of the home and plot of land (essentially the **landvalue**) typically influence the sale price (larger homes sell for more), yet only the **impvalue** variable appears significant. But when looking at the coefficients, the $b_{\text{landvalue}} \approx b_{\text{impvalue}} \approx 0.81$ so it appears the two variables contribute the same predictive ability. Likewise, **area** appears it will be the most important in terms of predicting the **saleprice** in terms of raw numbers. It turns out that the variability in the **impvalue** variable is masking the influence of the other variables.

This idea will make more sense with a plot. Using material from the first chapter we convert the predictor variables from wide to tall format to make a side-by-side

boxplot.

```
appraisal.tall <- appraisal %>%
  dplyr::select(-saleprice, -LandImprov) %>%
  gather(key="Variable", value="Value")
ggplot(appraisal.tall) +
  geom_boxplot(aes(x=Variable, y=Value))
```



Note the range for the `impvalue` variable compared to `area` and `landvalue`. Remember each value is also mapped to a `saleprice` value, the information provided in `impvalue` will overpower any information in `area` or `landvalue` since the variables are not on the same scale. Let's standardize each predictor variable by applying the `scale()` function to each column of the dataset.

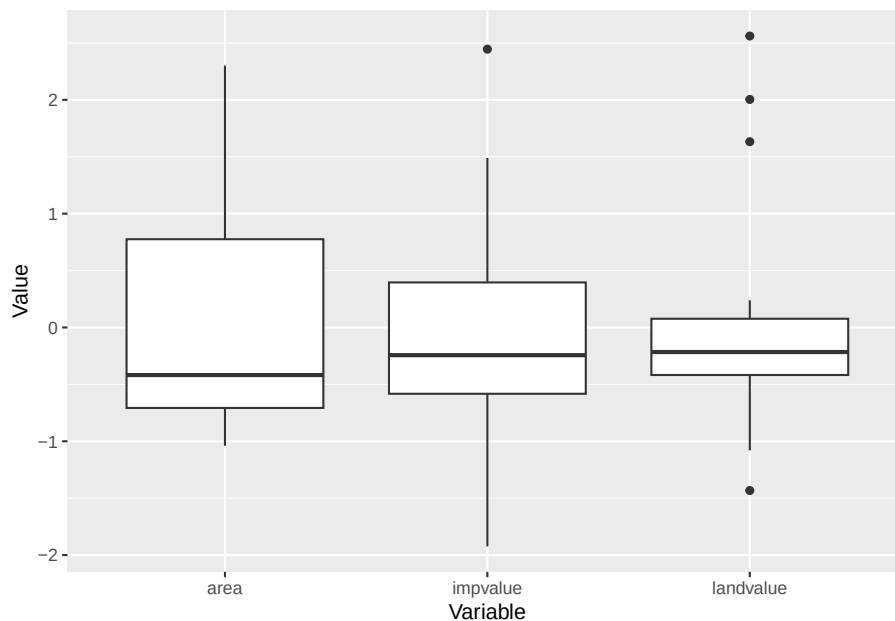
```
z.appraisal <- appraisal %>%
  mutate_at(c("landvalue", "impvalue", "area"), scale)
head(z.appraisal)
```

##	saleprice	landvalue	impvalue	area	LandImprov
## 1	68900	-0.6042621	0.630281	1.051338	50927
## 2	48500	-0.0395659	-0.484310	-0.977692	36860
## 3	55500	0.0533118	-0.251123	-0.552562	40939
## 4	62000	0.1461894	0.280078	-0.254112	49592
## 5	116500	1.6322320	2.445472	1.783507	90827
## 6	45000	-0.1324435	-0.519688	-1.012046	35817

We see the `landvalue`, `impvalue` and `area` variables all resemble *z*-scores. Now

for a regurgitation of the box-plot on the standardized variables

```
z.appraisal.tall <- z.appraisal %>%
  dplyr::select(-saleprice, -LandImprov) %>%
  gather(key="Variable", value="Value")
ggplot(z.appraisal.tall) +
  geom_boxplot(aes(x=Variable, y=Value))
```



Now the units on all three predictor variables is essentially the same. Let's fit our model again.

```
z.appraisal.fit <- lm(saleprice ~ landvalue + impvalue + area, data=z.appraisal)
summary(z.appraisal.fit)
```

```
##
## Call:
## lm(formula = saleprice ~ landvalue + impvalue + area, data = z.appraisal)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14688  -2026   1025    2717  15967
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    56660      1770   32.01  6.2e-16 ***
## landvalue       4403      2754    1.60   0.1294
```

```
## impvalue      12577      3234      3.89      0.0013 **
## area          6336      3060      2.07      0.0549 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7920 on 16 degrees of freedom
## Multiple R-squared:  0.898, Adjusted R-squared:  0.878
## F-statistic: 46.7 on 3 and 16 DF,  p-value: 3.87e-08
```

First note that the p -values are the same as before, likewise the overall F statistics, R^2 and R_{adj}^2 are all the same. However, now the coefficients are different. The largest coefficient corresponds to the variable that appears most important (in terms of significant testing). With all predictor variables on the same scale, we can now compare predictor variable impact.

Chapter 9

Model Selection

One of the important themes running through what we do in regression concerns model simplification. The principle of *parsimony*, by which we should abide, is attributed to the 14th century English logician William of Occam, who developed what is known as Occam's Razor):

Occam's Razor (paraphrased) All other things being equal, the simplest explanation tends to be the correct explanation.

The term razor refers to the act of shaving away unnecessary complications to get to the simplest explanation. In statistical models, application of Occam's Razor means that models should have as few parameters as possible, and should be pared down until they are "minimally adequate". This theme extends beyond this course (model parsimony is important in many areas of statistics).

Historically, the process of model simplification required the application of hypothesis testing in regression models. In general, a predictor variable X_j under investigation was retained in a model only if it was statistically significant (i.e., if the test for $H_0 : \beta_j = 0$ had a small p -value). However, one can show that this method of choosing predictors to retain is fraught with potential problems (e.g., consider multicollinearity), so luckily more robust methods have been developed for the task.

In our quest to simplify models, however, we must be careful not to throw out the baby with the bathwater. Too simple is not good, either! Building statistical models is as much an art as it is a science, so you must be aware of how your models are explaining the observed data at every step along the way to obtaining a final model.

9.1 Stepwise Procedures

Variable selection is intended to select some “best” subset of predictors. Why bother?

1. Occam’s Razor applied to regression implies that the smallest model that fits the data is best. Unnecessary predictors can add noise to the estimation of other quantities that we are interested in.
2. Multicollinearity may result by having too many variables trying to do the same job. Judicious removal of redundant variables can greatly improve estimation of the effects of the predictors on the response.
3. Cost considerations: if the model is to be used for prediction, we can save time and/or money by not measuring redundant predictors.

Prior to any variable selection, you should:

1. Identify outliers/influential points, perhaps excluding them at least temporarily.
2. Apply transformations to the variables that seem appropriate from a preliminary inspection of the data (standardizing predictor variables or squaring terms to handle curvature, etc...).

There are two useful measures for comparing the quality of the fits of regression models: R_{adj}^2 and the Akaike Information Criterion (AIC). We will do a demonstration of variable selection using both measures, but the AIC is preferable in practice because its concept is applicable to a broader array of model types.

Minimizing the “loss of information”. Before engaging in the construction of a regression model, we must first accept that there are no true models. Indeed, models only approximate reality. The question then is to find which model would best approximate reality given the data we have recorded. In other words, we are trying to minimize the loss of information. Kullback and Leibler addressed such issues in the 1950s and developed a measure to represent the information lost when approximating reality (i.e., a good model minimizes the loss of information). A few decades later, Japanese statistician Hirotugu Akaike proposed a method for variable selection. He established a relationship between the maximum likelihood, which is an estimation method used in many statistical analyses, and the Kullback-Leibler measure. The result is known as the Akaike Information Criterion, defined by

$$AIC = n \log(RSS/n) + 2k$$

The AIC penalizes a model for the addition of parameters (k), and thus selects a model that fits well but also has a minimum number of parameters (i.e., simplicity and parsimony).

By itself, the value of the AIC for a given data set has no meaning. It becomes interesting when it is compared to the AICs of a series of models. Specified in advance, the model with the lowest AIC is generally considered the best model among all models specified for the data at hand. Thus, if only poor models are considered, the AIC will select the best of the poor models. This highlights the importance of spending time to determine the set of candidate models based on previous investigations, as well as judgment and knowledge of the system under study.

Once appropriate transformations have been applied (if warranted), one may run each of the models and compute the AIC. The models can then be ranked from best to worse (i.e., low to high AIC values). But, be aware of the following:

- One should ensure that the same data set is used for each model; i.e., the same observations must be used for each analysis.
- Missing values for only certain variables in the data set can also lead to variations in the number of observations.
- Furthermore, the same response variable Y must be used for all models (i.e., it must be identical across models, consistently with or without transformation).

9.1.1 Backward Selection

Strategy for stepwise variable selection using AIC:

1. Start with all the candidate predictors in the model. Check assumptions and make corrections if necessary.
2. Run a whole-model F -test on this “global” model. Be sure it indicates that your model does have utility. If not, then none of your variables should be selected!
3. Find the AIC of the global model.
4. Look at all candidate models that result by the removal of one predictor from the global model. This is called “backward selection”. Calculate the AIC for each of these models.
5. Pick the model with the smallest AIC. (Akaike’s rule of thumb: two models are essentially indistinguishable if the difference in their AIC is less than 2.)
6. Return to step 4, and repeat the process starting with the revised model.
7. Continue the process until the deletion of any predictor results in a rise in the AIC.

Long-winded example To demonstrate the stepwise process let’s consider the housing appraisal dataset. We’ve used the full model fit multiple times.

```
AIC(appraisal.fit)
```

```
## [1] 421.356
```

Now let’s consider the three models where we remove a single predictor.

```
appraisal.no.land <- lm(saleprice ~ impvalue + area, data=appraisal)
appraisal.no.imp <- lm(saleprice ~ landvalue + area, data=appraisal)
appraisal.no.area <- lm(saleprice ~ landvalue + impvalue, data=appraisal)
AIC(appraisal.no.land)
```

```
## [1] 422.32
```

```
AIC(appraisal.no.imp)
```

```
## [1] 432.666
```

```
AIC(appraisal.no.area)
```

```
## [1] 424.105
```

We see that the full model has the best AIC (smallest) with a value of 421.3557. Removing any of the variables will make the model worse (in terms of AIC).

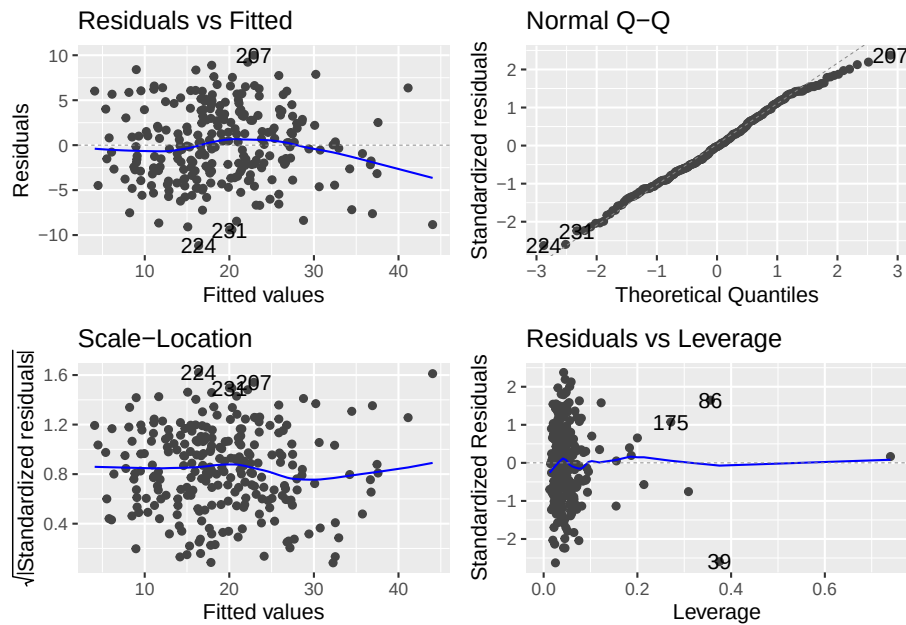
So here there is nothing to do. But you can quickly imagine the tediousness of repeating the process over and over again.

Performing the 7 step algorithm above would be redundant and tedious. Computers are excellent at performing redundant and tedious task. R has functions that will perform stepwise AIC model selection automatically. We highlight its use with another example.

Example: Estimating body fat percentage. Making accurate measurement of body fat is inconvenient and costly, so it is desirable to have methods of estimating body fat that are cheaper and easier to implement. The standard technique of underwater weighing computes body volume as the difference between body weight measured in air and weight measured during water submersion. Our goal is to develop a good predictive model for percentage of body fat that uses body measurements only; i.e., a model that gives body fat percentage estimates very close to the accurate measurements obtained via underwater weighing. Since underwater weighing is inconvenient and expensive, a good model based solely on body measurements only will be of much use in practice.

We first fit the full model and check the assumptions.

```
site <- "http://www.users.miamioh.edu/hughesmr/sta363/bodyfat.txt"
bodyfat <- read.table(site, header=TRUE)
bodyfat.fit <- lm(bodyfat.pct ~ ., data=bodyfat)
autoplot(bodyfat.fit)
```



The assumptions generally look fine here. We then test for whole-model utility:

```
summary(bodyfat.fit)
```

```
##
## Call:
## lm(formula = bodyfat.pct ~ ., data = bodyfat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.169  -2.864  -0.101   3.209  10.007
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -18.1885    17.3486  -1.05  0.2955
## age             0.0621     0.0323   1.92  0.0562 .
## weight        -0.0884     0.0535  -1.65  0.0998 .
## height        -0.0696     0.0960  -0.72  0.4693
## neck          -0.4706     0.2325  -2.02  0.0440 *
## chest         -0.0239     0.0991  -0.24  0.8100
## abdomen        0.9548     0.0864  11.04 <2e-16 ***
## hip           -0.2075     0.1459  -1.42  0.1562
## thigh          0.2361     0.1444   1.64  0.1033
## knee           0.0153     0.2420   0.06  0.9497
## ankle          0.1740     0.2215   0.79  0.4329
## biceps         0.1816     0.1711   1.06  0.2897
```

```
## forearm      0.4520      0.1991      2.27      0.0241 *
## wrist       -1.6206      0.5349     -3.03      0.0027 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.31 on 238 degrees of freedom
## Multiple R-squared:  0.749, Adjusted R-squared:  0.735
## F-statistic: 54.6 on 13 and 238 DF, p-value: <2e-16
```

The F -test for the whole model is significant ($F = 54.65$, $df=(13, 238)$, p -value < 0.0001), so we know at least one predictor is significant. We proceed to run a backward variable selection using AIC as our criterion. The R function `step()` does this nicely in one pass. Below we include all output of the `step()` function, it can be suppressed with the `trace=0` option.

```
step(bodyfat.fit, direction="backward")
```

```
## Start:  AIC=749.36
## bodyfat.pct ~ age + weight + height + neck + chest + abdomen +
##      hip + thigh + knee + ankle + biceps + forearm + wrist
##
##           Df Sum of Sq  RSS   AIC
## - knee      1      0.1 4412 747.4
## - chest      1      1.1 4413 747.4
## - height     1      9.7 4421 747.9
## - ankle      1     11.4 4423 748.0
## - biceps     1     20.9 4432 748.5
## <none>                4411 749.4
## - hip        1     37.5 4449 749.5
## - thigh      1     49.6 4461 750.2
## - weight     1     50.6 4462 750.2
## - age        1     68.3 4480 751.2
## - neck       1     76.0 4487 751.7
## - forearm    1     95.5 4507 752.8
## - wrist      1    170.1 4582 756.9
## - abdomen    1   2261.0 6672 851.6
##
## Step:  AIC=747.36
## bodyfat.pct ~ age + weight + height + neck + chest + abdomen +
##      hip + thigh + ankle + biceps + forearm + wrist
##
##           Df Sum of Sq  RSS   AIC
## - chest      1      1.1 4413 745.4
## - height     1      9.7 4421 745.9
## - ankle      1     12.1 4424 746.1
## - biceps     1     20.8 4432 746.5
## <none>                4412 747.4
```

```

## - hip      1      37.4 4449 747.5
## - weight   1      53.1 4465 748.4
## - thigh    1      54.9 4466 748.5
## - age      1      74.1 4486 749.6
## - neck     1      78.4 4490 749.8
## - forearm  1      96.8 4508 750.8
## - wrist    1     170.5 4582 754.9
## - abdomen  1    2269.9 6681 850.0
##
## Step: AIC=745.43
## bodyfat.pct ~ age + weight + height + neck + abdomen + hip +
##      thigh + ankle + biceps + forearm + wrist
##
##           Df Sum of Sq  RSS   AIC
## - height   1         8.7 4421 743.9
## - ankle    1        12.4 4425 744.1
## - biceps   1        20.1 4433 744.6
## <none>             4413 745.4
## - hip      1        36.3 4449 745.5
## - thigh    1        60.1 4473 746.8
## - weight   1        70.8 4483 747.4
## - age      1        73.8 4486 747.6
## - neck     1        79.5 4492 747.9
## - forearm  1        95.6 4508 748.8
## - wrist    1       170.0 4583 753.0
## - abdomen  1     2879.4 7292 870.0
##
## Step: AIC=743.92
## bodyfat.pct ~ age + weight + neck + abdomen + hip + thigh + ankle +
##      biceps + forearm + wrist
##
##           Df Sum of Sq  RSS   AIC
## - ankle    1         13 4435 742.7
## - biceps   1         22 4444 743.2
## - hip      1         30 4452 743.6
## <none>             4421 743.9
## - thigh    1         69 4490 745.8
## - neck     1         77 4498 746.3
## - age      1         81 4503 746.5
## - forearm  1         98 4519 747.5
## - weight   1        120 4541 748.6
## - wrist    1        181 4603 752.0
## - abdomen  1       3179 7600 878.4
##
## Step: AIC=742.68
## bodyfat.pct ~ age + weight + neck + abdomen + hip + thigh + biceps +

```

```
##      forearm + wrist
##
##              Df Sum of Sq  RSS   AIC
## - biceps      1         21 4455 741.9
## - hip          1         32 4466 742.5
## <none>         4435 742.7
## - thigh       1         72 4507 744.8
## - age         1         78 4512 745.1
## - neck        1         87 4522 745.6
## - forearm     1         97 4532 746.2
## - weight      1        107 4542 746.7
## - wrist       1        168 4603 750.0
## - abdomen     1       3182 7617 877.0
##
## Step:  AIC=741.85
## bodyfat.pct ~ age + weight + neck + abdomen + hip + thigh + forearm +
##      wrist
##
##              Df Sum of Sq  RSS   AIC
## <none>         4455 741.9
## - hip          1         37 4492 741.9
## - neck         1         79 4534 744.3
## - age          1         84 4539 744.5
## - weight       1         93 4548 745.1
## - thigh        1        101 4556 745.5
## - forearm      1        140 4596 747.7
## - wrist        1        167 4622 749.1
## - abdomen      1       3163 7618 875.0
##
##
## Call:
## lm(formula = bodyfat.pct ~ age + weight + neck + abdomen + hip +
##      thigh + forearm + wrist, data = bodyfat)
##
## Coefficients:
## (Intercept)          age          weight          neck          abdomen          hip
##    -22.6564         0.0658        -0.0899        -0.4666         0.9448       -0.1954
##      thigh      forearm      wrist
##     0.3024         0.5157        -1.5367
```

The full (“global”) model AIC is 749.36. The resulting AICs obtained via the deletion of one predictor is given for each predictor considered. Remember that lower AIC values are better. Moreover, the above list is ordered based on the lack of contribution for each predictor. For example:

- deleting *knee* from the global model will result in an AIC of 747.36 (a reduction of $749.36 - 747.36 = 2.0$ in AIC). This is the largest reduction

in AIC possible for any single variable deletion, so knee is at the top of the list.

- Remember: if the change in $AIC < 2$, then there is a no appreciable difference in the quality of the fit. Since the change in AIC here equals 2, there is a change in the fit quality ... since dropping knee lowers the AIC this much, the model can be said to be better by deleting it.
- Note that the `step()` function will NOT consider the $AIC < 2$ rule.

The steps then continue. The new “global” model in the next step is the 12-predictor model that excludes knee:

Next in line for deletion is `chest`. Note that the change in AIC by deleting chest will be $747.36 - 745.43 = 1.93$. This is less than 2, so the models with and without chest are not appreciably different. However, we now apply the principle of parsimony to default to the simpler model that excludes `chest`.

We continue on through until the process terminates, which occurs when the deletion of any remaining predictor results in a rise in the AIC.

This stepwise procedure terminates at an eight-predictor model.

```
##
## Call:
## lm(formula = bodyfat.pct ~ age + weight + neck + abdomen + hip +
##      thigh + forearm + wrist, data = bodyfat)
##
## Coefficients:
## (Intercept)      age      weight      neck      abdomen      hip
##    -22.6564     0.0658    -0.0899    -0.4666     0.9448    -0.1954
##      thigh    forearm      wrist
##     0.3024     0.5157    -1.5367
```

9.1.2 Forward selection

A similar method can occur when you build a model from the ground up. That is, start off with a “null” model, whereby the response variable is predicted with a mean only $\hat{Y} = b_0 = \bar{Y}$. Then continually add terms picking the parameters that improve AIC the most. Stop the process when adding terms worsens the AIC value.

In R, this can be accomplished with the `step` procedure but now we tell it to perform forward selection.

```
bodyfat.null <- lm(bodyfat.pct ~ 1, data=bodyfat)
step(bodyfat.null, scope=formula(bodyfat.fit), direction="forward")

## Start:  AIC=1071.75
## bodyfat.pct ~ 1
##
```

```

##           Df Sum of Sq  RSS    AIC
## + abdomen  1    11632  5947  800.6
## + chest    1     8678  8901  902.2
## + hip      1     6871 10708  948.8
## + weight   1     6593 10986  955.3
## + thigh    1     5505 12074  979.1
## + knee     1     4548 13031  998.3
## + biceps   1     4277 13302 1003.5
## + neck     1     4231 13348 1004.4
## + forearm  1     2296 15283 1038.5
## + wrist    1     2111 15468 1041.5
## + age      1     1493 16086 1051.4
## + ankle    1     1244 16335 1055.3
## + height   1        141 17438 1071.7
## <none>                17579 1071.7
##
## Step:  AIC=800.65
## bodyfat.pct ~ abdomen
##
##           Df Sum of Sq  RSS    AIC
## + weight   1    1004.2 4943 756.0
## + wrist    1     709.2 5238 770.6
## + neck     1     614.5 5333 775.2
## + hip      1     548.2 5399 778.3
## + height   1     458.8 5489 782.4
## + knee     1     318.7 5629 788.8
## + ankle    1     233.3 5714 792.6
## + age      1     200.9 5747 794.0
## + chest    1     195.5 5752 794.2
## + thigh    1     174.6 5773 795.1
## + biceps   1     135.3 5812 796.8
## + forearm  1      54.3 5893 800.3
## <none>                5947 800.6
##
## Step:  AIC=756.04
## bodyfat.pct ~ abdomen + weight
##
##           Df Sum of Sq  RSS    AIC
## + wrist    1     157.19 4786 749.9
## + neck     1      86.93 4856 753.6
## + thigh    1      81.36 4862 753.9
## + forearm  1      66.85 4876 754.6
## + biceps   1      63.81 4879 754.8
## + height   1      40.29 4903 756.0
## <none>                4943 756.0
## + knee     1       9.72 4934 757.5

```



```

## + age      1      1.94 4941 757.9
## + ankle    1      1.51 4942 758.0
## + chest    1      0.01 4943 758.0
## + hip      1      0.01 4943 758.0
##
## Step:  AIC=749.9
## bodyfat.pct ~ abdomen + weight + wrist
##
##           Df Sum of Sq  RSS   AIC
## + forearm  1    127.82 4658 745.1
## + biceps   1     88.73 4697 747.2
## + thigh    1     40.46 4746 749.8
## <none>                      4786 749.9
## + neck     1     25.18 4761 750.6
## + height   1     23.41 4763 750.7
## + age      1     21.15 4765 750.8
## + knee     1     20.54 4766 750.8
## + ankle    1     14.97 4771 751.1
## + hip      1      9.23 4777 751.4
## + chest    1      1.26 4785 751.8
##
## Step:  AIC=745.07
## bodyfat.pct ~ abdomen + weight + wrist + forearm
##
##           Df Sum of Sq  RSS   AIC
## + neck     1     51.07 4607 744.3
## + age      1     38.36 4620 745.0
## <none>                      4658 745.1
## + biceps   1     33.88 4624 745.2
## + thigh    1     27.22 4631 745.6
## + knee     1     19.83 4638 746.0
## + ankle    1     18.16 4640 746.1
## + height   1     18.05 4640 746.1
## + hip      1      3.53 4655 746.9
## + chest    1      0.49 4658 747.0
##
## Step:  AIC=744.3
## bodyfat.pct ~ abdomen + weight + wrist + forearm + neck
##
##           Df Sum of Sq  RSS   AIC
## + age      1     47.93 4559 743.7
## + biceps   1     45.93 4561 743.8
## <none>                      4607 744.3
## + thigh    1     25.10 4582 744.9
## + height   1     18.87 4588 745.3
## + hip      1     10.99 4596 745.7

```

```

## + ankle 1 10.66 4597 745.7
## + knee 1 10.40 4597 745.7
## + chest 1 0.01 4607 746.3
##
## Step: AIC=743.66
## bodyfat.pct ~ abdomen + weight + wrist + forearm + neck + age
##
##           Df Sum of Sq  RSS   AIC
## + thigh  1    67.39 4492 741.9
## + biceps  1    48.14 4511 743.0
## <none>          4559 743.7
## + height  1    18.98 4540 744.6
## + ankle  1    14.78 4544 744.8
## + knee   1     6.55 4553 745.3
## + hip    1     3.23 4556 745.5
## + chest  1     0.84 4558 745.6
##
## Step: AIC=741.91
## bodyfat.pct ~ abdomen + weight + wrist + forearm + neck + age +
##           thigh
##
##           Df Sum of Sq  RSS   AIC
## + hip      1    36.52 4455 741.9
## <none>          4492 741.9
## + biceps   1    25.49 4466 742.5
## + ankle    1    12.77 4479 743.2
## + height   1     4.33 4488 743.7
## + chest    1     0.76 4491 743.9
## + knee     1     0.00 4492 743.9
##
## Step: AIC=741.85
## bodyfat.pct ~ abdomen + weight + wrist + forearm + neck + age +
##           thigh + hip
##
##           Df Sum of Sq  RSS   AIC
## <none>          4455 741.9
## + biceps   1    20.712 4435 742.7
## + height   1    11.749 4444 743.2
## + ankle    1    11.620 4444 743.2
## + knee     1     0.037 4455 743.8
## + chest    1     0.000 4455 743.9
##
## Call:
## lm(formula = bodyfat.pct ~ abdomen + weight + wrist + forearm +
##     neck + age + thigh + hip, data = bodyfat)

```

```
##
## Coefficients:
## (Intercept)      abdomen      weight      wrist      forearm      neck
##      -22.6564      0.9448      -0.0899      -1.5367      0.5157      -0.4666
##           age           thigh           hip
##           0.0658           0.3024          -0.1954
```

Here, we see at the first step that adding **abdomen** improves the model the most. At the second step, a model with **abdomen** and **weight** looks best. The resulting model matches that of backward selection.

IMPORTANT NOTE It is possible that forward and backward selection will determine different models! If you have taken an elementary calculus course the concept of local versus global minimums should not be foreign. The stepwise procedures essentially find local minimums of AIC working in one direction, they can result in different models.

9.2 Best subsets

If there are p potential predictors, then there are 2^p possible models. This can get out of hand in a hurry. For example, the body fat example has 13 potential predictors, meaning we have $2^{13} = 8192$ main-effects linear models to consider!

The idea of a best-subsets selection method is to choose candidate models based on some objective criterion that measures the quality of the fit or quality of the predictions resulting from the model. Because there are so many potential candidates to consider, we usually select the best few models of each size to evaluate. There are several possible criteria we could use to compare candidate models, but the usual criterion of choice is R_{adj}^2 or BIC (recall from Chapter 6 BIC is a variant of AIC).

The `regsubsets()` function in the R add-on package **leaps** finds the optimal subset of predictors of each model size. By default, the function returns only optimal subsets and only computes subsets up to size 8; these defaults can be changed using the `nbest` and `nvmax` arguments, respectively.

```
bodyfat.gsub <- regsubsets(bodyfat.pct ~ ., data=bodyfat, nbest=3, nvmax=13)
```

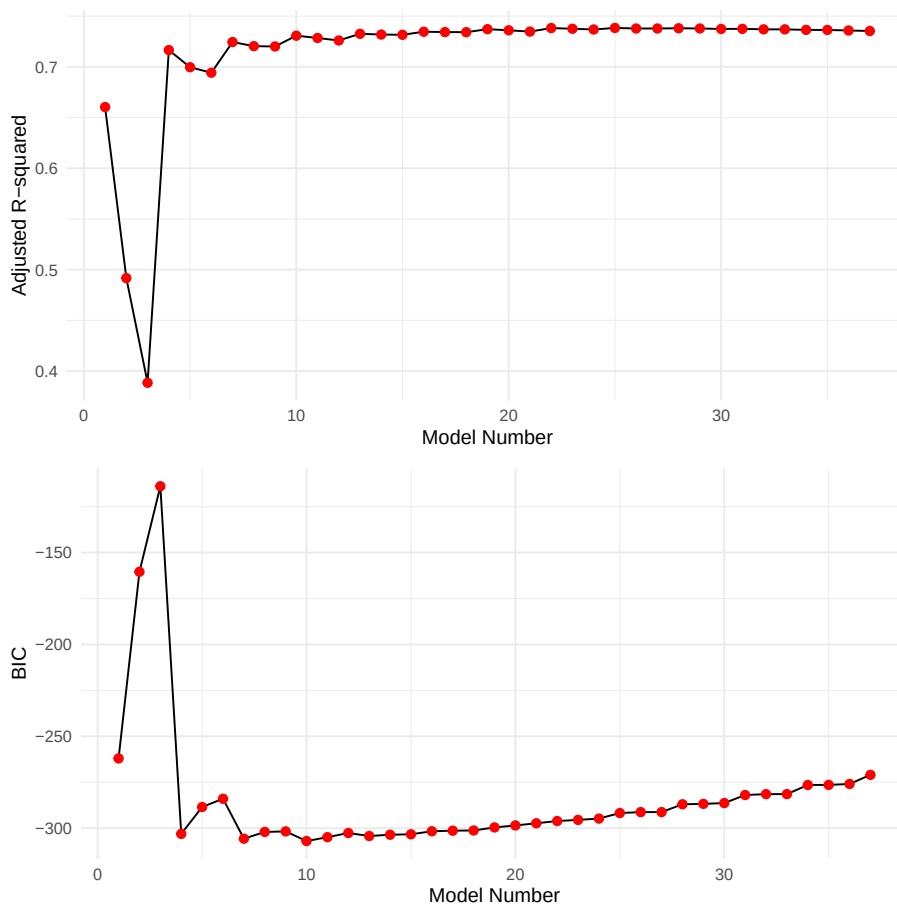
The object `bodyfat.gsub` contains a ton of information (we just fit upwards of 3×13 models). It works best to plot the R_{adj}^2 and BIC values to get a feel for which models to consider. The following code extracts the R_{adj}^2 and BIC values for each model fit.

```
stats <- summary(bodyfat.gsub)
gsub.df <- data.frame(Model.Number=1:length(stats$adjr2), Adjusted.R2=stats$adjr2, BIC=stats$bic)
p1 <- ggplot(gsub.df, aes(x=Model.Number, y=Adjusted.R2)) +
  geom_line() +
  geom_point(color="red", size=2) +
```

```

theme_minimal() +
  ylab("Adjusted R-squared") + xlab("Model Number")
p2 <- ggplot(gsub.df, aes(x=Model.Number, y=BIC)) +
  geom_line() +
  geom_point(color="red", size=2) +
  theme_minimal() +
  ylab("BIC") + xlab("Model Number")
grid.arrange(p1,p2, nrow=2)

```



First we note that these plots may include too much information. It is hard to tell exactly what is happening. Recall the objective, we want a large R^2_{adj} or small BIC values. From the computed R^2_{adj} and BIC values, model number 10 has the best BIC value and model 25 has the best R^2_{adj} . However we note from the plot that model 25 has essentially the same R^2_{adj} as all the other models nearby. In fact consider the following:

	Model	Adjusted.R2	BIC
Best Adjusted-R2	25	0.73835	-291.776
Best BIC	10	0.73072	-307.026

The best model in terms of R_{adj}^2 barely improves over the best BIC model, yet there is quite a bit of improvement in terms of BIC. Let's extract the coefficients from the two model fits.

```
coef(bodyfat.gsub, which.max(gsub.df$Adjusted.R2))
```

```
## (Intercept)      age      weight      neck      abdomen      hip
## -23.3049918  0.0634833 -0.0984253 -0.4932953  0.9492607 -0.1828710
##      thigh      biceps      forearm      wrist
##   0.2653788  0.1788900  0.4514962 -1.5420837
```

```
coef(bodyfat.gsub, which.min(gsub.df$BIC))
```

```
## (Intercept)      weight      abdomen      forearm      wrist
## -34.854074  -0.135631  0.995751  0.472928 -1.505562
```

The model based on R_{adj}^2 chose 9 predictor variables compared to only 4 for the best BIC model. So, there is no clear-cut answer, but that's the point: the "best subsets" idea allows you to consider the options and to weigh the relative trade-offs in using one model over another.

Some sanity checks when performing a variable selection procedure

Some important points worth noting:

- Variable selection is a means to an end, not an end unto itself. Too often, researchers use these techniques as a substitute for thinking about the problem, being content to "let a computer choose the variables" for them. Don't fall into that trap! Your aim is to construct a model that predicts well or explains the relationships in the data. Automatic variable selection is not guaranteed to be consistent with these goals. Use these methods as a guide only.
- Some models have a natural hierarchy. For example, in polynomial models, X^2 is a higher order term than X . When performing variable selection, it is important to respect hierarchies. Lower order terms should not be removed from the model before higher order terms in the same variable. Another example is when you have a model containing interaction terms, e.g. X_1X_2 . Any model containing X_1X_2 as a parameter must also contain main effect terms for X_1 and X_2 to respect the hierarchy.

Finally, it is entirely possible that there may be several models that fit (roughly) equally well. If this happens, you should consider:

1. Do the models have similar qualitative consequences?
2. Do they make similar predictions?
3. What is the practical cost of measuring the predictors?

4. Which has better diagnostics?

If you find models that seem (roughly) equally as good yet lead to quite different conclusions, then it is clear that the data cannot answer the question of interest without ambiguity.

9.3 Shrinkage Methods

We conclude this chapter with a short review of what are known as *shrinkage* methods for regression. These methods are utilized more in *modern* practice than the subsets or stepwise procedures described above. The details and implementation of these methods are outside the scope of this class but the topic is relevant enough for a short introduction.

First, recall the least squares estimation procedure associated with regression; that is, find the set $b_0, b_1, \dots, b_p$ that minimize

$$RSS = \sum_{i=1}^n (y_i - (b_0 + b_1 x_{1i} + b_2 x_{2i} + \dots + b_p x_{pi}))^2$$

for a set of predictor variables X_i , $i = 1, \dots, p$ and response variable Y .

Now consider the following modification to the standard regression ideas

- Assume all the X_i terms have been standardized (see the section [@ref\(#standardizingPredictors\)](#)).
- Since all X_i terms are standardize, all b_i terms are on the same scale

Since all b_i terms are on the same scale, the magnitude (i.e., $|b_i|$) reasonably corresponds to the *effect* X_i has on the response variable Y . So a good model selection method will pick non-zero magnitude b_i terms. This leads to two so-called *shrinkage* methods: Ridge regression and LASSO regression.

Ridge Regression

The idea of ridge regression is to minimize the equation

$$\sum_{i=1}^n (y_i - (b_0 + b_1 x_{1i} + b_2 x_{2i} + \dots + b_p x_{pi}))^2 + \lambda \sum_{j=1}^p b_j^2 = RSS + \lambda \sum_{j=1}^p b_j^2$$

The term on the right-hand side, $\lambda \sum_{j=1}^p b_j^2$ for $\lambda > 0$, essentially operates as a penalty term, *shrinking* the b_i terms towards zero; it is known as the shrinkage penalty. The parameter λ is known as the tuning parameter and must be estimated or specified by the user.

LASSO Regression

The idea of Least Absolute Shrinkage and Selection Operator, or simple LASSO, regression is similar but the object equation is a little different:

$$\sum_{i=1}^n (y_i - (b_0 + b_1 x_{1i} + b_2 x_{2i} + \dots + b_p x_{pi}))^2 + \lambda \sum_{j=1}^p |b_j| = RSS + \lambda \sum_{j=1}^p |b_j|$$

Here the only change is to the shrinkage penalty component of the question.

Summary of LASSO and Ridge Regression

Both ridge and LASSO regression are implemented in the `glmnet` package in R and we reference the associated documentation on its use. We summarize the two methods with some noteworthy properties of each.

- Ridge regression can be used to mitigate the effects of multicollinearity
- LASSO has the advantage of producing simpler models (it can be shown that LASSO can shrink some b_i terms to exactly zero)
- The tuning parameter, λ , needs to be specified by the user or selected via cross-validation (discussed in the next chapter)
- Like stepwise regression, both Ridge and LASSO can be automated.

See *An Introduction to Statistical Learning* by James, Witten, Hastie and Tibshirani for a more thorough treatment on the topic.

Chapter 10

Model Validation

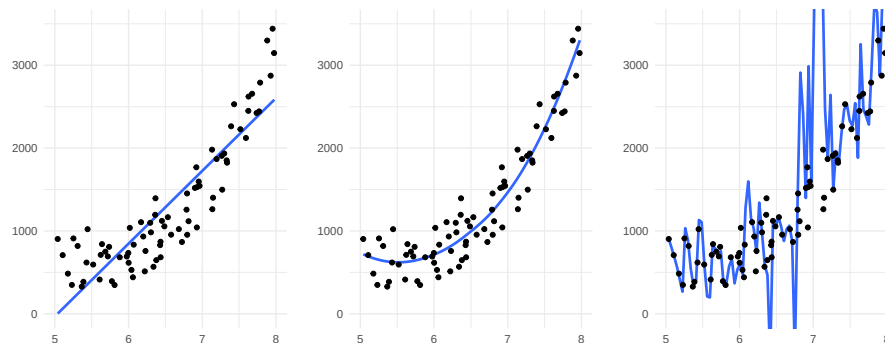
At this point we have covered various concepts of statistical modeling but one fundamental question remains, “Is my model any good?” Answering this question is of fundamental importance and there is no single way to determine the appropriateness of a model. We have covered techniques such as R^2 , R_{adj}^2 and AIC but all three of those measures essentially measure how well the fitted model *fits* the data you used to make the fit. Generally speaking, we should consider other measures to validate our models as well.

10.1 Underfitting vs. Overfitting Models

We have seen that increasing the complexity of a statistical model will always improve the explanatory power on a response variable. This is seen by the fact that R^2 will always improve as the number of predictors increases. In fact, you can show that if a regression model has $k = n$ parameters (i.e., if the number of β -coefficients in the model is the same as the sample size of the data it is being fit to), then you guarantee $R^2 = 100\%$!

Of course, all this means is that such a model perfectly fits the data *it was built with*. At the same time, this model may be a very poor predictor of new (unobserved) individuals. So, using the *same* data to assess the quality of a model is not exactly a great way to assess its predictive performance.

Consider the following three models fit to the same data points:



The model on the left is underfit: it misses the clear curvature in the relationship between the predictor and response. So, from a trend perspective, it would systematically mispredict future observations that are produced by the same process.

The model on the right, however, is overfit: if you look closely, you'll see that it falls much closer to the observed data values than either of the other two models. So this overly complicated model can predict its OWN data very well – but that is because the model you see is catching a lot of the specific random behavior in this particular data set. It isn't hard to imagine that a *new* data set, generated by the same process but exhibiting different random behavior, would be poorly predicted by even this complicated model!

The model in the middle appears to be the preferred one to generalize and make predictions from. This is because it captures the systemic trend in the predictor/response relationship, but *that's all*. It strikes a happy medium between two situations that can lead to poor predictive performance of a model on future observations.

10.1.1 The Bias-Variance Trade-off

So here is the dilemma:

- We want to avoid overfitting because it gives too much predictive power to specific quirks in our data.
- We want to avoid underfitting because we will ignore important general features in our data.

How do we balance the two? This is known as the bias-variance trade-off. *Bias* corresponds to underfitting (our predictions are too vague to account for general pattern that do exist in the sample) and *variance* corresponds to overfitting (our predictions are so specific that they only reflect our specific sample).

10.2 Validation Techniques

Overfitting results in low prediction error on the observed data but high prediction error on new (unobserved) data, while underfitting results in the opposite. Measuring these type of errors can be accommodated using a process called **cross-validation** (CV). CV comprises a set of techniques that enable us to measure the performance of a statistical model with regard to how well it can predict results in new datasets.

There are three general approaches to this, which are detailed below. They all involve splitting your data into partitions, and using some part(s) of the data to build models and the remaining part(s) to test your model's predictive performance. It should go without saying that the methods we discuss here require that you have fairly large data sets (many cases, large n) so that we have ample information with which to build models.

10.3 Basic Validation with a single holdout sample

The most basic idea behind CV involves dividing your data into two partitions, or subsets:

- A **training set** on which we build our model. It is called a *training set* because we use this data partition to *train* the model for prediction.
- A **test set** (or validation set) which is used to test our model by estimating the prediction error resulting from predicting new observations.

Commonly used split proportions in practice are 80% for training data and 20% for test data, though this can be altered. To assess model predictive performance, a good choice is to look at the model's **residual standard error** as calculated on the **test data**. Formulaically, the residual standard error is the square root of the mean squared prediction error values.

Example: Estimating Bodyfat Percentage. Let's revisit the bodyfat percentage problem from Chapter 9 in the textbook. Recall that the goal was to develop a model that would do well at predicting a man's bodyfat percentage by simply taking some selected circumference measurements around their body. Let's use this data to do some model validation.

The first thing we need to do is split the data set consisting of $n = 252$ men into a training set and a test set. This is done using the code below, creating a random 80/20 training/test split using R's `sample` function:

```
set.seed(54321)    # Set a seed value for reproducibility purposes in this document

# Randomly select 80% of the data (rows)
index <- sample(1:nrow(bodyfat), size=floor(.80*nrow(bodyfat)))
train <- bodyfat %>%
```

```

  filter(row_number() %in% index)
test  <- bodyfat %>%
  filter(!row_number() %in% index)

```

The vector `index` consists of randomly selecting row numbers (cases) from the `bodyfat` dataset. The size of the selection is set to be 80% of the number of rows in the dataset itself. Below we check to see how many cases landed in each of the training and test data sets:

```
nrow(bodyfat)
```

```
## [1] 252
```

```
nrow(train)
```

```
## [1] 201
```

```
nrow(test)
```

```
## [1] 51
```

We have split the original sample of $n = 252$ men into a training set of 201 men and a test (validation) set of 51 men.

10.3.1 Use the training data to fit and select models

We now use the training data (named `train` by us above) to build our model. We can use any or all of the techniques we have already covered to this point to build (“train”) our model: stepwise regression, variable deletion, transformations, etc. We use a best-subsets approach below, much like we did back in Chapter 9. **It is critical to remember that everything we do in the model fitting stage is done on the training data only.**

10.3.2 Model training:

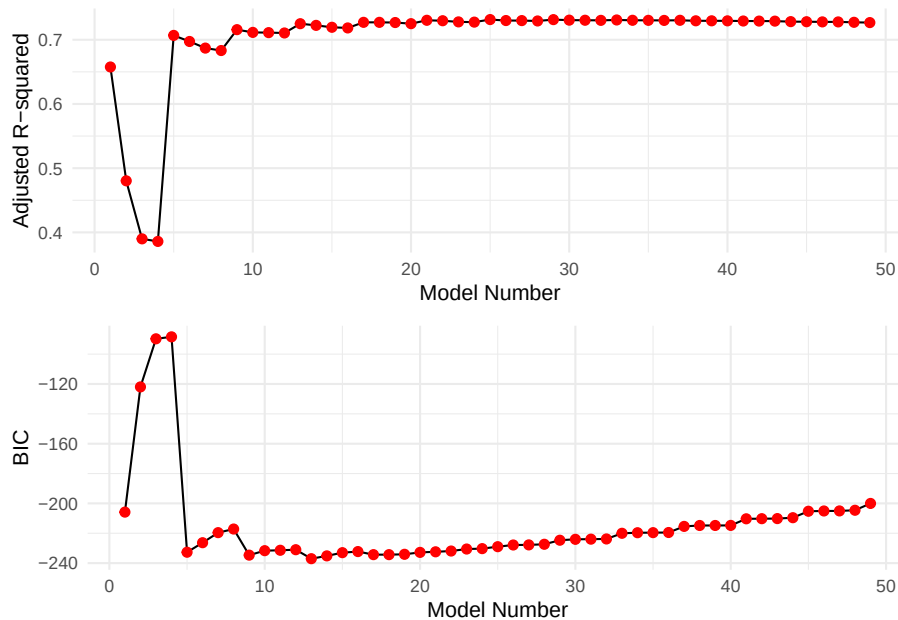
Check various models’ performance based on R^2_{adj} and BIC:

```

bodyfat.gsub <- regsubsets(bodyfat.pct ~ ., data=train, nbest=4, nvmax=13)
stats <- summary(bodyfat.gsub)
gsub.df <- data.frame(Model.Number=1:length(stats$adjr2), Adjusted.R2=stats$adjr2, BIC=stats$bic)
p1 <- ggplot(gsub.df, aes(x=Model.Number, y=Adjusted.R2)) +
  geom_line() +
  geom_point(color="red", size=2) +
  theme_minimal() +
  ylab("Adjusted R-squared") + xlab("Model Number")
p2 <- ggplot(gsub.df, aes(x=Model.Number, y=BIC)) +
  geom_line() +
  geom_point(color="red", size=2) +
  theme_minimal() +

```

```
ylab("BIC") + xlab("Model Number")
grid.arrange(p1,p2, nrow=2)
```



The estimated β -coefficients for the predictors of the best fitting model based on maximizing R_{adj}^2 are as follows:

```
coef(bodyfat.gsub, which.max(gsub.df$Adjusted.R2))
```

```
## (Intercept)      weight      neck      abdomen      ankle      biceps
## -36.886823    -0.151137   -0.370366    1.005815    0.360620    0.371678
##   forearm      wrist
##   0.464346   -1.609547
```

We see that this criterion selects a 10-predictor model. While this is OK, it would involve a lot of body measuring in practice, and so might not be the best choice from an implementation perspective. Its adjusted R_{adj}^2 is found as follows:

```
max(gsub.df$Adjusted.R2)
```

```
## [1] 0.731449
```

Now, let's look at what model is selected as optimal when minimizing BIC:

```
coef(bodyfat.gsub, which.min(gsub.df$BIC))
```

```
## (Intercept)      weight      abdomen      forearm      wrist
## -32.562236    -0.122294    0.969583    0.539201   -1.721664
```

The best fitting model based on BIC has four predictors: `weight`, `abdomen`, `biceps`, and `wrist`. As a corollary assessment, we can check the value of R_{adj}^2 for this 4-predictor model:

```
gsub.df$Adjusted.R2[which.min(gsub.df$BIC)]
```

```
## [1] 0.725006
```

The best fitting model based on BIC is much simpler (4 predictors instead of 10), and its adjusted R_{adj}^2 is not noticeably lower than the 10-predictor model (0.7427 vs. 0.7549). **So, let's choose to use the four predictor model.**

10.3.3 Model validation step:

Now, let's use this model to predict bodyfat percentages for the men in the holdout (test) dataset. First we fit the chosen model on the training dataset. Then we use that model to predict the holdout values in the testing set.

```
fit1 <- lm(bodyfat.pct ~ weight + abdomen + biceps + wrist, data=train)
test.predictions <- predict(fit1, newdata=test)
```

The residual standard error (or equivalently, the square root of the mean squared residuals – or **root mean squared error**) can be calculated on the test data to see how well our model predicts future observations. Below we manually calculate this value for explanation.

```
# Calculate observed - predicted bodyfat for test data
residuals <- test$bodyfat.pct - test.predictions

# Calculate and display the residual std error
test.rse <- sqrt(mean(residuals^2))
test.rse
```

```
## [1] 4.48002
```

This is an estimate of the average residual (prediction error) size for individuals in an independent sample of men. Using an Empirical Rule-style argument, we can be about 95% confident that our model will produce a predicted male bodyfat percentage that is within about $2 \times 4.68 = 9.36\%$ of the actual value.

Compare this result to the artificially optimistic residual standard error we get by naively predicting the results for the *same men* we used to fit the model, using all the original data:

```
# Fit model to all the data
full.sample.fit <- lm(bodyfat.pct ~ weight + abdomen + biceps + wrist, data=bodyfat)

# Predict all men in the same sample,
# and calculate their residuals and residual std error
full.predictions <- predict(full.sample.fit, newdata=bodyfat)
```

```
residuals <- bodyfat$bodyfat.pct - full.predictions
full.rse <- sqrt(mean(residuals^2))
full.rse
```

```
## [1] 4.31743
```

The result might look better, but it is biased toward the sample it came from!

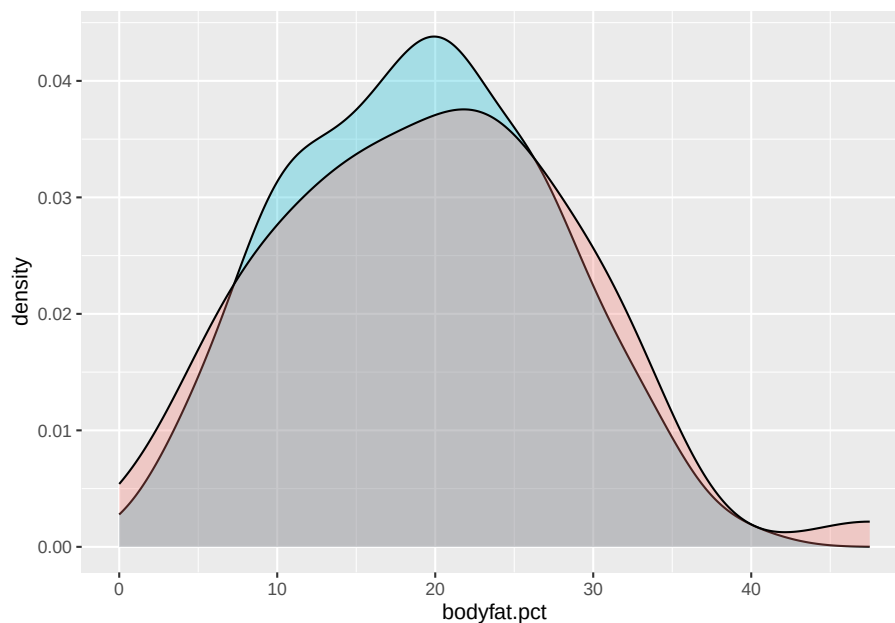
10.4 Hold-out sample validation using caret

The above example was quite involved and done so for the sake of explanation. We can instead use some features in the `tidyverse` and the add-on `caret` library to effectively do the same thing.

```
# Create a balanced 80/20 split of the sample based on the response variable
train.index <- bodyfat$bodyfat.pct %>%
  createDataPartition(p = 0.8, list = FALSE)

train.data <- bodyfat %>%
  filter(row_number() %in% train.index) # 80% goes into training data
test.data <- bodyfat %>%
  filter(!row_number() %in% train.index) # The rest goes into test data

# Verify balance between training and test data on the response variable:
ggplot() +
  geom_density(data=train.data, aes(x = bodyfat.pct),
    fill = "#00BCD8", alpha = 0.3) +
  geom_density(data=test.data, aes(x = bodyfat.pct),
    fill = "#F8766D", alpha = 0.3)
```



```
# Build/fit our model
fitted.model <- lm(bodyfat.pct ~ weight + abdomen + biceps + wrist, data=train.data)

# Make predictions and compute RMSE and MAE
predictions <- fitted.model %>%
  predict(test.data)
data.frame(RMSE = RMSE(predictions, test.data$bodyfat.pct),
           MAE = MAE(predictions, test.data$bodyfat.pct))

##      RMSE      MAE
## 1 4.32594 3.58625
```

The `caret` package can easily provide us with the root mean squared error RMSE (residual standard error) like before, but can also provide other measures of model performance. Two measures are typically employed:

- **Root Mean Squared Error (RMSE)**, which measures the average prediction error made by the model when predicting the outcome for a future observation. This is what we have already calculated. Lower values of RMSE are better.
- **Mean Absolute Error (MAE)**. This is an alternative to RMSE that is less sensitive to outliers in your data. It corresponds to the average *absolute* difference between observed and predicted outcomes. Lower values of MAE are also better.

Both of these are measured in the same units as the response variable. In the above example, the mean absolute error in bodyfat prediction is 3.265% and

the root means squared error is 3.9% (note this is different than our previous derivation because a different training and testing set were declared).

Disadvantages. The single holdout sample method is only useful when you have a large data set that can be partitioned. The key disadvantage, however, is that we build a model only using a fraction of the available data, which may possibly leave out some interesting information, leading to higher bias. We try to mitigate this by ensuring the training and test data sets are balanced with respect to the response variable (using `createDataPartition` from the `caret` package), but the potential for bias still exists as it could come from imbalance among the predictors, etc.

As a result, test error rates using a single holdout sample can be highly unstable depending on which observations are included in the training set. So, we’d like to consider using more comprehensive approaches.

10.5 “Leave one out” Cross-Validation (LOOCV)

More comprehensive methods **involve doing the training/testing in multiple stages across many partitions in the data**. One such method that admittedly takes the process to an extreme is called “leave-one-out” cross-validation. This method is as follows:

1. Leave out one data point (observation) and build the model on the remaining $n - 1$ data points.
2. Use the model from step 1 to predict the single data point that was left out. Record the test error associated with this prediction.
3. Repeat the process above for **all** n data points. This means you will fit n models!
4. Calculate the overall prediction error by averaging all the test errors recorded for the individual points through the process. We can still use RMSE or MAE for this.

This is a very intensive process as you might imagine, but it is actually very easy to do in `caret`:

```
# Set training control method as LOOCV
train.control <- trainControl(method = "LOOCV")

# Train the model
LOOCV.model <- train(bodyfat.pct ~ weight + abdomen + biceps + wrist,
  data = bodyfat,
  method = "lm",
  trControl = train.control)
```

```

# Display results
LOOCV.model

## Linear Regression
##
## 252 samples
## 4 predictor
##
## No pre-processing
## Resampling: Leave-One-Out Cross-Validation
## Summary of sample sizes: 251, 251, 251, 251, 251, 251, ...
## Resampling results:
##
## RMSE      Rsquared  MAE
## 4.43271  0.718507  3.65969
##
## Tuning parameter 'intercept' was held constant at a value of TRUE

```

The function `trainControl()` defines the partitioning method for the coming validation step. Then we use the `train()` function in `caret` to execute the process. We first specify the model form (we are still choosing to use the 4-predictor model here), run the process on the master data set (here, `bodyfat`), use linear regression (`method="lm"`), and specify the partitioning method to use.

The resulting output list the RMSE and MAE, along with a pseudo R-squared type measure (looking at predictor error compared to variance in the response).

LOOCV Disadvantages. On the surface, LOOCV seems like it might be the preferred approach since we use almost all the data ($n - 1$ data points) to independently predict every possible holdout. However, it can be computationally intensive, and might result in higher variation in the prediction error if some data points are outliers. So, ideally we will use a good ratio of testing data points ... a solution provided by the next method, which is a sort of compromise between a single holdout sample and LOOCV.

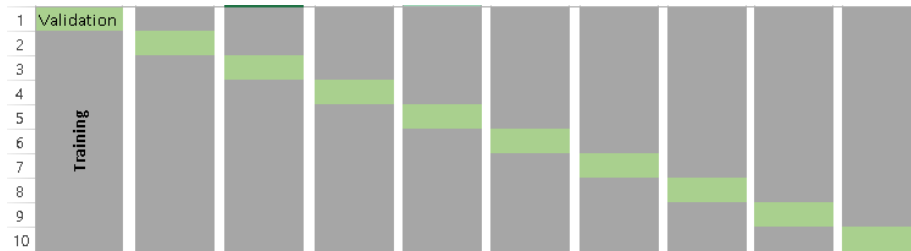
10.6 k -fold Cross-Validation

The k -fold cross-validation method evaluates model performance on different subsets of the training data and then calculates the average prediction error rate across all the different subsets. We purposefully choose a number of subsets, known as **folds** of the data, into which we partition the data. The process is as follow:

1. Randomly split the data into k subsets, or **folds**. Note that k is determined by the user; typically a value of 5 or 10 is used.

2. Hold out one fold, and train the model on all other folds combined
3. Test the model on the held-out fold and record the prediction errors
4. Repeat this process until each of the k folds has served as a test set
5. Calculate the average of the k recorded errors. This will be the performance measure for the model.

Visually, the folds can be thought of as something like the below example, a $k = 10$ fold segmentation.



The most obvious advantage of k -fold CV compared to LOOCV is computational. The question is: *what is a good value for k* ? Consider the following:

- **A low value of k (few folds) leads to more bias potential.** It is not hard to see that using a small value of k is not that much different than just using the first method described, a single holdout sample.
- **A high value of k (many folds) leads to more variance in the prediction error.** In fact, if $k = n$, then you are doing LOOCV.

It has been shown in practice that using $k = 5$ or 10 yields test error rates that do not appreciably suffer from excessive bias nor high prediction error variance.

The following example uses `caret` to perform a 5-fold cross validation to estimate the prediction error for predicting bodyfat percentage in men from the `weight`, `abdomen`, `biceps`, and `wrist` measurement predictors.

```
# Set training control method as 5-fold CV
train.control <- trainControl(method = "cv", number = 5)

# Train the model
kfoldCV.model <- train(bodyfat.pct ~ weight + abdomen + biceps + wrist,
  data = bodyfat,
  method = "lm",
  trControl = train.control)

# Display results
kfoldCV.model

## Linear Regression
##
## 252 samples
```

```
## 4 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 202, 202, 202, 202, 200
## Resampling results:
##
## RMSE      Rsquared  MAE
## 4.41851  0.731733  3.64661
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

Here we see similar output as the LOOCV but the procedure runs substantially quicker. k -fold cross-validation is generally recommended over the other two methods in practice due to its balance between variability, bias and computational run-time.

10.7 A final note

As with many topics in this text, we are merely *scratching the surface* on this topic. However, we have outlined the basic building blocks on modern model validation. Several variants exist including

- Segmenting your data into *training*, *tuning* and *testing* sets. Consider the brief discussion on selecting a *tuning* parameter for LASSO and Ridge regression in the previous chapter. The tuning set can be used to select that parameter, then the final model can be validated with the testing set.
- Repeating k -fold CV multiple times. Performing k -fold CV one time is not too different than the single training and testing set approach, data is segmented randomly into sets. A different random permutation will result in a different RMSE (we saw this above!). It is possible to repeat the k -fold CV multiple times and aggregate all the results. If computational power allows, this is typically done in practice.

Chapter 11

Statistical Odds

We now begin our discussion on Generalized Linear Models. But before moving into the specifics of the models, we take a quick tangent to discuss statistical odds. This is necessary in order to truly understand and appreciate the finer details of Logistic Regression.

Consider the game of Roulette:

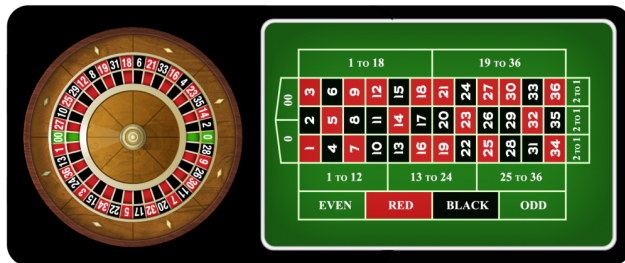


Figure 11.1: Image of American roulette table (with 0 and 00).

In this game a croupier spins the wheel in one direction, then spins a ball, known as *the pill*, in the opposite direction. The *pin* will stop on one of the 38 designated numbers (0, 00, 1, 2, ..., 36). You will note that zero values are colored green while the remaining numbers alternate the colors red or black. Players of the game will place bets on various parts of the board.

Our goal here is not to study games of chance. However, roulette provides a good example of basic probability and odds. Consider the simple case where the player bets on either **red** or **black**.

For discussion, suppose a player places a bet on **red**. What is the probability they win?

Solution There are 38 possible outcomes in the game, of which 18 are **red**. The probability the player wins the game is simply $18/38$, or 0.4737. The probability the player loses is the complement, which is $20/38$ or 0.5263.

11.1 Probability versus Odds

Odds are related to probabilities but are different. The *odds* of winning are calculated by counting the number of ways you win the game, and divide by the number of ways you lose the game. So in our working example there are 18 ways **red** occurs while there are 20 ways the outcome is not **red**. Thus the odds of red are $18/20 = 0.9$.

Note,

$$\text{Odds of red} = \frac{18}{20} = \frac{P(\text{red})}{P(\text{not red})} = \frac{18/38}{20/38} = \frac{18}{20} = 0.9 = \frac{9}{10}.$$

In the gambling world, one may say the odds of **red** are 9 to 10.

Thought exercise: For those who like games of chance, in a standard casino betting on **red** or **black** typically pays a 1 to 1 ratio, meaning if you bet \$100 on **black** and the outcome is **black**, you win \$100. *Should you play this game? Would you play this game?*

11.2 Odds ratios

Now that we have a basic understanding of *odds*, let consider a non-gambling (motivated by data) example.

On April 14, 1912, the RMS Titanic struck an Iceberg in the North Atlantic Ocean off the coast of Newfoundland. Roughly 3 hours after striking the iceberg, the Titanic sunk on April 15, 1912 at 2:20am.

In the paper, The “Unusual Episode” Data Revisited, published in the Journal of Statistics Education vol.3, no.3 (1995), records for 2201 passengers and crew were recorded with their ticket status (the Class variable), Age (categorized as Adult/Child), Gender (Female/Male) and whether they survived the sinking.

```
titanic <- read.csv("titanic.csv")
head(titanic)
```

```
##   X Class   Age Gender Survived
## 1 1      1 Adult   Male      Yes
## 2 2      1 Adult   Male      Yes
## 3 3      1 Adult   Male      Yes
## 4 4      1 Adult   Male      Yes
## 5 5      1 Adult   Male      Yes
## 6 6      1 Adult   Male      Yes
```

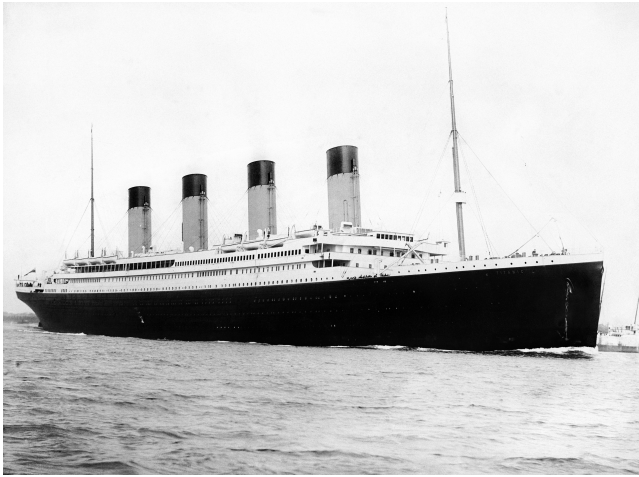


Figure 11.2: Image of the RMS Titanic (damage from iceberg not included)

In this example, we will explore the odds of surviving the sinking of the Titanic as a function of different variables. First consider the basic case of surviving, we will build a contingency table using the `xtabs()` function in R.

```
xtabs(~Survived, data=titanic)
```

```
## Survived
##   No  Yes
## 1490  711
```

What is the probability a randomly selected passenger survived the sinking of the Titanic?

For a randomly selected passenger, what are the odds of survival?

Solutions

The probability a randomly selected passenger survived is $\frac{711}{1490+711}$, or 0.323.

The odds of survival are $\frac{711}{1490}$ or 0.4772.

Thought exercise: If the odds of Roulette were similar to surviving the Titanic, would you play the game?

In the Titanic dataset we have other information besides whether a person survived or not. Consider a simple question: Does *Gender* help explain, or predict, a person's ability to survive this disaster?

We have studied many problems such as this. Does area influence the sale price of a house; does ACT score predict first year GPA; etc...

Here things are different because our response variable (survived or not) is

categorical (binary) as well as our predictor variable (gender). The most basic way to explore this sort of relationship is with a two-way contingency table:

```
xtabs( ~ Gender + Survived, data=titanic)
```

```
##           Survived
## Gender      No  Yes
##  Female  126  344
##   Male 1364  367
```

If you are a female, what are your odds of surviving?

$$\frac{344}{126} = 2.7302$$

If you are a male, what are your odds of surviving?

$$\frac{367}{1364} = 0.2691$$

Thought exercise – Consider the gambling ideas from above. If you know a person's gender, are you willing to bet on surviving or not?

It sure looks like there is a relationship, or association, between gender and surviving the sinking of the titanic.

Ultimately in this class we are interested in statistical modeling but the titanic dataset provides an example of categorical data analysis, an area of study that was briefly covered in your Introductory Statistics course. This analysis is still popular in several areas (Psychology and Sociology) and appropriate for some sorts of data.

In the above calculation regarding the Titanic, we essentially are determining an association between categories with a binary response (survived or died). There are several quantitative measures of association for categorical data. There are pros and cons to each. We will just list them here and encourage the interested reader to explore on their own the topic:

- Cramer's V
- The ϕ statistic
- Relative risk

The first two methods are based on the Chi-square test for association (typically covered in Intro Stat). They can be considered analogous measures of the correlation coefficient r we saw with regression. Relative risk is derived from the world of Biostatistics and can be a great tool for comparing these sorts of relationships. Basically you compare the conditional probabilities: $P(\text{survived} \mid \text{female}) = \frac{344}{344+126} = 0.7319$ and $P(\text{survived} \mid \text{male}) = \frac{367}{1731} = 0.212$

The relative risk of surviving for a female compared to a male is $\frac{344/470}{367/1731} = 3.4522$.

If there were no association between gender and surviving, this relative risk value would be approximately one. Here, we see a random female passenger

had greater than a $3\times$ chance of surviving compared to a male passenger.

Note This example is a bit strange because *risk* is typically a bad thing. In our above discussion, surviving is the outcome of interest. If I reworked things in terms of not surviving (i.e., death!), we see the relative risk of dying for a female compared to a male is 0.3402. So a randomly selected female would only perish about 1/3 of the rate of a male.

11.3 Ideas of modeling odds

In the above discussion, it sure seems like gender is a predictor for surviving. Other variables are included in the dataset. Some additional contingency table analysis provides additional insight.

```
xtabs(~ Age + Survived, data=titanic)
```

```
##           Survived
## Age         No  Yes
##  Adult 1438  654
##   Child   52   57
```

The odds of a child surviving are 1.0962 while that of an adult are 0.4548.

Although not ideal, it was better to be a child than an adult on the Titanic.

What about the `Class` variable. Here things get a little more interesting because there are multiple categories

```
xtabs(~ Class + Survived, data=titanic)
```

```
##           Survived
## Class      No  Yes
##   1      122 203
##   2      167 118
##   3      528 178
##  Crew     673 212
```

We see some obvious things:

- Best to be in first class
- If not first, take your chances in second class.
- Crew members and third class did not fare well.

Of course, all these factors may interact! We'll revisit that topic in the next chapter but for now we can build more complicated contingency tables. Here we wrap the output of `xtabs` in the `fTable()` function for more pleasing output. First consider `Class` and `Age` on surviving.

```
fTable(xtabs(~ Class + Age + Survived, data=titanic))
```

```
##           Survived  No  Yes
```

```
## Class Age
## 1      Adult      122 197
##        Child       0   6
## 2      Adult      167  94
##        Child       0  24
## 3      Adult      476 151
##        Child       52  27
## Crew   Adult      673 212
##        Child       0   0
```

We can see that being a child in first or second class turned out fairly well. The third class children did not survive at as higher of a frequency.

Similarly we can explore other interactions.

```
fable(xtabs( ~ Class + Gender + Survived, data=titanic))
```

```
##              Survived  No Yes
## Class Gender
## 1      Female         4 141
##        Male        118  62
## 2      Female         13  93
##        Male        154  25
## 3      Female        106  90
##        Male        422  88
## Crew   Female         3  20
##        Male        670 192
```

```
fable(xtabs( ~ Age + Gender + Survived, data=titanic))
```

```
##              Survived  No  Yes
## Age  Gender
## Adult Female        109 316
##        Male        1329 338
## Child Female         17  28
##        Male         35  29
```

Lastly, we can do a full interaction between all the variables

```
fable(xtabs( ~ Class + Age + Gender + Survived, data=titanic))
```

```
##              Survived  No Yes
## Class Age  Gender
## 1      Adult Female         4 140
##        Male        118  57
##        Child Female         0   1
##        Male         0   5
## 2      Adult Female         13  80
##        Male        154  14
```

##	Child Female	0	13
##	Male	0	11
## 3	Adult Female	89	76
##	Male	387	75
##	Child Female	17	14
##	Male	35	13
## Crew	Adult Female	3	20
##	Male	670	192
##	Child Female	0	0
##	Male	0	0

Of note here is the number of 0 counts that occur (all first and second class children survived, and there were no children on the crew). This will impact the feasible choice of models we consider in the next chapter.

Chapter 12

Logistic Regression

Earlier, we introduced diagnostics to check the assumptions in a standard regression analysis. Recall these assumptions were that the errors ε_i are independent $N(0, \sigma^2)$; i.e.,

1. the ε_i are independent.
2. the ε_i have constant variance,
3. the ε_i are normally distributed,

In many cases, remedial measures such as transformations, weighted least squares, etc., can help validate the assumptions. However, some kinds of responses by their very nature will violate these assumptions. An easy-to-see example is the dichotomous responses from the previous chapter (odds of surviving the sinking of the Titanic). In cases like this, it is much preferable to apply an appropriate modeling technique that acknowledges the true nature of the response, rather than putting the data through contortions in an attempt to “fit a square peg in a round hole.”

Example: Predicting an election outcome. Suppose we are interested in factors that influence whether or not a political candidate wins an election. Predictor variables might include the amount of money spent on the campaign, the amount of time spent campaigning negatively, whether or not the candidate is an incumbent, etc.

What makes this problem different from problems we’ve seen before? It’s that the outcome (i.e., the response variable Y) is not quantitative; rather, it is binary: the outcome is “win” or “lose”. We could code these numerically as 0 and 1 (lose = 0, win = 1). Because the response variable is binary, we need to use a model that handles binary outcomes correctly.

In situations as the above we use a **Generalized Linear Model**, or GLM. A GLM is a “generalization” of the ideas in the more typical “normal” regression modeling, but with the following added flexibility:

1. Errors, and thus the response variable, need not be normally distributed. For binary data, a more appropriate choice for the errors is the binomial distribution. For count data, the Poisson distribution may be a more appropriate choice. Most introductory statistics textbooks cover these distributions in detail.
2. Error variance need not be constant. In fact, part of the reason the binomial and Poisson distributions are more appropriate in certain cases is because in these distributions, the mean and variance are linked: so if the mean changes, so does the variance. It's a built-in property.
3. Linearity is not assumed. In GLM, we use what is known as a link function $g(Y)$ to connect the response Y to the linear combination of the predictors $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k$ on the right-hand side of our model.

If the response Y is binary (0 or 1), we can view the property of interest for the response to be the probability of an event occurring. For example, if win = 1 and lose = 0, then we can think of the underlying problem as modeling the probability of winning the election, $P(\text{win})$. We then use regression to model the likelihood of winning as a function of the predictors (amount of money spent on the campaign, etc.). We might even like to predict the probability of winning based on the specific amount of money spent on the campaign, the amount of time spent campaigning negatively, whether or not the candidate is an incumbent, etc. From this perspective, clearly the response $Y = P(\text{win})$ must be between 0 and 1, because probabilities must be between 0 and 1. However, standard regression methods do not ensure this will happen (e.g., what if your model predicted a 113% chance of winning the election?!). The use of an appropriate link function will ensure an appropriate prediction.

12.1 Logistic Model

From a practical standpoint, logistic regression and least squares regression are similar. Both methods produce prediction equations. In both cases, the regression β -coefficients measure the predictive capability of the predictor variables.

The response variable that characterizes logistic regression is what makes it special. With linear least squares regression, the response variable Y is a quantitative variable. With logistic regression, the response variable is an indicator of some characteristic; that is, a 0 or 1 variable. Logistic regression is used to determine whether other measurements are related to the presence or absence of some characteristic.

Here are a few examples of problems suited to logistic regression:

- Loan defaults. With the mortgage loan crisis afflicting the markets during the 2007-2009 recession, it is especially important for banks to know whether a prospective borrower will be at a high risk of defaulting on a loan. Historical data for past borrowers might include annual income, amount of the loan, amount in savings or assets, years of work, and credit

rating as predictors. The outcome of interest is whether or not a borrower defaulted on their loan.

- Heart attack risk. We wish to study the influence of age, gender and exercise on whether or not someone has a heart attack. Again, we have a binary response variable, whether or not a heart attack occurs.
- Graduate school admission. How do predictors such as GRE (Graduate Record Exam) scores, GPA, and prestige of the undergraduate program effect admission into graduate school? The response variable, whether or not a student is admitted, is a binary variable.

While the response variable in a logistic regression is a 0/1 variable, the logistic regression equation, which is a linear equation, does not predict the 0/1 variable itself. Instead, logistic regression predicts the probability that an indicator variable is equal to 1. To be more precise, a logistic regression equation does not directly predict the probability that the indicator is equal to 1, $P(Y = 1)$. Rather, it predicts the log odds that an observation will have an indicator equal to 1.

Odds. The odds of an event is defined as the ratio of the probability that an event occurs to the probability that it fails to occur. Let p denote the probability of an event of interest occurring, i.e., let $p = P(Y = 1)$. Then the odds of the event is given by $p/(1 - p)$

The general form of a (multiple) logistic regression model always expressed in terms of the log odds:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots \beta_k X_k$$

The link function $\log(p/(1 - p))$ is call the *logit* transformation. The logit ensures that our model will produce estimates of p between 0 and 1. The quantity $p/(1-p)$ is the odds of the event of interest occurring. The only downside of this transformation is that the linear predictor $\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots \beta_k X_k$ is in terms of “log odds”, which is not easy to interpret. However, once we fit a model, we can back-transform in order to express our predictions as probabilities.

Here are some facts to keep in mind:

- Probabilities of events (e.g. p) must lie between 0 and 1, with $\frac{1}{2}$ as a neutral value for which both outcomes are equally likely. The constraints at 0 and 1 make it impossible to construct a linear equation for predicting probabilities.
- Odds, on the other hand, lie between 0 and $+\infty$, with 1 as a neutral value for which both outcomes are equally likely.
- Odds are asymmetric: when the roles of the two outcomes are switched, each value in the range 0 to 1 is transformed by taking its inverse ($1/\text{value}$) to a value in the range 1 to $+\infty$. For example, if the odds of having a low birthweight baby are $1/4$, then the odds of not having a low birth weight baby is $4/1$.

- Log odds are symmetric. They lie in the range $-\infty$ to $+\infty$. The value for which both outcomes are equally likely is 0. When the roles of the two outcomes are switched, the log odds are multiplied by -1 , since $\log(a/b) = -\log(b/a)$. For example, if the log odds of having a low birth weight baby are -1.39 , the odds of not having a low birth weight baby are 1.39 .

Those new to log odds can take comfort in knowing that as the probability of something increases, the odds and log odds increase too. Talking about the behavior of the log odds an event is qualitatively the same thing as talking about the behavior of the probability of the event. Because log odds take on any value between $-\infty$ to $+\infty$, the β -coefficients from a logistic regression equation can be interpreted in the usual way, namely, they represent the change in log odds of the response per unit change in the predictor.

Example. Suppose we fit a logistic regression model to a sample of postmenopausal women, where the response variable Y equals 1 if a subject is osteoporotic and 0 otherwise. The predictor variable of interest is age.

Let p = the probability that a postmenopausal woman is osteoporotic. Suppose our fitted model is

$$\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = -4353 + 0.038(\text{age})$$

Since the coefficient for age is positive, the log odds (and, therefore, the probability) of osteoporosis increases with age. Taking anti-logarithms of both sides gives

$$\frac{\hat{p}}{1-\hat{p}} = e^{-4353+0.038(\text{age})}$$

With a little algebraic manipulation, one can express the fitted model in terms of the predicted probability of osteoporosis $\hat{\pi}$

$$\hat{p} = \frac{e^{-4353+0.038(\text{age})}}{1 + e^{-4353+0.038(\text{age})}}$$

We'll see what such an equation looks like plotted with a later example.

12.2 Fitting, Interpreting and assessing a logistic model

If b_1 is the “slope” estimate for age in the logistic regression example above, then e_1^b is the odds ratio corresponding to a one unit increase in age. For example, if age equals some amount w , then the predicted odds of having osteoporosis at age w is given by

$$e^{-4.53+0.038w}$$

while one year later (i.e. at age $w + 1$) the predicted odds are given by

$$e^{-4.53+0.038(w+1)}$$

Dividing one equation by the other yields something interesting:

$$\frac{\text{odds of osteoporosis at age } = w + 1}{\text{odds of osteoporosis at age } = w} = \frac{e^{-4.53+0.038(w+1)}}{e^{-4.53+0.038(w)}} = e^{0.038} = 1.0387$$

Thus, the odds that an older female has osteoporosis are 1.0387 times the odds of an female one year younger. This may alternatively be stated to mean that the odds of osteoporosis increases 3.87% over that of a younger individual with each year of age. Note that this percentage change in the odds is $e^b - 1$. For a 10 year age difference, the increase is $e^{b \times 10} = e^{(0.038 \times 10)} = 1.0387^{10} = 1.46$, or a 46% increase.

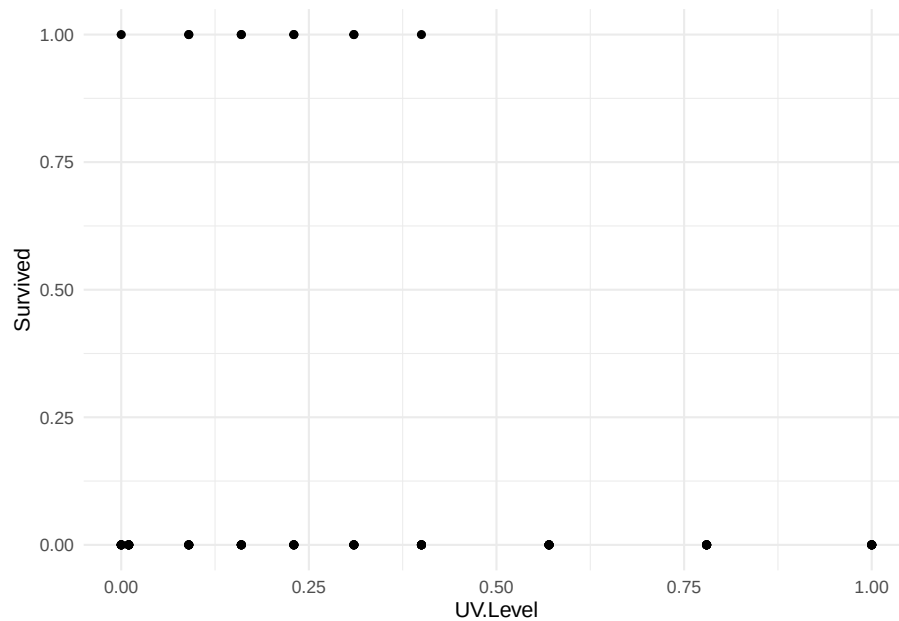
Example: Tadpole exposure to UV light. An experiment was performed that involved the exposure of 80 tadpoles to different level of ultraviolet light. There were 10 levels of UV light exposure (expressed as percentage of full exposure): 0% (dark), 1%, 9%, 16%, 23%, 31%, 40%, 57%, 78%, and 100%. The survival status of each tadpole is recorded after 21 days (variable Survival: 0=died, 1=survived). The researcher is most interested in looking at how well UV exposure predicts mortality. (The data are courtesy of Claire Meikle, and appear in the R workspace tadpoleUV.RData.)

```
load("tadpoleUV.RData")
head(tadUV)
```

##	Day.Exposed	UV.Level	Days.survived	Survived
## 1	2	0.01	7	0
## 2	2	1.00	1	0
## 3	2	0.23	2	0
## 4	1	0.00	9	0
## 5	1	0.16	19	0
## 6	1	0.16	5	0

First, we must recognize that the response variable, **Survived**, is a binary variable. The predictor of survival in the model is **UV.Level**. A plot of the data is below, and might look a little strange compared to data plots we've seen to date.

```
ggplot(tadUV) +
  geom_point(aes(x=UV.Level, y=Survived)) +
  theme_minimal()
```



One thing we can see is that there is a higher prevalence of mortality under higher UV conditions. So there may be an association between survival and exposure.

We model $p = P(\text{survival})$ as a function of UV level as follows:

```
tadpole.fit <- glm(Survived ~ UV.Level, data=tadUV, family=binomial)
summary(tadpole.fit)
```

```
##
## Call:
## glm(formula = Survived ~ UV.Level, family = binomial, data = tadUV)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.300     0.464  -2.80   0.0051 **
## UV.Level      -2.330     1.471  -1.58   0.1131
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 60.283  on 79  degrees of freedom
## Residual deviance: 57.022  on 78  degrees of freedom
## AIC: 61.02
##
```

```
## Number of Fisher Scoring iterations: 5
```

The fitted logistic regression model is $\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = -1.300 - 0.0233(\text{UV level})$.

Here is some model info and interpretation:

- In the summary table, the column Pr(>|z|) contains the p-values for individual tests of model parameters. The row labeled `UV.Level` is a test of $H_0 : \beta_1 = 0$, i.e. whether UV level is a significant predictor of the probability of survival. Since the p -value is large (0.113) we conclude that there is no significant relationship between the probability of survival and UV level. This is known as the *Wald* test for parameter β_j .
- Even though there is not a significant relationship, we can still quantify this relationship by exponentiating $b_1 = -0.0233$

```
exp(coef(tadpole.fit)[2])
```

```
## UV.Level
## 0.0972602
```

Recall that the percentage change in the odds for each one unit increase in the predictor can be found via $e^b - 1$. Thus, percentage change in the odds of survival for each 1% increase in UV is $0.9769658 - 1 = -0.023$; i.e., a 2.3% decrease in the survival odds per 1% increase in UV exposure.

- We can find Wald based confidence intervals for the percentage change in the odds by using the R function `confint()`

```
exp(confint(tadpole.fit)) - 1
```

```
## Waiting for profiling to be done...
```

```
##           2.5 %    97.5 %
## (Intercept) -0.897605 -0.351891
## UV.Level    -0.996857  0.192072
```

Note that the CI for the percentage change in survival odds for a 1% increase in UV level contains 0, i.e. there is no significant effect on survival odds.

There is no whole-model F -test in logistic regression, but we can construct a whole-model test called a likelihood ratio chi-square test using the `anova()` model comparison function:

```
null.model <- glm(Survived ~ 1, data=tadUV, family=binomial)
anova(null.model, tadpole.fit, test="Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: Survived ~ 1
## Model 2: Survived ~ UV.Level
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1         79        60.28
```

```
## 2          78          57.02  1      3.262   0.0709 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The p -value is large (0.07092), indicating that the model fits are not significantly different; i.e., this result says that the model using UV level as a predictor is not significantly better than a model containing no predictors at all. This corroborates the earlier test result for β_1 .

Probability prediction. Since odds ratios can be hard to interpret, we can use predicted probabilities to interpret results. For example, here is how to have R obtain the predicted probability of tadpole survival at 40% UV exposure and 75% UV exposure:

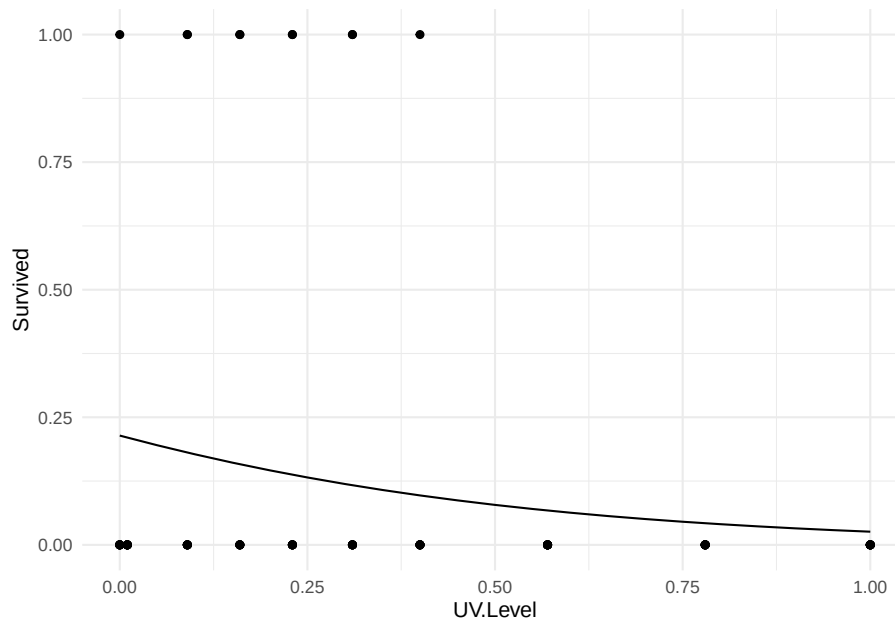
```
predict(tadpole.fit, newdata=data.frame(UV.Level=0.40), type="response")
```

```
##          1
## 0.0968759
predict(tadpole.fit, newdata=data.frame(UV.Level=0.75), type="response")
```

```
##          1
## 0.0453014
```

Making a 2D plot. We can coerce R into making a plot of the fitted logistic regression model. Here is code to do it in the present example.

```
fake.tadpoles <- data.frame(UV.Level = seq(0, 1, 0.05) )
fake.tadpoles <- fake.tadpoles %>%
  mutate(Fitted = predict(tadpole.fit, newdata=fake.tadpoles, type="response"))
ggplot(tadUV) +
  geom_point(aes(x=UV.Level, y=Survived) ) +
  geom_line(data=fake.tadpoles, aes(x=UV.Level, y=Fitted) ) +
  theme_minimal()
```



The notions of logistic regression extend into multiple predictors, ANCOVA settings, variable selection, etc. We will see some of these applications in the below case study.

12.3 Case Study - Titanic Dataset

We are now going to use logistic regression for the Titanic dataset. Recall the basic format of the dataset

```
tail(titanic)
```

```
##           X Class  Age Gender Survived
## 2196 2196  Crew Adult Female      Yes
## 2197 2197  Crew Adult Female      Yes
## 2198 2198  Crew Adult Female      Yes
## 2199 2199  Crew Adult Female       No
## 2200 2200  Crew Adult Female       No
## 2201 2201  Crew Adult Female       No
```

By default, if we performed a logistic regression with the `Survived` variable as the response, R would consider “No” to be a success (1) and “Yes” to be a failure (0) due to alphabetical ordering. We typically think of problems like this as a success being surviving! So we will create a new variable correspond to how we think of the problem.

```
titanic <- titanic %>%
  mutate(Survive = ifelse(Survived=="Yes", 1, 0) )
```

Now we will perform a logistic regression on the new variable `Survive` with `Class`, `Age` and `Gender` all being predictor variables. We note that all three predictor variables are factors.

```
titanic.fit1 <- glm(Survive ~ Class + Gender + Age, data=titanic, family="binomial")
summary(titanic.fit1)
```

```
##
## Call:
## glm(formula = Survive ~ Class + Gender + Age, family = "binomial",
##      data = titanic)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    2.044      0.168   12.17 < 2e-16 ***
## Class2         -1.018      0.196   -5.19 2.1e-07 ***
## Class3         -1.778      0.172  -10.36 < 2e-16 ***
## ClassCrew      -0.858      0.157   -5.45 5.0e-08 ***
## GenderMale     -2.420      0.140  -17.24 < 2e-16 ***
## AgeChild       1.062      0.244    4.35 1.4e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2769.5  on 2200  degrees of freedom
## Residual deviance: 2210.1  on 2195  degrees of freedom
## AIC: 2222
##
## Number of Fisher Scoring iterations: 4
```

Not surprising given the contingency tables in the previous chapter, every predictor variable is significant. We need to consider the coefficients carefully.

- **(Intercept)** - This corresponds to a first class adult female. We see the $b_0 = 2.044$ which corresponds to a probability of surviving as $e^{2.044}/(e^{2.044} + 1) = 0.88534$, so roughly a 90% probability of surviving (recall from the previous chapter that 140 out of 144 first class adult female passengers survived, the difference has to do with the model structure compared to just calculated an observed relative frequency).
- **GenderMale** - This is the influence of being a male compared to a first class adult women. So for a first class adult male, the probability of survival is $e^{2.044-2.420}/(e^{2.044-2.420} + 1) = 0.407092$ and recall that 57 of the 175 first class adult males survived. Again, the difference between the observed

relative frequency and the result of the logistic regression is the model structure (compared to just reporting a number).

- Other variable can be interpreted in a similar fashion.

In the previous section we saw that some of these predictor variable may interact. That is, there seemed to be connection between some of the predictor variables. However, we also saw that some of the interactions resulted in a large number of 0 occurrences. As such, here we consider a model with Gender and Class interacting (being part of the Crew or Third class and male was not good for your chances of surviving). So now we will consider a model with some interaction.

```
titanic.fit2 <- glm(Survive ~ Class * Gender + Age, data=titanic, family="binomial")
summary(titanic.fit2)
```

```
##
## Call:
## glm(formula = Survive ~ Class * Gender + Age, family = "binomial",
##      data = titanic)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      3.558      0.507   7.02 2.3e-12 ***
## Class2          -1.681      0.588  -2.86  0.0042 **
## Class3          -3.885      0.529  -7.35 2.0e-13 ***
## ClassCrew       -1.661      0.800  -2.08  0.0380 *
## GenderMale      -4.233      0.531  -7.97 1.6e-15 ***
## AgeChild         1.054      0.230   4.57 4.8e-06 ***
## Class2:GenderMale  0.448      0.646   0.69  0.4877
## Class3:GenderMale  2.863      0.563   5.08 3.7e-07 ***
## ClassCrew:GenderMale 1.086      0.820   1.33  0.1852
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2769.5  on 2200  degrees of freedom
## Residual deviance: 2143.4  on 2192  degrees of freedom
## AIC: 2161
##
## Number of Fisher Scoring iterations: 6
```

Before proceeding to interpret this model, let's first ask if it improves over the previous model. You'll note that `titanic.fit1` is a simpler version of `titanic.fit2`. We use the `anova()` function to test if adding the interaction terms significantly improves the model.

```
anova(titanic.fit1, titanic.fit2, test="Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: Survive ~ Class + Gender + Age
## Model 2: Survive ~ Class * Gender + Age
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1      2195      2210
## 2      2192      2143   3    66.67 2.21e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

We see overwhelming support that the interactive model improves over the main effects model (p -value near zero). Interpreting this model is complicated:

- **(Intercept)** - This corresponds to a first class adult female, and the probability of surviving is $e^{3.558}/(e^{3.558} + 1) = 0.972294$ which is much closer to the observed relative frequency of $140/144 = 0.972222$.
- **AgeChild** - This is the impact of being a child compared to a first class adult female. So $e^{3.558+1.054}/(e^{3.558+1.054} + 1) = 0.991066$ is the probability of a first class female child surviving (there was only 1 on the titanic, and they survived).
- **Class3:GenderMale** - Interpreting the interaction terms is wonky and here is why. First consider the full model

$$\text{logit}(p) = \beta_0 + \beta_1(\text{Class2}) + \beta_3(\text{Class3}) + \beta_4(\text{Crew}) + \beta_5(\text{Male}) + \beta_6(\text{Child}) + \beta_7(\text{Class2Male}) + \beta_8(C$$

For the **Class3:Male** term to be in effect both **Class2** and **GenderMale** will be activated (remember everything is a 1 or 0 dummy variable). So interpretation of this interaction term actually involves many terms. The probability of a second class adult male surviving is

$$\frac{e^{3.558-3.885-4.233+2.863}}{1 + e^{3.558-3.885-4.233+2.863}} = 0.154857$$

which is relatively close to the $75/(75 + 387) = 0.162338$ that did survive.

Overall we see this more complex model appears to more closely match the observed relative frequencies from the previous chapter. This is not too surprising given the measure of **Residual deviance**. Before explaining, let's look at both fitted models one more time

```
summary(titanic.fit1)
```

```
##
## Call:
## glm(formula = Survive ~ Class + Gender + Age, family = "binomial",
##      data = titanic)
```



```
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)   2.044     0.168   12.17 < 2e-16 ***
## Class2       -1.018     0.196   -5.19 2.1e-07 ***
## Class3       -1.778     0.172  -10.36 < 2e-16 ***
## ClassCrew    -0.858     0.157   -5.45 5.0e-08 ***
## GenderMale   -2.420     0.140  -17.24 < 2e-16 ***
## AgeChild      1.062     0.244    4.35 1.4e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2769.5  on 2200  degrees of freedom
## Residual deviance: 2210.1  on 2195  degrees of freedom
## AIC: 2222
##
## Number of Fisher Scoring iterations: 4
summary(titanic.fit2)

##
## Call:
## glm(formula = Survive ~ Class * Gender + Age, family = "binomial",
##      data = titanic)
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)     3.558     0.507    7.02 2.3e-12 ***
## Class2          -1.681     0.588   -2.86  0.0042 **
## Class3          -3.885     0.529   -7.35 2.0e-13 ***
## ClassCrew       -1.661     0.800   -2.08  0.0380 *
## GenderMale      -4.233     0.531   -7.97 1.6e-15 ***
## AgeChild         1.054     0.230    4.57 4.8e-06 ***
## Class2:GenderMale  0.448     0.646    0.69  0.4877
## Class3:GenderMale  2.863     0.563    5.08 3.7e-07 ***
## ClassCrew:GenderMale 1.086     0.820    1.33  0.1852
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2769.5  on 2200  degrees of freedom
## Residual deviance: 2143.4  on 2192  degrees of freedom
## AIC: 2161
```

```
##
```

```
## Number of Fisher Scoring iterations: 6
```

Both model report a **Null deviance: 2769.5** which can be loosely interpreted as the “error” in the most basic model, **Survive ~ 1** (an intercept only model). The **Residual deviance** corresponds to the error of the fitted models. You’ll note that **titanic.fit1** has a deviance of 2210.1 while **titanic.fit2** has a smaller deviance of 2143.4, this we can argue is a better fit. The **anova()** function output tells us it is a significantly better fit.

Chapter 13

Generalized Linear Models

In the previous chapter we introduced the idea of a **Generalized Linear Model**, where we fit a linear regression model to a response variable via a link function. Logistic regression highlights the case of a binary outcome (TRUE/FALSE, Success/Failure, Alive/Dead, 1/0). The ideas expressed in that chapter can be extended to a multitude of situations. This chapter offers an overview of the topic concentrating on *count* responses.

Example: Prevalence of salamanders. An observational field study is conducted in which the number of salamanders in different locations is counted. For this dataset, salamander count (the response variable) takes on non-negative integer values. This response is to be modeled as a function of two quantitative predictor variables: forest age and percentage of canopy cover.

What makes this problem different from problems we saw in the previous chapters? Since the response variable Y is a count (integers 0, 1, 2, ...), it is discrete and bounded below by 0. A common characteristic of counts is that the variability in the count increases with the mean. For example:

- If we had sampled sites with field conditions that produced low salamander counts, the counts would tend to be very similar because of the physical count boundary of 0.
- Conversely, sampled sites with field conditions that produce higher salamander counts would tend to be less similar (i.e. more variable).

Count variables by their very nature violate the usual constant variance assumption from regression. Furthermore, if the response Y is a count, then it must be greater than or equal to 0 to be valid. A standard regression method may not ensure this will happen either. Using an appropriate link function will ensure valid predictions.

For count data such as this, it is often more appropriate to assume the response variable Y follows a Poisson or Negative Binomial distribution, as compared to

a Normal distribution utilized in standard regression.

13.1 Poisson Regression

Before getting into the details of Poisson regression we will briefly introduce the Poisson distribution. This is a common distribution that is covered in many Introductory Statistics books (although the topic tends to be omitted from many Intro Stat classes).

13.1.1 Poisson distribution

Suppose you had a binomial random variable X . A common example is a coin flip where you observed the number of heads (X) out of n trials. Here we would say

$$X \sim \text{Binomial}(n, p = 0.5)$$

where $p = P(\text{Heads})$ on any given coin flip.

Now, suppose that the number of trials n is very large (say, $n \rightarrow \infty$) and the probability of success is relatively small such that as n grows large, $np \rightarrow c$, where c is some constant. In situations such as this, the observed value of X can be considered a count, 0, 1, 2, 3, ... and is unbounded as n grows.

Mathematics can show that in this situation X follows the following distribution

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

which is known as the Poisson distribution with rate parameter λ . The rate parameter corresponds to $\lambda = np$ in the aforementioned binomial situation and it can be shown that the theoretical mean of a Poisson random variable X is $E[X] = \lambda$. Not only that, but $\mu = E[X] = \lambda = \text{Var}(X)$. That is, for a Poisson random variable, its mean and its variance are the same - this fact is important later.

Simplified. A Poisson distribution is appropriate in some applications when the variable of interest is a count. Examples may include the number of customers who enter a bank in a given hour, the number of earthquakes observed along a fault line, and as we'll see later, the number of tropical cyclones that develop in the Atlantic Ocean.

13.1.2 Poisson Regression Development

The basic construct of the Poisson Regression model is to use a log link function. Namely, we are interested in modeling $E[Y] = \mu = \lambda$ and that is achieved via

$$\log(\mu) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k$$

Where β_j is the coefficient on the j^{th} predictor variable X_j . It is easy to “un-transform” this model with an exponent. Namely

$$E[Y] = \mu = \lambda = e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k} \quad (13.1)$$

Important Notes. We have already established that for Poisson data $\lambda = E[Y] = \text{Var}(Y)$; i.e., the mean and the variance are equal. In many applications it may be the case that the observed variance is greater than the mean; this phenomenon is known as *overdispersion* and can be an indication a Poisson regression is not appropriate (more on that later).

13.1.3 Example - Tropical Cyclone Counts in the North Atlantic

A tropical cyclone is a rapidly rotating storm system characterized by a low-pressure center, a closed low-level atmospheric circulation, strong winds, and a spiral arrangement of thunderstorms that produce heavy rain. Depending on its location and strength, a tropical cyclone is referred to by different names, including hurricane, typhoon, tropical storm, cyclonic storm, tropical depression, and simply cyclone. A hurricane or tropical storm is a tropical cyclone that occurs in the Atlantic Ocean and northeastern Pacific Ocean, and a typhoon occurs in the northwestern Pacific Ocean; while in the south Pacific or Indian Ocean, comparable storms are referred to simply as “tropical cyclones” or “severe cyclonic storms”. We will consider the tropical cyclone record for the North Atlantic Basin.

```
atlantic <- read.csv("atlanticCycloneRecord.csv")
head(atlantic)
```

```
##   Year TS H1 H2 H3 H4 H5
## 1 1851  3  2  0  1  0  0
## 2 1852  0  3  1  1  0  0
## 3 1853  4  1  1  1  1  0
## 4 1854  2  1  1  1  0  0
## 5 1855  1  1  2  1  0  0
## 6 1856  2  2  0  1  1  0
```

```
tail(atlantic)
```

```
##   Year TS H1 H2 H3 H4 H5
## 162 2012  9  5  3  2  0  0
## 163 2013 12  2  0  0  0  0
## 164 2014  2  3  1  1  1  0
## 165 2015  7  2  0  1  1  0
## 166 2016  8  3  0  2  1  1
## 167 2017  7  2  2  2  2  2
```

Consider 2017, news of hurricanes could not be missed (Harvey in Houston, Irma in Florida and Maria in Puerto Rico). In terms of impact on the United States, 2017 seemed extreme. But in a broader sense, one may ask, was 2017 that abnormal?

The answer to this question is a complex problem that cannot but tackled in a single textbook example but we will look at the simple case of modeling the number of storms (we are not concentrating on storm intensity or accumulated cyclone energy) and determined if 2017 seemed abnormal. In particular, we will consider the Southern Oscillation Index (SOI) as a predictor variable on tropical cyclone counts. The SOI is a standardized index based on the observed sea level pressure differences between Tahiti and Darwin, Australia. The SOI is one measure of the large-scale fluctuations in air pressure occurring between the western and eastern tropical Pacific (i.e., the state of the Southern Oscillation) during El Niño and La Niña episodes. In general, smoothed time series of the SOI correspond very well with changes in ocean temperatures across the eastern tropical Pacific. The negative phase of the SOI represents below-normal air pressure at Tahiti and above-normal air pressure at Darwin. Prolonged periods of negative (positive) SOI values coincide with abnormally warm (cold) ocean waters across the eastern tropical Pacific typical of El Niño (La Niña) episodes.

We will attempt to tackle the question of whether the SOI predict cyclone counts.

We begin by creating a total storm count variable

```
atlantic <- atlantic %>%
  mutate(Storms=TS+H1+H2+H3+H4+H5)
tail(atlantic)
```

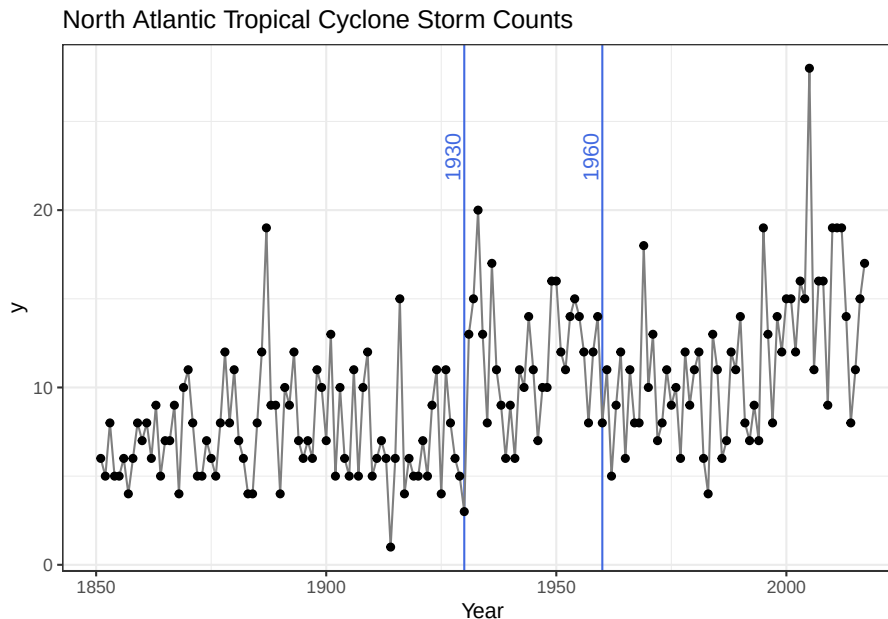
##	Year	TS	H1	H2	H3	H4	H5	Storms
## 162	2012	9	5	3	2	0	0	19
## 163	2013	12	2	0	0	0	0	14
## 164	2014	2	3	1	1	1	0	8
## 165	2015	7	2	0	1	1	0	11
## 166	2016	8	3	0	2	1	1	15
## 167	2017	7	2	2	2	2	2	17

The variable `Storms` is the total number of tropical storms and hurricanes in a given year. Let's plot the storm counts in time to look at the historic record.

```
df.annotate <- data.frame(Year=c(1930,1960),
  y=rep(23,2),
  labs=c("1930", "1960"))

ggplot(atlantic) +
  geom_vline(xintercept=c(1930,1960), col="royalblue") +
  geom_text(data=df.annotate, aes(x=Year, y=y, label=labs), angle=90, nudge_x=-2.5, col="black") +
  geom_line(aes(x=Year, y=Storms), col="gray50") +
  geom_point(aes(x=Year, y=Storms)) +
  ggtitle("North Atlantic Tropical Cyclone Storm Counts") +
```

```
theme_bw()
```



In our plot we have the number of tropical cyclones per year with 1930 and 1960 highlighted for the following reasons: Overall we see some distinct change in the record (highlighted by blue vertical lines).

- Circa 1930 - Aircraft reconnaissance! Before this we only knew of a storm if it made land fall or a ship saw it.
- Circa 1960 - Satellites! Since the early 1960s we have eyes on the ocean all the time. Counts after 1960 are considered much more reliable.

Thus we remove all data before 1961 and calculate some summary statistics

```
atlantic <- atlantic %>%
  filter(Year>1960)
atlantic %>% summarize(Avg.Storm = mean(Storms),
                      Var.Storm = var(Storms))
```

```
##   Avg.Storm Var.Storm
## 1    11.614   19.9555
```

We note that on average there are 11.6 storms per year with a variance of near 20. We have some indication that maybe our response variable suffers from overdispersion, we will revisit that a little later.

Now we load in the SOI data.

```
soi <- read.table("soi.txt", header=TRUE)
head(soi)
```

```
##   YEAR  JAN  FEB  MAR  APR  MAY  JUN  JUL  AUG  SEP  OCT  NOV  DEC
## 1 1951  1.5  0.9 -0.1 -0.3 -0.7  0.2 -1.0 -0.2 -1.1 -1.0 -0.8 -0.7
## 2 1952 -0.9 -0.6  0.5 -0.2  0.8  0.7  0.5  0.1 -0.2  0.4  0.0 -1.2
## 3 1953  0.3 -0.5 -0.2  0.2 -1.7  0.1  0.0 -1.2 -1.2  0.1 -0.3 -0.5
## 4 1954  0.7 -0.3  0.3  0.6  0.5  0.1  0.4  1.1  0.2  0.3  0.1  1.4
## 5 1955 -0.5  1.9  0.6 -0.1  1.0  1.3  1.6  1.5  1.3  1.5  1.2  1.0
## 6 1956  1.3  1.6  1.3  0.9  1.4  1.1  1.1  1.2  0.1  1.8  0.2  1.1
```

Since our storm counts are by year we need a single SOI value per year. The North Atlantic “Hurricane Season” is June through November, so we will consider the mean SOI value in those months. We will create the new variable using material from Chapter 1.

We take the SOI data and `group_by` YEAR. Then within a given year we calculate the mean SOI level for June, July, August, September, October and November. Lastly we select the variables of interest for our analysis and rename YEAR to Year (thus matching the cyclone counts).

```
soi <- soi %>%
  group_by(YEAR) %>%
  mutate(SOI = mean(c(JUN, JUL, AUG, SEP, OCT, NOV))) %>%
  dplyr::select(YEAR, SOI) %>%
  rename(Year=YEAR)
tail(soi)
```

```
## # A tibble: 6 x 2
## # Groups:   Year [6]
##   Year    SOI
##   <int> <dbl>
## 1  2012  0.0333
## 2  2013  0.517
## 3  2014 -0.483
## 4  2015 -1.15
## 5  2016  0.417
## 6  2017  0.55
```

Now we need to combine our Atlantic Hurricane data with the SOI measures. Essentially we want to *join* or *merge* the two datasets linking observations by Year. For those who know SQL, this is a common operation known as a *join*. R includes several functions to do this. Today we will use an `inner_join` so only rows with a common year between the two datasets will be included.

First we select the necessary variables from the `atlantic` dataset and then `inner_join` it with the SOI.


```
atlantic <- atlantic %>%
  dplyr::select(Year, Storms)
combine <- inner_join(atlantic, soi, by="Year")
head(combine)
```

```
##   Year Storms      SOI
## 1 1961      11 0.133333
## 2 1962       5 0.516667
## 3 1963       9 -0.533333
## 4 1964      12 0.950000
## 5 1965       6 -1.133333
## 6 1966      11 0.116667
```

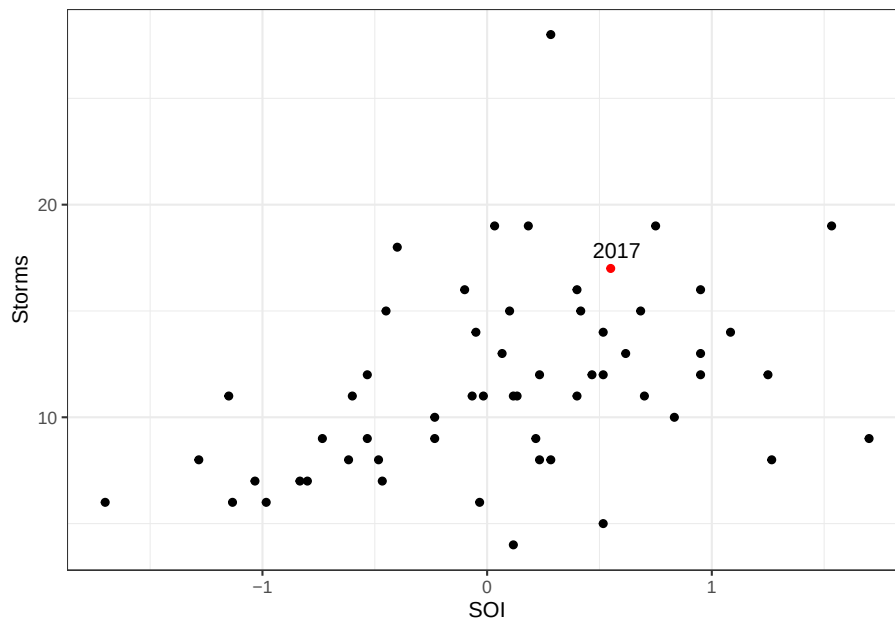
```
tail(combine)
```

```
##   Year Storms      SOI
## 52 2012      19 0.0333333
## 53 2013      14 0.5166667
## 54 2014       8 -0.4833333
## 55 2015      11 -1.1500000
## 56 2016      15 0.4166667
## 57 2017      17 0.5500000
```

Let's plot our response variable `Storms` against our predictor variable `Yearly.SOI`. Here we use some extra features in `ggplot` to highlight 2017

First we create a variable called `Highlight` if the year is 2017. Then we save another dataset with just that observation so we can annotate the point with text. We use the aesthetic option `color` to highlight 2017 as a different color.

```
combine <- combine %>%
  mutate(Highlight = ifelse(Year==2017, "highlight", "normal"))
recent.2017 <- combine %>%
  filter(Year==2017)
ggplot(combine) +
  geom_point(aes(x=SOI, y=Storms, color=Highlight)) +
  scale_color_manual("Status", values=c("highlight"="red", "normal"="black")) +
  geom_text(data=recent.2017, aes(x=SOI*1.05, y=Storms*1.05, label="2017")) +
  theme_bw() +
  theme(legend.position = "none")
```



Visually we see that 2017 does not appear to be all that different compared to the other observations (think back to the ideas of leverage and influence – 2017 is in the middle of the pattern, not an outlier in any real way). We also see that the **Storms** appear to increase with a positive SOI value. Let's test this with a poisson model; we use the `glm` function as before with logistic regression, but here we specify the family to be `poisson`.

```
storm.fit <- glm(Storms ~ SOI, data=combine, family=poisson)
summary(storm.fit)
```

```
##
## Call:
## glm(formula = Storms ~ SOI, family = poisson, data = combine)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  2.4247    0.0402   60.37 < 2e-16 ***
## SOI          0.2050    0.0541    3.79  0.00015 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 92.187  on 56  degrees of freedom
## Residual deviance: 77.728  on 55  degrees of freedom
## AIC: 323.1
```

```
##
## Number of Fisher Scoring iterations: 4
```

To interpret the model I need to recognize that our model is

$$\log(\lambda) = 2.42473 + 0.20502 \times SOI$$

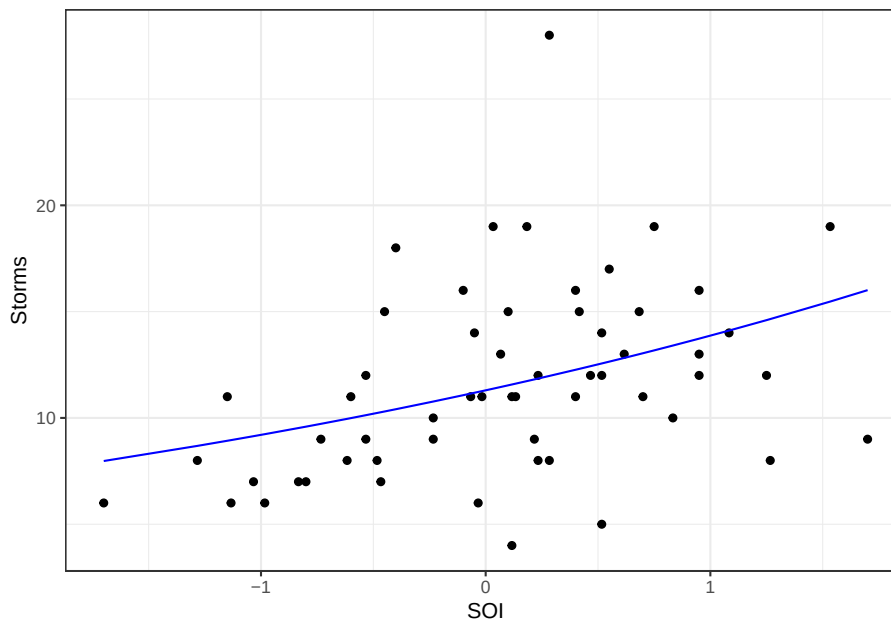
So we must “un”-log.

The intercept of 2.42473 corresponds to $\exp(2.42473) = 11.299178$ storms when the SOI value is 0.

The slope term must be interpretation on a log scale. That is, $2.42473 + 0.20502 \times (\text{one unit of SOI})$ is a mean number of storm of 13.870302.

We also note that the covariate predictor variable SOI is significant according to the Wald test ($p\text{-value}=0.000152$). So we have evidence that SOI does in fact explain storm counts. Let’s look at the predictive ability of the model. First we will plot the fitted model against the data.

```
combine <- combine %>%
  mutate(Fitted = predict(storm.fit, type="response"))
ggplot(combine) +
  geom_point(aes(x=SOI, y=Storms)) +
  geom_line(aes(x=SOI, y=Fitted), col="blue") +
  theme_bw()
```



Note the slight curvature in the fitted “line” (recall the model is $e^{\beta_0 + \beta_1 X_1}$). Using the model what do we predict of 2017?

```
predict(storm.fit, newdata=recent.2017, type="response")
```

```
##          1
## 12.6479
```

Compared to the observed value of 17 we see that 2017 does appear to deviate from the predicted value. We may want to consider building an interval around this predicted value. To do this, we can tell the `predict` function to calculate a standard error for the predicted value:

```
predict(storm.fit, newdata=recent.2017, type="response", se.fit=TRUE)
```

```
## $fit
##          1
## 12.6479
##
## $se.fit
##          1
## 0.550723
##
## $residual.scale
## [1] 1
```

Now, using *empirical rule* type arguments from your Intro Statistics class (± 2 standard errors is approximately 95%), we see that the predicted number of storms is between 11.546433 and 13.749327, so 2017 is still outside the predicted value.

Note: We want to remind the reader that we are performing a *simple* regression model here. Modeling complicated systems such as tropical cyclones is not a trivial exercise and would require greater expertise and (likely) more predictor variables than included here. However, we include this example as it is a practical example of Poisson regression, is scientifically interesting and relevant to modern society.

13.1.3.1 Overdispersion

Let’s revisit the issue of overdispersion. Early we saw some indication of potential overdispersion in our data

```
atlantic %>% summarize(Avg.Storm = mean(Storms),
                      Var.Storm = var(Storms))
```

```
## Avg.Storm Var.Storm
## 1      11.614    19.9555
```

Here the variance appears to be nearly twice as big as the mean. However we

must step back and realize this is BEFORE we fit our model. It is possible the covariate information (that is, the SOI variable) will explain some of this variability. A quick check for overdispersion in Poisson models is to compare the **Residual deviance** to its degrees of freedom. In the ideal situation, the ratio should be 1. In cases when it is much greater than 1 (typically a value of 2 or more is considered a problem), we have indication of overdispersion. Here we have the following output

```
Residual deviance: 77.728 on 55 degrees of freedom
```

and note $77.728/55 = 1.413236$ which is not that much greater than 1. In fact, a formal hypothesis test confirms we do not have overdispersion, but we exclude formal hypothesis testing of overdispersion from this introductory text.

13.2 Handling overdispersion

Although Poisson regression can be appropriate for *count* data, there are many cases when the highly restrictive property $\lambda = E[Y] = Var(Y)$ is violated. In such situations, a different modeling approach is necessary. Two common approaches are to use a “Quasi-Poisson” model or a Negative Binomial Regression.

In the former, a Poisson model is fit with an added variable such that $Var(Y) = c\lambda$. That is, we let the variance be a scalar multiple of the mean. This value c is known as the *dispersion* parameter (in regular Poisson regression, $c = 1$). This slight modification adds flexibility to Poisson regression model. To fit this sort of model in R, we simply state `family=quasipoisson` in the `glm` function. One thing to note is that the mean response (i.e., the fitted model) is the same for Poisson and Quasi-Poisson models. The only difference is in the calculation of standard errors and prediction intervals.

Another approach to handle overdispersion is known as Negative Binomial regression. It is based on properties of the Negative Binomial distribution that is typically covered in a formal Probability course. However, here we do not need all those details for our discussion. The key element for our understanding is that the Negative Binomial distribution allows for a second parameter, and thus loosens the restriction that $E[Y] = Var(Y)$. Unlike Quasi-Poisson, it does not use a linear relationship between mean and variance, but rather a quadratic relationship. Formulaically the negative binomial model is the same equation as (??) except an additional *dispersion* parameter is estimated. We walk through an example.

13.2.1 Example – Attendance Records

School administrators study the attendance behavior of high school juniors at two schools. Predictors of the number of days of absence include the type of

program in which the student is enrolled and a standardized test in math. Attendance data on 314 high school juniors from two urban high schools in the file `attendance.RData`. The response variable of interest is days absent, `daysabs`. The variable `math` is the standardized math score for each student. The variable `prog` is a three-level nominal variable indicating the type of instructional program in which the student is enrolled (it is a factor but coded as a numeric).

```
load("attendance.RData")
glimpse(attendance)
```

```
## Rows: 314
## Columns: 5
## $ id      <dbl> 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 10~
## $ gender  <fct> male, male, female, female, female, female, female, male, male~
## $ math    <dbl> 63, 27, 20, 16, 2, 71, 63, 3, 51, 49, 31, 22, 73, 77, 10, 89, ~
## $ daysabs <dbl> 4, 4, 2, 3, 3, 13, 11, 7, 10, 9, 4, 5, 5, 6, 1, 0, 1, 0, 5, 24~
## $ prog    <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 2, 2, 1, 3, 2, 2, 2, 1, 2, 2,~
```

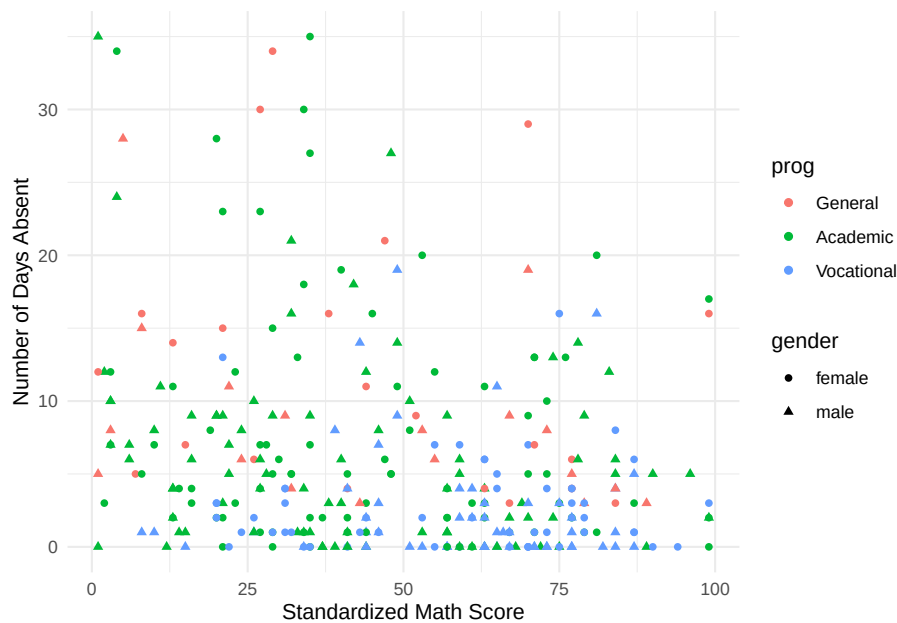
As a first step we want to do a little data cleaning (making sure R properly handles the categorical variables). We add `labels` to the three `prog` levels and make `id` a factor.

```
attendance <- attendance %>%
  mutate(prog = factor(prog, levels = 1:3,
                        labels = c("General", "Academic", "Vocational")),
         id = factor(id) )
glimpse(attendance)
```

```
## Rows: 314
## Columns: 5
## $ id      <fct> 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 10~
## $ gender  <fct> male, male, female, female, female, female, female, male, male~
## $ math    <dbl> 63, 27, 20, 16, 2, 71, 63, 3, 51, 49, 31, 22, 73, 77, 10, 89, ~
## $ daysabs <dbl> 4, 4, 2, 3, 3, 13, 11, 7, 10, 9, 4, 5, 5, 6, 1, 0, 1, 0, 5, 24~
## $ prog    <fct> Academic, Academic, Academic, Academic, Academic, Academic, Ac~
```

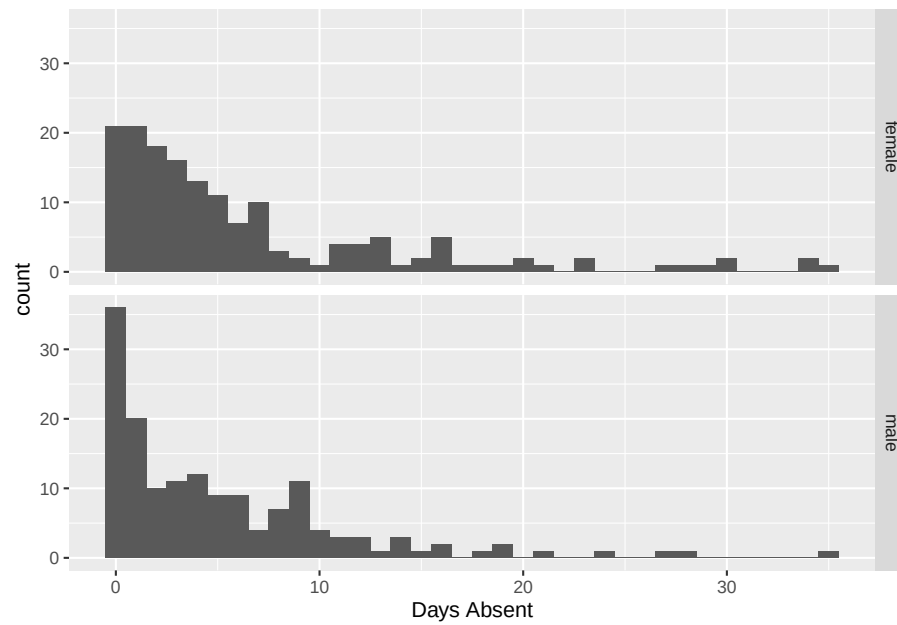
Now we graphically explore the data graphically. We build an advanced scatterplot where the shape of the points is determined by the gender and the color of the points is determined by the program.

```
ggplot(attendance) +
  geom_point(aes(x=math, y=daysabs, color=prog, shape=gender)) +
  theme_minimal() +
  xlab("Standardized Math Score") + ylab("Number of Days Absent")
```



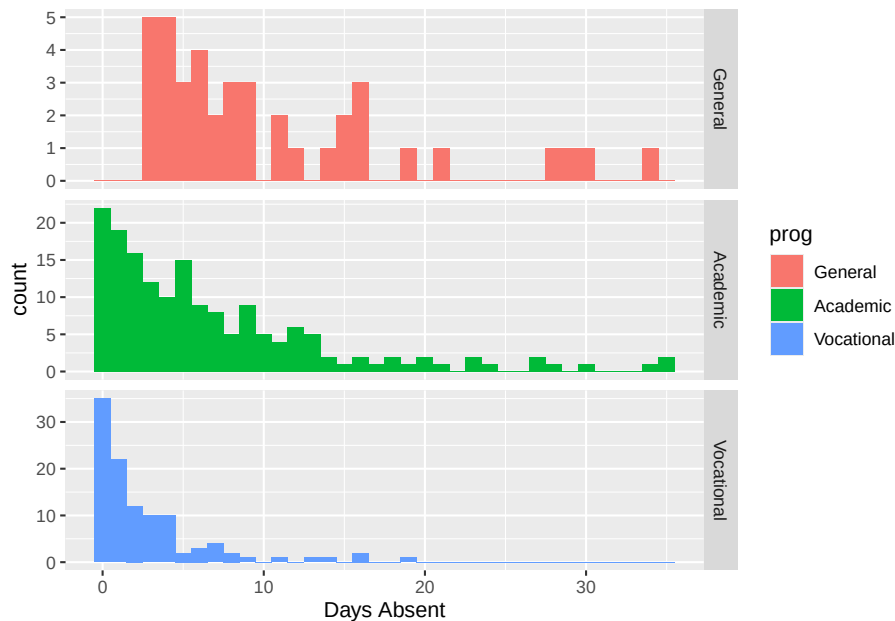
It takes a minute to comprehend everything in this plot. Visually it is difficult to distinguish any differences in gender (shapes of the point). There does appear to be a relationship between attendance and the program (visually speaking “Vocational” and “Academic” appear to be lower on the days missed compared to “Gender”). We take a deeper dive at looking at these two categorical variables by comparing histograms.

```
ggplot(attendance) +
  geom_histogram(aes(x=daysabs), binwidth = 1) +
  facet_grid(gender~.) + xlab("Days Absent")
```



Here we see little difference between the number of days absent and gender (coincides with the scatterplot). Next we explore the relationship with the respective program.

```
ggplot(attendance) +  
  geom_histogram(aes(x=daysabs, fill=prog), binwidth=1) +  
  facet_grid(prog~., scales="free") +  
  xlab("Days Absent")
```

Here we see three distinct shapes in the histograms. For the “General” program it is fairly uniformly distributed (compared to the other two) with all recorded students having several days absent. For “Academic” and “Vocational” we see more a decay type shape (also sometimes known as a Pareto shape). The “Academic” program definitely has more days absent compare to the “Vocational” program.

13.2.2 Incorrect Poisson Model

We have a count response variable and previously we learned about Poisson regression so let’s attempt to fit that model.

```
attend.pois <- glm(daysabs ~ math + gender + prog, data=attendance, family=poisson)
summary(attend.pois)
```

```
##
## Call:
## glm(formula = daysabs ~ math + gender + prog, family = poisson,
##      data = attendance)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   2.759479   0.063773  43.27 < 2e-16 ***
## math         -0.006956   0.000935  -7.44 1.0e-13 ***
## gendermale    -0.242476   0.046777  -5.18 2.2e-07 ***
## progAcademic  -0.426033   0.056731  -7.51 5.9e-14 ***
```

```
## progVocational -1.270720  0.077914  -16.31  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 2217.7  on 313  degrees of freedom
## Residual deviance: 1746.8  on 309  degrees of freedom
## AIC: 2640
##
## Number of Fisher Scoring iterations: 5
```

In the response we see that all the predictor variables are significant (all p -values are less than 10^{-6}) but there is a glaring problem. The **Residual deviance** is recorded as 1746.8 on 309 degrees of freedom. This is a ratio value of 5.653074 which is much greater than 1. There are some major concerns about overdispersion. We could see this visually as well. Revisit the scatter plot of the data, with lower math scores, the range of the days absent looks larger than with higher math scores (an indication the variance changes given the mean response).

So in this case, a Poisson regression model is not appropriate.

13.2.3 A quasi-Poisson approach

We now fit a Quasi-Poisson model to the data.

```
attend.qpois <- glm(daysabs ~ math + gender + prog, data=attendance, family=quasipoisson)
summary(attend.qpois)
```

```
##
## Call:
## glm(formula = daysabs ~ math + gender + prog, family = quasipoisson,
##      data = attendance)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.75948    0.16282   16.95 < 2e-16 ***
## math          -0.00696    0.00239   -2.91  0.0038 **
## gendermale     -0.24248    0.11942   -2.03  0.0432 *
## progAcademic   -0.42603    0.14484   -2.94  0.0035 **
## progVocational -1.27072    0.19892   -6.39 6.2e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 6.51816)
##
##      Null deviance: 2217.7  on 313  degrees of freedom
```

```
## Residual deviance: 1746.8  on 309  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 5
```

Note, the estimated model is the same as the Poisson! Intercept of 2.759479, *math* slope of -0.006956 and so on. However, the standard errors are now different, along with the associated *p*-values. In fact, now we see that the *gendermale* variable is borderline significant with a *p*-value of 0.043.

Unfortunately, assessing the goodness-of-fit for a Quasi-Poisson regression is a difficult task. It is considered more of a “band-aid” type fix to Poisson regression, although there are practical applications where it is better than Negative Binomial regression.

13.2.4 Fitting a Negative Binomial regression

The negative binomial model is sometimes considered to be more of a *direct* model for overdispersed data. That is, the Negative Binomial distribution allows for a larger variance than the mean whereas Quasi-Poisson is a fix to Poisson regression. To fit a negative binomial model we use the `glm.nb()` function in the MASS package. The function largely works the same as `lm` and `glm`. Below we fit the full model

```
library(MASS)
attend.negbin1 <- glm.nb(daysabs ~ math + gender + prog, data=attendance)
summary(attend.negbin1)
```

```
##
## Call:
## glm.nb(formula = daysabs ~ math + gender + prog, data = attendance,
##       init.theta = 1.047288915, link = log)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    2.70748    0.20427   13.25 < 2e-16 ***
## math          -0.00624    0.00249   -2.50  0.012 *
## gendermale     -0.21109    0.12199   -1.73  0.084 .
## progAcademic   -0.42454    0.18173   -2.34  0.019 *
## progVocational -1.25262    0.19970   -6.27 3.6e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(1.0473) family taken to be 1)
##
##      Null deviance: 431.67  on 313  degrees of freedom
## Residual deviance: 358.87  on 309  degrees of freedom
```

```
## AIC: 1740
##
## Number of Fisher Scoring iterations: 1
##
##
##              Theta:  1.047
##              Std. Err.:  0.108
##
##  2 x log-likelihood:  -1728.307
```

In the summary output, we now see that the `gender` variable does not appear significant (this matches our visual plots). Just as important, we see a **Residual deviance** of 358.87 with 309 degrees of freedom, a ratio of 1.161392 much closer to the value of 1 we hope to see. Based on a quick glance this model seems more appropriate. Recapping why

- The model output now more closely matches our exploratory data analysis (gender not important)
- A quick check of the residual deviance suggest a better fit

13.2.5 Picking between Quasi-Poisson and Negative Binomial

There is no general or clear answer on which model is preferred, it all depends on the application. There are datasets that are overdispersed but in such a way that a Quasi-Poisson approach is more appropriate than a negative binomial. However, in other cases, the Negative Binomial is better and more natural. A non-technical comparison between the two approaches (with an application to counting the number of harbor seals) is available here: <https://doi.org/10.1890/07-0043.1>

13.2.6 Inference on predictor variables

Using the negative binomial regression model, we see that the `gender` variable does not appear significant. Let's fit a reduced model:

```
attend.negbin2 <- glm.nb(daysabs ~ math + prog, data=attendance)
summary(attend.negbin2)
```

```
##
## Call:
## glm.nb(formula = daysabs ~ math + prog, data = attendance, init.theta = 1.032713156
##      link = log)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    2.61527    0.19746   13.24 < 2e-16 ***
```

```
## math          -0.00599    0.00251   -2.39    0.017 *
## progAcademic  -0.44076    0.18261   -2.41    0.016 *
## progVocational -1.27865    0.20072   -6.37   1.9e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(1.0327) family taken to be 1)
##
##      Null deviance: 427.54  on 313  degrees of freedom
## Residual deviance: 358.52  on 310  degrees of freedom
## AIC: 1741
##
## Number of Fisher Scoring iterations: 1
##
##
##              Theta:  1.033
##             Std. Err.:  0.106
##
## 2 x log-likelihood:  -1731.258
```

We note that residual deviance has actually decreased in this reduced model! The other variables are still significant. We can formally test for significance with the `anova` function as we have before:

```
anova(attend.negbin1, attend.negbin2, test="Chisq")
```

```
## Likelihood ratio tests of Negative Binomial Models
##
## Response: daysabs
##      Model   theta Resid. df    2 x log-lik.  Test      df LR stat.
## 1      math + prog 1.03271    310      -1731.26
## 2 math + gender + prog 1.04729    309      -1728.31 1 vs 2      1    2.9507
##      Pr(Chi)
## 1
## 2 0.0858406
```

We see that the `math` variable does **not** significantly improve the model (p -value=0.0858). Arguably, its inclusion in the model was making things worse!

13.2.7 Plotting fitted model

Lastly, we will plot the fitted model. Even though the data used to fit the model is technically 5 dimensional (days absent, math score, and 3 factor values), since we only have two numeric terms, we can use aesthetic options in `ggplot` to plot the chosen fitted model. We begin by creating some *fake* data that will results in a smooth fitted line:

```

newdata2 <- data.frame(
  math = rep(seq(from = min(attendance$math),
                     to = max(attendance$math),
                     length.out = 100), 3),
  prog = factor(rep(1:3, each = 100), levels = 1:3, labels =
                 levels(attendance$prog)))
glimpse(newdata2)

## Rows: 300
## Columns: 2
## $ math <dbl> 1.00000, 1.98990, 2.97980, 3.96970, 4.95960, 5.94949, 6.93939, 7.~
## $ prog <fct> General, General, General, General, General, General, General, Ge~

```

You'll note this dataset has 300 observations and the math scores range from 1 to 99 in small increments, repeating for each of the three factor levels in prog. We then use this data in the `predict` function. Here we specify the `type="response"` so that output is in the original units.

```

newdata3 <- predict(attend.negbin2, newdata2, type = "response", se.fit=TRUE)
glimpse(newdata3)

## List of 3
## $ fit : Named num [1:300] 13.6 13.5 13.4 13.3 13.3 ...
## .. attr(*, "names")= chr [1:300] "1" "2" "3" "4" ...
## $ se.fit : Named num [1:300] 2.66 2.63 2.6 2.56 2.53 ...
## .. attr(*, "names")= chr [1:300] "1" "2" "3" "4" ...
## $ residual.scale: num 1

```

For each predicted response value (called `fit`) we have a standard error estimate (called `se.fit`). Using a standard empirical rule Z -based confidence interval from your Intro Statistics course (± 2 standard errors), we can build a confidence band around the fitted line.

```

newdata2 <- newdata2 %>%
  mutate(DaysAbsent.Fitted = newdata3$fit,
         lo = (newdata3$fit - 1.96*newdata3$se.fit),
         up = (newdata3$fit + 1.96*newdata3$se.fit))
glimpse(newdata2)

```

```

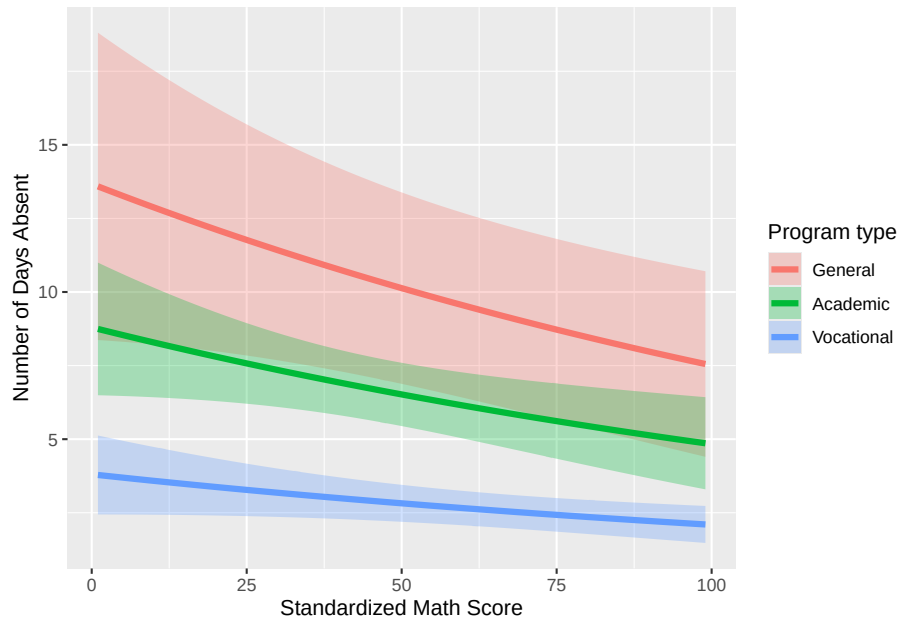
## Rows: 300
## Columns: 5
## $ math <dbl> 1.00000, 1.98990, 2.97980, 3.96970, 4.95960, 5.94949~
## $ prog <fct> General, General, General, General, General, General~
## $ DaysAbsent.Fitted <dbl> 13.5892, 13.5088, 13.4289, 13.3494, 13.2705, 13.1920~
## $ lo <dbl> 8.36713, 8.35375, 8.33964, 8.32482, 8.30928, 8.29302~
## $ up <dbl> 18.8112, 18.6638, 18.5181, 18.3741, 18.2317, 18.0910~

```

Note the code specifying `lo` and `up`, it follows standard confidence intervals

from your Intro Statistics course. Lastly, we can plot the fitted model with some aesthetic options.

```
ggplot(newdata2) +
  geom_ribbon(aes(x=math, ymin=lo, max=up, fill=prog), alpha=0.3) +
  geom_line(aes(x=math, y=DaysAbsent.Fitted, color=prog), size=1.5) +
  labs(fill="Program type", color="Program type", x="Standardized Math Score", y="Number of Days
```



Thus, we have plotted our fitted model. The final plot helps tell the underlying story with this data

- Program type has an effect on the number of days absent (vocational is less than academic which is less than general).
- Number of days absent decreases with higher standardized math scores.